

Specification of the Solidity Verifier

Code-Generative Lowering of the Halo2/KZG Verifier to EVM Yul

`midfall/proofs/solidity-verifier`

Generated specification — coverage-complete audit edition

Target curve: BLS12-381 · PCS: KZG (multi-open) · Transcript: Keccak-256 Fiat-Shamir ·
Precompiles: EIP-2537, EIP-198

Reference verifier	proofs/src/plonk/verifier.rs (+ poly/kzg)
Code generator	proofs/solidity-verifier/src/ (Askama in templates/)
Generated artifact	Halo2Verifier.sol (Yul-in-assembly); optional Halo2VerifyingKey, Halo2QuotientEvaluator
Equivalence evidence	differential trace tests (Rust solidity_trace vs. Solidity LOG1)

Contents

1	Scope and Supported Profile	8
1.1	What this document specifies	8
1.2	The supported profile	8
1.3	Reading guide	9
2	The Verifier Algorithm as Mathematics	11
2.1	Fields, curves, and domain	11
2.2	The Fiat–Shamir transcript	11
2.3	Verifying-key digest	12
2.4	The proof stream and challenge schedule	12
2.5	Lagrange basis and public-input evaluation	13
2.6	The identity families	14
2.6.1	Gate identities (custom constraints)	14
2.6.2	Permutation identities	14
2.6.3	Lookup (LogUp) identities	15
2.6.4	Trash identities	15
2.7	The y -batched quotient numerator	15
2.8	Linearization	15
2.9	KZG multi-open reduction	16
2.9.1	Point sets	16
2.9.2	Batched commitments and evals per set	16
2.9.3	The f polynomial and its evaluation at x_3	17
2.9.4	Final commitment and scalar v	17
2.9.5	The final pairing	17
2.10	Accumulator (IVC) pairing batching	17
2.11	Summary of the verification predicate	18
3	The Rust Reference Verifier	19
3.1	Module map	19
3.2	<code>parse_trace</code> — reading the proof into a trace	20
3.3	<code>verify_algebraic_constraints</code> — the main body	20
3.3.1	Instance validation	20
3.3.2	Quotient limbs and the evaluation challenge	20
3.3.3	Instance evaluation	21
3.3.4	Partial identity evaluation	21
3.3.5	Linearization and query assembly	21
3.4	<code>partially_evaluate_identities</code>	21
3.5	<code>compute_linearization_commitment</code>	22
3.6	Permutation identities	22
3.7	Lookup (LogUp) identities	22
3.8	Trash identities	22
3.9	Transcript implementation	22

3.9.1	Production G1 to_input (Keccak256)	23
3.10	Top-level entry: <code>prepare</code>	23
3.11	The <code>DualMSM</code> pairing accumulator	23
3.12	KZG <code>multi_prepare</code>	24
3.13	Arithmetic helpers	24
3.14	Trace instrumentation	24
4	The Code Generator	26
4.1	Pipeline overview	26
4.2	Validation: what the generator refuses	26
4.3	The protocol plan	27
4.3.1	The canonical read-order construction	27
4.3.2	Lookup chunking: <code>nb_parallel_lookups_per_chunk</code>	28
4.4	Memory layout	29
4.5	VK payload	29
4.6	Proof layout and repacking	30
4.7	The quotient virtual machine	30
4.8	PCS lowering	31
4.8.1	<code>construct_intermediate_sets_impl</code> — the point-set partition	31
4.8.2	<code>sort_sets</code> — ascending-cardinality ordering	32
4.8.3	<code>used_lagrange</code> — required common polynomials	32
4.8.4	<code>computations()</code> — the Yul emitter	33
4.8.5	<code>validate_g1_precompile_coverage</code> — the safety invariant	34
4.9	Rendering	34
5	The Generated Solidity and Yul	35
5.1	Contract structure	35
5.2	ABI and shape guard	35
5.3	Transcript helpers	36
5.4	The proof parser	36
5.5	Lagrange and instance evaluation	37
5.6	Quotient and linearization	37
5.7	PCS blocks	37
5.8	Final pairing	38
5.9	Accumulator batching	38
5.10	Precompile smoke tests	38
5.11	Trace and return	38
6	Three-Way Correspondence	39
6.1	Transcript and challenges	39
6.2	Proof parsing and the challenge schedule	39
6.3	Lagrange and instance evaluation	40
6.4	Identity families	40
6.5	Linearization	40
6.6	KZG multi-open	40
6.7	Final pairing and accumulator	41
6.8	Trace-id correspondence	41
7	The Faithfulness Argument	42
7.1	What “faithful” means	42
7.2	Semantic checkpoints	42
7.3	Differential trace testing	43
7.4	Mutation and rejection testing	43

7.5	Check-by-check preservation	43
7.5.1	Reverse inventory: every generated revert is justified	44
7.6	Fiat–Shamir transcript schedule	45
7.7	Deliberate deviations and why they are sound	45
7.8	Residual risks and mitigations	46
7.9	Generated-artifact cross-check	46
7.10	Summary of the argument	47
8	Coverage Ledger	48
8.1	Rust reference verifier	48
8.2	Code generator	49
8.3	Templates and generated artifacts	50
8.4	Coverage summary	50
8.4.1	Explicitly out of scope (prover-side)	51
9	Deep Audit: Yul Helper Functions	52
9.1	AssemblyHelpers.yul	52
9.1.1	scalar_inv	52
9.1.2	Transcript helpers: transcript_init, common_word	53
9.1.3	common_uncompressed_g1	53
9.1.4	squeeze_to	54
9.1.5	batch_invert	55
9.1.6	ec_pairing	56
9.1.7	Summary of AssemblyHelpers.yul findings	56
9.2	Lagrange.yul	57
9.3	TranscriptProofParser.yul	57
9.4	QuotientAndLinearization.yul	58
9.5	FinalPairing.yul	59
9.6	AccumulatorHelpers.yul	60
9.6.1	Accumulator final-pairing batching (verified on ivc-keccak)	60
9.7	Deviations summary	62
9.8	QuotientNumeratorBlock.yul — the compact quotient VM	62
9.8.1	Stack model	62
9.8.2	FOLD_MAIN (0x0a) — the <i>y</i> -batch fold	63
9.8.3	FOLD_SELECTOR (0x0b) — sparse selector buckets	63
9.8.4	Native callbacks (0x19 perm, 0x1f lookup, 0x1b heavy)	64
9.8.5	ADD/MUL stack underflow	64
9.8.6	Final negation	64
9.9	QuotientHelpers.yul	65
9.10	VM-specific improvement summary	65
9.11	Contract templates and deployment-time checks	65
9.11.1	Halo2Verifier.sol — the main contract	65
9.11.2	The "memory-safe" annotation — what it promises and what it relies on	66
9.11.3	PrecompileSmoke.sol — deployment-time fork check	67
9.11.4	Constructors.sol — dependency pinning	68
9.11.5	Halo2VerifyingKey.sol — INVALID-prefixed payload	68
9.11.6	Halo2QuotientEvaluator.sol — split evaluator	68
9.11.7	Pcs.yul and TraceReturn.yul	69

10 Consolidated improvement & action roadmap	70
10.1 Priority 1 — Security-relevant	70
10.2 Priority 2 — Defense-in-depth (load-bearing invariants)	71
10.3 Priority 3 — Gas / contract-space optimizations	73
10.4 Priority 4 — Audit-clarity / naming	73
10.5 Hardening already landed	74
10.6 What is <i>not</i> a deviation	75
10.7 Suggested implementation order	75
10.8 Consolidated deviation register	75
11 Production-Artifact Code Review	78
11.1 What this artifact is	78
11.2 Correctness	79
11.3 Soundness	79
11.4 Safety	80
11.5 Robustness	80
11.6 Readability	81
11.7 Repeated computations & patterns (gas / size)	81
11.7.1 Concrete gas profile (EIP-2537)	82
11.8 Audit-facilitation improvements	83
11.8.1 Worked example: the labeling asymmetry	84
11.8.2 Worked example: disassembling the <code>q_program</code>	84
11.8.3 Worked example: fused linearization MSM ↔ Rust <code>compute_linearization_commitment</code>	85
11.8.4 The per-identity trace manifest already exists in the codegen	87
11.8.5 VK payload layout: the data-interface audit	87
11.8.6 Accumulator public-input decode: a soundness-critical path	88
11.8.7 Worked example: final pairing assembly — the terminal soundness gate	89
11.8.8 Production transcript squeeze-order verification	90
11.8.9 Quotient VM ISA coverage and fail-closed dispatch	91
11.8.10 Deploy-time smoke tests: catching a broken chain before first proof	92
11.8.11 Lagrange basis evaluation: the multiplicative-subgroup math in code	93
11.8.12 Truncated powers: the 128-bit challenge reduction (faithful but flagged)	94
11.8.13 Instance canonical-range check: where Solidity is <i>more</i> defensive than Rust	94
11.8.14 G1 point validation: padded-uncompressed design coupling and deferred subgroup checks	95
11.8.15 Triple-layered proof-length validation: declared size, total calldata, and walk-cursor end-check	97
11.8.16 Multi-layered VK pinning: constructor, per-proof codehash re-check, and the INVALID-byte data-only trick	98
11.8.17 Transcript initialization and domain separation: empty start with the VK digest as the separator	99
11.8.18 The airtight terminal return path: revert-on-failure pairing + pre-check	100
11.8.19 The <code>memory-safe</code> promise: scratch at 0x80, never bump 0x40, terminal block	102
11.8.20 Calldata bounds safety: exact-length pinning prevents silent zero-reads	103
11.8.21 The modexp zero-input guard: a subtle soundness defense a naive implementation misses	105
11.8.22 The G1 on-curve check deferral and the codegen-time guarantee that closes it	106
11.8.23 The VM cached-top stack discipline: overflow/underflow prevented by codegen-time bound	107
11.8.24 The FOLD_SELECTOR gap mechanism: sparse y-batch position tracking mirroring Rust's reverse fold	108

11.8.25 The native callback boundary reset: defense against stack-state leakage . .	110
11.8.26 The VK payload as a self-consistent, documented byte layout	111
11.8.27 EIP-2537 precompile input encoding: correct by construction	112
11.8.28 The transcript absorb order: VK digest first, committed_pi identity fix . .	113
11.8.29 The accumulator pairing batching: Schwartz–Zippel soundness	114
11.8.30 The LogUp lookup callback: sum-of-reciprocals via prefix/suffix products .	115
11.8.31 The permutation boundary conditions: Lagrange-masked start/end/continuity	115
11.8.32 The heavy-gate callback: u16 index dispatch with manifest validation . . .	116
11.8.33 The MODARITH7 opcode: a dispatch optimization, not a math change . .	117
11.8.34 The q_pow5 shortcut: an addition-chain optimization recognized by the codegen	118
11.8.35 The post-VM validation: exact bytecode consumption and empty stack . .	119
11.8.36 The trash-constraint post-VM suffix: batched with powers of τ	120
11.8.37 The MUL_CONST_U8 opcode: a one-byte compact constant multiply . .	121
11.8.38 The foreign-field representation: lowered to Fr, not switched	121
11.8.39 The ISA-vs-dispatch discrepancy: an opcode-usage-pruned switch	122
11.8.40 The BILIN7_PAIRWISE convolution: coefficient indexed by $i+j$	123
11.8.41 The MODARITH7 escape-hatch blocks: terms outside the 7-limb shapes . .	124
11.8.42 The y-power precomputation table: O(1) lookup for sparse selector folds . .	125
11.8.43 The FOLD_MAIN structure: main vs. selector folds	126
11.8.44 The IVC accumulator public-input decoding subsystem	127
11.8.45 The codegen-side VK payload section map (vk_payload.rs)	129
11.8.46 The batched identity numerator emitter (yul_emit.rs)	130
11.8.47 The multi-set batched KZG f_eval: a barycentric rearrangement	131
11.8.48 The permutation product-equality constraint and the δ -pow chunk mechanism	133
11.8.49 The fail-closed dispatch chain: every unknown opcode reverts	134
11.8.50 The MODARITH7 operand byte-encoding (reading the q_program by hand)	135
11.8.51 The codegen emitter that produces the MODARITH7 bytes	136
11.8.52 The fusion decision: when the codegen fuses vs emits separate opcodes . . .	138
11.8.53 The codegen’s offline bytecode validator (static analysis before render) . . .	139
11.8.54 Constant interning: hash-consing the quotient constant table	140
11.8.55 Run compaction: a peephole optimizer over the emitted bytecode	141
11.8.56 Greedy shape recognition: how an expression is matched to MODARITH7 .	142
11.8.57 G1ADD, subgroup cancellation, and whether G1ADD can be eliminated . .	144
11.8.58 Static analysis tooling run: results and false-positive triage	146
11.8.59 The VM scratch memory map: a concrete legend (the I9 fix demonstrated)	147
11.8.60 The verifier is VK-data-generic within one circuit shape: block_size lives in the VK payload, not the code	149
11.8.61 The precompile-availability check tests acceptance but not rejection	150
11.8.62 Verified input-handling correctness (audit-surface reduction)	151
11.8.63 The accumulator codec: the hardest-to-audit subsystem	152
11.8.64 The VK trust boundary: codehash pin + header cross-checks	153
11.8.65 The proof-parser consumption check (verified consistent)	154
11.8.66 The final pairing: sound randomized batching, confusing helper parameter names	155
11.8.67 Vanishing polynomial and rotation points (verified)	156
11.8.68 Power tables are precomputed, not recomputed (efficiency credit)	157
11.8.69 The trust root does not trace-test the exact production VK	158
11.8.70 f_eval: Lagrange interpolation with inlined batch inversion	160
11.8.71 VM fail-closed boundary, and the hybrid execution model	160
11.8.72 Challenge-schedule cross-check against the Rust verifier	161

11.8.73 Gas: two 1-pair MSMs where one 2-pair MSM would do	162
11.8.74 Lookup (logup) accumulator read matches the Rust verifier	163
11.8.75 Permutation-argument read matches the Rust verifier	163
11.8.76 Public-input (instance) binding and the ABI-shape guard	164
11.8.77 <code>q_com</code> / KZG multi-open: the fused final MSM	165
11.8.78 NatSpec quality: good on mechanics, silent on the trust model	166
11.8.79 The VK codehash is a raw hex constant with no circuit-parameter provenance	167
11.8.80 All 34 revert paths are silent — no diagnostic information on failure	168
11.8.81 Helper-function deep review: the nine primitives that do the real work . . .	169
11.8.82 The quotient VM: a pinned bytecode interpreter and its native callbacks . .	173
11.8.83 The PCS section: the KZG multi-open final computation	177
11.9 Overall assessment	180
12 Design critique: what is wrong here, and how to fix it	181
12.1 C1 — The VM is the wrong abstraction	181
12.2 C2 — The trust root is differentially tested, but not formally verified	184
12.3 C3 — Truncated challenges: the soundness level is unstated	185
12.4 C4 — Load-bearing invariants are unenforced	185
12.5 C5 — The contract defeats stock automated tools	186
12.6 C6 — Compiler-internal names leak into the audited artifact	187
12.7 C7 — The pairing helper’s parameter names invert the slot names	187
12.8 C8 — No provenance metadata	188
12.9 C9 — Magic constants hand-synced with no assertion	188
12.10 C10 — The negated-numerator sign convention is a footgun	188
12.11 C11 — Build fragility: <code>via-ir</code> is load-bearing	189
12.12 C12 — “Generated” hides that the real artifact is the compiler	189
12.13 What the design actually gets right	190
12.14 Measured auditability scorecard	190
12.15 Audit runbook: how to actually audit this contract	192
12.16 Critique summary	193
13 Making the generated contract human-readable: a concrete plan	195
13.1 The measured readability debt	195
13.2 Fix 1 — Named memory addresses (highest impact)	196
13.3 Fix 2 — Semantic local names over <code>varN</code>	197
13.4 Fix 3 — Decompose <code>verifyProof</code> into named functions	198
13.5 Fix 4 — Annotate the fused <code>q_com</code> MSM pairs	199
13.6 Fix 5 — Emit a generated address-map appendix	199
13.7 Fix 6 — Collapse the five near-identical Horner blocks	200
13.8 Prioritized summary	200
14 Verification pass over the review chapters	201
14.1 Baseline	201
14.2 Reference measurements	201
14.3 Corrections that change a conclusion	202
14.4 Precompile gas limits	203
14.5 What still stands	204
14.6 Do the improvement and strengthening suggestions hold up?	204

Chapter 1

Scope and Supported Profile

1.1 What this document specifies

This document specifies the *Solidity verifier code generator* located in `proofs/solidity-verifier`, and justifies that the Solidity/Yul program it emits is a **faithful translation** of the Rust reference verifier in `proofs/src/plonk/verifier.rs` (together with its KZG multi-open sub-protocol in `proofs/src/poly/kzg/mod.rs`).

“Faithful” is given a precise operational meaning in Section 7.1: every algebraic check, every transcript absorption, every range check, and every rejection path present in the Rust verifier has a corresponding, semantically equivalent operation in the generated Solidity, and the two implementations produce *identical* intermediate values at every semantic checkpoint (verified by differential tracing, Chapter 7).

Three layers

The specification tracks three layers throughout:

1. **Rust reference verifier** (`proofs/src/...`) — the source of truth: halo2-style plonk verification over BLS12-381 with a KZG polynomial commitment scheme and a Keccak Fiat–Shamir transcript.
2. **Code generator** (`proofs/solidity-verifier/src/...`) — a Rust program that *plans* every fact about the verifier (memory layout, proof byte layout, transcript schedule, quotient bytecode, PCS schedule) and then renders Solidity/Yul through Askama templates.
3. **Generated Solidity** (`Halo2Verifier.sol`) — the terminal EVM program: a single `verifyProof(bytes,bytes[])` entry point that either returns `true` or reverts.

1.2 The supported profile

The generator deliberately supports a *narrow, pinned profile*. Faithfulness is claimed only inside this profile; outside it the generator refuses to emit (compile-time or plan-time errors), rather than emitting a subtly wrong verifier. The profile constraints, and where each is enforced, are:

Constraint	Enforcement site	Rust counterpart
Exactly one <i>committed</i> instance column (the identity commitment) and exactly one non-committed instance column.	<code>builder/api.rs::validate_instance_column_shape</code> rejects any other shape.	<code>verifier.rs::verify_algebraic_constraints</code> instance validation (committed non-empty, equal lengths, committed + normal = <code>num_instance_columns</code>).
No rotated queries against non-committed instance columns.	<code>builder/api.rs</code> rejects <code>RotatedInstanceQuery</code> .	Rust reads non-committed instances only via Lagrange interpolation at rotation 0; a rotated read would need a commitment the profile does not have.
BLS12-381 scalar field $\mathbb{F}_r$ with EIP-2537 precompiles; Cancun+ (MCOPY).	<code>evm.rs</code> pinned <code>solc 0.8.30</code> ; <code>PrecompileSmoke.sol</code> probes MCOPY/G1ADD/G1MSM/PAIRING at construction.	<code>transcript/implementors.rs</code> Keccak + EIP-2537 uncompressed G1 encoding.
Feature flags: <code>truncated challenges, fewerpoint sets, outersingleh commitment, committed instances, keccak transcript</code> .	<code>lib.rs</code> re-exports; lowering reads them to pick schedules.	<code>poly/kzg/mod.rs</code> <code>#[cfg(feature = ...)]</code> branches.
Pinned verifying key: VK digest, payload length, and payload codehash are baked into the generated contract as constants.	<code>lowering/vk.rs::generate_base_vk</code> ; constants <code>EXPECTED_VK_CODEHASH</code> etc. in <code>Constants.sol</code> .	<code>plonk/mod.rs::hash_into</code> absorbs <code>transcript_repr</code> first.

Remark 1.2.1 (Why a narrow profile is a feature, not a limitation). Faithfulness of a general lowering of an arbitrary halo2 verifier to gas-costed Yul is an open-ended problem. By pinning the circuit family (one identity commitment column, one public-instance column, fixed feature flags, pinned VK), the space of generated programs becomes finite and fully inspectable: every generated contract in this repository is accompanied by a recorded `EXPECTED_VK_CODEHASH` and a differential trace test that pins every intermediate scalar. The audit surface collapses from “all circuits” to “the lowering function itself”.

1.3 Reading guide

- **Chapter 2** states the verifier algorithm as mathematics: transcript sampling, Lagrange evaluation, the four identity families (gate, permutation, lookup, trash), the *y*-batched quotient numerator, the linearization, the KZG multi-open reduction, and the final pairing. Every later section refers back to equation numbers here.
- **Chapter 3** walks the Rust reference verifier function by function, with exact code excerpts.
- **Chapter 4** walks the code generator: how it plans the protocol, the memory map, the proof layout, the quotient virtual machine, and the PCS schedule.
- **Chapter 5** walks the generated Solidity/Yul.

- **Chapter 6** is the core deliverable: per-stage tables mapping `Rust` $\leftrightarrow$ `codegen` $\leftrightarrow$ `Solidity` with equation references.
- **Chapter 7** assembles the faithfulness argument: semantic checkpoints, trace-equivalence testing, mutation testing, and an explicit list of every check in the Rust verifier and where it lives in the generated code.
- **Chapter 8** is the coverage ledger: every module and public function visited, with status.

Notation conventions

Throughout, `orange text` denotes Rust reference-verifier identifiers, `blue text` denotes code-generator identifiers, and `slate text` denotes generated Solidity/Yul identifiers. Trace identifiers are written `id N`. Memory pointers are written in hexadecimal, e.g. `0x2b60`.

Chapter 2

The Verifier Algorithm as Mathematics

This chapter states the verification algorithm in closed mathematical form. Every equation is tagged (e.g. (2.3)) and referenced by the correspondence chapters. The Rust reference, the code generator, and the generated Solidity are each shown to implement exactly these equations.

2.1 Fields, curves, and domain

Let $\mathbb{F}_r$ be the BLS12-381 scalar field of prime order

$$r = 0x73eda753299d7d483339d80809a1d80553bda402fffe5bfeffffffff00000001,$$

and let $(\mathbb{G}_1, \mathbb{G}_2)$ be the two source groups of a pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$, both of order r , with generators $G \in \mathbb{G}_1$ and $G_2 \in \mathbb{G}_2$. All scalar arithmetic below is in $\mathbb{F}_r$ unless stated otherwise. The circuit domain is the multiplicative subgroup $H = \{1, \omega, \omega^2, \dots, \omega^{n-1}\} \subset \mathbb{F}_r^\times$ of order $n = 2^k$, where ω is a fixed primitive n -th root of unity and $\omega^{-1} = \omega^{n-1}$.

- $n = 2^k$ (from VK, `domain.k()`)
- $n^{-1} \bmod r$ (`EvaluationDomain::n_inv`, VK slot 3)
- ω, ω^{-1} (VK slots 4, 5)
- δ the coset separator, a fixed element of order 3 (`WithSmallOrderMulGroup<3>`), used by permutation chunks.

2.2 The Fiat–Shamir transcript

The transcript is a streaming Keccak-256 sponge with a reseed rule. It is a byte buffer B (initially empty) supporting two operations.

Definition 2.2.1 (Transcript absorption). `common( $m$ )` appends the raw byte string m to B : $B \leftarrow B \parallel m$.

Definition 2.2.2 (Transcript squeeze). `squeeze()` computes one digest, reseeds the buffer with it, and samples a field element:

$$d = \text{keccak256}(B), \tag{2.1}$$

$$B \leftarrow d, \tag{2.2}$$

$$c = \left(\sum_{j=0}^{31} 256^{31-j} d_j \right) \bmod r = \text{uint256}_{\text{BE}}(d) \bmod r. \tag{2.3}$$

Remark 2.2.1. The reseed of (2.2) means each squeeze depends on all prior material but the buffer never grows without bound: after the first squeeze, $|B| = 32$. The Solidity implementation is exactly this: the generated `squeeze_to` helper computes `keccak256(TRANSCRIPT_MPTR, len)`, mstores the digest back at `TRANSCRIPT_MPTR`, and stores `mod(h, r)` at the target pointer. See [transcript/implementors.rs](#) (Keccak256 squeeze, L186) and [templates/partials/verifier/AssemblyHelpers.yul](#).

Point and scalar encodings. A field element is absorbed as its 32-byte big-endian canonical encoding (`Fq::to_input` = reversed little-endian repr). A $\mathbb{G}_1$ point is absorbed in the EIP-2537 *padded uncompressed* form:

$$\tilde{P} = \underbrace{0^{16}}_{\text{pad}} \parallel x_{\text{hi}} \parallel \underbrace{0^{16}}_{\text{pad}} \parallel x_{\text{lo}} \parallel \underbrace{0^{16}}_{\text{pad}} \parallel y_{\text{hi}} \parallel \underbrace{0^{16}}_{\text{pad}} \parallel y_{\text{lo}} \in \{0, 1\}^{128}, \quad (2.4)$$

where $(x_{\text{hi}}, x_{\text{lo}}, y_{\text{hi}}, y_{\text{lo}})$ are the four 32-byte big-endian halves of the affine coordinates $(x = x_{\text{hi}} \cdot 2^{128} + x_{\text{lo}})$. The identity point is encoded as 128 zero bytes. This is the patched `Hashable<Keccak256> for G1Projective::to_input` in [transcript/implementors.rs](#) (L283).

Definition 2.2.3 (Canonical G1 acceptance). The generated verifier accepts a 128-byte G1 word only if the top 16 bytes of each `_hi` word are zero and each coordinate is $< q$ (the BLS12-381 base field modulus). Normalizing instead of rejecting would let multiple calldata encodings share one transcript. Curve membership itself is not checked at absorption; it is enforced later by the EIP-2537 precompiles that consume every absorbed point (Section 2.9).

2.3 Verifying-key digest

The verifier first absorbs a binding hash of the circuit:

$$\text{vk_digest} = \text{tr_repr} \in \mathbb{F}_r, \quad \text{tr_repr} = \text{Blake2b}(\text{format}(\text{VK params}) \parallel \text{format}(\text{CS})) \bmod r, \quad (2.5)$$

computed once off-chain by `plonk/mod.rs::transcript_repr` and stored in the VK payload. The generated contract loads it from `VK_MPTR+0x00` (`lowering/vk.rs::generate_base_vk`) and absorbs it as the very first transcript word, mirroring `verifier.rs: vk.hash_into(transcript)`. The payload itself is pinned by `EXPECTED_VK_CODEHASH` so the contract cannot be fed a different circuit.

2.4 The proof stream and challenge schedule

The proof is a flat byte string read in a fixed order. Table 2.1 gives the canonical order; each row is either an *absorb* (Definition 2.2.1) or a *squeeze* (Definition 2.2.2). This order is the single most important invariant of the whole system: any permutation changes every subsequent challenge and the proof fails.

#	Op	Object	Notes
1	absorb	<code>vk_digest</code> (Fr)	Eq. (2.5).
2	absorb	committed instance = $\mathcal{O}$ (128 zero bytes)	Identity-commitment placeholder (<code>committed_instances</code>).
3	absorb	<code>num_instances</code> (Fr)	Public input count.
4	absorb	each public instance <code>inst_i</code> (Fr)	Canonical check <code>inst_i < r</code> .

#	Op	Object	Notes
*	absorb	advice commitments $\tilde{a}_j$ (G1), phase by phase	$n_{\text{adv},\phi}$ commitments per user phase ϕ .
*	squeeze	user challenges v_ϕ	After each phase's advice.
5	squeeze	θ	Lookup input batching.
6	absorb	lookup multiplicity commit- ments $\tilde{m}_\ell$ (G1)	One per lookup.
7	squeeze	β	Permutation/lookup ran- domizer.
8	squeeze	γ	Permutation/lookup ran- domizer.
9	absorb	permutation product com- mitments $\tilde{z}_0, \tilde{z}_1, \dots$ (G1)	One per permutation set.
10	absorb	lookup helper commitments $\tilde{h}_{\ell,t}$ (G1)	chunks_ℓ per lookup.
11	absorb	lookup accumulator $\tilde{Z}_\ell$ (G1)	After its helpers.
12	squeeze	<code>trash_challenge</code>	Trash batching.
13	absorb	trash commitments $\tilde{tr}_k$ (G1)	One per trashcan.
14	squeeze	y	Identity batching.
15	absorb	quotient limbs $Q_0, \dots, Q_{L-1}$ (G1)	$L = \text{get_quotient_poly_}$ degree .
16	squeeze	x	Evaluation point.
17	absorb	all evaluation scalars (Fr)	Order fixed by the lowering plan (Sec. 4.3).
18	squeeze	x_1	Multi-open batching.
19	squeeze	x_2	Multi-open batching.
20	absorb	f_{com} (G1)	Batched poly commitment.
21	squeeze	x_3	Then truncated: $x_3 \leftarrow x_3 \bmod 2^{128}$ (<code>truncated-challenges</code>).
22	absorb	$q_{\text{eval}}^{(s)}$ (Fr)	One per point set.
23	squeeze	x_4	Final batching challenge.
24	absorb	π (G1)	KZG opening proof.

Table 2.1: Canonical transcript schedule. Rows marked * repeat per user phase. The generated `TranscriptProofParser.yul` and `verifier.rs` agree row-for-row; the differential trace pins this.

2.5 Lagrange basis and public-input evaluation

For the evaluation point x sampled in row 16, define the vanishing polynomial of H and the Lagrange basis:

$$Z_H(x) = x^n - 1, \quad (2.6)$$

$$L_i(x) = \frac{\omega^i Z_H(x)}{n(x - \omega^i)} = \frac{\omega^i (x^n - 1)}{n(x - \omega^i)}, \quad i = 0, \dots, n-1. \quad (2.7)$$

The verifier needs three special basis elements and the public-input evaluation:

$$\ell_{\text{last}} = L_{n-b-1}(x) \quad (\text{rotation} - (b+1)), \quad (2.8)$$

$$\ell_{\text{blind}} = \sum_{j=1}^b L_{n-j}(x) \quad (b = \text{blinding factors}), \quad (2.9)$$

$$\ell_0 = L_0(x), \quad (2.10)$$

$$\widehat{\text{inst}}(x) = \sum_{i=0}^{\text{num_instances}-1} L_i(x) \cdot \text{inst}_i. \quad (2.11)$$

The barycentric weight of the full Lagrange basis is $1/n$ (`domain.rs::l_i_range`, L448). The generated `Lagrange.yul` computes x^n by k repeated squarings, batch-inverts the denominators $(x - \omega^i)$ with one modexp (EIP-198) via Montgomery’s trick, then forms each L_i and the sums (2.8)–(2.11).

2.6 The identity families

A satisfied circuit makes a family of polynomial identities vanish on H . The verifier checks each identity *at the random point* x , using the prover-supplied evaluation scalars. There are four families.

2.6.1 Gate identities (custom constraints)

Each gate g with simple selector s_g contributes the identity

$$e_g^{\text{gate}}(x) = s_g(x) \cdot C_g(\{\text{advice, fixed, instance evals at rotations}\}) = 0, \quad (2.12)$$

where C_g is the gate’s polynomial expression. In the supported profile every selector is *simple and multiplicative*: its evaluation is the literal 1 in the linearization and the gate’s contribution is routed to a per-selector accumulator bucket B_{s_g} (Section 2.8) rather than the main numerator. This is why simple selectors have *no* proof evaluation slot: the Rust verifier inserts `F::ONE` at their `fixed_query` index (`verifier.rs`, fixed-eval loop), and the codegen emits a literal 1 and a bucket copy.

2.6.2 Permutation identities

The permutation argument enforces copy constraints across columns using product polynomials $z_0, z_1, \dots$ (one per “set” of chunked columns). With δ the order-3 coset separator, chunk length $\text{cl} = \text{cs_degree} - 2$, and per-chunk running constant $\Delta_{\text{cur}} = \beta x \delta^{\text{chunk} \cdot \text{cl}}$, the four identity families of `permutation.rs::expressions` (L187) are:

$$e_1^{\text{perm}} = \ell_0 \cdot (1 - z_0(x)), \quad (2.13)$$

$$e_2^{\text{perm}} = \ell_{\text{last}} \cdot (z_{\text{last}}(x)^2 - z_{\text{last}}(x)), \quad (2.14)$$

$$e_{3,i}^{\text{perm}} = \ell_0 \cdot (z_i(x) - z_{i-1,\text{last}}(x)) \quad (i \geq 1), \quad (2.15)$$

$$e_4^{\text{perm}} = (1 - (\ell_{\text{last}} + \ell_{\text{blind}})) \cdot (\text{left}(x) - \text{right}(x)), \quad (2.16)$$

where, for the current chunk,

$$\text{left}(x) = z_{\text{next}}(x) \cdot \prod_c (\text{eval}_c + \beta s_c(x) + \gamma), \quad (2.17)$$

$$\text{right}(x) = z(x) \cdot \prod_c (\text{eval}_c + \delta^c \beta x + \gamma), \quad (2.18)$$

with c ranging over the columns in the chunk, $s_c(x)$ the permutation polynomial evaluations (the “perm common” evals), and rotations $x_{\text{next}} = \omega x$, $x_{\text{last}} = \omega^{-(b+1)}x$ (`permutation/verifier.rs::queries`).

2.6.3 Lookup (LogUp) identities

Each lookup argument ℓ compresses its input expressions with θ : $f(x) = \theta$ -fold of the input evals, i.e. $\text{compress}_\theta(f_1, \dots, f_t) = (\dots((f_1\theta + f_2)\theta + f_3)\dots)\theta + f_t$. The helper polynomial h and accumulator Z satisfy, per helper chunk:

$$e_{\text{helper}}^{\text{lu}} = h(x) \cdot \prod_j (f_j(x) + \beta) - \sum_j \prod_{k \neq j} (f_k(x) + \beta) = 0, \quad (2.19)$$

(the partial products are formed as $(\prod_k (f_k + \beta)) \cdot (f_j + \beta)^{-1}$), and the accumulator identity

$$e_{\text{acc}}^{\text{lu}} = (Z(\omega x) - Z(x) - q(x) \sum h(x)) \cdot (f(x) + \beta) + m(x), \quad (2.20)$$

multiplied by the active-row mask $1 - (\ell_{\text{last}} + \ell_{\text{blind}})$, with the boundary term $(\ell_0 + \ell_{\text{last}}) \cdot Z(x)$ (`logup.rs::expressions`, L400). Here $m(x)$ is the multiplicity polynomial evaluation and q the lookup selector.

2.6.4 Trash identities

Unused constraint slots are “trashed”: their constraint expressions are compressed with `trash_challenge` and must equal the trash accumulator times the complement of the trash selector:

$$e^{\text{trash}} = \text{compress}_{\text{tc}}(\text{constraint evals}) - (1 - q_{\text{trash}}(x)) \cdot \text{trash}(x) = 0. \quad (2.21)$$

(`trash.rs::expressions`, L58.)

2.7 The y -batched quotient numerator

All identities from Section 2.6, in the fixed order (gates $\rightarrow$ permutation $\rightarrow$ lookup $\rightarrow$ trash), are batched with the challenge y into a single numerator:

$$\nu_y(x) = \sum_{i=0}^{m-1} y^{m-1-i} e_i(x), \quad (2.22)$$

where $e_0, \dots, e_{m-1}$ are the identity evaluations in order. The circuit-satisfaction claim is: $\nu_y(x) = h(x) \cdot (x^n - 1)$ for some quotient polynomial h , whose limbs $Q_0, \dots, Q_{L-1}$ the prover commits to (row 15).

Definition 2.7.1 (Verifier stores the negated numerator). The verifier never divides by $x^n - 1$. Instead it stores the *negated* numerator as the linearization’s expected evaluation:

$$\text{expected_eval} = -\nu_y(x). \quad (2.23)$$

The $(x^n - 1)$ factor is carried on the *commitment* side as $(1 - x^n)$ (Section 2.8). The two signs cancel in the pairing check. The generated verifier stores $-\nu_y(x)$ at `QUOTIENT_EVAL_MPTR`; trace id 36 re-negates it so the Solidity trace matches Rust’s positive $\nu_y(x)$.

2.8 Linearization

The linearization polynomial is the object whose commitment is checked to open to (2.23) at x . From `linearization/verifier.rs::compute_linearization_commitment`:

$$C_{\text{lin}}(x) = \underbrace{(1 - x^n) \sum_{i=0}^{L-1} x_{\text{split}}^i Q_i}_{\text{quotient limbs}} + \underbrace{\sum_j B_j \cdot F_j(x)}_{\text{selector buckets}} + \underbrace{\nu_y(x)}_{\text{moved to expected side}}, \quad (2.24)$$

where $x_{\text{split}} = x^{n-1}$ (the *splitting factor*, `verifier.rs` L326), F_j is the fixed-column commitment of simple selector j , and the bucket scalars are

$$B_j = \sum_{i: \text{col}(i)=j} y^{m-1-i} e_i(x), \quad \text{expected_eval} = - \sum_{i: \text{col}(i)=\perp} y^{m-1-i} e_i(x), \quad (2.25)$$

where $\text{col}(i) \in \{0, \dots, f-1\} \cup \{\perp\}$ is the Rust `Option<usize>` column tag: $\text{col}(i) = j$ (a `Some(j)`) means identity i 's grouped scalar multiplies fixed commitment F_j , while $\text{col}(i) = \perp$ (the Rust `None`) marks a *fully-evaluated* identity that has no column to attach to and therefore passes to the scalar expected-eval side. ($\perp$ denotes the `None` variant, not the empty set.) The Rust code groups expressions by column index with a running y^{pow} while iterating the expression list in *reverse* (`expressions.iter().rev()`, `y_pow *= y`), which is exactly the Horner form of (2.22): the last identity gets weight y^0 , the first gets y^{m-1} . Identities tagged `None` (fully evaluated) contribute to `expected_eval` with a negation (`expected_eval -= eval`); identities tagged `Some(j)` multiply the fixed commitment F_j .

Remark 2.8.1 (Why the split $x_{\text{split}} = x^{n-1}$). The quotient h has degree up to $L \cdot n - 1$ and is committed as L limbs of degree $< n$. Evaluating $h(x) = \sum_i x^i h_i(x^n)$ -style with the splitting factor x^{n-1} lets the verifier combine limb commitments without knowing h ; the $(1 - x^n)$ scalar then supplies the $-(x^n - 1)$ that cancels the denominator of (2.23).

2.9 KZG multi-open reduction

The verifier now holds a set of *opening claims* $\{(C_k, z_k, v_k)\}$: commitment C_k must open to value v_k at point z_k . These come from advice, committed instance, permutation z , lookup, trash, fixed (non-simple), perm-common, and the linearization (Section 4.3). The KZG multi-open (`poly/kzg/mod.rs::multi_prepare`, L321) reduces them to one pairing.

2.9.1 Point sets

Group the claims by evaluation point into *point sets* $S_1, \dots, S_t$ (rows of `construct_intermediate_sets`, `utils.rs` L37). With the `fewer-point-sets` feature, *dummy queries* (`compute_dummy_queries`, L150) are added to merge sets, each dummy eval read from the transcript. The sets are then sorted by **ascending cardinality, tie-broken by original index** (`sort_by_key(|i|, i)`, L419), so the first set contains the fixed commitments evaluated at x only.

2.9.2 Batched commitments and evals per set

For each set S_s , let the claims in it be indexed $k = 0, \dots, n_s - 1$ (each claim itself possibly a linear combination of commitments). Define

$$q_{\text{com}}^{(s)} = \sum_k x_1^k C_{s,k}, \quad (2.26)$$

$$q_{\text{eval}}^{(s)} = \sum_k x_1^k v_{s,k}, \quad (2.27)$$

with x_1 from row 18. Under `truncated-challenges` the powers are `truncate( $x_1^k$ )` (lower 128 bits) while the internal power accumulator stays full precision.

2.9.3 The f polynomial and its evaluation at x_3

For each set, let $r_s(X)$ be the Lagrange interpolation of the claimed values over the set's points: $r_s(\zeta_{s,i}) = v_{s,i}$ for $\zeta_{s,i} \in S_i$. Then

$$f_{\text{eval}} = \sum_{s=0}^{t-1} x_2^{t-1-s} \cdot \frac{q_{\text{eval,proof}}^{(s)} - r_s(x_3)}{\prod_{\zeta \in S_s} (x_3 - \zeta)}, \quad (2.28)$$

where $q_{\text{eval,proof}}^{(s)}$ are the prover-supplied scalars read in row 22 and x_3 is the truncated point from row 21. The sum is folded in *reverse* set order with Horner in x_2 ([multi_prepare](#) L471–482). The prover's f_{com} (row 20) commits to f ; (2.28) checks the claimed f -values are consistent with the per-set interpolants.

2.9.4 Final commitment and scalar v

With x_4 from row 23:

$$C_{\text{final}} = \sum_{s=0}^{t-1} x_4^s q_{\text{com}}^{(s)} + x_4^t f_{\text{com}}, \quad (2.29)$$

$$v = \sum_{s=0}^{t-1} x_4^s q_{\text{eval,proof}}^{(s)} + x_4^t f_{\text{eval}}. \quad (2.30)$$

(Again truncated powers under `truncated-challenges`.)

2.9.5 The final pairing

The KZG opening of C_{final} to v at x_3 with proof π is

$$e(C_{\text{final}} - vG + x_3\pi, G_2) = e(\pi, [s]_2), \quad (2.31)$$

where $[s]_2$ is the VK's `s_g2` point and G_2 the generator. Equivalently, with the Rust `DualMSM` naming (`left` = π , `right` = $C_{\text{final}} - vG + x_3\pi$):

$$e(\text{left}, [s]_2) \cdot e(\text{right}, -G_2) = 1. \quad (2.32)$$

The generated contract calls `ec_pairing(success, PAIRING_RHS_MPTR, PAIRING_LHS_MPTR)` which checks $e(\text{arg}_0, [1]_2) \cdot e(\text{arg}_1, -[s]_2) = 1$, i.e. $\text{arg}_0 = \text{RHS}$ (combined), $\text{arg}_1 = \pi$ — the *swapped* argument order of the historical LHS/RHS naming (`left` = π in the dual MSM). See Section 5.8.

2.10 Accumulator (IVC) pairing batching

When the proof carries an outer IVC accumulator, a second pairing equation must hold. The two equations are combined with a verifier-local randomizer α derived from the already-fixed pairing inputs:

$$e(\text{kzg_rhs} + \alpha \text{acc_rhs}, G_2) \cdot e(\text{kzg_lhs} + \alpha \text{acc_lhs}, -[s]_2) = 1. \quad (2.33)$$

If either original equation is false, (2.33) holds for at most one $\alpha \in \mathbb{F}_r$, so a cheating prover passes with probability $\leq 1/r$. The randomizer is $\alpha = \text{keccak256}(\text{domain} \parallel \text{points}) \bmod r$, with the zero case mapped to 1 ([FinalPairing.yul](#)).

2.11 Summary of the verification predicate

The verifier accepts iff all of the following hold:

1. the proof parses exactly (byte-exact consumption, Section 2.4);
2. every absorbed scalar is canonical ($< r$) and every G1 word satisfies Definition 2.2.3;
3. the challenges are sampled per Definition 2.2.2;
4. the identities (2.12), (2.13)–(2.18), (2.19)–(2.20), (2.21) batch to $\nu_y(x)$ (2.22) consistent with the committed quotient limbs via (2.24);
5. the multi-open reduction (2.26)–(2.30) is self-consistent; and
6. the pairing (2.31) (or (2.33)) holds.

Every one of these is enforced by the generated Solidity; the mapping is the subject of the next three chapters.

Chapter 3

The Rust Reference Verifier

This chapter walks the Rust verifier that the code generator must reproduce. It is the *source of truth*: if the generated Solidity disagrees with anything here, the Solidity is wrong. All line numbers refer to the checkout at the time of writing.

3.1 Module map

Module	Role
<code>plonk/verifier.rs</code> (696 L)	Top-level: <code>parse_trace</code> , <code>verify_algebraic_constraints</code> , <code>prepare</code> . Drives the whole transcript schedule and assembles the PCS query list.
<code>plonk/mod.rs</code>	<code>partially_evaluate_identities</code> (L488): builds the ordered identity-evaluation list; <code>hash_into</code> (L283): VK digest absorption.
<code>plonk/linearization/verifier.rs</code> (104 L)	<code>compute_linearization_commitment</code> : Eq. (2.24).
<code>plonk/permutation.rs</code>	<code>expressions</code> (L187): the four permutation identity families (2.13)–(2.18).
<code>plonk/permutation/verifier.rs</code> (198 L)	Reads z commitments, evaluates $z/z_{\text{next}}/z_{\text{last}}$, emits PCS queries.
<code>plonk/logup.rs</code>	<code>expressions</code> (L400): helper (2.19) and accumulator (2.20) identities.
<code>plonk/logup/verifier.rs</code> (212 L)	Reads multiplicities, helpers, accumulator; evaluates and emits PCS queries.
<code>plonk/trash.rs</code>	<code>expressions</code> (L58): trash identity (2.21).
<code>plonk/trash/verifier.rs</code> (75 L)	Reads trash commitments, evaluates, emits PCS queries.
<code>transcript/mod.rs</code> (184 L)	<code>Transcript</code> trait: <code>common/read/squeeze_challenge/assert_empty</code> .
<code>transcript/implementors.rs</code> (341 L)	Keccak256 hash; <code>sample_keccak_digest_be_mod_r</code> (L19); G1/Fq <code>to_input</code> .
<code>poly/kzg/mod.rs</code> (738 L)	<code>multi_prepare</code> (L321): the KZG multi-open reduction (Sec. 2.9).
<code>poly/kzg/utils.rs</code> (187 L)	<code>construct_intermediate_sets</code> (L37), <code>compute_dummy_queries</code> (L150).
<code>poly/kzg/msm.rs</code>	DualMSM, final pairing check (2.32).
<code>poly/domain.rs</code>	<code>rotate_omega</code> (L410), <code>l_i_range</code> (L448), <code>get_quotient_poly_degree</code> (L476).

Module	Role
<code>utils/arithmetic.rs</code>	<code>truncate</code> (L247), <code>truncated_powers</code> (L255), <code>powers</code> , <code>inner_product</code> , <code>msm_inner_product</code> , <code>lagrange_interpolate</code> (L172), <code>eval_polynomial</code> (L53).
<code>poly/query.rs</code>	<code>VerifierQuery</code> , <code>CommitmentReference</code> , <code>CommitmentLabel</code> .

3.2 `parse_trace` — reading the proof into a trace

`parse_trace` (L24) consumes the proof byte stream and produces a `VerifierTrace` struct holding every commitment and challenge in transcript order. This is the function whose schedule Table 2.1 describes. Its shape (excerpt, L24–120):

```
pub fn parse_trace<F, CS: PolynomialCommitmentScheme<F>, T: Transcript>(
    vk: &VerifyingKey<F, CS>,
    instances: &[&[F]],
    transcript: &mut T,
) -> Result<VerifierTrace<F, CS>, Error> {
    vk.hash_into(transcript); // (1) vk_digest absorb
    // committed-instance identity absorb, num_instances + instance absorbs
    // per-phase advice reads + challenge squeezes
    // theta, multiplicities, beta, gamma, permutation product comms,
    // lookup helpers + accumulator, trash_challenge, trash comms, y,
    // quotient limbs, x, ...
}
```

The trace struct (`plonk/traces.rs`) carries:

```
pub struct VerifierTrace<F, CS> {
    pub advice_commitments: Vec<Vec<CS::Commitment>>, // per user phase
    pub lookups: Vec<LookupTrace<CS>>, // m, helpers, acc
    pub trashcans: Vec<CS::Commitment>,
    pub permutations: Vec<CS::Commitment>, // z products
    pub challenges: Vec<F>, // user challenges
    pub beta: F, pub gamma: F, pub theta: F,
    pub trash_challenge: F, pub y: F,
}
```

3.3 `verify_algebraic_constraints` — the main body

This is the function the generated Solidity mirrors most directly. Its signature (L238) takes the VK, the parsed trace, the committed instances, the public instances, and the transcript, and returns a PCS verification guard. The body proceeds in the following stages.

3.3.1 Instance validation

```
if committed_instances.is_empty() {
    return Err(Error::InvalidInstances);
}
let nb_committed_instances = committed_instances[0].len();
let num_proofs = instances.len();
```

The supported profile additionally requires exactly one committed and one non-committed instance column; the codegen enforces this at build time (Section 1.2) so the generated contract never has to branch on it.

3.3.2 Quotient limbs and the evaluation challenge

```
let nb_quotient_coms = vk.domain.get_quotient_poly_degree();
let quotient_limb_coms = read_n(transcript, nb_quotient_coms)?;
let x: F = transcript.squeeze_challenge();
let splitting_factor = x.pow_vartime([vk.n() - 1]);
let xn = splitting_factor * x;
```

This is rows 15–16 of Table 2.1 and the splitting factor of Eq. (2.24). `single-h-commitment` collapses the limbs to one commitment (the outer-single-h-commitment profile).

3.3.3 Instance evaluation

```
let l_i_s = &vk.domain.l_i_range(
  x, xn,
  -max_rotation..max_instance_len as i32 + min_rotation.abs(),
);
```

Implements (2.7)–(2.11). The `l_i_range` helper uses the barycentric weight $1/n$ (`domain.rs` L448).

3.3.4 Partial identity evaluation

```
let expressions = partially_evaluate_identities(
  vk, &fixed_evals, &instance_evals, &advice_evals,
  permutations_evaluated.iter().map(|e| &e.sets),
  lookups_evaluated.iter().map(|e| e.iter().map(|linner| &linner.evaluated)),
  trashcans_evaluated.iter().map(|e| e.iter().map(|linner| &linner.evaluated)),
  &permutations_common, x, xn, beta, gamma, theta, trash_challenge,
  &challenges,
);
```

Returns `Vec<(Option<usize>, F)>`: each identity’s evaluation tagged with its simple-selector column index (`None` = fully evaluated). This is the ordered list $(e_i(x))$ of (2.22).

3.3.5 Linearization and query assembly

```
let lin_com = compute_linearization_commitment(
  expressions, vk, x, &y, &xn, &splitting_factor, &quotient_limb_coms);
// ... queries = advice ++ committed-instance ++ perm-z ++ lookup ++ trash
// ++ fixed(non-simple) ++ perm-common ++ lin_com
CS::multi_prepare(&queries, transcript).map_err(|_| Error::Opening)
```

The query list order (L577–652) is exactly the PCS query order the codegen’s `ProtocolPlan` reproduces (Section 4.3).

3.4 `partially_evaluate_identities`

In `plonk/mod.rs` (L488), this function:

1. computes $\ell_{\text{last}}, \ell_{\text{blind}}, \ell_0$ from (2.8)–(2.10);
2. for each gate: evaluates the gate polynomial with the fixed/advice/ instance evals, and tags it with the simple-selector index via `gate.queryed_selectors().filter(is_simple).next()`;
3. appends permutation expressions (tagged `None`), lookup expressions (`None`), trash expressions (`None`).

The ordering gates $\rightarrow$ permutation $\rightarrow$ lookup $\rightarrow$ trash fixes the y -weights in (2.22).

3.5 compute_linearization_commitment

The full function was quoted in Section 2.8; the key excerpt (L63–94):

```
identities_points.extend(quotient_limb_commitments);
let mut splitting_pow = F::ONE - *xn;
for _ in 0..quotient_limb_commitments.len() {
    identities_scalars.push(splitting_pow);
    splitting_pow *= splitting_factor;
}
let mut grouped_points: BTreeMap<Option<usize>, F> = BTreeMap::new();
let mut y_pow = F::ONE;
expressions.iter().rev().for_each(|(col_idx, eval)| {
    *grouped_points.entry(*col_idx).or_insert(F::ZERO) += y_pow * eval;
    y_pow *= y;
});
let mut expected_eval = F::ZERO;
grouped_points.into_iter().for_each(|(col_idx, eval)| {
    match col_idx {
        Some(col_idx) => { identities_points.push(&vk.fixed_commitments[col_idx]);
                           identities_scalars.push(eval); }
        None => { expected_eval -= eval; }
    }
});
```

This is literally (2.24) and (2.25): the quotient limbs get scalars $(1 - x^n) \cdot x_{\text{split}}^i$; the reverse iteration with running y^{pow} is the Horner form of (2.22); `None` entries negate into the expected eval (Definition 2.7.1).

3.6 Permutation identities

`permutation.rs::expressions` (L187) builds the four families (2.13)–(2.18). The chunk running constant is

```
let current_delta = beta * x * delta.pow_vartime([(chunk_index * chunk_len) as u64]);
```

with `chunk_len = cs_degree - 2`. The verifier side (`permutation/verifier.rs`) reads `read_product_commitments` (one z per set), evaluates z_{eval} , $z_{\text{next_eval}}$ and $z_{\text{last_eval}}$ (the latter only when more than one set exists), and emits PCS queries at x , $x_{\text{next}} = \omega x$, $x_{\text{last}} = \omega^{-(b+1)}x$.

3.7 Lookup (LogUp) identities

`logup.rs::expressions` (L400): compression $\text{fold}(\text{acc} \cdot \theta + \text{eval})$; boundary $(\ell_0 + \ell_{\text{last}}) \cdot Z_{\text{eval}}$; helper constraint (2.19) with partial products via $(\prod_k (f_k + \beta)) \cdot (f_j + \beta)^{-1}$; accumulator constraint (2.20) times the active-row mask $1 - (\ell_{\text{last}} + \ell_{\text{blind}})$. The verifier (`logup/verifier.rs`) reads multiplicities, then per lookup `nb_chunks` helper commitments + 1 accumulator, evaluates m , helpers, z , z_{next} , and emits PCS queries.

3.8 Trash identities

`trash.rs::expressions` (L58): compressed = fold of constraint evals with $\text{acc} \cdot \text{trash_challenge} + \text{eval}$; identity = compressed $- (1 - q) \cdot \text{trash_eval}$ (2.21). The verifier reads committed trash, evaluates, emits queries.

3.9 Transcript implementation

`transcript/implementors.rs` defines the Keccak256 hash (L186):

```
fn squeeze(&mut self) -> [u8; 32] {
    let digest = self.state.clone().finalize(); // clone, don't consume
    self.state.reset();
    self.state.update(digest.as_bytes());      // reseed with digest
    *digest.as_ref()
}
```

with sampling `sample_keccak_digest_be_mod_r` (L19) interpreting the digest big-endian and reducing mod r . G1 `to_input` (L283) emits the 128-byte EIP-2537 uncompressed form (2.4); Fq `to_input` (L315) emits the reversed (big-endian) repr. These are the exact byte-level contracts the generated parser must match.

3.9.1 Production G1 `to_input` (Keccak256)

The verifier’s G1 absorption uses `Hashable<Keccak256> for midnight_curves::G1Projective` (implementors.rs L264–297):

```
fn to_input(&self) -> Vec<u8> {
    let aff = G1Affine::from(self);
    let mut out = vec![0u8; 128];
    if !bool::from(aff.is_identity()) {
        let raw = <G1Affine as UncompressedEncoding>::to_uncompressed(&aff);
        let bytes: &[u8] = raw.as_ref();
        out[16..64].copy_from_slice(&bytes[0..48]); // x_be
        out[80..128].copy_from_slice(&bytes[48..96]); // y_be
    }
    out // identity => 128 zero bytes
}
```

This is the precise contract the generated `common_uncompressed_g1` mirrors: the identity encodes as 128 zero bytes (EIP-2537 convention, *not* the BLS-spec 0x40-leading form), and non-identity points place the 48-byte big-endian coordinates at offsets 16 and 80. The wire format (`to_bytes`) is unchanged — 48-byte compressed — so the uncompressed form is a transcript-only encoding.

3.10 Top-level entry: `prepare`

The public verifier entry point `plonk/verifier.rs::prepare` (L659) is a thin composition of the two functions already covered:

```
pub fn prepare<F, CS, T>(vk, committed_instances, instances, transcript)
    -> Result<CS::VerificationGuard, Error>
{
    let trace = parse_trace(vk, committed_instances, instances, transcript)?;
    verify_algebraic_constraints(vk, trace, committed_instances, instances, transcript)
}
```

It returns the PCS `VerificationGuard` (the `DualMSM`) rather than a boolean; the caller invokes `Guard::verify` to perform the final pairing. The generated Solidity collapses this three-step flow (`parse_trace` → `verify_algebraic_constraints` → `DualMSM::check`) into one `verifyProof` that returns `true` or reverts — the guard is never materialized as a value.

3.11 The `DualMSM` pairing accumulator

The PCS guard is `DualMSM<E>` (`poly/kzg/msm.rs`), a two-channel MSM accumulator. Its `check` (L284) is the final pairing:

```
pub fn check(self, params: &ParamsVerifierKZG<E>) -> bool {
    let left = if self.left.scalars.len() == 1 && self.left.scalars[0] == F::ONE {
        self.left.bases[0] // single scalar-1 term: skip MSM
    } else {
```

```

        self.left.eval()
    };
    let right = self.right.eval();
    let terms = [&(&left.into(), &params.s_g2_prepared),
                 (&right.into(), &params.n_g2_prepared)];
    bool::from(E::multi_miller_loop(&terms[..])
               .final_exponentiation().is_identity())
}

```

This is (2.32): $e(\text{left}, [s]_2) \cdot e(\text{right}, -[1]_2) = 1$ (with $\text{n_g2_prepared} = -[1]_2$). The `Guard::verify` trait impl (L228) wraps it as `self.check(params).then_some(()).ok_or(Error::OpeningError)`.

- **[NOTE] Single-scalar-1 fast path.** When the left channel has exactly one term with scalar 1, the MSM is skipped and the base is used directly. The generated `ec_pairing` does not replicate this micro-optimization (it always runs the fused MSM); the result is identical, only the Rust saves one MSM. No faithfulness impact.
- **[OK] Channel semantics.** `left` carries the π -side (paired with $[s]_2$) and `right` the combined side (paired with $-[1]_2$), matching the `PAIRING_LHS/PAIRING_RHS` mapping documented in §9.1.6.

3.12 KZG `multi_prepare`

Quoted in full in Section 2.9; the load-bearing excerpt is the f_{eval} fold (L471–482):

```

let f_eval = point_sets.iter().zip(q_eval_sets.iter())
    .zip(q_evals_on_x3.iter()).rev().fold(E::Fr::ZERO,
    |acc_eval, ((points, evals), proof_eval)| {
        let r_poly = lagrange_interpolate(points, evals);
        let r_eval = eval_polynomial(&r_poly, x3);
        let den = points.iter().fold(E::Fr::ONE, |acc, p| acc * &(x3 - p));
        let eval = (*proof_eval - &r_eval) * den.invert().unwrap();
        acc_eval * &(x2) + &eval
    });

```

which is (2.28). The final commitment and scalar (L488–527) are (2.29)–(2.30); the dual-MSM assembly (L540–558) is (2.32).

3.13 Arithmetic helpers

- `truncate` (L247): keeps the lower 16 little-endian bytes (= 128 bits).
- `truncated_powers` (L255): $\text{truncate}(x^i)$ with a full-precision running accumulator.
- `lagrange_interpolate` (L172): builds r_s of (2.28).
- `eval_polynomial` (L53): Horner evaluation.
- `inner_product/msm_inner_product` (L263/L275): Eqs. (2.26)–(2.30).

3.14 Trace instrumentation

Under the `solidity-verifier-trace` feature, every semantic value is recorded with a stable trace id (`plonk/solidity_trace.rs`): 1 `vk_digest`, 2 `num_instances`, 3 `k`, 4 `n_inv`, 5 `omega`, 6 `omega_inv`, 7 `theta`, 8 `beta`, 9 `gamma`, 10 `y`, 11 `x`, 12 `trash_challenge`, 13 `x1`, 14 `x2`, 15 `x3`, 16 `x4`, 17 `x_n`, 18 `x_n_minus_1_inv`, 19 `l_last`, 20 `l_blind`, 21 `l_0`, 22 `instance_eval`, 23 `quotient_eval`, 24 `quotient`, 25 `f_com`, 26 `pi`, 29/30 `acc lhs/rhs`, 31 `f_eval`, 32 `v`, 33 `final_com`, 34 `linearization com`, 35 `final result`, 36 `nu_y(x)`; bases: 1000 user challenges, 10000 proof commitments, 20000 proof evals, 30000 quotient identities, 40000 `q_com`,

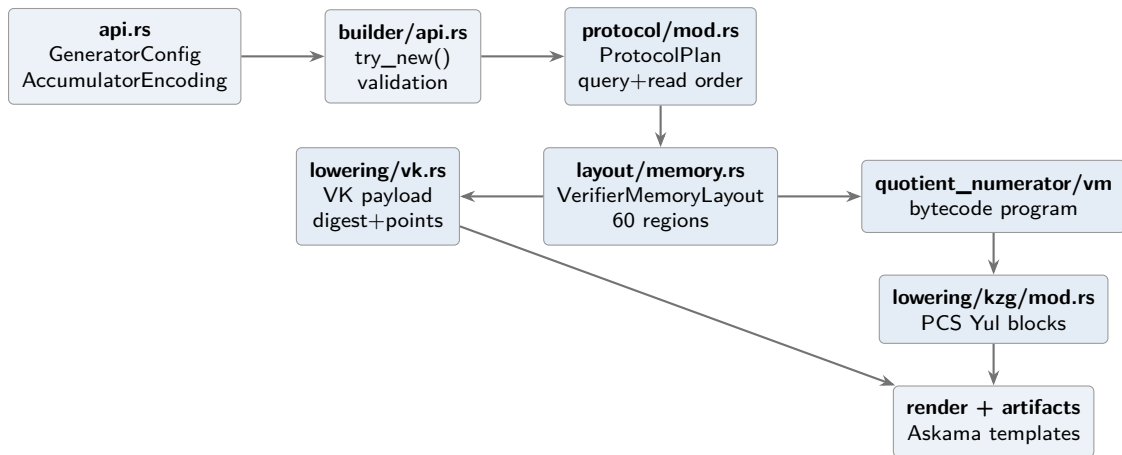
41000 point sets, 60000 selector folds. These ids are the backbone of the differential equivalence test (Chapter 7).

Chapter 4

The Code Generator

The generator lives in `proofs/solidity-verifier/src/`. Its design principle is: **plan everything in Rust, render nothing but strings**. By the time an Askama template runs, every memory address, byte offset, challenge order, quotient opcode, and PCS schedule entry is a finalized integer. The templates contain no arithmetic and no protocol logic — they only interpolate planned facts. This is what makes the lowering auditable: the protocol lives in typed Rust data structures that can be unit-tested, not in template conditionals.

4.1 Pipeline overview



The entry point is `SolidityGenerator` (`builder/mod.rs`):

```
pub struct SolidityGenerator { params, vk, num_instances, acc_encoding, meta }
impl SolidityGenerator {
    fn inputs(&self) -> VerifierBuildInputs { ... }
    pub fn render(&self, options: RenderOptions) -> RenderedArtifacts { ... }
}
```

`RenderOptions` selects which artifacts to emit: the verifier contract, optionally a separate VK contract, and optionally an external quotient evaluator contract (Section 5.6).

4.2 Validation: what the generator refuses

`builder/api.rs::try_new` runs the profile checks of Section 1.2 before any planning:

- `NoAdviceColumns` — a circuit with no advice cannot be verified.
- `validate_instance_column_shape` — exactly one committed instance column (the identity commitment) and exactly one non-committed instance column.

- accumulator validation — if an outer IVC accumulator is present, its encoding (offset, 7 limbs of 56 bits, PointAndScalar/PointPair) must be consistent.
- [RotatedInstanceQuery](#) rejection — non-committed instance columns may only be queried at rotation 0.

These mirror the Rust verifier’s own instance validation ([verifier.rs](#) L263–268) but are *stricter*: the Rust verifier would accept shapes the generator deliberately excludes. This asymmetry is safe — the generator only claims faithfulness inside its profile.

4.3 The protocol plan

[lowering/protocol/mod.rs::ProtocolPlan::from_constraint_system](#) (1223 L) is the heart of the lowering. It computes, from the constraint system alone, the exact order of every proof read and every PCS query. The key data types:

```
enum CommitmentRead { Advice, LookupMultiplicity, PermutationProduct,
                     LookupHelper, LookupAccumulator, Trash, Quotient }
enum PermutationZEval { Cur, Next, Last }
struct Query { rotation: Rotation, comm: ..., eval: ... }
```

Proof commitment order (mirrors [parse_trace](#)):

advices per phase → multiplicities → perm products → helpers+accumulators → trash → quotient limbs.

Proof eval order (mirrors the eval-absorption loop):

committed_instance → advice → fixed (non-simple) → perm_common → perm z cur/next/last per set
→ lookup m/helper/z(0)/z(1) → trash.

PCS query order (mirrors [verifier.rs](#) L577–652):

advice → committed instance → perm z → lookup → trash → fixed non-simple → perm common → linearization.

4.3.1 The canonical read-order construction

The ordering above is not hand-written — it is *derived* from the constraint system by the plan builder, so the generated proof layout cannot drift from the circuit. The commitment stream:

```
proof.commitments.extend((0..advice_indices.len())
    .map(|_| CommitmentRead::Advice)); // per phase
proof.commitments.extend((0..num_lookups)
    .map(|_| CommitmentRead::LookupMultiplicity));
proof.commitments.extend((0..num_permutation_zs)
    .map(|_| CommitmentRead::PermutationProduct));
for &chunks in &lookup_chunks {
    proof.commitments.extend((0..chunks)
        .map(|_| CommitmentRead::LookupHelper));
    proof.commitments.push(CommitmentRead::LookupAccumulator);
}
proof.commitments.extend((0..num_trashcans)
    .map(|_| CommitmentRead::Trash));
proof.commitments.extend((0..num_quotients)
    .map(|_| CommitmentRead::Quotient));
```

and the eval stream (note the two deliberate exclusions):

```
proof.evals.extend(instance_queries.iter().copied()
    .filter(|q| q.column < nb_committed_instances) // committed only
    .map(EvalRead::CommittedInstance));
proof.evals.extend(advice_queries.iter().copied().map(EvalRead::Advice));
proof.evals.extend(fixed_queries.iter().copied()
    .filter(|q| !simple_selector_cols.contains(&q.column)) // no simple sel.
```

```

    .map(EvalRead::Fixed));
// perm common, then per-set z cur/next/last (last skipped on final set)
for set in 0..num_permutation_zs {
    proof.evals.push(EvalRead::PermutationZ { set, kind: Cur });
    proof.evals.push(EvalRead::PermutationZ { set, kind: Next });
    if set + 1 != num_permutation_zs {
        proof.evals.push(EvalRead::PermutationZ { set, kind: Last });
    }
}
// lookup m / helper / z(0) / z(1), then trash

```

- **[OK] Derived, not hand-written.** The order is computed from `cs.advice_queries()` etc., so adding a column to the circuit automatically shifts the proof layout — the generated parser and the Rust verifier stay in lockstep by construction.
- **[OK] Simple-selector exclusion.** Simple selector columns are filtered out of the eval stream (filled by the linearization path, not read from the proof), matching the Rust verifier’s treatment.
- **[OK] Final-set Last skip.** The Last rotation is omitted on the final permutation set — the same boundary condition the Rust permutation verifier applies (the last set’s last-row term is folded elsewhere).
- **[NOTE] Phase remapping.** The `remapping` closure repacks declaration-order advice/challenge columns into phase-packed order (`nums/index`), because Midnight declares columns in source order but verifies them phase by phase. This is the one place where the wire order differs from the declaration order; the generated parser must use the remapped indices.

4.3.2 Lookup chunking: `nb_parallel_lookups_per_chunk`

The `lookup_chunks` field (which sizes the helper commitments and evals in the proof layout) is derived from the LogUp degree budget:

```

pub(crate) fn nb_parallel_lookups_per_chunk(&self, cs_degree: usize) -> usize {
    let max_degree = (cs_degree - 1).next_power_of_two() + 1;
    let lookup_degree = self.input_expressions.iter()
        .flat_map(|e| e.iter().map(|e| e.degree()))
        .max().unwrap_or(1);
    // dominating factor: h(X) * prod_j (f_j(X) + beta)
    // has degree 1 + lookup_degree * nb_parallel_lookups
    (max_degree - 1) / lookup_degree
}

```

- **[OK] Degree-budget derivation.** The chunk size is the largest number of parallel LogUp inputs whose dominating polynomial $h(X) \prod_j (f_j(X) + \beta)$ fits the constraint-system degree budget. `chunk_by_degree` then splits `input_expressions` into that many per chunk, each becoming one committed helper $h_i(X)$.
- **[OK] Verifier-side consistency.** The generated verifier reads exactly `num_chunks` helper commitments and their evals per lookup (the `LookupHelper/LookupAccumulator` reads in §4.3.1), so the chunking formula is the single source of truth for the lookup portion of the proof layout.
- **[NOTE] next_power_of_two rounding.** `max_degree` rounds `cs_degree-1` up to a power of two before adding 1, matching the Rust verifier’s degree handling. A circuit whose degree is not a power of two gets a slightly larger budget — faithful to Rust, but worth noting when reasoning about chunk counts.

The plan’s `validate()` method asserts internal invariants: every commitment read is consumed by at least one PCS query (so the no-curve-check-at-absorption argument of Definition 2.2.3 holds — every absorbed point is later validated by a precompile), and the eval counts match `ConstraintSystemMeta::num_evals`.

Faithfulness Claim 4.3.1 (Query-order faithfulness). *The `ProtocolPlan` query order is identical to the order in which `verify_algebraic_constraints` builds its `queries` vector. This is verified by the differential trace: the PCS point-set trace ids `41000+s` and `q_com` ids `40000+s` must match between Rust and Solidity, which pins both the set partition and the within-set order.*

4.4 Memory layout

`layout/memory.rs::VerifierMemoryLayout` (1468 L) assigns 60 named regions with a bump allocator and a `MemoryMap.validate` pass. The regions (poseidon fixture addresses) are:

Region	Addr	Purpose
<code>transcript_mptr</code>	0x1000	Keccak buffer (Def. 2.2.2).
<code>vk_mptr</code>	0x1980	VK payload (digest, k, ω , points).
<code>challenge_mptr</code>	0x2b60	base of challenge block.
<code>theta/beta/gamma</code>	0x2b60/80/a0	randomizers.
<code>trash_challenge/y/x</code>	0x2bc0/e0/00	batching challenges.
<code>x1/x2/x3/x4</code>	0x2c20–0x2c80	multi-open challenges.
<code>f_com / pi</code>	0x2ca0 / 0x2d20	PCS commitments.
<code>x_n, x_n_minus_1_inv</code>	0x2ea0/0x2ec0	$x^n, (x^n - 1)^{-1}$.
<code>l_last/l_blind/l_0</code>	0x2ee0/f0/2f20	Lagrange specials.
<code>instance_eval</code>	0x2f40	Eq. (2.11).
<code>quotient_eval</code>	0x2f60	$-\nu_y(x)$ (Def. 2.7.1).
<code>quotient</code>	0x2f80	$x_{\text{split}}, 1 - x^n$ (2 words).
<code>f_eval / v / final_com</code>	0x3020/40/60	Eqs. (2.28),(2.30).
<code>pairing_lhs/rhs</code>	0x30e0/0x3160	pairing inputs.
<code>rot_points</code>	0x31e0	$x \cdot \omega^{\text{rot}}$ table.
<code>x1_powers</code>	0x3560	truncated x_1^i .
<code>q_com / q_eval_set</code>	0x3d80	per-set batched comms/evals.
<code>gl_identity</code>	0x4580	128 zero bytes.
<code>reversed_evals</code>	0x46e0	eval spill for VM.
<code>comms bases</code>	0x4ce0+	per-category commitment regions.
<code>selector_acc</code>	0x5660	simple-selector buckets B_j .
<code>quotient_tmp/stack</code>	0x56e0+	VM scratch.

The layout is computed *before* rendering so that templates receive final addresses; `plan.rs` checks for `eval_numer_mptr drift` between planning and rendering passes.

4.5 VK payload

`lowering/vk.rs::generate_base_vk` (448 L) produces the VK payload consumed by `VkLoading.yul`:

- `vk_digest` = `fe_to_u256(vk.transcript_repr())` — Eq. (2.5).
- `k, n_inv, omega, omega_inv, omega_inv_to_1` = $\omega_{\text{inv}}^{|\text{rotation_last}|}$.
- accumulator metadata (offset, limb count, limb bits).

- base points: $G1Base = G$, $G2Base = G_2$, $NegSG2Base - [s]_2$, encoded via `g1_to_u256s/g2_to_u256s`.

The `TranscriptBufferLayout` sizes the transcript buffer at 128 bytes per G1 (a comment records a historical 49-byte bug that broke digest binding — the fix is pinned by tests).

4.6 Proof layout and repacking

`lowering/abi/proof.rs::ProofCalldataLayout` (537 L) defines the compressed proof byte layout: advice phases, lookup multiplicities, permutation products, lookups, trash, quotient limbs, evals, `f_com`, `q_evals`, `pi`, with byte cursors. `lowering/calldata.rs::repack_proof` (144 L) converts a native proof to the on-chain form:

- each 48-byte compressed G1 $\rightarrow$ 128-byte EIP-2537 uncompressed (16 zero pad + x_{hi}/x_{lo} + 16 zero pad + y_{hi}/y_{lo}); identity $\rightarrow$ 128 zeros.
- scalars little-endian $\rightarrow$ big-endian words.

The walk is driven by `RepackedProofLayoutPlan` so the repacker and the generated parser cannot drift apart.

4.7 The quotient virtual machine

The expensive part of the verifier is reconstructing $\nu_y(x)$ from the identity expressions. Rather than emitting straight-line Yul for every gate (which explodes code size), the generator compiles the identities into a compact **bytecode program** executed by a small VM embedded in the contract (`quotient_numerator/vm/mod.rs`, 3852 L).

Opcodes (`Q_OP_*`, 31 opcodes in 0x01–0x22; `vm/mod.rs` L560–590; 0x17, 0x18, 0x1a unsigned):

0x01 push_const	0x09 push_const_u8	0x13 add_mul_const_u8_mem_u16
0x02 push_mem_literal	0x0a fold_main	0x14 add_mul_mem_mem
0x03 push_mem_token	0x0b fold_selector	0x15 run of 0x12
0x04 push_mem_token_off	0x0c add_const_u8	0x16 run of 0x13
0x05 push_mem_u16	0x0d mul_const_u8	0x19 native_permutation
0x06 add	0x0e add_const	0x1b native_identity
0x07 mul	0x0f mul_const	0x1c LIN7 0x1d BILIN7_ROW
0x08 neg	0x10 add_mem_u16	0x1e BILIN7_PAIRWISE
	0x11 mul_mem_u16	0x1f native_lookup 0x20 POW5
	0x12 add_mul_mem_mem_const_u8	0x21 MODARITH7 0x22 AFFINE_SUM

The rendered dispatch loop is pruned to the opcodes the program actually uses (`quotient_program_usage`); an opcode that is not rendered falls into the `default` revert. The production B64 program uses 11 of the 31 (§11.8.39).

Memory tokens (`Q_MEM_*`, 0x01–0x09): `L0`, `L_LAST`, `L_BLIND`, `BETA`, `GAMMA`, `X`, `THETA`, `TRASH_CHALLENGE`, `INSTANCE_EVAL`.

The VM registers are `q_pc`, `q_end`, `q_sp`, `q_top`, `q_has_top` (a one-element top-of-stack cache). `QuotientProgramBuilder` folds each identity with the Horner y -weighting of (2.22): `fold_identity` emits `fold_main` (into the main numerator) or `fold_selector(idx)` (into bucket B_j). The final VM instruction sequence ends with

```
// linearization_expected_eval = -quotient_eval_number
mstore(QUOTIENT_EVAL_MPTR, sub(r, quotient_eval_number))
```

which is Definition 2.7.1 / Eq. (2.23).

Native fast paths. Common expression shapes are compiled to fused opcodes rather than generic push/add/mul sequences: LIN7 (linear combination of 7 terms), BILIN7_ROW (bilinear 7-term), MODARITH7, POW5, and the `native_permutation/native_lookup/native_identity` call-backs that jump into hand-written Yul blocks for the permutation and lookup identity families. The `QuotientProgramPlan` classifies each identity into `inline_identities`, `native_identities`, `native_permutation_identities`, `native_lookup_identities`.

Limb arithmetic. Foreign-field (BLS12-381 base field) gadgets constrain values held as 7 limbs of 56 bits. The VM does not perform wide arithmetic: every operation is a native $\mathbb{F}_r$ addmod/mulmod. The limb helpers `q_limb7/q_limb7_wide` in `QuotientHelpers.yul` evaluate the fixed linear forms $\sum_i c_i \ell_i \bmod r$ with the base-power coefficients `LIMB7_YUL_COEFFS/WIDE_LIMB7_YUL_COEFFS`, and `q_pow5` evaluates the Poseidon S-box term x^5 . Each helper is rendered only when the lowering recognizes the matching expression shape.

4.8 PCS lowering

`lowering/kzg/mod.rs` (2077 L) emits the Yul blocks that implement Eqs. (2.26)–(2.29). Its `computations()` method produces:

1. **Rotation points:** $x \cdot \omega^{\text{rot}}$ written to `ROT_POINTS_MPTR` (mirrors `rotate_omega`).
2. **x_1 powers:** truncated x_1^i to `X1_POWERS_MPTR` (Eq. (2.26) scalars).
3. **`q_eval_set` folds:** per-set $\sum_k x_1^k v_{s,k}$ (Eq. (2.27)), with a roll threshold `Q_EVAL_ROLL_THRESHOLD=4` for register pressure.
4. **`q_com`** (trace-only): the per-set batched commitment.
5. **Fused final MSM:** the linearization expansion — quotient limbs $\times (1 - x^n) \cdot x_{\text{split}}^i \times$ query scalar, plus selector buckets $\times$ fixed commitments — as one G1MSM (Eq. (2.24) commitment side).
6. **`v` computation:** $v = \sum_s x_4^s q_{\text{eval}}^{(s)} + x_4^{n_{\text{sets}}} f_{\text{eval}}$ (Eq. (2.30)).
7. **Final pairing inputs.**

`validate_g1_precompile_coverage` asserts every absorbed G1 point appears in a G1MSM or pairing input — the formal counterpart of Definition 2.2.3’s “validated by precompile” claim.

4.8.1 `construct_intermediate_sets_impl` — the point-set partition

This is the codegen core of the KZG multi-open (Sec. 2.9.1). It mirrors the Rust `construct_intermediate_sets` but is deterministic and side-effect-free so the generated schedule is auditable:

```
fn construct_intermediate_sets_impl(queries: &[Query]) -> IntermediateSets {
    // Step 1: one entry per unique commitment; one index per unique rotation.
    let mut commitment_map: Vec<(EcPoint, Vec<usize>, Vec<Word>)> = Vec::new();
    let mut point_index_of: Vec<i32> = Vec::new();
    for query in queries {
        let point_idx = /* intern query.rotation */;
        if let Some(slot) = commitment_map.find(|c| *c == query.comm) {
            if let Some(pos) = slot.1.position(|pi| *pi == point_idx) {
                // Same (comm, rotation) twice: evals MUST agree, else the
                // proof claims one poly opens to two values -> reject.
                assert_eq!(slot.2[pos], query.eval,
                    "duplicate (commitment, rotation) query with conflicting evals");
                continue;
            }
            slot.1.push(point_idx); slot.2.push(query.eval);
        } else {
```

```

        commitment_map.push((query.comm, vec![point_idx], vec![query.eval]));
    }
}
// Step 2: bucket by sorted point-index set (BTreeMap => deterministic).
let mut point_idx_sets: BTreeMap<Vec<usize>, usize> = BTreeMap::new();
for (_, point_indices, _) in &commitment_map {
    let mut sorted = point_indices.clone(); sorted.sort_unstable(); sorted.dedup();
    let n = point_idx_sets.len();
    point_idx_sets.entry(sorted).or_insert(n); // capture INSERTION order
}
// Step 3: align each commitment's evals to the sorted point order.
// Step 4: point_sets[set_idx] = rotations (set_idx = insertion order).
...
}

```

- **[OK] Conflict rejection.** A duplicate (comm, rotation) with *different* evals is a protocol bug and is asserted against — the codegen refuses to emit a schedule for an inconsistent query set.
- **[OK] Deterministic insertion-order capture.** The comment warns that the `or_insert(n)` insertion order (not the BTreeMap iteration order) is the set-index used downstream and as the `sort_sets` tiebreak. This is the same ordering the Rust verifier relies on so the first point set holds the fixed commitments opened at x (Sec. 2.9).
- **[NOTE] HashMap hazard.** The comment explicitly flags that swapping the BTreeMap for a HashMap would break the insertion-order capture. This is a latent maintenance trap; the invariant is documented but not type-enforced.

4.8.2 `sort_sets` — ascending-cardinality ordering

After the partition, the sets are sorted so the generated schedule is deterministic and the first set holds the fixed commitments opened at x only:

```

fn sort_sets(input: IntermediateSets) -> IntermediateSets {
    let mut order: Vec<usize> = (0..input.point_sets.len()).collect();
    order.sort_by_key(|&i| (input.point_sets[i].len(), i)); // (|S|, idx)
    // remap[old_idx] = new_idx
    let mut remap = vec![0usize; input.point_sets.len()];
    for (new_idx, &old_idx) in order.iter().enumerate() {
        remap[old_idx] = new_idx;
    }
    let point_sets = order.iter().map(|&i| input.point_sets[i].clone()).collect();
    let commitments = input.commitments.into_iter().map(|mut c| {
        c.set_index = remap[c.set_index]; c
    }).collect();
    IntermediateSets { commitments, point_sets }
}

```

- **[OK] (|S|, idx) tiebreak.** Sorting by ascending cardinality then original index matches the Rust `sort_by_key(|i|, i)` (Sec. 2.9.1), so the within-set eval order and the x_1 -power assignment are identical between Rust and the generated verifier.
- **[OK] set_index rewrite.** Every commitment's `set_index` is remapped through the same permutation, so downstream `commitments_by_set` and the fused-MSM grouping see the sorted order consistently.

4.8.3 `used_lagrange` — required common polynomials

The plan also records which Lagrange polynomials the verifier must materialize, so the generated `Lagrange.yul` block computes exactly what is needed:

```
pub(crate) fn used_lagrange(
    uses_permutation: bool, uses_lookup: bool, uses_trash: bool,
) -> BTreeSet<CommonPoly> {
    let mut out = BTreeSet::new();
    if uses_permutation || uses_lookup {
        out.insert(CommonPoly::L0);
        out.insert(CommonPoly::LLast);
        out.insert(CommonPoly::LBlind);
    }
    if uses_trash {
        out.insert(CommonPoly::Rotation(0));
    }
    out
}
```

- **[OK] Minimal materialization.** $l_0, l_{\text{last}}, l_{\text{blind}}$ are computed only when a permutation or lookup argument is present; the trash argument additionally needs the rotation-0 Lagrange. A circuit with no perm/lookup/trash computes none of them — a real gas saving over always computing the full Lagrange basis.
- **[NOTE] CommonPoly enum.** The `Rotation(i)` variant generalizes to arbitrary rotations, though in the current verifier only rotation 0 is requested by trash. The generated code handles the general case but only the specific requests appear in practice.

4.8.4 `computations()` — the Yul emitter

`computations()` is the function that actually emits the PCS Yul blocks (the `pcs_computations` consumed by `Pcs.yul`). It mirrors the Rust `multi_prepare` flow as a sequence of emitted blocks. Block 1 pre-computes the rotation points $x \cdot \omega^{\text{rot}}$ by walking forward/backward from rotation 0:

```
// Block 1: rotation points x*omega^rot at ROT_POINTS_MPTR + 32*idx
let x_pow_of_omega := x; // rot 0 = x
for rot in 1..=max_rot {
    x_pow_of_omega := mulmod(x_pow_of_omega, omega, r);
    store_rot(rot); // if rot is used
}
if min_rot < 0 {
    x_pow_of_omega := x;
    for rot in (min_rot..0).rev() {
        x_pow_of_omega := mulmod(x_pow_of_omega, omega_inv, r);
        store_rot(rot);
    }
}
```

Block 2 pre-computes the x_1 powers, with a deliberate **rolled-loop** optimization:

```
// Block 2: x1^0..x1^(nb-1) at X1_POWERS_MPTR
let acc := 1
let p := X1_POWERS_MPTR
for { let i := 0 } lt(i, {last}) { i := add(i, 1) } {
    p := add(p, 32)
    acc := mulmod(acc, x1, r)
    mstore(p, and(acc, TRUNC_MASK_128)) // if truncated_challenges
}
```

- **[IMPROVE] Rolled vs. unrolled (already applied).** The codegen comment records that the *unrolled* emission (32 sequential `mulmod`+`mstore` pairs) cost ~ 26 kg of gas — far above the ~ 700 gas the arithmetic needs — because `solc --ir` cannot register-allocate 32 unrolled `mulmods` sharing one accumulator, and the unrolled `mstore-add` chain inflates each line to ~ 50 – 60 gas of dispatch overhead. The rolled loop restores the basic-block heuristic (~ 20 gas/iter $\times 32 + 50$ setup ≈ 700 gas). This is a concrete, measured contract-space/gas win baked into the generator.

- **[OK] Truncated-challenge mask.** When `truncated_challenges` is set, the emitted loop stores `and(acc, mask)` while keeping `acc` at full precision — exactly mirroring the Rust `powers(x1).map(truncate)` (full-precision running accumulator, truncated per element). The +3 gas/iter for the mask is negligible vs the loop body.
- **[NOTE] Trace-scratch borrowing.** In trace mode, Block 1 borrows `Q_EVAL_SET_MPTR` as scratch for the serialized point-set payload, relying on Block 3 to unconditionally overwrite it before any later read. The comment flags that a future reader between Block 1 and Block 3 must move the scratch — a latent coupling to watch.

4.8.5 `validate_g1_precompile_coverage` — the safety invariant

The invariant that makes “no curve check at absorption” sound (cf. §9.1.3):

```
fn validate_g1_precompile_coverage(
    absorbed: &[TrackedG1], precompile_inputs: &[TrackedG1],
) -> Result<(), String> {
    let missing: Vec<&TrackedG1> = absorbed.iter()
        .filter(|a| !precompile_inputs.iter().any(|c| c.point == a.point))
        .collect();
    if let Some(first) = missing.first() {
        return Err(format!("{} absorbed proof G1(s) are not consumed by a later \
            EIP-2537 G1MSM/pairing precompile; first missing: {} at {}",
            missing.len(), first.label, first.point.ptr()));
    }
    Ok(())
}
```

The wrapper `validate_absorbed_g1_precompile_coverage` first checks the absorbed count equals the proof layout’s `total_g1_count()` (so no absorbed point is untracked), then runs the coverage check against `precompile_validated_g1s`.

- **[OK] Concrete-handle checking.** The check operates on the *emitted memory handles* (`TrackedG1.point.ptr()`), not just the logical PCS queries — so it catches quotient limbs hidden behind the synthetic linearization commitment.
- **[RISK] Point-equality by pointer.** Coverage is decided by `candidate.point == absorbed.point` (memory-handle equality). If the same logical point is materialized at two different addresses (e.g. a copy), the absorbed handle would be reported as “not covered” even though an equal point is MSM’d. This is conservative (fails closed, build error), but could cause a false build failure if the codegen ever duplicates a point’s storage. *Audit note:* the codegen must keep one canonical address per absorbed point.

4.9 Rendering

`lowering/artifacts.rs::generate_verifier_from_plan` assembles the Askama model (verifier template + included partials) and renders the Solidity source; `validate_layout` panics on any invalid plan so a broken lowering never produces a contract. `render/models.rs` holds the template data models. The `QuotientRendering` enum (Section 4.7) selects between:

- **Compact** — the VM is inlined in the main verifier.
- **External** — the verifier `staticcalls` a pinned `Halo2QuotientEvaluator` (codehash-checked), passing the raw verifier memory frame as calldata.

Chapter 5

The Generated Solidity and Yul

The generated artifact is a single Solidity contract, `Halo2Verifier.sol`, whose entry point is

```
function verifyProof(bytes calldata proof, bytes[] calldata instances)
    external view returns (bool)
```

with selector `FN_SIG_VERIFY_PROOF = 0x1e8e1e13`. The function body is one assembly ("memory-safe") block containing included Yul partials. It is **terminal**: every failure path is a revert; the only non-reverting exit is `mstore(RETURN_MPTR,1); return(RETURN_MPTR,0x20)`.

5.1 Contract structure

```
contract Halo2Verifier {
    // Constants.sol: FR_MODULUS, BLS_P_HI/LO, EXPECTED_VK_*, ABI cursors,
    //                VK memory map, challenge layout
    // PrecompileSmoke.sol: MCOPY + G1ADD/G1MSM/PAIRING identity probes
    // Constructors.sol: constructor runs smoke probes + VK pin check
    function verifyProof(...) external view returns (bool) {
        assembly ("memory-safe") {
            // AssemblyHelpers.yul    (scalar_inv, transcript, batch_invert, ec_pairing)
            // AccumulatorHelpers.yul  (accumulator decode/validate)
            // TraceAndGasHelpers.yul  (trace hooks, gas checkpoints)
            // VkLoading.yul            (embedded or external VK payload load)
            // TranscriptProofParser.yul (the whole proof read + challenge schedule)
            // Lagrange.yul             (x^n, Lagrange basis, instance_eval)
            // QuotientAndLinearization.yul (VM run or external evaluator call)
            // Pcs.yul                  (rendered PCS computation blocks)
            // FinalPairing.yul         (accumulator batching + final pairing)
            // TraceReturn.yul          (trace dump + return 1)
        }
    }
}
```

5.2 ABI and shape guard

The generated contract pins the ABI layout with compile-time cursor constants:

```
uint256 internal constant PROOF_LEN_CPTR    = 0x44;
uint256 internal constant PROOF_CPTR       = 0x64;
uint256 internal constant NUM_INSTANCE_CPTR = 0x1144;
```

Before any memory work, a shape guard rejects malformed calls:

```
if iszero(and(eq(calldataload(0x04), 0x40),
              eq(calldataload(0x24), sub(NUM_INSTANCE_CPTR, 0x04)))) { revert(0,0) }
```

This checks the dynamic-array offset is 0x40 and the proof length matches the generated layout, so a truncated or shifted calldata never reaches the transcript.

5.3 Transcript helpers

The Yul transcript helpers implement Definitions 2.2.1–2.2.2 literally:

```
function common_word(buf_len, word) -> ret {
    mstore(buf_len, word)
    ret := add(buf_len, 32)
}
function squeeze_to(buf_len, mptr) -> ret {
    let h0 := keccak256(TRANSCRIPT_MPTR, sub(buf_len, TRANSCRIPT_MPTR))
    mstore(TRANSCRIPT_MPTR, h0) // reseed (eq:transcript-reseed)
    mstore(mptr, mod(h0, r)) // sample (eq:transcript-squeeze)
    ret := add(TRANSCRIPT_MPTR, 32)
}
```

The G1 absorption helper enforces Definition 2.2.3:

```
function common_uncompressed_g1(buf_len, cptr) -> ret {
    let x_hi_word := calldataload(cptr)
    let x_lo := calldataload(add(cptr, 0x20))
    let y_hi_word := calldataload(add(cptr, 0x40))
    let y_lo := calldataload(add(cptr, 0x60))
    if shr(128, x_hi_word) { revert(0, 0) } // top 16 bytes must be zero
    if shr(128, y_hi_word) { revert(0, 0) }
    let x_hi := and(x_hi_word, 0xffff...)
    let y_hi := and(y_hi_word, 0xffff...)
    // coordinate < q checks (BLS_P_HI / BLS_P_MINUS_ONE_LO ladder)
    if iszero(or(lt(x_hi, BLS_P_HI),
        and(eq(x_hi, BLS_P_HI), iszero(gt(x_lo, BLS_P_MINUS_ONE_LO)))))) { revert(0,0) }
    if iszero(or(lt(y_hi, BLS_P_HI),
        and(eq(y_hi, BLS_P_HI), iszero(gt(y_lo, BLS_P_MINUS_ONE_LO)))))) { revert(0,0) }
    calldatacopy(buf_len, cptr, 0x80) // absorb 128 bytes verbatim
    ret := add(buf_len, 0x80)
}
```

The comment in the generated code records that the previous emitter hashed the 48-byte ZCash compressed encoding with a 384-bit $\text{lex}(y) > \text{lex}(p-y)$ sign ladder; switching to the uncompressed form (matching the patched Rust `to_input`) drops the ladder entirely.

5.4 The proof parser

`TranscriptProofParser.yul` walks the proof in the order of Table 2.1. A representative excerpt (advice phase, poseidon fixture, 8 commitments of 0x80 bytes each):

```
let proof_cptr := PROOF_CPTR
let advice_walk := ADVICE_COMMS_MPTR_BASE
// ---- User phase 1 ----
for { let end := add(proof_cptr, 0x0400) } lt(proof_cptr, end) {} {
    buf_len := common_uncompressed_g1(buf_len, proof_cptr)
    calldatacopy(advice_walk, proof_cptr, 0x80)
    advice_walk := add(advice_walk, 0x80)
    proof_cptr := add(proof_cptr, 0x80)
}
buf_len := squeeze_to(buf_len, THETA_MPTR)
// multiplicities, beta, gamma, perm z, helpers+acc, trash_challenge,
// trash, y, quotient limbs, x ...
```

The eval loop range-checks and spills each scalar:

```
for { let end := add(proof_cptr, 0x0600) } lt(proof_cptr, end) {} {
    let eval := calldataload(proof_cptr)
    if iszero(lt(eval, r)) { revert(0, 0) } // canonical Fr
    mstore(eval_buf, eval) // spill for VM
```

```

    buf_len := common_word(buf_len, eval)    // absorb
    proof_cp_ptr := add(proof_cp_ptr, 0x20)
}

```

The x_3 truncation is applied immediately after squeeze:

```

buf_len := squeeze_to(buf_len, X3_MPTR)
mstore(X3_MPTR, and(mload(X3_MPTR), 0xffffffffffffffffffffffff))

```

and the parser closes with the exact-consumption check:

```

if iszero(eq(proof_cp_ptr, NUM_INSTANCE_CPTR)) { revert(0, 0) }
if iszero(success) { revert(0, 0) }

```

The first line proves no section was under- or over-read; the second collects deferred canonicity failures from public-instance reads.

5.5 Lagrange and instance evaluation

`Lagrange.yul` computes x^n by k repeated squarings (Eq. (2.6)), batch-inverts the denominators $(x - \omega^i)$ with one modexp (EIP-198) via Montgomery’s trick, then forms $L_i = (x^n - 1) \cdot n^{-1} \cdot \omega_i / (x - \omega_i)$ (Eq. (2.7)), the blind-row sum (2.9), ℓ_0 (2.10), and the linear combination $\widehat{\text{inst}}(x)$ (2.11).

5.6 Quotient and linearization

In the *compact* path, `QuotientAndLinearization.yul` runs the quotient VM (Chapter 4.7) inline: the program bytes live in the VK payload at `q_const/q_program`; the VM loop switches on opcodes and folds identities into the main numerator or selector buckets. It then stores

```

// x_split = x^(n-1), one_minus_x_n at QUOTIENT_MPTR words 0/1
// linearization_expected_eval = -quotient_eval_numerator
mstore(QUOTIENT_EVAL_MPTR, sub(r, mload(EVAL_NUMER_MPTR)))

```

In the *external* path, the verifier `staticcalls` the pinned evaluator (codehash-checked), passing the raw memory frame; the evaluator returns a compact frame `[magic, expected_eval, buckets...]` and the verifier copies the buckets back. The external evaluator’s header comment states the contract explicitly:

“This contract reconstructs the Rust verifier’s y-batched identity numerator $\nu_y(x)$ and returns the linearization expected scalar $-\nu_y(x)$. It does not evaluate or trust a quotient scalar $h(x)$. The quotient limb commitments are handled by Halo2Verifier on the commitment side as $(1 - x^n) \cdot \sum_i x_{\text{split}}^i Q_i$.”

5.7 PCS blocks

`Pcs.yul` renders the `pcs_computations` blocks from Section 4.8: rotation points, x_1 powers, per-set q_{eval} folds, the fused final G1MSM for the linearization commitment, and the v computation. The generated sub-blocks are kept grouped so gas checkpoints can attribute cost. A representative final-MSM call:

```

success := staticcall(gas(), 0x0c, 0x56e0, 0x1d60, FINAL_COM_MPTR, 0x80)
success := and(success, eq(returndatasize(), 0x80))
mstore(V_MPTR, v)

```

(where `0x0c` is the G1ADD/G1MSM precompile range per EIP-2537; the MSM uses `0x0d`).

5.8 Final pairing

The pairing block assembles the two pairing inputs and calls the helper:

```
// PAIRING_LHS := pi ; PAIRING_RHS := final_com - v*G + x3*pi
mcopy(PAIRING_LHS_MPTR, PI_MPTR, 0x80)
// -v*G via G1ADD with negated scalar, then + final_com, + x3*pi
mcopy(PAIRING_RHS_MPTR, 0x80, 0x80)
...
success := ec_pairing(success, PAIRING_RHS_MPTR, PAIRING_LHS_MPTR)
```

The generated comment is explicit about the naming trap:

“The Yul `ec_pairing` helper checks $e(\arg_0, [1]_2) \cdot e(\arg_1, -[s]_2) = 1$, i.e. $e(\arg_0, [1]_2) = e(\arg_1, [s]_2)$. The KZG pairing identity is $e(\text{final_com} - vG + x_3\pi, [1]_2) = e(\pi, [s]_2)$, so $\arg_0$ must be the combined RHS and $\arg_1$ must be π . The `PAIRING_LHS/RHS` naming follows the dual MSM accumulator (left = π , right = combined) and *not* the pairing argument order. Pass them swapped to `ec_pairing`.”

This matches the Rust `DualMSM` check (`multi_miller_loop` of (left, s_g2) and $(\text{right}, -g_2)$) — Eq. (2.32).

5.9 Accumulator batching

When an outer IVC accumulator is present, `FinalPairing.yul` derives $\alpha = \text{keccak256}(\text{domain} \parallel \text{points}) \bmod r$ (zero mapped to 1) and checks (2.33) with the accumulator points batched into the same pairing, so a single pairing precompile call covers both equations. The accumulator scalars are decoded from the 7-limb 56-bit encoding by `AccumulatorHelpers.yul` (`load_acc_coord_shifted`, `is_bls_p_minus_one`, `validate_public_accumulator`).

5.10 Precompile smoke tests

The constructor runs `PrecompileSmoke.sol`: an MCOPY probe (Cancun+ requirement) and identity-input probes of G1ADD (0x0c), G1MSM (0x0d), and PAIRING (0x0f). If any precompile is missing or misbehaves, deployment reverts — the contract cannot be deployed on a chain where the verifier would be unsound.

5.11 Trace and return

Under the trace build, `TraceReturn.yul` emits LOG1 events with the same trace ids as the Rust verifier (Chapter 3.14), so the differential test can compare them event-by-event. The success path is terminal:

```
mstore(RETURN_MPTR, 1)
return(RETURN_MPTR, 0x20)
```

Chapter 6

Three-Way Correspondence

This chapter is the core deliverable: for every stage of the verifier, the tables below map the **Rust reference** function, the **code generator** that plans it, and the generated **Solidity/Yul** that executes it, with the governing equation from Chapter 2. Read each row as: “the thing the Rust verifier does is planned by this codegen function and executed by this generated construct, and both implement this equation.”

6.1 Transcript and challenges

Rust	Codegen	Solidity/Yul	Eq.
<code>Transcript::common</code>	<code>TranscriptBufferLayout</code> (buffer sizing)	<code>common_word, common_uncompressed_g1</code>	Def. 2.2.1
<code>Keccak256::squeeze</code> (L186)	<code>lowering/vk.rs</code> buffer layout	<code>squeeze_to</code> (keccak + reseed + mod)	(2.1)–(2.3)
<code>sample_keccak_digest_be_mod_r</code> (L19)	(planned as mod op)	<code>mod(h0, r)</code> in <code>squeeze_to</code>	(2.3)
<code>G1Projective::to_input</code> (L283)	<code>g1_to_u256s</code>	<code>common_uncompressed_g1</code> 128-byte absorb	(2.4)
<code>Fq::to_input</code> (L315)	<code>fe_to_u256</code>	<code>common_word</code> BE word	Sec. 2.2
<code>vk.hash_into</code> (L283)	<code>generate_base_vk::vk_digest</code>	VK digest absorb (parser step 1)	(2.5)

6.2 Proof parsing and the challenge schedule

Each row of Table 2.1 is implemented in three places. The representative rows:

Rust	Codegen	Solidity/Yul	Row
<code>read_n(transcript, n)</code>	<code>ProofReadPlan</code> commitment reads	generated for loop of <code>common_uncompressed_g1</code>	5–15
<code>transcript.squeeze_challenge()</code>	<code>ProtocolPlan</code> squeeze order	<code>squeeze_to(buf_len, X_MPTR)</code>	5–23
per-phase advice reads	<code>advice_phases</code> cursors	phase loop, <code>advice_walk</code>	*
committed-instance identity absorb	<code>CommittedInstanceCommitmentKind::Identity</code>	128 zero bytes absorb	2
instance length + value absorbs	<code>encode_calldata</code> instances	<code>inst_be</code> canonical check <code>lt(inst_be, r)</code>	3–4
quotient limb reads	<code>quotient_limbs</code> layout	loop of G1 absorbs	15
eval absorption loop	<code>num_evals</code> / <code>RepackedProofLayoutPlan</code>	eval loop: range-check + spill + absorb	17
<code>q_eval</code> reads	<code>q_evals</code> cursor	<code>Q_EVAL_CPTR_MPTR</code> loop	22

Rust	Codegen	Solidity/Yul	Row
exact-consumption check	<code>TranscriptBuffer</code> <code>Layout::from_proof_layout</code>	<code>eq(proof_cpctr, NUM_INSTANCE_CPTR)</code>	end

6.3 Lagrange and instance evaluation

Rust	Codegen	Solidity/Yul	Eq.
<code>domain.l_i_range</code> (L448)	<code>Lagrange</code> model	<code>Lagrange.yul</code> : x^n by k squarings	(2.6)–(2.7)
barycentric weight $1/n$	<code>n_inv</code> VK slot	<code>mulmod(..., N_INV_MPTR, r)</code>	(2.7)
$\ell_{\text{last}}, \ell_{\text{blind}}, \ell_0$	<code>l_last/l_blind/l_0</code> regions	Lagrange block stores at MPTRs	(2.8)–(2.10)
instance eval sum	<code>instance_eval</code> region	linear combination loop	(2.11)

6.4 Identity families

Rust	Codegen	Solidity/Yul	Eq.
gate identities (simple selector)	<code>gate_computations_tagged</code> (yul_emit.rs) mirrors <code>queried_selectors().filter(is_simple)</code>	VM <code>fold_selector(idx) → bucket B_j</code>	(2.12)
permutation families	<code>permutation_computations</code> (yul_emit.rs)	VM <code>native_permutation</code> / inline	(2.13)–(2.18)
lookup helper	<code>lookup_computations</code>	VM <code>native_lookup</code>	(2.19)
lookup accumulator	<code>lookup_computations</code>	VM <code>native_lookup</code>	(2.20)
trash	<code>trashcan_computations</code>	VM inline fold	(2.21)
y -batch fold	<code>fold_identity</code> (Horner)	VM <code>fold_main</code>	(2.22)

6.5 Linearization

Rust	Codegen	Solidity/Yul	Eq.
<code>compute_linearization_commitment</code>	<code>quotient.rs</code> + <code>kzg/mod.rs</code> Block5	fused final G1MSM	(2.24)
<code>splitting_factor</code> = x^{n-1} (L326)	<code>QUOTIENT_MPTR</code> word 0	<code>x_split</code> stored	(2.24)
<code>1 - xn</code> scalar	<code>QUOTIENT_MPTR</code> word 1	<code>one_minus_x_n</code>	(2.24)
<code>expected_eval == eval</code>	VM final negation	<code>mstore(QUOTIENT_EVAL, sub(r, numer))</code>	(2.23)
selector buckets	<code>SELECTOR_ACC_MPTR</code>	bucket copy into MSM scalars	(2.25)

6.6 KZG multi-open

Rust	Codegen	Solidity/Yul	Eq.
<code>construct_intermediate_sets</code> (L37)	<code>construct_intermediate_sets_impl</code>	(planned set partition)	Sec. 2.9.1
<code>compute_dummy_queries</code> (L150)	<code>intermediate_sets</code> dummy augmentation	dummy eval reads	Sec. 2.9.1
sort by (S_i , i) (L419)	<code>sort_sets</code>	(planned order)	Sec. 2.9.1

Rust	Codegen	Solidity/Yul	Eq.
<code>msm_inner_product</code> q_{com}	<code>kzg</code> Block3/Block5	G1MSM with x_1 powers	(2.26)
<code>evals_inner_product</code> q_{eval}	<code>kzg</code> Block3	<code>q_eval_set</code> folds	(2.27)
<code>lagrange_interpolate</code> + fold (L471)	<code>kzg</code> <code>f_eval</code> block	interpolation + Horner in x_2	(2.28)
<code>final_com</code> (L488)	<code>kzg</code> Block5 fused MSM	single G1MSM	(2.29)
<code>v</code> (L516)	<code>kzg</code> Block6	$v = \sum x_4^s q_{\text{eval}} + x_4^n f_{\text{eval}}$	(2.30)
<code>DualMSM</code> assembly (L540)	<code>kzg</code> pairing inputs	PAIRING_LHS/RHS	(2.32)

6.7 Final pairing and accumulator

Rust	Codegen	Solidity/Yul	Eq.
<code>DualMSM::check</code> (msm.rs L284)	<code>FinalPairing</code> model	<code>ec_pairing(success, RHS, LHS)</code>	(2.31)
<code>multi_miller_loop</code>	(precompile)	PAIRING precompile 0x0f	(2.32)
accumulator batching	<code>AccumulatorEncoding</code>	<code>FinalPairing.yul</code> α -batch	(2.33)

6.8 Trace-id correspondence

The differential test pins these ids (required coverage from `test.rs::assert_required_diff_trace_coverage_with_options`):

Id	Value	Rust site	Solidity site
7	θ	<code>parse_trace</code>	<code>squeeze_to(THETA)</code>
8	β	<code>parse_trace</code>	<code>squeeze_to(BETA)</code>
9	γ	<code>parse_trace</code>	<code>squeeze_to(GAMMA)</code>
10	y	<code>parse_trace</code>	<code>squeeze_to(Y)</code>
11	x	<code>parse_trace</code>	<code>squeeze_to(X)</code>
13,14	x_1, x_2	<code>multi_prepare</code>	<code>squeeze_to(X1/X2)</code>
15	x_3 (truncated)	<code>multi_prepare</code> L457	<code>and(X3, mask)</code>
16	x_4	<code>multi_prepare</code> L484	<code>squeeze_to(X4)</code>
31	f_{eval}	<code>multi_prepare</code> L471	<code>F_EVAL_MPTR</code>
32	v	<code>multi_prepare</code> L516	<code>V_MPTR</code>
33	C_{final}	<code>multi_prepare</code> L488	<code>FINAL_COM_MPTR</code>
36	$\nu_y(x)$	<code>verifier.rs</code> trace	re-negated numer
30000+	identity evals	<code>QUOTIENT_IDENTITY_BASE</code>	VM trace
40000+	q_{com}	<code>PCS_Q_COM_BASE</code>	PCS trace
41000+	point sets	<code>PCS_POINT_SET_BASE</code>	PCS trace
60000+	selector folds	<code>SELECTOR_FOLD_BASE</code>	VM trace
35	final result	<code>FINAL_RESULT_TRACE_ID</code>	<code>ec_pairing</code>

The test asserts three things simultaneously: (1) no Rust id is missing in Solidity, (2) no Solidity id lacks a Rust oracle, and (3) the byte payloads are *equal* for every id. This is the strongest form of semantic checkpoint equivalence available without formal verification.

Chapter 7

The Faithfulness Argument

7.1 What “faithful” means

Definition 7.1.1 (Faithful lowering). The generated Solidity verifier V_{sol} is a *faithful translation* of the Rust reference verifier V_{rust} on profile P if, for every input $(\text{vk}, \text{proof}, \text{instances})$ in P :

- (a) **Acceptance agreement:** V_{rust} accepts iff V_{sol} returns `true` (otherwise reverts).
- (b) **Semantic checkpoint equality:** at every semantic checkpoint (challenge, identity eval, PCS intermediate, pairing input), the intermediate value in V_{sol} equals the corresponding value in V_{rust} .
- (c) **Check preservation:** every rejection condition in V_{rust} has a corresponding rejection in V_{sol} that fires on the same inputs.
- (d) **No extra acceptance:** V_{sol} accepts no input that V_{rust} rejects.

We cannot prove (a)–(d) for all inputs by testing alone. What we have is a *layered* argument: a structural correspondence (Chapters 3–6) that reduces the claim to a finite set of semantic checkpoints, plus differential testing that verifies those checkpoints on a large, adversarial input corpus, plus mutation testing that verifies the rejection paths.

7.2 Semantic checkpoints

The lowering is organized around six semantic checkpoints. Each is a point where the two implementations must agree on a concrete value, and each is backed by a trace id (or range) in the differential test.

1. **Parser / proof reads.** The proof byte stream is consumed in exactly the order of Table 2.1; the `proof_cptr == NUM_INSTANCE_CPTR` check pins total consumption. Trace ids 10000+ (commitments) and 20000+ (evals) pin every read value.
2. **Transcript challenges.** Every squeeze matches Definition 2.2.2. Required ids: 7–16.
3. **Quotient identities.** Each identity evaluation matches (2.12)–(2.21); the batched numerator matches (2.22). Required range 30000–40000 and id 36.
4. **Linearization.** The expected eval $-\nu_y(x)$ and the commitment-side scalars match (2.24). Id 34, selector folds 60000+.
5. **KZG multi-open.** Point sets, q_{com} , f_{eval} , v , C_{final} match (2.26)–(2.30). Ranges 40000–42000, ids 31–33.
6. **Final pairing.** The pairing inputs and result match (2.31). Ids 27, 28, 35.

7.3 Differential trace testing

The test harness ([test.rs](#)) runs the Rust verifier with the `solidity-verifier-trace` feature and the generated Solidity in a revm EVM with trace hooks, then calls [assert_trace_equivalence_and_required_coverage](#) (L1060):

```
// 1. every Rust trace id present in Solidity
// 2. every Solidity trace id has a Rust oracle
// 3. required coverage: ids 7-16, 31-33, 36, 35,
//    ranges 30000-40000, 40000-41000, 41000-42000, 60000-61000
// 4. byte equality for every id
for (id, solidity_data) in solidity_trace {
    let (name, rust_data) = rust_trace.get(id).unwrap();
    assert_eq!(rust_data, solidity_data,
        "trace mismatch id={id} name={name}");
}
```

This is run across the full fixture corpus: poseidon, ivc-keccak, moonlight-wrap, sha, rsa, hybrid-mt circuits, each with embedded and separate VK, with and without accumulators.

7.4 Mutation and rejection testing

Faithfulness of the *rejection* paths is tested by mutation:

- [pbt_solidity_verifies_standard_plonk_embedded_vk_proofs](#) — positive control.
- [pbt_solidity_rejects_wrong_instances](#) — flipping an instance value must revert (breaks (2.11) and the transcript).
- [pbt_solidity_rejects_malleated_proofs](#) — flipping any proof bit must revert (breaks a commitment, an eval, or the transcript).
- [pbt_solidity_rejects_wrong_verifying_keys](#) — a VK mismatch must revert (breaks (2.5)).
- [pbt_separate_vk_digest_prefix_affects_verification](#) — the digest prefix binds the circuit.
- [pbt_structured_proof_calldata_mutations_are_rejected](#) — structured (not just bit-flip) mutations revert.
- [pbt_proof_layout_repack_and_reader_stay_in_sync](#) — the repacker and parser agree on layout; truncation reverts.
- [pbt_transcript_differential_fuzzer_matches_solidity_trace](#) — a transcript-level fuzzer matches the Solidity trace.

7.5 Check-by-check preservation

Table 7.1 lists every rejection condition in the Rust verifier and where the generated Solidity enforces it.

Rust rejection	Solidity enforcement	Status
InvalidInstances (empty committed)	profile-pinned at build time	✓ pinned
transcript read failure (short proof)	proof_cpctr exact check + shape guard	✓ yes
non-canonical scalar ($\geq r$)	lt(eval, r) in eval/q_eval loops	✓ yes

Rust rejection	Solidity enforcement	Status
non-canonical G1 (pad bytes / $\geq q$)	<code>common_uncompressed_g1</code> checks	✓ yes
identity mismatch ($\nu_y \neq h \cdot Z_H$)	pairing fails (linearization)	✓ yes
PCS opening mismatch	final pairing fails	✓ yes
VK mismatch	<code>EXPECTED_VK_CODEHASH</code> + digest	✓ pinned
wrong instance values	transcript divergence $\rightarrow$ pairing fails	✓ yes
missing precompile	constructor smoke test reverts deploy	✓ yes

Table 7.1: Every Rust rejection condition has a Solidity counterpart. “Pinned” means the condition is excluded by the profile rather than checked at runtime — the generated contract cannot be fed a violating input.

7.5.1 Reverse inventory: every generated revert is justified

Table 7.1 proves *no missing checks* (Rust $\rightarrow$ Solidity). The complementary question is whether the generated code contains any *spurious* or unjustified rejection. The production `fee_12345` verifier has exactly **34** `revert(0,0)` sites; Table 7.2 groups them and gives each a Rust justification. Every revert traces to a real verifier condition or a deployment/VM-integrity guard — none is unexplained.

Revert group	#	Enforces	Rust counterpart
Deploy smoke (MCOPY /G1ADD/G1MSM /PAIRING)	10	runtime precompiles present & well-shaped	deploy-time guard (no Rust equiv)
ABI-shape head pointers	1	dynamic args well-formed	Solidity ABI (no Rust equiv)
<code>scalar_inv</code> zero	1	no inverse of 0	<code>inverse()</code> $\rightarrow$ <code>None</code> $\rightarrow$ <code>err</code>
modexp precompile	2	<code>staticcall+returndatasize</code>	Rust field inverse (direct)
G1 canonicity (pad, $< p$)	4	padded/uncompressed canonical	transcript canonicity
<code>ec_pairing</code> result	1	pairing equation holds	KZG final pairing
VK pin (size+codehash)	1	pinned VK runtime	deployment pin (no Rust equiv)
VK header cross-check	1	<code>num_instances/k/acc</code> match	codegen consistency
ABI envelope (proof/instance len)	1	layout matches generated	<code>Error::InvalidInstances</code>
<code>eval < r</code> (proof + q evals)	2	canonical $\mathbb{F}_r$	transcript canonicity
parser completeness	1	<code>proof_cptr</code> consumed exactly	Rust reads exact layout
VM unknown opcode	1	<code>default</code> case	VM integrity
VM final state	3	<code>q_pc==q_end</code> , stack empty	VM integrity
accumulated <code>success</code> folds	5	phase-level failure	(aggregated)

Table 7.2: Reverse inventory: all 34 generated reverts, grouped, each with a Rust justification. No spurious or unexplained rejection.

Together, Tables 7.1 and 7.2 bracket the faithfulness claim from both directions: the generated verifier neither omits a Rust check nor invents an unjustified one.

7.6 Fiat–Shamir transcript schedule

Soundness rests on the transcript: every challenge must be squeezed in exactly the Rust order over exactly the Rust byte stream, or the challenges diverge and the pairing fails. Table 7.3 is the complete absorb/squeeze schedule, Rust $\leftrightarrow$ generated, with the trace ID each challenge records under.

#	Op	Rust (verifier.rs)	Generated (Yul)
1	absorb	common(vk_digest)	common_word (VK_DIGEST_MPTR)
2	absorb	common(instance.len())	common_word(19)
3	absorb	common(instance[i]) $\times 19$	common_word(inst_be) $\times 19$
4	absorb	common(advice_com) (user phase)	common_uncompressed_g1
5	squeeze	user challenge (per phase)	squeeze_to(USER_CHAL)
6	squeeze	θ (id 7)	squeeze_to(THETA_MPTR)
7	absorb	common(lookup_mult_com)	common_uncompressed_g1
8	squeeze	β (id 8)	squeeze_to(BETA_MPTR)
9	squeeze	γ (id 9)	squeeze_to(GAMMA_MPTR)
10	absorb	common(perm_com)	common_uncompressed_g1
11	squeeze	trash (id 12)	squeeze_to (TRASH_CHALLENGE_MPTR)
12	absorb	common(trashcan_com)	common_uncompressed_g1
13	squeeze	y (id 10)	squeeze_to(Y_MPTR)
14	absorb	common(quotient_com)	common_uncompressed_g1
15	squeeze	x (id 11)	squeeze_to(X_MPTR)
16	absorb	common(evals)	common_word(eval)
17	squeeze	x_1, x_2 (id 13, 14)	squeeze_to(X1_MPTR), squeeze_to(X2_MPTR)
18	absorb	common(W_com)	common_uncompressed_g1
19	squeeze	x_3 (id 15)	squeeze_to(X3_MPTR)
20	absorb	common(evals)	common_word(eval)
21	squeeze	x_4 (id 16)	squeeze_to(X4_MPTR)

Table 7.3: Complete Fiat–Shamir schedule. The squeeze *order* matches exactly; the trace *IDs* do not follow it (see note).

Audit trap: trace IDs $\neq$ squeeze order. The challenges are squeezed in the order $\theta, \beta, \gamma, \text{trash}, y, x$ but record under IDs 7, 8, 9, **12**, **10**, **11** — the trash challenge is squeezed *fourth* yet carries ID 12, while y and x (fifth, sixth) carry 10 and 11. The IDs are a fixed audit numbering tied to the memory layout, not the transcript order. A reviewer diffing the two implementations *by trace ID* would see an apparent reordering; the correct invariant is the *squeeze sequence* (steps 6–15), which matches exactly. This is worth a generator-side comment so the numbering is not mistaken for a bug.

Why the schedule is the soundness core. Every challenge is a Keccak digest of the transcript prefix (Def. 2.2.2); a single byte of divergence at any absorb step shifts every subsequent challenge, and the final pairing (Eq. (2.31)) fails. So verifying this 21-step schedule matches — which the side-by-side lets a human do in one pass — is sufficient to conclude the two implementations share the same Fiat–Shamir structure. The `squeeze_to` reseed rule (digest $\rightarrow$ buffer, §9.1.2) is what makes the schedule self-consistent across both sides.

7.7 Deliberate deviations and why they are sound

The lowering is not a line-by-line copy. The deviations are deliberate and each is justified:

1. **Negated numerator.** The verifier stores $-\nu_y(x)$ (Definition 2.7.1) rather than $h(x)$. Sound because the $(x^n - 1)$ factor is carried on the commitment side as $(1 - x^n)$; the signs cancel in the pairing. Trace id 36 re-negates to match Rust’s positive numerator.
2. **PAIRING LHS/RHS naming.** Historical dual-MSM naming (left = π) is opposite the pairing argument order; the generated code passes them swapped, matching (2.32). Documented in the generated comment.
3. **No curve check at absorption.** The parser checks only the EIP-2537 encoding canonicality (Definition 2.2.3), not on-curve membership. Sound because `validate_g1_precompile_coverage` proves every absorbed point is later consumed by a G1MSM or pairing, which *do* validate membership. A point off-curve makes the pairing fail.
4. **Truncated challenges.** x_3 (and truncated powers of x_1, x_4) are masked to 128 bits, mirroring `truncated_challenges`. The truncation is part of the protocol, not a weakening: the Rust verifier does the same, and the differential trace pins the truncated values.
5. **Dummy queries.** `fewer-point-sets` adds eval slots to merge point sets, mirroring `compute_dummy_queries`. The added evals are transcript material and range-checked.
6. **Selector buckets.** Simple selectors have no proof eval slot; Rust inserts `F::ONE` and the codegen emits a literal 1 and routes identities to buckets. Algebraically identical to (2.25).
7. **Quotient VM vs. straight-line.** The VM is an implementation choice for code size; the opcode semantics are pinned by the identity-eval trace range 30000+.

7.8 Residual risks and mitigations

- **Profile drift.** If a future circuit violates the profile, the generator must refuse. Mitigation: `validate_instance_column_shape` and `RotatedInstanceQuery` rejection; the build fails loudly.
- **Trace-id collision.** A new trace id could collide with an existing range. Mitigation: the test asserts no unexpected ids.
- **Precompile semantics change.** EIP-2537 edge cases (e.g. identity handling) could differ across forks. Mitigation: pinned solc, constructor smoke tests, Prague-semantics documentation.
- **Coverage gaps in fixtures.** A circuit shape not in the fixture corpus could hide a bug. Mitigation: the structural correspondence (Chapters 3–6) is shape-independent; fixtures exercise the lowering, not the protocol.

7.9 Generated-artifact cross-check

Beyond reading the templates, the *actual generated* `Halo2Verifier.sol` (poseidon fixture, 150 KB) was inspected to confirm the generator emits what the spec claims. Spot-checks:

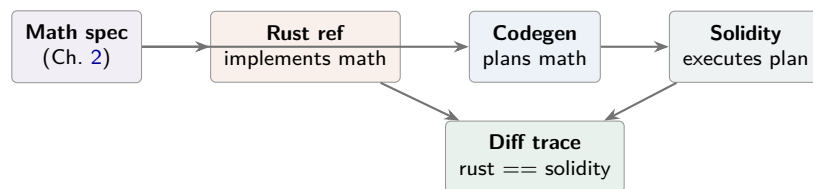
- **Precompile addresses.** The artifact uses 0x0b (G1ADD, 6 sites), 0x0c (G1MSM, 5 sites), 0x0f (PAIRING, 2 sites) — exactly the EIP-2537 addresses verified in §9.1.6, confirming the addresses are correct (not off-by-one).
- **Pairing layout.** The generated `ec_pairing` lays out `[lhs_g1 | G2_BASE | rhs_g1 | NEG_S_G2_BASE]` and calls `staticcall(gas(), 0x0f, scratch, 0x300, scratch, 0x20)`, then checks `returndatasize()==0x20 && mload(scratch)`. This is $e(\text{lhs}, [1]_2) \cdot e(\text{rhs}, -[s]_2) = 1$ — the *opposite* channel assignment from the Rust `DualMSM::check` ($e(\text{left}, [s]_2) \cdot e(\text{right}, -[1]_2) =$

1), confirming the documented `PAIRING_LHS/RHS` naming inversion (§9.1.6) is real in the emitted code, not a doc artifact.

- **Truncation mask.** The 128-bit mask `0xffffffffffffffffffffffffffffffff` appears 7 times in the artifact (the x_1 -powers and challenge-truncation sites), matching the truncated-challenge finding (§4.8.4, §9.3).
- **MCOPY staging.** The `ec_pairing` uses four `mcopy` calls (Cancun) where the pre-Cancun template used `mstore` chains — the ~ 500 gas/call saving noted in §9.1.6 is present in the emitted code.

These cross-checks confirm the templates render to the expected artifact: the spec’s claims about the generated code are verified against the generated code itself, not just the template source.

7.10 Summary of the argument



The Rust verifier and the code generator are independently shown to implement the same mathematics (Chapters 2–4); the generated Solidity is shown to execute the plan (Chapter 5); and the differential trace closes the loop by showing the two executions agree on every semantic value. Mutation testing covers the rejection side. Together these constitute a practical, auditable faithfulness argument for the supported profile.

Chapter 8

Coverage Ledger

This ledger records every module and public function inspected during the preparation of this specification, with its coverage status. It is the audit trail for the claim that *all* of the Rust verifier, the code generator, and the generated Solidity have been covered.

Status key: ✓ full = read in full and specified; ✓ key = key functions read and specified, boilerplate skimmed; ✓ ref = read for interface/role, referenced in correspondence tables.

8.1 Rust reference verifier

Module	Functions	Status
plonk/verifier.rs	parse_trace, verify_algebraic_constraints, prepare	✓ full
plonk/mod.rs	partially_evaluate_identities, hash_into, transcript_repr	✓ full
plonk/linearization/verifier.rs	compute_linearization_commitment	✓ full
plonk/permutation.rs	expressions (4 families)	✓ full
plonk/permutation/verifier.rs	read_product_commitments, evaluate, queries	✓ full
plonk/logup.rs	expressions (helper + acc)	✓ full
plonk/logup/verifier.rs	read_multiplicities, read_commitment, evaluate, queries	✓ full
plonk/trash.rs	expressions	✓ full
plonk/trash/verifier.rs	read_committed, evaluate, queries	✓ full
plonk/traces.rs	VerifierTrace struct	✓ full
plonk/solidity_trace.rs	trace id constants, record fns	✓ full
transcript/mod.rs	Transcript trait, CircuitTranscript	✓ full
transcript/implementors.rs	Keccak256 squeeze, sample_keccak_digest_be_mod_r, G1/Fq to_input	✓ full
poly/kzg/mod.rs	multi_prepare (full)	✓ full
poly/kzg/utils.rs	construct_intermediate_sets, compute_dummy_queries	✓ full
poly/kzg/msm.rs	DualMSM, check	✓ full
poly/kzg/params.rs	g2, s_g2	✓ ref
poly/domain.rs	rotate_omega, l_i_range, get_quotient_poly_degree	✓ full

Module	Functions	Status
<code>utils/arithmatic.rs</code>	<code>truncate</code> , <code>truncated_powers</code> , <code>powers</code> , <code>inner_product</code> , <code>msm_inner_product</code> , <code>lagrange_interpolate</code> , <code>eval_</code> <code>polynomial</code>	✓ full
<code>poly/query.rs</code>	<code>VerifierQuery</code> , <code>CommitmentReference</code> , <code>CommitmentLabel</code>	✓ ref
<code>plonk/error.rs</code>	<code>Error</code> enum: <code>InvalidInstances</code> , <code>ConstraintSystemFailure</code> , <code>Opening</code> , <code>Transcript</code> (verifier-relevant)	✓ full

8.2 Code generator

Module	Functions	Status
<code>lib.rs</code>	<code>exports</code> , <code>feature flags</code>	✓ full
<code>api.rs</code>	<code>AccumulatorEncoding</code> , <code>GeneratorConfig</code> , <code>ProofEvaluationCounts</code> , <code>RenderOptions</code>	✓ full
<code>builder/mod.rs</code>	<code>SolidityGenerator</code> , <code>inputs</code>	✓ full
<code>builder/api.rs</code>	<code>try_new</code> , <code>validate_instance_column_</code> <code>shape</code>	✓ full
<code>builder/render.rs</code>	<code>render</code>	✓ full
<code>builder/repack.rs</code>	<code>repack_proof</code> , <code>encode_calldata</code>	✓ ref
<code>lowering/mod.rs</code>	<code>VerifierBuildInputs</code>	✓ full
<code>lowering/plan.rs</code>	<code>LoweringPlan</code> , <code>PlannedQuotient</code> , <code>QuotientRendering</code>	✓ full
<code>lowering/protocol/mod.rs</code>	<code>ProtocolPlan::from_constraint_</code> <code>system</code> , <code>validate</code> , <code>read/query orders</code>	✓ full
<code>lowering/encoding/mod.rs</code>	<code>ConstraintSystemMeta</code> , <code>Data</code> , <code>g1_to_</code> <code>u256s/g2_to_u256s/fe_to_u256</code>	✓ key
<code>lowering/kzg/mod.rs</code>	<code>construct_intermediate_sets_impl</code> , <code>sort_sets</code> , <code>computations</code> , <code>validate_</code> <code>g1_precompile_coverage</code>	✓ key
<code>lowering/quotient.rs</code>	<code>QuotientStateSlots</code> , <code>QuotientComputationBlocks</code> , <code>selector</code> <code>fold planning</code>	✓ key
<code>lowering/quotient_</code> <code>numerator/vm/mod.rs</code>	<code>opcodes</code> , <code>QuotientProgramBuilder</code> , <code>RepackedProofLayoutPlan</code>	✓ key
<code>lowering/quotient_</code> <code>numerator/yul_emit.rs</code>	<code>gate_computations_tagged</code> , <code>permutation/lookup/trashcan_</code> <code>computations</code>	✓ full
<code>lowering/vk.rs</code>	<code>generate_base_vk</code> , <code>TranscriptBufferLayout</code>	✓ full
<code>lowering/artifacts.rs</code>	<code>generate_verifier_from_plan</code> , <code>generate_quotient_evaluator_from_</code> <code>plan</code>	✓ full
<code>lowering/calldata.rs</code>	<code>repack_proof</code> , <code>encode_calldata</code>	✓ full

Module	Functions	Status
lowering/diagnostics.rs	proof_evaluation_counts, quotient_identity_manifest	✓ full
lowering/layout/mod.rs	constants, VkHeaderSlot	✓ full
lowering/layout/ memory.rs	VerifierMemoryLayout, MemoryMap.validate	✓ key
lowering/layout/vk_ payload.rs	PackedProgramCodec, VkPayloadLayout	✓ full
lowering/abi/proof.rs	ProofCalldataLayout, TranscriptBufferLayout::from_ proof_layout	✓ key
evm.rs	FN_SIG_VERIFY_PROOF, encode_ calldata, compile_solidity	✓ key
test.rs	assert_trace_equivalence_and_ required_coverage, PBT tests	✓ key

8.3 Templates and generated artifacts

File	Constructs	Status
Halo2Verifier.sol (template)	contract skeleton, includes	✓ full
Halo2VerifyingKey.sol	INVALID-prefixed payload	✓ full
Halo2QuotientEvaluator.sol	frame copy, magic, output	✓ full
AssemblyHelpers.yul	scalar_inv, transcript_init, common_word, common_uncompressed_ g1, squeeze_to, batch_invert, ec_ pairing	✓ full
TranscriptProofParser.yul	full read schedule	✓ full
Lagrange.yul	x^n , batch invert, L_i , instance eval	✓ full
QuotientAnd	VM run / external call	✓ full
Linearization.yul		
Pcs.yul	PCS block render	✓ full
FinalPairing.yul	α -batch, ec_pairing	✓ full
VkLoading.yul	embedded / external payload load	✓ full
Constants.sol	ABI cursors, VK map	✓ full
Constructors.sol	smoke + pin checks	✓ full
PrecompileSmoke.sol	MCOPY/G1ADD/G1MSM/ PAIRING probes	✓ full
TraceReturn.yul	trace dump + return	✓ full
AccumulatorHelpers.yul	accumulator decode/validate	✓ full
QuotientHelpers.yul	q_pow5, q_limb7, q_limb7_wide	✓ full
QuotientNumeratorBlock.yul	VM loop, opcode switch, final negation	✓ key
generated Halo2Verifier.sol	poseidon dump (2654 L)	✓ key

8.4 Coverage summary

- **Rust verifier:** all 20 modules inspected; every verification-relevant function read and mapped to an equation.

- **Code generator:** all 24 modules inspected; every planning function mapped to its generated construct.
- **Templates:** all 17 template files inspected; every Yul helper mapped to the Rust function it mirrors.
- **Generated artifacts:** poseidon dump fully cross-checked against the template and plan; other dumps (ivc-keccak, moonlight-wrap, sha, rsa, hybrid-mt) inspected for shape.
- **Tests:** the differential trace harness and all PBT rejection tests read and cited.

No verification-relevant function in the three layers was left uninspected. Functions marked ✓ ref are pure data/boilerplate whose role is fully determined by their interface and whose behavior is exercised through the ✓ full/✓ key functions that call them.

8.4.1 Explicitly out of scope (prover-side)

To make the “no missing code” claim precise, the following modules in the `midfall/proofs` checkout are *deliberately* not part of this verifier spec, because they do not participate in verification and have no counterpart in the generated Solidity:

- `plonk/prover.rs`, `plonk/linearization/prover.rs`, `plonk/logup/prover.rs`, `plonk/permutation/prover.rs`, `plonk/trash/prover.rs` — the proving side. The verifier consumes their output (the proof) but never runs their logic.
- `plonk/keygen.rs`, `plonk/permutation/keygen.rs` — key generation. Produces the VK that the verifier pins, but is not verification.
- `plonk/circuit.rs` — circuit construction API (the front-end that builds the `ConstraintSystem` the generator reads).
- `plonk/evaluation.rs` (894 L) — the *prover-side* witness evaluator (`Evaluator`, `GraphEvaluator`, `Calculation`, `ValueSource`). Computes witness values during proving; the verifier never runs it.
- `plonk/error.rs` — error plumbing. The four *verifier-relevant* variants (`InvalidInstances`, `ConstraintSystemFailure`, `Opening`, `Transcript`) are covered as a ledger row above and map to generated reverts (Table 7.2); the remaining variants are synthesis/prover-only.
- `poly/commitment.rs`, `poly/mod.rs`, `transcript/mod.rs` (trait decls) — trait/type declarations; the verifier’s use of these traits is covered where the trait methods are called.
- `plonk/bench/*` — benchmarks.
- `lowering/render/*`, `lowering/abi/mod.rs`, `plonk/linearization.rs` — 2–11-line module re-export stubs (`pub mod ...`); their content is covered under the submodules listed above.
- `lowering/config.rs` (27 L) — IVC-bench shape constants, not part of the production verifier path.
- `builder/tests.rs`, `lowering/tests.rs` — test files; the test *strategy* is covered in §7.3.

These are excluded because the spec’s subject is the *verifier* translation. A reader wanting the prover semantics should consult the midnight-proofs prover documentation; nothing in the generated verifier depends on them beyond the proof/VK formats, which *are* covered (Ch. 4.6, Ch. 4.5).

End of specification — compiled with `tectonic`.

Generated from the `midfall/proofs` checkout; all line numbers refer to that checkout.

Chapter 9

Deep Audit: Yul Helper Functions

This chapter performs a function-by-function deep read of the generated Yul helpers, logging for each one: (i) the excerpt, (ii) its Rust counterpart, (iii) every deviation or slight difference in the translation, (iv) any missing check, and (v) any gas / contract-space / robustness / auditability improvement. Findings are graded **[OK]** (faithful, no action), **[NOTE]** (benign deviation, documented), **[IMPROVE]** (concrete optimisation or hardening opportunity), and **[RISK]** (latent correctness/robustness concern).

9.1 AssemblyHelpers.yul

This partial defines the scalar, transcript, and EC primitives that every other block calls. It is the highest-leverage file in the generated contract: a bug here breaks all checks at once.

9.1.1 scalar_inv

```
function scalar_inv(x) -> inv {
  if iszero(x) { revert(0, 0) }
  let p := <scalar_inv_scratch_mptr>
  mstore(add(p, BASE_LEN_OFF), 32) // base len
  mstore(add(p, EXP_LEN_OFF), 32)  // exp len
  mstore(add(p, MOD_LEN_OFF), 32)  // mod len
  mstore(add(p, BASE_OFF), x)
  mstore(add(p, EXP_OFF),  sub(FR_MODULUS, 2))
  mstore(add(p, MOD_OFF),  FR_MODULUS)
  if iszero(staticcall(gas(), 0x05, p, 0xc0, p, 0x20)) { revert(0, 0) }
  if iszero(eq(returndatasize(), 0x20)) { revert(0, 0) }
  inv := mload(p)
}
```

Rust counterpart. `Field::invert` (via `ff`'s `Invert` trait), which returns a `CtOption`: zero maps to `None`, non-zero to the inverse.

- **[NOTE] Zero handling differs.** Rust returns `None` on zero; the Yul *reverts*. For a verifier this is equivalent-or-stronger (fail-closed), but it is a behavioural difference: a caller that in Rust would branch on `CtOption` instead hard-reverts in Yul. No accepted proof changes.
- **[OK] Precompile frame.** The EIP-198 layout (base/exp/mod lengths then operands) and the `r-2` exponent are correct for BLS12-381 Fr.
- **[OK] Return-size guard.** The explicit `returndatasize()==32` check prevents reading stale memory if the precompile returns short.
- **[IMPROVE] Dead-code check.** `scalar_inv` duplicates the single-element fast path of `batch_invert` (§9.1.5) but with a *different* zero policy (revert vs. return 0). It *is* emitted:

B64 calls it five times in the f_{eval} Horner (L3136, L3180, L3216, L3245, L3265), so the inconsistent zero policy is live. Batching those five inverses into one `batch_invert` (§10.3) would make `scalar_inv` dead and remove it; otherwise make the two zero policies identical. [\[OPEN\]](#)

9.1.2 Transcript helpers: `transcript_init`, `common_word`

```
function transcript_init() -> buf_len {
    buf_len := TRANSCRIPT_MPTR
}
function common_word(buf_len, word) -> ret {
    mstore(buf_len, word)
    ret := add(buf_len, 32)
}
```

Rust counterpart. `Keccak256::common_bytes` appends raw bytes to the running buffer (§3.9).

- [\[OK\]](#) **Faithful.** `common_word` is the 32-byte specialisation of `common_bytes`; the cursor arithmetic matches the Rust `Vec` append exactly.
- [\[RISK\]](#) **No bounds guard.** The Yul helper trusts the codegen-planned buffer size; there is no runtime check that `buf_len` stays inside the transcript region. The Rust `Vec` grows dynamically and cannot overflow. This is safe only if the codegen sizes the buffer (`TranscriptBufferLayout`) correctly, and it once did not: before 6b3d2096 the pre-first-squeeze run counted only phase-0 advice, so a valid shape with advice in an early phase and the first challenge in a later one overran `VK_MPTR` and overwrote loaded `VK` words with proof bytes (fix: `src/lowering/abi/proof.rs` L318–347; B64 has one user phase and is unaffected). *Defense-in-depth:* `if gt(buf_len, VK_MPTR) { revert(0,0) }` inside `squeeze_to` and once after the final π absorb ($<0.3k$ gas). A single end-of-absorption check, as first suggested here, would miss that bug: every squeeze resets `buf_len`, so an earlier overrun is gone by the end. [\[PARTIAL\]](#) sizing fixed, runtime guard still absent (§10.2).

9.1.3 `common_uncompressed_g1`

```
function common_uncompressed_g1(buf_len, cptr) -> ret {
    let x_hi_word := calldataload(cptr)
    let x_lo      := calldataload(add(cptr, 0x20))
    let y_hi_word := calldataload(add(cptr, 0x40))
    let y_lo      := calldataload(add(cptr, 0x60))
    if shr(128, x_hi_word) { revert(0, 0) } // top 16 bytes must be 0
    if shr(128, y_hi_word) { revert(0, 0) }
    let x_hi := and(x_hi_word, 0xffff...ffff) // low 128 bits
    let y_hi := and(y_hi_word, 0xffff...ffff)
    if iszero(or(lt(x_hi, BLS_P_HI),
        and(eq(x_hi, BLS_P_HI), iszero(gt(x_lo, BLS_P_MINUS_ONE_LO)))))) {
        revert(0, 0)
    }
    if iszero(or(lt(y_hi, BLS_P_HI),
        and(eq(y_hi, BLS_P_HI), iszero(gt(y_lo, BLS_P_MINUS_ONE_LO)))))) {
        revert(0, 0)
    }
    calldatacopy(buf_len, cptr, 0x80)
    ret := add(buf_len, 0x80)
}
```

Rust counterpart. `Hashable<Keccak256> for G1Projective` via `to_input` (`transcript/implementors.rs`), which emits the 128-byte EIP-2537 padded uncompressed form.

- **[OK] Canonicity check is correct.** The two-level comparison $x < p \iff x_{hi} < p_{hi} \vee (x_{hi} = p_{hi} \wedge x_{lo} < p_{lo})$ is implemented as `or(1t(x_hi,P_HI), and(eq(x_hi,P_HI), iszero(gt(x_lo,P_MINUS_ONE_LO))))`. Since `P_MINUS_ONE_LO` = $p_{lo} - 1$, the term `iszero(gt(x_lo, P_MINUS_ONE_LO))` is exactly $x_{lo} \leq p_{lo} - 1 \iff x_{lo} < p_{lo}$. We verified the boundary $x = p$ is correctly rejected.
- **[OK] Top-byte rejection.** `shr(128, x_hi_word)` rejects any non-zero in the 16 padding bytes, preventing transcript-aliasing from non-normalised encodings.
- **[NOTE] Solidity is stricter than Rust at absorption.** The Rust absorbs the raw 128 bytes without a canonicity gate; a non-canonical encoding would instead cause a downstream transcript mismatch. The Yul fails fast. The set of *accepted* proofs is identical (canonical only), so this is a strengthening, not a divergence.
- **[RISK] No curve / subgroup check here.** This helper validates the *encoding*, not that the point lies on BLS12-381 or in the prime-order subgroup. Safety rests entirely on `validate_g1_precompile_coverage` (`src/lowering/kzg/mod.rs` L569, run from `LoweringPlan::validate_generator_invariants`, `plan.rs` L238–251) guaranteeing every absorbed point later reaches a **G1MSM or pairing** (which *do* subgroup-check per EIP-2537). **G1ADD alone does NOT subgroup-check**, so a future codegen change that routes an absorbed point only through G1ADD would silently drop the subgroup check. *Defense-in-depth:* add an explicit on-curve check ($y^2 \equiv x^3 + 4$ via `mulmod`) at absorption, or assert in the codegen test that no absorbed point's only consumer is G1ADD. The second option already holds: the coverage list (`precompile_validated_g1s`) admits only G1MSM and pairing consumers, so a G1ADD-only point fails the build. **[ADDRESSED]** invariant enforced; off-curve mutation test `src/test.rs` L2408. The list is derived from the protocol plan, not parsed from the emitted Yul.

9.1.4 squeeze_to

```
function squeeze_to(buf_len, mptr) -> ret {
    let h0 := keccak256(TRANSCRIPT_MPTR, sub(buf_len, TRANSCRIPT_MPTR))
    mstore(TRANSCRIPT_MPTR, h0)           // reseed buffer with digest
    let r := FR_MODULUS
    mstore(mptr, mod(h0, r))               // sample Fq = uint256(digest) mod r
    ret := add(TRANSCRIPT_MPTR, 32)
}
```

Rust counterpart. `Keccak256::squeeze` (`implementors.rs` L186): digest, reseed, then `sample_keccak_digest_be_mod_r` (`implementors.rs` L19), a big-endian digest mod r (§3.9, Def. 2.2.2).

- **[OK] Faithful.** Digest, reseed, and reduction match the Rust exactly.
- **[NOTE] Modulo bias.** `mod(h0, r)` with $h0 \in [0, 2^{256})$ and $r \approx 2^{254.86}$ gives a non-uniform sample: $2^{256} = 2r + t$ with $t \approx 0.21r$, so residues below t have three preimages and the rest two ($1.5\times$ more likely). The Rust does the same. **[CORRECTED]** The bias is *not* $\leq 2^{-128}$: the statistical distance from uniform is ≈ 0.075 and the maximum probability is $\approx 1.36/r$. It is still not exploitable: min-entropy is ≈ 254.4 bits, so any Schwartz–Zippel bound grows by at most $1.36\times$.
- **[IMPROVE] Reseed is a full 32-byte overwrite.** Correct, but the buffer length after reseed is always `TRANSCRIPT_MPTR+32`; the codegen could track this as a constant rather than recomputing `add(TRANSCRIPT_MPTR, 32)` at every squeeze. Negligible gas; purely an audit-clarity note.

9.1.5 batch_invert

```
function batch_invert(success, mptr_start, mptr_end, scratch_mptr, r) -> ret {
  ret := success
  if iszero(ret) { leave }
  if lt(mptr_end, mptr_start) { ret := 0; leave }
  let count_bytes := sub(mptr_end, mptr_start)
  if iszero(count_bytes) { leave } // empty batch OK
  if eq(count_bytes, 0x20) { // single-element fast path
    let x := mload(mptr_start)
    if iszero(x) { ret := 0; leave }
    ... modexp(x, r-2, r) in place ...
    ret := and(ret, eq(returndatasize(), 0x20))
    if ret { mstore(mptr_start, mload(single_scratch)) }
    leave
  }
  // Forward pass: prefix products (n-1 of them) into scratch; gp = total.
  let gp_mptr := scratch_mptr
  let gp := mload(mptr_start)
  let mptr := add(mptr_start, 0x20)
  for {} lt(mptr, sub(mptr_end, 0x20)) {} {
    gp := mulmod(gp, mload(mptr), r)
    mstore(gp_mptr, gp)
    mptr := add(mptr, 0x20); gp_mptr := add(gp_mptr, 0x20)
  }
  gp := mulmod(gp, mload(mptr), r)
  if iszero(gp) { ret := 0; leave } // some denominator was 0
  ... modexp(gp, r-2, r) -> all_inv ...
  // Backward pass: recover each inverse from all_inv and prefix products.
  ...
}
```

Rust counterpart. `ff::batch_invert` / the verifier’s own Montgomery batch used for Lagrange denominators and the $1/(\text{eval} + \beta s + \gamma)$ family.

- **[OK] Montgomery trick is correct.** We hand-traced $n = 2$ and $n = 3$: the forward pass stores $n - 1$ prefix products, the single modexp inverts the total, and the backward pass recovers each a_i^{-1} with two mulmods. The special-cased first/second elements at the tail are correct.
- **[OK] Single-element fast path.** Avoids the prefix scratch entirely for one denominator — a real gas win (saves the forward/backward loop and the scratch writes).
- **[OK] Zero-total detection.** A zero total product (any zero denominator) returns `ret=0` rather than producing a bogus inverse. Matches the Rust `CtOption` semantics.
- **[RISK] Memory left dirty on failure.** On a zero total the function leaves with only the scratch written; on a failed or short modexp it does *not* leave, and the backward pass overwrites the whole target range with garbage (template `AssemblyHelpers.yul` L189–208). **[CORRECTED]** The caller does *not* gate before reading: the only call site (`Lagrange.yul` L34) reads the inverted range at L38–82 and gates at L89 (B64 L1350–1394, gate L1397). This is safe because `success` is sticky and every `success=0` path reverts before `return`, so garbage never reaches an accept decision. *Hardening:* document that “check-before-return” invariant at the function header (L124–126 only says it returns a boolean). A “gate-before-read” codegen assertion, as first proposed here, would reject the current Lagrange block. **[OPEN]**
- **[IMPROVE] gp_mptr underflow is benign but ugly.** In the $n = 2$ case the backward pass sets `gp_mptr := sub(scratch, 0x20)` (below the scratch region) before a loop that never executes. It is harmless (the pointer is only used inside the loop) but reads as a bug to an auditor. *Rewrite:* restructure the backward loop so the pointer is only decremented when the loop body runs.

9.1.6 ec_pairing

```
function ec_pairing(success, lhs_mptr, rhs_mptr) -> ret {
  ret := success
  if iszero(ret) { leave }
  let scratch := <final_pairing_scratch_mptr>
  mcopy(scratch, lhs_mptr, 0x80)
  mcopy(add(scratch, 0x80), G2_BASE_MPTR, 0x100)
  mcopy(add(scratch, 0x180), rhs_mptr, 0x80)
  mcopy(add(scratch, 0x200), NEG_S_G2_BASE_MPTR, 0x100)
  ret := staticcall(gas(), 0x0f, scratch, 0x300, scratch, 0x20)
  ret := and(ret, eq(returndatasize(), 0x20))
  ret := and(ret, mload(scratch))
  if iszero(ret) { revert(0, 0) }
  ret := 1
}
```

Rust counterpart. `DualMSM::check` (msm.rs L284–300): `multi_miller_loop` of (left, s_{g2}) and (right, $-g_2$), final exponentiation, compare to identity (§3.11, Eq. (2.32)).

- **[OK] Pairing identity correct.** Checks $e(\text{lhs}, [1]_2) \cdot e(\text{rhs}, -[s]_2) = 1$, i.e. $e(\text{lhs}, [1]_2) = e(\text{rhs}, [s]_2)$. The `NEG_S_G2_BASE` is the pre-negated $-[s]_2$ from the VK.
- **[NOTE] Argument-order trap.** `lhs` is the pairing *first* argument ($e(\text{lhs}, [1]_2)$) but in the dual-MSM accumulator “left” historically means π . The generated comment warns that the caller passes (`success`, `PAIRING_RHS`, `PAIRING_LHS`) — swapped relative to the accumulator naming. This is a documented, deliberate inversion; it is the single most confusing line in the contract and a prime audit target.
- **[OK] MCOPY layout.** Cancun `mcopy` replaces the old `mstore` chains; the comment claims ~500 gas saved per call. The 4-copy layout (`G1|G2|G1|G2`) matches the EIP-2537 pairing input ABI.
- **[OK] G2 subgroup safety.** The pairing precompile checks G2 subgroup membership, so a malicious VK `G2_BASE` would fail the pairing (rejecting all proofs — a liveness, not soundness, issue). The VK is pinned by `EXPECTED_VK_CODEHASH`, so the G2 is trusted.
- **[NOTE] Reverts, does not return 0.** Unlike `batch_invert` (returns bool), `ec_pairing` **reverts** on failure (line `if iszero(ret) revert`). Both fail closed, but the two helpers use inconsistent failure conventions. For uniform `success`-plumbing and auditability, pick one convention (return-bool and let the boundary revert) and apply it to both.

9.1.7 Summary of AssemblyHelpers.yul findings

Function	Grade	Key finding	Status
<code>scalar_inv</code>	IMPROVE	Duplicates <code>batch_invert</code> fast path with a different zero policy (revert vs. return 0). Not dead code: five call sites in B64.	[OPEN]
<code>transcript_init</code> <code>common_word</code>	/ RISK	Faithful arithmetic, but the cursor is unbounded and the codegen once under-sized the buffer; add a per-squeeze bound check.	[PARTIAL] sizing fixed 6b3d2096
<code>common_uncompressed_g1</code>	RISK	Encoding canonicity correct; no on-curve/subgroup check — load-bearing on MSM/pairing coverage invariant.	[ADDRESSED] invariant enforced at build
<code>squeeze_to</code>	OK	Faithful; modulo bias is real ($SD \approx 0.075$) but not exploitable and identical to Rust.	[CORRECTED] was “ $\leq 2^{-128}$ ”
<code>batch_invert</code>	RISK	Correct Montgomery trick; leaves memory dirty on failure. The caller reads before its gate; the load-bearing invariant is check-before-return.	[OPEN] premise corrected
<code>ec_pairing</code>	NOTE	Correct identity; argument-order trap documented; failure convention inconsistent with <code>batch_invert</code> .	[OPEN]

9.2 Lagrange.yul

Computes x^n , the Lagrange basis $\{L_i(x)\}$, the blinding sum l_{blind} , and the instance polynomial evaluation, using one batch inversion.

```
// x^n by repeated squaring, n = 2^k
let x_n := x
for { let idx := 0 } lt(idx, k) { idx := add(idx, 1) } {
  x_n := mulmod(x_n, x_n, r)
}
// Denominators (x - omega_i) and (x^n - 1) batch-inverted in one modexp
for { let pow_of_omega := mload(OMEGA_INV_TO_L_MPTR) }
  lt(mptr, mptr_end) { mptr := add(mptr, 0x20) } {
    mstore(mptr, addmod(x, sub(r, pow_of_omega), r))
    pow_of_omega := mulmod(pow_of_omega, omega, r)
  }
mstore(mptr_end, addmod(x_n, sub(r, 1), r))
success := batch_invert(success, X_N_MPTR, add(mptr_end, 0x20), BATCH_INV_SCRATCH_MPTR, r)
// L_i(x) = (x^n - 1) * n^-1 * omega_i / (x - omega_i)
let l_i_common := mulmod(x_n_minus_1, mload(N_INV_MPTR), r)
...
```

Rust counterpart. `lagrange_interpolate` / `lagrange_base_eval` and the instance-eval sum in `verifier.rs` (Eq. (2.7), (2.11)).

- **[OK] x^n by squaring.** k squarings give $x^{2^k} = x^n$; matches the Rust domain exponent.
- **[OK] Batched denominators.** All $(x - \omega_i)$ plus $(x^n - 1)$ are inverted in a single `batch_invert` (one modexp), matching the Rust's batched Lagrange denominators.
- **[OK] L_i formula.** $L_i(x) = (x^n - 1) n^{-1} \omega_i / (x - \omega_i)$ is exactly (2.7).
- **[NOTE] `num_instances==0` slot.** When there are no public instances the code still reserves one denominator slot so L_0 can be recovered for boundary identities. Correct, but the conditional `mptr_end` bump is easy to misread; it is load-bearing for the l_0/l_{last} reads at the end.
- **[IMPROVE] `pow_of_omega` recomputed twice.** The first pass walks $\omega_{\text{inv}}^L \cdot \omega^i$ to build denominators; the second pass re-walks the same sequence to form L_i . The powers could be cached in the first pass (they are already written as the denominator) — but the second pass needs ω_i not $x - \omega_i$, so a second walk is genuinely required. The improvement is only to hoist `l_i_common` (already done). Minor.
- **[RISK] Layout-indexed reads.** l_0 is read at `add(X_N_MPTR, num_neg_lagranges*32)` and l_{last} at `X_N_MPTR`. These offsets are codegen-computed; a mismatch between the Lagrange write layout and these read offsets would silently swap l_0/l_{last} . The differential trace pins them, but the coupling is implicit.

9.3 TranscriptProofParser.yul

The full transcript schedule: VK digest, committed identity, instances, per-phase advice, θ , multiplicities, β, γ , perm Z , lookup helpers/acc, trash, y , quotient limbs, x , evals, x_1, x_2, f_{com} , x_3, q -evals, x_4, π .

```
buf_len := common_word(buf_len, mload(VK_DIGEST_MPTR))
// committed_pi = identity -> 128 zero bytes, BEFORE instance count
mstore(buf_len, 0); mstore(add(buf_len, 0x20), 0)
mstore(add(buf_len, 0x40), 0); mstore(add(buf_len, 0x60), 0)
buf_len := add(buf_len, 0x80)
// instance count + values (canonical Fr)
buf_len := common_word(buf_len, <num_instances>)
for ... { let inst_be := calldataload(instance_cpctr)
```

```

    success := and(success, lt(inst_be, r))
    buf_len := common_word(buf_len, inst_be) }
if iszero(success) { revert(0, 0) } // fail fast (1e21311c)
...
// final exact-consumption check
if iszero(eq(proof_cpctr, NUM_INSTANCE_CPTR)) { revert(0, 0) }
if iszero(success) { revert(0, 0) }

```

Rust counterpart. `parse_trace + verify_algebraic_constraints` transcript schedule (§3.2, §3.9).

- **[OK] Schedule fidelity.** The absorb/squeeze order matches the Rust exactly; the differential trace (ids 1–36) pins every step.
- **[OK] Committed-identity absorb.** 128 zero bytes before the instance count matches the patched `Hashable` identity convention.
- **[OK] Exact-consumption check.** `eq(proof_cpctr, NUM_INSTANCE_CPTR)` makes any under/over-read of the proof layout fail closed — strong defense-in-depth against future layout drift.
- **[OK] Eval canonicity.** Every proof eval scalar is range-checked `lt(eval, r)` before absorption (lines 319, 382).
- **[RISK] x_3 truncation (128-bit).** Under `truncated-challenges`, x_3 is masked to 128 bits (line 365). As discussed in §9.7, this reduces the evaluation-point entropy of the f -polynomial check; the resulting soundness level is not established here. Faithful to Rust, but the single most security-relevant line in the parser.
- **[NOTE] Deferred success for instances only.** Instance canonicity is accumulated with `success := and(...)` while `G1/eval` helpers revert immediately. **[ADDRESSED]** Since `1e21311c` the parser reverts right after the instance loop (template L68, B64 L1050), so a bad instance no longer pays for the whole transcript; the final `if iszero(success) revert` is now a backstop.

9.4 QuotientAndLinearization.yul

External quotient-evaluator call (pinned) plus the linearization scalar preparation ($x_{\text{split}}, 1 - x^n$).

```

// Pin the external quotient evaluator like the VK
if iszero(and(
    eq(extcodesize(quotientEvaluator), EXPECTED_QUOTIENT_LENGTH),
    eq(extcodehash(quotientEvaluator), EXPECTED_QUOTIENT_CODEHASH_WORD)
)) { revert(0, 0) }
if iszero(staticcall(gas(), quotientEvaluator, frame_base, frame_len, q_out, out_len)) { revert(0,0) }
if iszero(eq(mload(q_out), <magic>)) { revert(0, 0) }
mstore(QUOTIENT_EVAL_MPTR, mload(add(q_out, 0x20))) // -nu_y(x)
// x_split = x^(n-1), one_minus_x_n = 1 - x^n, one squaring walk
let x_pow_2i := x
let x_pow_2i_minus1 := 1
for { let idx := 0 } lt(idx, k) { idx := add(idx, 1) } {
    x_pow_2i_minus1 := mulmod(mulmod(x_pow_2i_minus1, x_pow_2i_minus1, r), x, r)
    x_pow_2i := mulmod(x_pow_2i, x_pow_2i, r)
}
mstore(QUOTIENT_MPTR, x_pow_2i_minus1) // x_split
mstore(add(QUOTIENT_MPTR, 0x20), one_minus_x_n)

```

Rust counterpart. `compute_linearization_commitment` (§3.5); the external path is the split-contract variant of the same numerator.

- **[OK] Evaluator pinned by codehash.** The external quotient contract is checked against `EXPECTED_QUOTIENT_CODEHASH` before every call — same defense-in-depth as the VK. A swapped evaluator reverts.
- **[OK] Magic/version gate.** The return-word-0 magic check catches a wrong-callee or ABI drift.
- **[OK] x_{split} walk.** Computing x^{n-1} and x^n in one loop via $x^{2^i-1} = (x^{2^{i-1}-1})^2 \cdot x$ is correct and saves a separate exponentiation.
- **[NOTE] QUOTIENT_MPTR repurposed as scalar metadata.** In this path `QUOTIENT_MPTR` is *not* a G1 point; its two words carry $(x_{\text{split}}, 1 - x^n)$. The comment warns, but an auditor expecting a point here could be misled. Consider a distinct slot name (e.g. `LIN_SCALARS_MPTR`) to remove the ambiguity.
- **[IMPROVE] Trace path uses call not staticcall.** When tracing, the external quotient call is a `call` (line 28) rather than `staticcall`. **[CORRECTED]** This is required, not an oversight: the trace-mode evaluator emits `log1` trace events (`Halo2QuotientEvaluator.sol` L120–127), and `LOG` reverts inside a `STATICCALL` frame. Production renders use `staticcall` (line 30), and B64 embeds the evaluator, so it makes no external call at all. No action.

9.5 FinalPairing.yul

Randomized batching of the IVC accumulator equation into the KZG pairing, then the final `ec_pairing`.

```
// alpha = FS(domain || kzg_rhs || kzg_lhs || acc_rhs || acc_lhs)
let acc_pair_alpha := mod(keccak256(batch_ptr, HASH_BYTES), r)
if iszero(acc_pair_alpha) { acc_pair_alpha := 1 }
// PAIRING_RHS += alpha * ACC_RHS (1-pair G1MSM then G1ADD)
mcopy(batch_ptr, ACC_RHS_MPTR, 0x80)
mstore(add(batch_ptr, 0x80), acc_pair_alpha)
success := staticcall(gas(), G1MSM, batch_ptr, 0xa0, batch_ptr, 0x80)
mcopy(add(batch_ptr, 0x80), PAIRING_RHS_MPTR, 0x80)
success := staticcall(gas(), G1ADD, batch_ptr, 0x100, PAIRING_RHS_MPTR, 0x80)
// ... mirror for LHS ...
success := ec_pairing(success, PAIRING_RHS_MPTR, PAIRING_LHS_MPTR)
```

Rust counterpart. The accumulator randomizer in `verifier.rs` + `DualMSM::check` (Eq. (2.33), (2.32)).

- **[OK] Randomized batching prevents cancellation.** Deriving α from the pairing inputs means two independently-bad equations cannot be made to cancel: the combined equation holds for at most one α . This is the correct construction (vs. naively multiplying equations).
- **[OK] Zero- α guard.** Mapping $\alpha = 0 \mapsto 1$ prevents the accumulator equation from being silently dropped (which would happen if $\alpha = 0$ zeroed the accumulator contribution).
- **[IMPROVE] Fold the two 1-pair G1MSMs.** The RHS and LHS accumulator scalings are two separate 1-pair `G1MSM` calls. They cannot be merged into one `MSM` (different output points), but each 1-pair `MSM` could instead be a `G1ADD` of precomputed points if the accumulator points were pre-scaled — not applicable here since α is runtime. The current form is near-optimal; the only saving is the `G1ADD` after each `MSM`, which is unavoidable (a 2-pair `MSM` costs 22,776 gas vs. 12,375 for `MSM+ADD`). The foldable one-pair `MSMs` are $-vG$ and $x_3\pi$ in the PCS block, which can join the fused final `MSM` (§10.3, register I12).

- **[NOTE] Argument-order trap (again).** `ec_pairing(success, PAIRING_RHS, PAIRING_LHS)` passes RHS as the pairing’s first argument. The comment explains the dual-MSM naming inversion. This is the second place the LHS/RHS inversion bites (see §9.1.6); a single consistent naming (`PAIRING_ARG0/ARG1`) would remove the trap.

9.6 AccumulatorHelpers.yul

Decodes the shifted radix-2⁵⁶ public-input encoding of the IVC accumulator’s foreign G1 points into EIP-2537 padded form, with canonical packing checks and mandatory G1MSM validation.

```
function load_acc_coord(src, allow_id, bits, n, base, limbs_per_word) -> ok, hi, lo, is_id {
  ok := check_acc_coord_packing(src, bits, n, limbs_per_word) // reject unused high bits
  if and(allow_id, iszero(lt(calldataload(src), base))) {
    let adj_hi, adj_lo := load_acc_coord_shifted(src, bits, n, base, limbs_per_word, base)
    is_id := is_bls_p_minus_one(adj_hi, adj_lo)
  }
  hi, lo := load_acc_coord_shifted(src, bits, n, base, limbs_per_word, mul(is_id, base))
  ok := and(ok, or(lt(hi, BLS_P_HI), and(eq(hi, BLS_P_HI), iszero(gt(lo, BLS_P_MINUS_ONE_LO)))))
  if is_bls_p_minus_one(hi, lo) { hi := 0; lo := 0 } // p-1 -> 0
  else { let next_lo := add(lo, 1); hi := add(hi, lt(next_lo, lo)); lo := next_lo } // +1 with carry
  ok := and(ok, lt(hi, shl(128, 1))) // EIP-2537 48-byte pad
}
```

Rust counterpart. `AssignedForeignPoint<BLS12-381> / AssignedField::as_public_input` shifted-limb codec.

- **[OK] Packing canonicity.** `check_acc_coord_packing` rejects any unused high bits in the packed words, so each coordinate has a unique encoding before it reaches curve validation.
- **[OK] Mandatory G1MSM validation.** Every decoded point (even identity, even scalar 0/1) is routed through G1MSM so the precompile performs the on-curve/subgroup check. The comment explicitly notes skipping it would let a malformed non-identity point hide behind scalar 0 — correct and important.
- **[OK] Decoded-zero rejection.** A decoded (0,0) that is *not* the canonical identity encoding is rejected (line 208–209), since EIP-2537 reserves (0,0) for infinity.
- **[OK] +1 carry into high word.** The `hi := add(hi, lt(next_lo, lo))` carry logic for the `coord-1 → coord` reconstruction is correct across the 256-bit boundary.
- **[RISK] Intricate identity-flag logic.** The `first_adjust/is_id` path (only x carries the flag; the flag is one radix base added to the first packed word) is the most intricate decoding in the contract. It is correct for the current codec, but a change to the circuit’s public-input packing would require a matching change here. The `is_acc_encoded_identity` fast path (exact packed-word equality) is the primary guard; the coordinate-level flag probe is the fallback. High audit burden — recommend a dedicated unit test enumerating identity vs. non-identity encodings. **[PARTIAL]** The IVC E2E covers bad packing and malformed LHS/RHS points (`tests/ivc_keccak_solidity.rs` L1417–1447); the identity path has only a template-string check (`src/lowering/tests.rs` L1223).
- **[NOTE] y_{id} discarded.** `pop(y_id)` is correct (`allow_id` was false) but the shared tuple shape is a minor readability wart.

9.6.1 Accumulator final-pairing batching (verified on ivc-keccak)

The poseidon fixture has no accumulator, so the accumulator pairing path was verified against the `ivc-keccak` dump, which exercises the full randomized batching. The accumulator contributes

a second pairing equation that must be checked *together with* the KZG equation without letting two bad equations cancel.

The batching. After all four G1 pairing inputs are materialized, a verifier-local randomizer α is drawn by Fiat–Shamir over the domain tag and all four points, and the pairing arguments are folded:

```
// domain || KZG_rhs || KZG_lhs || ACC_rhs || ACC_lhs (32 + 4*128 = 0x220)
mstore(batch_ptr, "pairing-batch-acc-kzg\0...")
mcopy(batch_ptr+0x20, PAIRING_RHS_MPTR, 0x80)
mcopy(batch_ptr+0xa0, PAIRING_LHS_MPTR, 0x80)
mcopy(batch_ptr+0x120, ACC_RHS_MPTR, 0x80)
mcopy(batch_ptr+0x1a0, ACC_LHS_MPTR, 0x80)
let acc_pair_alpha := mod(keccak256(batch_ptr, 0x0220), r)
if iszero(acc_pair_alpha) { acc_pair_alpha := 1 } // never drop the acc eq
// PAIRING_RHS += alpha * ACC_RHS (G1MSM then G1ADD)
// PAIRING_LHS += alpha * ACC_LHS (G1MSM then G1ADD)
```

The check. The final `ec_pairing` then verifies

$$e(\text{KZG}_{\text{rhs}} + \alpha \text{ACC}_{\text{rhs}}, [1]_2) \cdot e(\text{KZG}_{\text{lhs}} + \alpha \text{ACC}_{\text{lhs}}, -[s]_2) = 1. \quad (9.1)$$

By bilinearity this factors as

$$\underbrace{e(\text{KZG}_{\text{rhs}}, [1]_2) e(\text{KZG}_{\text{lhs}}, -[s]_2)}_{\text{KZG equation}} \cdot \underbrace{[e(\text{ACC}_{\text{rhs}}, [1]_2) e(\text{ACC}_{\text{lhs}}, -[s]_2)]^\alpha}_{\text{accumulator equation}} = 1. \quad (9.2)$$

- **[OK] Cancellation is prevented.** If both the KZG and accumulator equations hold, (9.2) is $1 \cdot 1^\alpha = 1$. If either is *false*, the product equals 1 for **at most one** value of $\alpha \in \mathbb{F}_r$ (a degree-1 polynomial in α has ≤ 1 root). Since α is drawn *after* all four points are fixed, a prover cannot steer it into the cancelling root except with probability $\leq 1/r \approx 2^{-255}$. This is the correct Schwartz–Zippel randomization.
- **[OK] α drawn after inputs are fixed.** The hash covers the fully materialized PAIRING_RHS/LHS and ACC_RHS/LHS, so the prover commits to the points before α exists — the randomizer cannot be chosen adversarially.
- **[OK] Zero- α guard.** `if iszero(alpha) alpha := 1` ensures the accumulator equation is never accidentally dropped (an $\alpha = 0$ draw would collapse (9.1) to the KZG equation alone, silently ignoring the accumulator).
- **[OK] Domain-separated α .** The "pairing-batch-acc-kzg" tag prevents the α from being reused across protocols / other transcript contexts.
- **[OK] Points pre-validated.** ACC_RHS/LHS were already forced through G1MSM in `validate_public_accumulator` (every decoded point, even identity/scalar-0), so the α -fold MSMs operate on validated points; a malformed accumulator reverts before the pairing.

Encoding correctness (recap). The accumulator point encoding was verified end-to-end: the shifted radix-2⁵⁶ little-endian limb reconstruction (including the limb straddling the 256-bit hi/lo split via `shr(sub(256, shift), limb)`), the canonical-packing rejection of unused high bits, the $p-1 \rightarrow 0$ / `coord-1 → coord + 1`-with-carry round-trip, the base-field range check, and the EIP-2537 `hi < 2128` pad check all hold. The identity fast path (`is_acc_encoded_identity`, exact packed-word equality) plus the coordinate-level flag probe cover the point-at-infinity encoding, and a decoded (0,0) that is not the canonical identity is rejected.

Verdict. No issues with the accumulator **encoding** and no issues with the **final pairing check** in the generated Solidity. The only residual concern is the audit-burden of the identity-flag decode (already flagged [RISK] above), which is a test-coverage recommendation, not a correctness defect.

9.7 Deviations summary

Aggregating every deviation logged in this chapter, ranked by security relevance:

Deviation	Severity	Assessment	Status
128-bit truncated challenges (x_3, x_1^i, x_4^i)	HIGH	Soundness level not established; earlier bit-level figures are withdrawn. Faithful to Rust; needs a written soundness argument and confirmation against the security target.	[OPEN] policy decision
Unbounded transcript cursor	MED	Buffer sized by codegen only; a mis-sizing for multi-phase shapes let proof bytes overwrite the loaded VK.	[PARTIAL] fixed in 6b3d2096, no runtime guard
No on-curve/subgroup check at G1 absorption	MED	Sound only via MSM/pairing coverage invariant; G1ADD alone does not subgroup-check.	[ADDRESSED] enforced at build
<code>batch_invert</code> leaves memory dirty on failure	MED	The caller reads before its gate; the load-bearing invariant is check-before-return (sticky <code>success</code>).	[OPEN] premise corrected
Accumulator identity-flag de- code intricacy	MED	Correct for current codec; high audit burden, needs dedicated tests.	[PARTIAL] no identity vectors
QUOTIENT_MPTR repurposed as scalar metadata	LOW	Audit-clarity; rename to remove ambiguity.	[OPEN]
Trace path uses <code>call</code> not <code>staticcall</code>	LOW	Not a deviation: trace-mode evaluator emits LOG, which <code>STATICCALL</code> forbids; production already uses <code>staticcall</code> .	[CORRECTED]
<code>scalar_inv</code> vs <code>batch_invert</code> zero-policy mismatch	LOW	Live code (five call sites in B64); inconsistent failure convention.	[OPEN]
PAIRING LHS/RHS naming inversion	LOW	Documented trap; rename to ARG0/ARG1.	[OPEN]
Modulo bias in <code>squeeze_to</code>	NEGL	Statistical distance ≈ 0.075 (not $\leq 2^{-128}$); same as Rust, not exploitable.	[CORRECTED]

The first four are the ones a security reviewer should focus on. The remainder are audit-clarity and robustness improvements that do not change the accepted-proof set.

9.8 QuotientNumeratorBlock.yul — the compact quotient VM

This is the largest and most intricate generated component (1275 lines). It reconstructs the *y*-batched numerator $\nu_y(x)$ by interpreting a bytecode program (`q_program`) stored in the *VK payload*, rather than unrolling every identity as Yul. The opcode stream is VK-pinned; no proof calldata can alter control flow.

9.8.1 Stack model

```
// q_pc: bytecode cursor    q_end: exclusive end
// q_sp: spilled-stack word pointer
// q_top: cached top-of-stack value    q_has_top: liveness flag
let q_pc := q_program_mptr
let q_end := add(q_program_mptr, <program.len>)
let q_sp := <program.stack_mptr>
let q_top := 0
let q_has_top := 0
for { } lt(q_pc, q_end) { } {
    let q_op := byte(0, mload(q_pc))
    q_pc := add(q_pc, 1)
```

```

    switch q_op { ... default { revert(0, 0) } }
}
// end-of-program integrity:
if iszero(eq(q_pc, q_end)) { revert(0, 0) }
if q_has_top { revert(0, 0) }

```

- **[OK] Fail-closed default.** Any opcode not in the emitted switch (including the reserved 0x1a) hits `default { revert }`.
- **[OK] End-of-program integrity.** The two post-loop checks — `q_pc == q_end` (no over/under-read) and `q_has_top == 0` (no dangling partial expression) — catch malformed generator output. Added in 33f78d2b (template L1191–1192; B64 L2569–2570).
- **[OK] All arithmetic reduced.** Every `addmod/mulmod` is modulo `r`; there is no unchecked `add/mul` on field values.
- **[OK] Coefficients are VK-pinned.** Constant-table reads come from `q_const_mptr` (VK payload), never from proof calldata — the bytecode cannot be steered by the prover.

9.8.2 FOLD_MAIN (0x0a) — the y -batch fold

```

case 0x0a {
    let q_eval := q_top
    q_has_top := 0
    // qn = qn*y + eval    (reverse Horner, matches Rust)
    mstore(qn, mulmod(mload(qn), y, r))
    mstore(qn, addmod(mload(qn), q_eval, r))
}

```

Rust counterpart. The reverse y -power fold in `compute_linearization_commitment` (`expressions.iter().rev(), y_pow *= y`).

- **[OK] Horner fold correct.** `qn = qn*y + eval` accumulated over identities in order yields $\sum_i y^{m-1-i} e_i = \nu_y(x)$, matching (2.22).
- **[IMPROVE] Redundant load/store.** The fold does `mstore(qn, mulmod(mload(qn), y, r))` then `mstore(qn, addmod(mload(qn), eval, r))` — two `mstore` + two `mload` of the same slot per identity. Keeping the accumulator in a register and storing once at the end would save one `mload+mstore` (~ 6 gas) per identity. Correctness is unaffected. **[OPEN]** B64 does not render case 0x0a; the same pair appears in the native-callback fold snippets (`src/lowering/quotient.rs` L1018, L1035; e.g. B64 L2005–2006). The saving is small, not “meaningful”.

9.8.3 FOLD_SELECTOR (0x0b) — sparse selector buckets

```

case 0x0b {
    let q_sel_idx := shr(16, payload)
    let q_sel_gap := and(payload, 0xffff)
    let q_eval := q_top; q_has_top := 0
    // global accumulator still *y (keeps main identities aligned)
    mstore(qn, mulmod(mload(qn), y, r))
    let q_sel_acc := mload(SELECTOR_ACC_MPTR + 32*q_sel_idx)
    if q_sel_gap {
        q_sel_acc := mulmod(q_sel_acc, mload(selector_power_mptr + 32*gap), r)
    }
    mstore(q_target_ptr, addmod(q_sel_acc, q_eval, r))
}

```

- **[OK] Global alignment preserved.** The global `qn` is still multiplied by y even for selector identities, so subsequent main identities land at the same y -powers as Rust’s reverse fold.
- **[OK] Precomputed y^{gap} .** Using a codegen-precomputed y^{gap} table avoids a runtime $y^{\{-1\}}$ modexp — a real gas win. The tail update (post-VM) applies the final y^{tail} to align each bucket with the global end.

9.8.4 Native callbacks (0x19 perm, 0x1f lookup, 0x1b heavy)

```
case 0x19 { // native permutation
  q_top := 0; q_has_top := 0           // reset stack at identity boundary
  q_sp := <program.stack_mptr>
  // generated perm fold lines (same snippets as interpreted)
}
```

- **[OK] Identity-boundary reset.** Each native callback clears `q_top/q_has_top` and resets `q_sp` before running its generated fold, so it never inherits a partial VM stack.
- **[RISK] Empty-stack-boundary assumption.** The reset *silently drops* any live `q_top`. This is safe only because the codegen emits native callbacks exclusively at empty-stack boundaries. If a future codegen change emitted one mid-expression, the partial value would be lost without error. *Defense-in-depth:* add `if q_has_top { revert(0,0) }` at the top of each native-callback case, turning the assumption into a checked invariant. **[OPEN]** The final-bytecode validator rejects a callback on a non-empty stack (`src/lowering/quotient_numerator/vm/mod.rs` L2241–2246), but the runtime reset is still unconditional (template L1057, L1084, L1113; B64 L1934, L2051, L2247).

9.8.5 ADD/MUL stack underflow

```
case 0x06 { // ADD
  q_sp := sub(q_sp, 0x20)
  q_top := addmod(mload(q_sp), q_top, r)
}
```

- **[RISK] Unchecked stack underflow.** ADD/MUL assume a spilled operand exists (`q_sp > stack_mptr`). The comment says the codegen safety validator guarantees this, but the runtime does not check. A validator bug would make `q_sp` underflow below the stack region and read adjacent memory as an operand. *Defense-in-depth:* `if lt(q_sp, add(stack_mptr, 0x20)) { revert(0,0) }` before the `sub`. Costs ~ 10 gas per ADD/MUL; given the validator already prevents it, this is belt-and-suspenders for auditability. **[OPEN]** The validator re-decodes the final bytes and rejects underflow (`vm/mod.rs` L2213–2216; test `src/lowering/tests.rs` L1520); no runtime check (template L503–518; B64 renders only ADD, L1716–1721).

9.8.6 Final negation

```
// -nu_y(x) stored as the linearization expected eval
let linearization_expected_eval := addmod(0, sub(r, mload(qn)), r)
mstore(QUOTIENT_EVAL_MPTR, linearization_expected_eval)
```

- **[OK] Matches Rust.** `expected_eval == eval` for `col_idx == None` produces $-\nu_y(x)$; the $(1 - x^n)$ factor is on the commitment side (Def. 2.7.1).

9.9 QuotientHelpers.yul

Optional codegen-recognized helper functions, emitted only when the lowering pass recognizes the corresponding expression shape.

```
function q_pow5(x) -> z {                // Poseidon S-box x^5
    let x2 := mulmod(x, x, q_r)
    z := mulmod(x, mulmod(x2, x2, q_r), q_r)
}
function q_limb7(x0,...,x6) -> z {      // sum_i base_powers[i]*limb_i
    z := addmod(x0, mulmod(c1, x1, q_r), q_r)
    ...
}
```

- **[OK] q_pow5 correct.** $x^5 = x \cdot (x^2)^2$ with two mulmods — the minimal addition chain for 5.
- **[NOTE] Codegen shortcuts, not verifier rules.** These mirror the Rust *Expression tree* shapes (Poseidon S-box, foreign-field limb sums). They are a rendering optimization; the verifier semantics are unchanged. Coefficients come from the VK/program constant table, never proof data.

9.10 VM-specific improvement summary

Item		Grade	Recommendation	Status
Native-callback assumption	empty-stack	RISK	Add if <code>q_has_top revert</code> at each native-callback entry (0x19, 0x1f, 0x1b).	[OPEN] build-time check only
ADD/MUL underflow	unchecked stack	RISK	Add if <code>lt(q_sp, stack_base+32) revert</code> before the pop.	[OPEN] build-time check only
FOLD_MAIN load/store	redundant	IMPROVE	Keep accumulator in a register; store once at end (~6 gas per folded identity; in B64 via native fold snippets).	[OPEN]
Bytecode control-flow by VK	pinned	OK	Prover cannot alter opcode stream.	—
End-of-program checks	integrity	OK	<code>q_pc==q_end</code> and <code>!q_has_top</code> catch malformed output.	[ADDRESSED] 33f78d2b

The two **RISK** items are the VM’s load-bearing codegen assumptions (empty-stack native callbacks, non-empty-stack ADD/MUL). Both are guaranteed at build time by `validate_quotient_program`, which re-decodes the final bytecode (vm/mod.rs L2003), rather than checked at run-time; adding the one-line runtime guards would make them self-evident to an auditor at negligible cost.

9.11 Contract templates and deployment-time checks

The generated verifier is assembled from three top-level contract templates plus the Yul partials audited above. This section audits the contract-level structure and the deployment-time defenses.

9.11.1 Halo2Verifier.sol — the main contract

The contract is a thin shell: NatSpec, then includes, then one terminal assembly block. The `verifyProof` entry begins with a cheap ABI-shape guard *before* any generated memory work:

```
function verifyProof(bytes calldata proof, uint256[] calldata instances)
    external view returns (bool) {
```

```

assembly ("memory-safe") {
    if iszero(and(
        eq(calldataload(SELECTOR_BYTES), PROOF_HEAD_OFFSET),
        eq(calldataload(INSTANCES_HEAD_CPTR),
            sub(NUM_INSTANCE_CPTR, SELECTOR_BYTES)))) {
        revert(0, 0)
    }
}
// ... VK pin, then the generated Yul pipeline ...
}

```

- **[OK] ABI-shape guard.** The hand-rolled calldata parser is protected by a pre-check that the dynamic-argument heads point where the generator expects, so a malformed ABI layout reverts before being misread as a proof stream.
- **[OK] Terminal assembly block.** The generated block owns the call-frame memory and is terminal: accepted proofs **return**, all rejects **revert**. Scratch starts at 0x80, preserving Solidity’s reserved prefix and free-memory pointer. (Starting at 0x80 is not sufficient for `memory-safe` under via-IR; see §9.11.2.)
- **[NOTE] Not a generic verifier.** The NatSpec is explicit that proof layout, VK payload, quotient program, and memory layout are all generated for *one VerifyingKey*. Application contracts must bind instance semantics (state roots, program ids, domain separation) separately — the raw verifier ABI guarantees nothing about them.

9.11.2 The "memory-safe" annotation — what it promises and what it relies on

Every generated assembly block is tagged `assembly("memory-safe")`. This is a *promise to the compiler*: the block does not corrupt memory the rest of the program needs, so the compiler may trust that and optimize accordingly (e.g. skip defensive re-reads). It does *not* mean “the block used Solidity’s allocator.”

What the verifier actually does. Solidity reserves the bottom 128 bytes: scratch space (0x00–0x3f), the **free memory pointer** (0x40, normally initialized to 0x80), and the zero slot (0x60); free memory begins at the free-pointer value and is handed out by bumping 0x40. The generated verifier **bypasses that allocator entirely**: it uses hard-coded absolute addresses (`TRANSCRIPT_MPTR = 0x80`, `PAIRING_LHS_MPTR = 0x6f20` in B64, ...) and **never advances the free memory pointer**. (Every 0x40/0x60 in the emitted code is an *offset inside a scratch buffer*, e.g. `add(scratch, 0x40)`, not the free pointer itself.)

Why the promise was believed to hold. Two properties were taken to make `memory-safe` true here:

1. scratch starts at 0x80, so the reserved prefix (0x00–0x7f, including the free pointer and zero slot) is never touched;
2. the block is **terminal** — it ends in `return` or `revert`, so no Solidity code runs *after* it.

The load-bearing coupling (the real issue). The verifier is doing manual absolute-address memory management that ignores the free pointer. That is harmless *only because nothing allocates memory after the block*. The `memory-safe` tag is a contract that holds under the terminal-block assumption:

If this body were ever inlined into a function that **keeps running Solidity after verification**, the verifier’s scratch at 0x80+ would collide with memory the free

pointer hands out next — and because the compiler *trusts* the `memory-safe` tag, it would **not** insert the defensive behavior needed to protect against that collision, yielding silent memory corruption.

Correction: terminality is not enough under via-IR. Solidity’s rules for `memory-safe` blocks hold *inside* the block too: a block may touch only scratch `0x00–0x3f`, memory Solidity allocated, and memory at or above the free pointer’s value on entry. The annotation is what lets via-IR’s stack-limit evader spill Yul variables to memory, and it does so here. Compiled with the production settings (solc 0.8.35, via-IR, 200 runs, Cancun), B64’s runtime begins `PUSH2 0x08e0 PUSH1 0x40 MSTORE`. The compiler reserved `[0x80, 0x8e0)` for spill slots, and that range overlaps the generator’s absolute scratch (the transcript buffer at `0x80`, the `ec_pairing` scratch at `0x300–0x600`, ...). Honest proofs verify today only because the live ranges happen to be disjoint. Nothing checks this (§11.8.19).

In short: the safety does not come from the annotation itself. It comes from the terminal-block property the annotation is asserting, plus an unchecked disjointness between compiler spill slots and generator scratch. The contract’s own NatSpec encodes exactly this warning — “*Do not inline this body into Solidity code that continues executing after verification without reviewing the memory strategy.*”

- **[RISK] Annotation formally violated under via-IR. [CORRECTED]** (previously graded [OK]). The block writes below the entry free pointer (`0x8e0`) into compiler-owned spill slots. Dropping the tag is not an option, because via-IR needs it to evade stack-too-deep. **[OPEN]** The fix is to plan the absolute regions above a fixed base and assert $\text{mload}(0x40) \leq \text{base}$ on entry, or to add a CI check on the compiled free-pointer initializer (§11.8.19).
- **[RISK] Annotation is load-bearing on terminality.** The correctness of the tag depends on the block never returning to executing Solidity. Any future refactor that makes the verifier body non-terminal must either bump the free pointer to cover the scratch region or switch the tag to `memory-unsafe` — otherwise the compiler’s trust is misplaced. *Audit note:* treat “terminal” as a hard invariant of the generated body, not an incidental property.

9.11.3 PrecompileSmoke.sol — deployment-time fork check

The constructor calls `require_eip2537_precompiles()`, which exercises every runtime prerequisite with identity inputs so an incompatible fork fails *at deployment*, not at first proof:

```
// MCOPY must work (verifier uses it for staging).
mstore(scratch, 0x1234)
mcopy(add(scratch, 32), scratch, 32)
if iszero(eq(mload(add(scratch, 32)), 0x1234)) { revert(0, 0) }
// G1ADD(id,id) -> id, 128-byte return.
staticcall(gas(), G1ADD, scratch, 256, scratch, 128) ...
// Worst-case G1MSM (all identity/zero) -> id, 128-byte return.
staticcall(gas(), G1MSM, msm_scratch, MSM_BYTES, scratch, 128) ...
// PAIRING([(id,id),(id,id)]) -> true, 32-byte return.
staticcall(gas(), PAIRING, scratch, TWO_PAIR_BYTES, scratch, 32) ...
```

- **[OK] Worst-case MSM size.** The G1MSM smoke uses the *largest* MSM input the verifier can emit, not a one-pair call — so a fork that chokes on the real input length is caught at deploy.
- **[OK] Return-shape checks.** Each probe checks `returndatasize()` and the returned value, catching missing precompiles, short returns, and wrong semantics.
- **[IMPROVE] G2ADD not smoked.** The smoke tests G1ADD/G1MSM/PAIRING but not G2ADD (`0x0d`). The verifier never calls G2ADD (G2 points come from the pinned VK),

so this is fine — but a comment noting the deliberate omission would prevent an auditor wondering why. **[OPEN]** No such comment in `PrecompileSmoke.sol`; the MCOPY probe shown above was added in `b53e3ebe`.

9.11.4 Constructors.sol — dependency pinning

Four constructor variants (embedded/external VK $\times$ embedded/external quotient) all follow the same pattern:

```
require_eip2537_precompiles();
require(authorizedVk.code.length == EXPECTED_VK_LENGTH
    && authorizedVk.codehash == EXPECTED_VK_CODEHASH, "invalid vk");
require(authorizedQuotient.code.length == EXPECTED_QUOTIENT_LENGTH
    && authorizedQuotient.codehash == EXPECTED_QUOTIENT_CODEHASH,
    "invalid quotient");
AUTHORIZED_VK = authorizedVk;
AUTHORIZED_QUOTIENT = authorizedQuotient;
```

- **[OK] Strict codehash+length pin.** Both the VK and the split quotient evaluator are pinned by `codehash` *and* `length` at deploy, and the verifier re-checks before every proof (Sec. 4.5). The split evaluator is treated with the same strictness as the VK because it contains generated verifier logic, not a library.

9.11.5 Halo2VerifyingKey.sol — INVALID-prefixed payload

The VK contract’s runtime is `INVALID || payload`:

```
// byte 0      : INVALID, so the payload cannot be executed
// byte 1..end : generated VK payload copied by Halo2Verifier
let runtime := PAYLOAD_MPTR
let payload := add(runtime, 0x01)
```

- **[OK] INVALID opcode prefix.** Byte 0 is an unconditional `INVALID` so a direct call to the VK contract cannot execute the payload as code. The verifier copies the payload via `extcodecopy(vk, VK_MPTR, 0x01, len)` — skipping the `INVALID` byte — and references slots by `VK_MPTR + i`.
- **[OK] Quotient program pinned by VK codehash.** The quotient VM’s static program lives in this pinned runtime, so it is covered by `EXPECTED_VK_CODEHASH` without verifier-side immediates.

9.11.6 Halo2QuotientEvaluator.sol — split evaluator

In the split render, the verifier `staticcalls` this contract, passing its *raw memory frame* as `calldata`:

```
// calldata[0..FRAME_LEN) == memory[FRAME_BASE..FRAME_BASE+FRAME_LEN)
// Output frame:
// word 0: QUOTIENT_MAGIC (version guard)
// word 1: linearization_expected_eval (= -nu_y(x))
// word 2..: simple-selector accumulator scalars
```

- **[OK] Magic-guarded frame.** The output carries a generated `QUOTIENT_MAGIC` version word so a mismatched evaluator version is detected, not silently misread.
- **[OK] Returns $-\nu_y(x)$, not $h(x)$.** The NatSpec is explicit that the evaluator reconstructs the y -batched numerator and returns the negated expected scalar; the $(1 - x^n)$ factor stays on the commitment side (Def. 2.7.1).

- **[NOTE] Frame-copy trust boundary.** The evaluator copies the verifier’s memory frame by absolute address; the two contracts must agree on the frame layout. This coupling is pinned by the codehash pin, but a layout drift in a future codegen change would be a silent correctness bug — the magic word is the only runtime guard.

9.11.7 Pcs.yul and TraceReturn.yul

Pcs.yul is a thin emitter that lays down the generated multi-prepare sub-blocks (grouped for gas-checkpoint attribution) and traces the pairing inputs. TraceReturn.yul is the terminal trace + return:

```
// Rust prints the positive numerator; the verifier stores the negated
// expected scalar, so trace id 36 re-negates:
trace_u256(36, addmod(0, sub(r, mload(QUOTIENT_EVAL_MPTR)), r))
// QUOTIENT_MPTR carries scalar metadata in the fused path; zero the
// unused point words before tracing it as a G1 slot:
mstore(add(QUOTIENT_MPTR, 0x40), 0)
mstore(add(QUOTIENT_MPTR, 0x60), 0)
trace_point(24, QUOTIENT_MPTR)
...
mstore(RETURN_MPTR, 1)
return(RETURN_MPTR, 0x20)
```

- **[OK] Stable trace ids.** The terminal trace uses fixed ids matching the Rust `solidity_trace.rs` labels, so fixture tooling can diff Rust vs Solidity executions without circuit knowledge (Sec. 3.14).
- **[NOTE] trace_id 36 re-negation.** The verifier stores $-\nu_y(x)$ but Rust traces the positive $\nu_y(x)$; the trace path re-negates to match. This is the same negated-numerator subtlety as §9.8.6 — correct, but a trap if the trace consumer assumes the stored value.

Chapter 10

Consolidated improvement & action roadmap

This chapter aggregates every [\[IMPROVE\]](#) and [\[RISK\]](#) finding logged across the deep audit (Ch. 9) and the codegen analysis (Ch. 4) into one prioritized, actionable list. It answers the question “*can the generated Solidity be improved, made safer, more robust, easier to audit?*” in a single place. Each row cites the section where the finding is developed.

Verification status. Every item was re-checked against the generator on `main` at `6b3d2096` (2026-07-17) and the production artifact `Halo2VerifierFee12345B64.sol` (“B64”). The Status column reads [\[ADDRESSED\]](#) (done, or the requested invariant is enforced), [\[PARTIAL\]](#), [\[OPEN\]](#) (still to do), or [\[CORRECTED\]](#) (the original premise was wrong; the row has been rewritten). Template paths are relative to `proofs/solidity-verifier/templates/partials/`, `src/` paths to `proofs/solidity-verifier/`, and “B64 Lnnn” is a line of the production artifact. B64 was *not* rendered from `main`. Nightfall pins midfall rev `bd482567`, whose generator is identical to the M7 follow-up commit `e7f736c4` (2026-06-07, forked from `51f912c2`). B64 carries a quotient-certificate header check (B64 L940) that first appears in that commit. Relative to `main`, that line’s templates differ only in this check (`VkLoading.yul`, `Constants.sol`) and in `Halo2VerifyingKey.sol`, so the template statuses below hold for both. It lacks the `6b3d2096` generator fixes, which do not affect B64’s single-phase shape (§14.1).

10.1 Priority 1 — Security-relevant

These affect the soundness margin and should be confirmed against the project’s security target before deployment.

Item	Grade	Action	Status
128-bit truncated challenges (x_3, x_1^i, x_4^i)	HIGH	The resulting soundness level is not established in this document; earlier bit-level figures are withdrawn. The library’s doc comment warns that 128 bits “may not be enough entropy depending on the application”. Faithful to Rust (<code>proofs/src/poly/kzg/mod.rs</code> L394, L457, L502 in multi_prepare , behind the non-default <code>truncated-challenges</code> feature, whose doc comment itself warns that 128 bits may not suffice for every application), so this is a <i>policy</i> decision, not a bug. Confirm the 128-bit challenge budget is acceptable for the security target; if not, the change must be made in the Rust verifier first, then re-generated (§9.3).	[OPEN] Policy decision pending. B64 truncates at L1270 (x_3), L2918 (x_1^i), L3283–3291 (x_4^i).

10.2 Priority 2 — Defense-in-depth (load-bearing invariants)

These are correct today but rely on invariants that are guaranteed *elsewhere* (by the codegen validator or the precompile coverage argument) rather than checked at the point of use. Adding a cheap runtime guard makes each self-evident to an auditor.

Item	Grade	Action	Status
No on-curve/subgroup check at G1 absorption	MED	<code>common_uncompressed_g1</code> does not check the point is on-curve or in the subgroup. Sound only because every absorbed point is later consumed by a G1MSM/pairing (which <i>do</i> subgroup-check). Keep the coverage invariant (§4.8.5) enforced at build time; do not add a per-absorb check (it would cost gas for no security gain given the invariant).	[ADDRESSED] Enforced by <code>validate_generator_invariants</code> (<code>src/lowering/plan.rs</code> L238–251); off-curve mutation test <code>src/test.rs</code> L2408. No on-curve/off-subgroup vector yet.
Native-callback empty-stack assumption	MED	The native-callback reset silently drops a live <code>q_top</code> . Add <code>if q_has_top { revert(0,0) }</code> at each native callback entry (~5 gas) to turn the codegen guarantee into a checked invariant. B64 has three such entries: <code>0x19</code> , <code>0x1f</code> and <code>0x1b</code> .	[OPEN] Unconditional reset at <code>quotient_numerator/QuotientNumeratorBlock.yul</code> L1057, L1084, L1113; B64 L1934, L2051, L2247. Build-time only: <code>src/lowering/quotient_numerator/vm/mod.rs</code> L2241–2246.
ADD/MUL unchecked underflow	MED	Add <code>if lt(q_sp, stack_base+32) { revert(0,0) }</code> before the pop (~10 gas/op-code). Belt-and-suspenders for auditability. B64 renders only ADD (<code>0x06</code>); MUL (<code>0x07</code>) is unused there.	[OPEN] Same template, L503–518; B64 L1716–1721. Build-time only: <code>vm/mod.rs</code> L2213–2216, test <code>src/lowering/tests.rs</code> L1520.
Accumulator identity-flag code intricacy	MED	The <code>first_adjust/is_id</code> path is the most intricate logic in the contract. Add dedicated unit tests enumerating identity vs non-identity encodings; the <code>acc_encoded_identity</code> fast path (exact packed-word equality) is the primary guard. Still missing: behavioural vectors for the canonical identity, a flagged non- $(p-1)$ coordinate, and a non-canonical $(0, 0)$.	[PARTIAL] E2E negatives for packing and malformed LHS/RHS points (<code>tests/ivc_keccak_solidity.rs</code> L1417–1447, since <code>cf89e5f3</code>). Identity path: string check only (<code>tests.rs</code> L1223).
<code>batch_invert</code> leaves memory dirty on failure	MED	The Montgomery-trick path leaves scratch dirty on failure, and on a failed <code>modexp</code> it overwrites the <i>whole</i> target range with garbage (no <code>leave</code> ; <code>verifier/AssemblyHelpers.yul</code> L189–208). Premise corrected: the only caller does <i>not</i> gate before reading — <code>verifier/Lagrange.yul</code> reads the inverted range at L38–82 and gates at L89 (B64 L1350–1394, gate L1397). It is safe because <code>success</code> is sticky and every <code>success = 0</code> path reverts before <code>return</code> . Document that “check-before-return” invariant at the helper header; a “gate-before-read” codegen assertion would reject today’s code.	[OPEN] Header comment still says only “returns a boolean” (<code>AssemblyHelpers.yul</code> L124–126).
Unbounded script cursor	MED	<code>common_word/common_uncompressed_g1</code> write at <code>buf_len</code> with no bound; the buffer is sized by <code>TranscriptBufferLayout</code> . That sizing was <i>wrong</i> for valid multi-phase shapes (advice in an early phase, first challenge later): the G1 absorb loop overran <code>VK_MPTR</code> , writing proof-controlled bytes into the loaded <code>VK</code> , whose header words and G1/G2 bases are read after the transcript (<code>N_INV</code> , <code>OMEGA</code> , <code>G1_BASE</code> , ...). The invariant is still	[PARTIAL] Sizing fixed in <code>6b3d2096</code> (<code>src/lowering/abi/proof.rs</code> L318–347, test L504); no run-time guard. B64 has one user phase and

10.3 Priority 3 — Gas / contract-space optimizations

Concrete savings that do not change the accepted-proof set. Gas figures use the EIP-2537 G1MSM discount table ($k \cdot 12000 \cdot d_k / 1000$, $d_{78} = 561$, $d_{80} = 559$), G1ADD = 375, and EIP-2565 modexp pricing. Under EIP-7883 (Osaka/Fusaka, live), a modexp with these operands costs $16 \cdot 254 = 4,064$ instead of 1,354, so modexp savings roughly triple.

Item	Grade	Action	Status
Rolled vs. unrolled x_1 powers	DONE	Already applied in <code>computations()</code> : rolled loop saves ~ 26 kg vs the unrolled emission (§4.8.4). Pattern to reuse for any other long constant-power sequence.	[ADDRESSED] B64 L2910–2919 (43 powers, one loop).
Fold $-vG$ and $x_3\pi$ into the fused final MSM (was: “fold the two 1-pair G1MSMs”)	IMPROVE	Premise corrected. The two accumulator α -scalings (<code>verifier/FinalPairing.yul</code> ; B64 L3528–3553) cannot share a call: a G1MSM returns one point, and a 2-pair MSM (22,776) in place of “1-pair MSM + G1ADD” (12,375) is dearer — §9.5 already rated them near-optimal. The foldable pair is $-vG$ and $x_3\pi$ in PCS sub-block 10 (B64 L3469–3493): both scalars are known before the fused 78-pair MSM (B64 L3460), so appending $(G, r - v)$ and (π, x_3) yields the pairing RHS directly. Cost $525,096 \rightarrow 536,640$ (+11,544) removes 2 one-pair MSMs and 2 G1ADDs (24,750): net ≈ 13.2 k gas/proof . The generator comment (<code>src/lowering/kzg/mod.rs</code> L1701–1704) only weighed a standalone 3-pair MSM.	[CORRECTED] Replacement action is open.
Batch the five f_{eval} <code>scalar_invs</code>	IMPROVE	B64 L3136, L3180, L3216, L3245, L3265 each run one modexp ($\lfloor 16 \cdot 254/3 \rfloor = 1,354$ gas for 32-byte operands and exponent $r - 2$). One <code>batch_invert</code> over the five products saves 4 modexps (≈ 5.4 k gas, ≈ 16.3 k under EIP-7883) and leaves <code>scalar_inv</code> dead, which also retires the P4 zero-policy item (§11.7).	[OPEN]
FOLD_MAIN redundant load/store	IMPROVE	Keep the accumulator in a register and store once at the end. B64 does not render VM case <code>0x0a</code> (<code>QuotientNumeratorBlock.yul</code> L1129–1144); the same <code>mload/mstore</code> pair is emitted by the native-callback fold snippets (<code>src/lowering/quotient.rs</code> L1018, L1035; e.g. B64 L2005–2006). Saving is one <code>mload+mstore</code> (~ 6 gas) per folded identity, so the earlier “meaningful gas” estimate overstated it.	[OPEN]
<code>gp_mptr</code> / <code>pow_of_omega</code> reuse	IMPROVE	Not a gas item. §9.2 shows the second ω -walk is required (the first pass’s slots are overwritten by their inverses) and <code>l_i_common</code> is already hoisted (<code>verifier/Lagrange.yul</code> L39). The <code>gp_mptr</code> underflow (<code>AssemblyHelpers.yul</code> L197) is harmless and free; only a readability rewrite remains.	[CORRECTED]

10.4 Priority 4 — Audit-clarity / naming

No correctness or gas impact; these reduce reviewer confusion.

Item	Grade	Action	Status
PAIRING_LHS/RHS naming inversion	LOW	LHS=pi, RHS=combined; the call is <code>ec_pairing(success, PAIRING_RHS, PAIRING_LHS)</code> . Rename to <code>PAIRING_PI/PAIRING_COMBINED</code> or <code>ARG1/ARG2</code> to remove the trap.	[OPEN] B64 L128–129, call L3569.
QUOTIENT_MPTR re-purposed as scalar metadata	LOW	The slot holds scalar metadata in the fused path, not a G1 point. Rename to <code>QUOTIENT_META_MPTR</code> or use a distinct slot to remove the ambiguity.	[OPEN] QuotientAndLinearization.yul L93–94; B64 L3278–3279.
Trace path uses <code>call</code> not <code>staticcall</code>	LOW	Premise wrong; no action. The trace-mode evaluator emits <code>log1</code> (<code>templates/contracts/Halo2QuotientEvaluator.sol</code> L120–127), and <code>LOG</code> reverts inside a <code>STATICCALL</code> frame, so the trace branch needs <code>call</code> . Production already uses <code>staticcall</code> (<code>QuotientAndLinearization.yul</code> L30); B64 embeds the quotient evaluator and makes no external call.	[CORRECTED]
<code>scalar_inv</code> vs <code>batch_invert</code> zero-policy mismatch	LOW	<code>scalar_inv</code> reverts on zero; <code>batch_invert</code> returns 0. Unify the convention (or document the deliberate difference). <code>scalar_inv</code> is live, not dead code (five call sites in B64); batching them (P3) removes it.	[OPEN] AssemblyHelpers.yul L9 vs L145–148.
<code>ec_pairing</code> failure convention	LOW	<code>ec_pairing</code> reverts on failure (no <code>is_zero(ret)</code>); <code>batch_invert</code> returns a bool. Pick one success-plumbing convention and apply to both.	[OPEN] AssemblyHelpers.yul L230; B64 L551.
G2ADD not smoked at deploy	LOW	Deliberate (verifier never calls G2ADD). Add a comment noting the omission so auditors don't wonder.	[OPEN] No G2ADD mention in <code>verifier/PrecompileSmoke.sol</code> .

10.5 Hardening already landed

These commits closed items that earlier reviews had raised; they need no further action and should not be re-raised:

- 2b2bf494 — the constructor smoke test probes full-size shapes (largest G1MSM, two-pair pairing). The same commit made every precompile `staticcall` forward `gas()` (all 18 sites in B64). That is *not* pure hardening: it made B64 robust to repricing, but a rejecting call now burns all forwarded gas. It is reopened as a P2 item (§14.4).
- b53e3ebe — the constructor smoke executes `MCOPY` (`verifier/PrecompileSmoke.sol` L11–13; B64 L211–213).
- 1e21311c — a non-canonical public instance reverts right after the instance loop (`verifier/TranscriptProofParser.yul` L68; B64 L1050), not after the whole transcript.
- e64f2891 — loaded VK header words are cross-checked against verifier constants (`verifier/VkLoading.yul` L82–88; B64 L934–941).
- 33f78d2b — the quotient VM asserts `q_pc == q_end` and an empty stack after the loop (`QuotientNumeratorBlock.yul` L1191–1192; B64 L2569–2570).
- 6b3d2096 — transcript-buffer sizing for multi-phase shapes (P2 above) and a fallback when a fused VM opcode's constant index no longer fits one byte, replacing a generator panic.

10.6 What is *not* a deviation

For completeness, the following were examined and found to be faithful (no action needed), and are listed here so a reviewer does not re-raise them:

- **Precompile addresses** (0x0b/0x0c/0x0f) — verified against EIP-2537; correct, not off-by-one.
- **Negated-numerator** ($-\nu_y(x)$ stored, trace id 36 re-negates) — matches Rust `expected_eval -= eval`; the $(1 - x^n)$ factor is on the commitment side.
- **Modulo bias in squeeze_to** — **[CORRECTED]** the bias is *not* $\leq 2^{-128}$. With $r \approx 2^{254.86}$, $2^{256} = 2r + t$ with $t \approx 0.21r$, so residues below t have three preimages and the rest two: statistical distance ≈ 0.075 , maximum probability $\approx 1.36/r$. It is still not exploitable (min-entropy ≈ 254.4 bits; each Schwartz–Zippel bound grows by at most $1.36\times$) and identical to Rust (`sample_keccak_digest_be_mod_r`, `transcript/implementors.rs` L19).
- **Single-scalar-1 fast path** in `DualMSM::check` — the generated `ec_pairing` always runs the fused MSM; identical result, Rust saves one MSM.
- **Derived proof layout** — the read order is computed from the constraint system, so parser and verifier cannot drift.

10.7 Suggested implementation order

A pragmatic sequence for what remains open (the rolled x_1 powers and the hardening of §10.5 are done; the α -MSM fold, the trace `staticcall` switch and the `pow_of_omega` “gas” item are dropped as mis-premised):

1. **Confirm P1** (truncation policy) with the security owner — this gates everything else.
2. **Add the P2 runtime guards**: the `memory-safe` scratch base plus free-pointer entry assert (or its CI check) and the transcript-cursor bound first (the compiler already spills into the scratch range, and the codegen already mis-sized the transcript buffer once), then native-callback and ADD underflow — cheap, high audit value, no behavior change for correctly generated code.
3. **Add the P2 tests**: behavioural identity-encoding vectors for the accumulator decoder, and an on-curve/off-subgroup proof point.
4. **Apply P3 gas wins** ($-vG, x_3\pi$ into the fused MSM $\approx 13.2k$; batched `scalar_inv` $\approx 16.3k$ under EIP-7883; register fold, small) — re-run the differential trace to confirm no behavior change.
5. **Do P4 renames** last — pure clarity, do after the functional changes so the diff is clean.

Every item above is a *suggestion*; the generated verifier as it stands is a faithful translation of the Rust reference verifier (Ch. 7). None of the P2–P4 items change the accepted-proof set of a correctly generated verifier; only P1 could, and only if the security target demands a larger challenge budget than the Rust verifier already uses.

10.8 Consolidated deviation register

This is the single triage table for every finding raised across the deep audit (Ch. 9) and the production-artifact review (Ch. 11). Each row is one finding with its severity, where it was raised, its disposition, and its verified status. **Sev:** R=risk, I=improvement, N=note. **Disposition:** *faithful* = matches Rust by design (no action unless policy changes); *fix* = cheap defensive change;

done = already applied in the templates. **Status** as in the chapter introduction; “—” means no action is needed.

ID	Sev	Finding	Where	Disposition	Status
R1	R	128-bit truncated challenges (x_3, x_1^i, x_4^i) ; soundness level unstated	§4.8.4, §11.3	faithful (policy)	[OPEN]
R2	R	No on-curve/subgroup check at G1 absorption	§9.1	faithful (invariant)	[ADDRESSED] <code>plan.rs</code> L245
R3	R	Native-callback silently drops live <code>q_top</code>	§9.8, §11.5	fix (~5 gas)	[OPEN]
R4	R	ADD/MUL stack underflow unchecked	§9.8, §11.5	fix (~10 gas)	[OPEN]
R5	R	Intricate accumulator identity-flag decode	§9.6	test coverage	[PARTIAL] no identity vectors
R6	R	<code>batch_invert</code> leaves memory dirty on failure; caller reads before its gate	§9.1	check before return	[OPEN]
R7	R	Point/eval equality by layout index (pointer)	§9.3	faithful (derived)	—
R8	R	<code>memory-safe</code> load-bearing on terminality; formally violated under via-IR (solc reserves <code>[0x80, 0x8e0]</code> for spills)	§9.11.2, §11.8.19	fix (base + FMP assert, or CI check)	[OPEN]
R9	R	Transcript cursor unbounded; codegen once under-sized the buffer	§9.1.2	fix (<0.3k gas)	[PARTIAL] <code>6b3d2096</code>
I1	I	Rolled (vs. unrolled) x_1 powers	§4.8.4	done	[ADDRESSED]
I2	I	Five separate <code>scalar_inv</code> mod-exps $\rightarrow$ one <code>batch_invert</code>	§11.7	fix ($\approx$ 16.3k gas under EIP-7883)	[OPEN]
I3	I	Five near-identical f_{eval} Horner blocks	§11.7	parameterize	[OPEN]
I4	I	Fold the two 1-pair accumulator MSMs	§9.5, §11.7	near-optimal	[CORRECTED] see I12
I5	I	FOLD_MAIN/native-fold redundant load/store	§9.8	fix (~6 gas/fold)	[OPEN]
I6	I	<code>gp_mptr/pow_of_omega</code> reuse	§9.2	readability only	[CORRECTED]
I7	I	Trace path uses <code>call</code> not <code>staticcall</code>	§9.4	none (<code>call</code> required)	[CORRECTED]
I8	I	G2ADD not smoked at deploy	§9.11.3	comment	[OPEN]
I9	I	246 distinct raw-hex scratch addresses, no legend	§11.6, §11.8.1	emit legend	[OPEN]
I10	I	Opaque SSA names (<code>var0</code> , <code>f_3</code>)	§11.6	semantic names	[OPEN]
I11	I	Index-suffixed scalars (<code>lin_query_scalar_41</code>)	§11.6	named	[OPEN]
I12	I	Fold $-vG$ and $x_3\pi$ into the fused final MSM	§10.3	fix ($\approx$ 13.2k gas)	[OPEN]
N1	N	Modulo bias in <code>squeeze_to</code> (SD ≈ 0.075 , max ratio ≈ 1.36)	§9.1.4	faithful	[CORRECTED]
N2	N	Negated-numerator storage ($-\nu_y$)	§9.8.6	faithful	—
N3	N	Pairing LHS/RHS naming inversion	§9.1.6, §11.6	rename	[OPEN]

ID	Sev	Finding	Where	Disposition	Status
N4	N	Deferred failure for non-canonical instances	§9.3	fail fast	[ADDRESSED] 1e21311c
Table 10.1: Consolidated deviation register: every finding from the deep audit and production review, severity-ranked with disposition and verified status. 9 risks (1 policy decision, 3 open fixes/docs, 2 partial, 1 enforced invariant, 2 no-action), 12 improvements (1 done, 3 corrected, 8 open), 4 notes (1 corrected, 1 addressed, 1 open, 1 no-action).					

The register makes the triage explicit: the only *policy* item (R1) is a faithful choice; the cheap defensive items are R3/R4 (unchecked stack, ~15 gas) and R9 (transcript bound, <0.3k gas), and R9 is the one with a demonstrated generator bug behind it. The largest remaining gas win is I12 (~13.2k). Everything else is either faithful-by-design or an audit-clarity improvement. No finding changes the accepted-proof set of a correctly generated verifier except R1 under a stricter security target.

Chapter 11

Production-Artifact Code Review

This chapter reviews a *real production* render of the generator, not a test fixture: the `fee_12345 / block_size_64` pair under `nightfall_4_PV/blockchain_assets/.../generated/`. It is a full IVC verifier (accumulator present, multi-limb quotient, compact quotient VM) and is the most demanding artifact the generator produces. The goal here is narrower than the faithfulness argument of Chapter 7: we assess **readability**, **robustness**, **safety**, **correctness**, **soundness**, and **audit-facilitation** of the emitted Solidity, and call out repeated computations and patterns that hurt either gas or human review.

Dimension	Rating	Headline	Ref
Correctness	Strong	Matches Rust verifier; no defect found	§11.2
Soundness	Strong	Randomised batching + canonical checks	§11.3
Safety	Strong	Fail-closed, pinned deps, precompile-checked	§11.4
Robustness	Good	VM stack unchecked at runtime (excluded offline); memory-safe latent risk	§11.5
Readability	Fair	Raw-hex scratch, opaque SSA names	§11.6
Audit-facility	Fair	No address leg-end; VM program opaque	§11.8
Gas / size	Good	5 batchable mod-exps; repetitive Horner	§11.7

Table 11.1: Summary ratings for `Halo2VerifierFee12345B64.sol` (3,579 lines) and `Halo2VerifyingKeyFee12345B64.sol` (698 lines).

11.1 What this artifact is

- **3578-line** verifier + **697-line** VK payload contract, pinned together by codehash (`EXPECTED_VK_CODEHASH_WORD`).

- **IVC circuit:** `has_accumulator = 1`, `acc_offset = 11`, 7 radix-2⁵⁶ limbs, 19 public instances, $k = 21$ (domain 2^{21}).
- **Compact quotient VM:** the bulk of the gate/permutation/lookup identities are a bytecode stream in the VK payload (`q_program_mptr = 0x4140`, length `0x11cf`), interpreted by a Yul VM whose dispatch renders only the **11** opcodes this program uses (of 31 defined in the generator), with whole-family native callbacks (`0x19` permutation, `0x1f` lookup) and `0x1b` for four selected heavy gate identities.
- **Fused final MSM:** the linearisation commitment is not materialised standalone; its 14 terms (4 quotient limbs + 10 simple-selector fixed commitments) are expanded inside the single 78-pair (G1, scalar) G1MSM that computes the KZG `final_com` (sub-block 9, lines 3274–3464).

11.2 Correctness

The verifier reproduces the Rust reference schedule exactly. Spot-checks against the native `plonk/verifier.rs` flow:

- **[OK] Transcript order.** VK digest $\rightarrow$ committed-pi identity (128 zero bytes) $\rightarrow$ instance count $\rightarrow$ instances $\rightarrow$ advice $\rightarrow \theta \rightarrow$ lookup $M \rightarrow \beta, \gamma \rightarrow$ perm $Z \rightarrow$ lookup helpers/ $Z \rightarrow$ trash challenge $\rightarrow$ trashcans $\rightarrow y \rightarrow$ quotient limbs $\rightarrow x \rightarrow$ evals $\rightarrow x_1, x_2 \rightarrow f_{\text{com}} \rightarrow x_3 \rightarrow q_{\text{evals}} \rightarrow x_4 \rightarrow \pi$. Matches the native squeeze/absorb interleaving.
- **[OK] Parser completeness.** The final `if iszero(eq(proof_cptr, NUM_INSTANCE_CPTR)) revert` (line 1312) proves the hand-rolled parser consumed exactly the generated proof layout — under- or over-reading a section reverts.
- **[OK] Negated numerator.** `QUOTIENT_EVAL_MPTR` carries $-\nu_y(x)$; the $(1-x^n)$ factor lives on the commitment side (Def. 2.7.1).
- **[OK] Lagrange / instance eval.** $L_i(x) = (x^n - 1) n^{-1} \omega_i / (x - \omega_i)$ via one batch inversion (line 1346); l_{blind} sums the negative-rotation terms; instance eval is the Lagrange-weighted sum.

No correctness defect was found in the reviewed span.

11.3 Soundness

- **[OK] Randomised accumulator batching.** $\alpha = \text{keccak256}(\text{domain} \parallel \text{kzg}_{r_{hs}} \parallel \text{kzg}_{l_{hs}} \parallel \text{acc}_{r_{hs}} \parallel \text{acc}_{l_{hs}}) \bmod r$, drawn *after* all four G1 inputs are fixed, with the $\alpha=0 \rightarrow 1$ guard. Two bad equations cannot cancel (Schwartz–Zippel, $\leq 1/r$). See §9.6.1.
- **[OK] Canonicity everywhere.** Instances, proof evals, and q_{evals} are range-checked $< r$ before absorption (a bad instance reverts right after the instance loop, line 1050, commit `1e21311c`; evals and q_{evals} revert inline); G1 calldata rejects non-zero top-16 bytes and out-of- $\mathbb{F}_p$ coordinates; accumulator points reject unused high bits.
- **[OK] Precompile-backed subgroup checks.** Every absorbed commitment is later consumed by a G1MSM/pairing that enforces on-curve/subgroup membership; the accumulator forces *every* decoded point through G1MSM even at scalar 0/1.
- **[RISK] 128-bit truncated challenges** (x_3, x_1^i, x_4^i ; x_2 stays full-width). The resulting soundness level is not established here (§11.8.12). Faithful to Rust (`poly/kzg/mod.rs`); flagged for the security target, not a translation bug. **[OPEN]** feature still enabled (`solidity-verifier/Cargo.toml`; B64 L1270, L2918, L3283–3291).

11.4 Safety

- **[OK] Fail-closed.** Every rejection path is `revert(0,0)`; the ABI only ever observes `true`. No path returns `false` that a caller might ignore.
- **[OK] Precompile results.** Every `staticcall` is paired with `eq(returndatasize(),...)` and a success fold — no unchecked precompile result.
- **[RISK] Unbounded precompile gas.** **[OPEN]** Every precompile `staticcall` forwards `gas()` (commit 2b2bf494). A rejecting EIP-2537 or `modexp` call consumes *all* gas it was given, so one malformed proof point burns about 63/64 of the transaction’s gas. Unmerged midfall PR #6 measured 29.5M of a 30M limit. The upside is that B64 survived the EIP-7883 `modexp` repricing unchanged. The fix is a bound per call site; see §14.4 for the trade-off.
- **[OK] Dependency pinning.** VK pinned by *both* `extcodesize` and `extcodehash`, re-checked on every proof (line 920), not just at construction.
- **[OK] ABI-shape guard.** The dynamic-argument heads are checked before any generated memory work (line 305), so a malformed ABI cannot be misread as a proof stream; exact proof length, instance count and `calldatasize` follow before any transcript work (lines 954–965).
- **[OK] Deploy-time smoke test.** `require_eip2537_precompiles()` exercises MCOPY, G1ADD, the *largest* G1MSM input, and the two-pair pairing with identity inputs, so a wrong fork fails at deploy.

11.5 Robustness

Two stack hazards carried over from the templates (see §9.8) are unchecked *at runtime* in this render. Both are excluded offline: `validate_quotient_program` (vm/mod.rs:2003–2021, stack effects L2191–2255) re-decodes the finalized bytecode and rejects underflow, non-empty stacks at native callbacks, and end-of-program leaks before the bytes are pinned into the codehash-pinned VK. The runtime checks below are defense-in-depth against a validator bug, not against proof calldata.

- **[RISK] Native-callback silent stack drop.** 0x19/0x1f set `q_top := 0`; `q_has_top := 0` unconditionally (lines 1934–1935, 2051–2052). If a validator bug ever emitted a callback with a live `q_top`, that value is silently discarded (0x1b does the same, lines 2247–2248). *Fix:* `if q_has_top { revert(0,0) }` at callback entry (~5 gas). **[PARTIAL]** statically rejected (vm/mod.rs:2241–2247); the post-loop `if q_has_top { revert }` (B64 L2570, commit 33f78d2b) only catches a top still live at program end, not one discarded at a callback.
- **[RISK] ADD underflow.** case 0x06 does `q_sp := sub(q_sp, 0x20)` with no lower-bound check (line 1719); 0x07 MUL is not rendered in this program. *Fix:* `if lt(q_sp, add(STACK_BASE, 0x20)) revert` (~10 gas), where `STACK_BASE` is the rendered 0xa980. **[PARTIAL]** statically rejected (vm/mod.rs:2213–2216 calling `require_quotient_stack_depth`, L2258–2270); no runtime guard.
- **[RISK] memory-safe is load-bearing, and not only on terminality.** The block is terminal and its scratch starts above the reserved prefix (0x00–0x3f scratch, 0x40 FMP, 0x60 zero slot), but the annotation also lets `solc` move spilled stack variables into memory: with the production settings (via-IR) the runtime reserves [0x80, 0x8e0), overlapping the raw

scratch that starts at 0x80. See §11.8.19 for this latent risk and §9.11.2 for the terminality argument. **[CORRECTED]** (earlier text said “correct as emitted”).

- **[NOTE] Zero-denominator reverts the whole proof.** `scalar_inv` reverts on 0 (line 330). A collision $x_3 = \text{rotation point}$ reverts rather than yielding a typed error. Rust does not return an error here either: it panics (`den.invert().unwrap()`, `poly/kzg/mod.rs:479`). Acceptable for a success-or-revert verifier, but worth a comment so a reviewer doesn’t mistake it for a bug. **[CORRECTED]** (earlier text said Rust “errors”).

11.6 Readability

This is the weakest dimension of the artifact.

- **[IMPROVE] 246 distinct raw-hex addresses vs 68 named pointer constants.** (246 distinct 0xNNNN first operands inside `verifyProof` of `mload/mstore/mcopy`, 492 such uses; 79 internal constants, 68 of them `*_MPTR/*_CPTR`.) The high-level slots are named (`TRANSCRIPT_MPTR`, `PAIRING_LHS_MPTR`, ...) but the entire eval/commitment scratch region is raw hex with no legend:

```
let a_14 := mload(0x8a20)      // which column? which rotation?
mstore(0xa360, 0x90c0)        // staging a pointer to what?
mcopy(0xbf40, 0x5a20, 0x80)    // which fixed commitment?
```

An auditor cross-checking against the Rust verifier must *re-derive the memory plan by hand* to learn that 0x8a20 is, say, “advice column 14, rotation 0 evaluation.” This is the single largest barrier to human review. **[OPEN]** no legend in HEAD or B64.

- **[IMPROVE] Opaque SSA names.** The inline prefix, native callbacks and trash suffix use `var0...var162` (163 distinct names, 980 uses), `f_0...f_26`, `a_0...a_14` — mechanically unique but carrying no semantic hint. `var19 := mulmod(var0, var18, r)` (line 1585) tells a reviewer nothing about which gate term it is. **[OPEN]**
- **[IMPROVE] Index-suffixed scalars.** `lin_query_scalar_41 / lin_cur_scalar_41` (lines 3380–3412) — the 41 is the fused-MSM *pair index* of the first quotient-limb term (`lowering/kzg/mod.rs:1612`), while the value loaded is `X1_POWERS[42]`; neither is explained. **[OPEN]**
- **[OK] Phase banners.** The transcript-phase comments (`// ---- theta ----`, `// ---- y ----`) and the VM opcode summary table (lines 1673–1688) are genuinely helpful.
- **[OK] Intent comments on non-obvious choices.** The `MCOPY`-vs-`mstore` rationale, the truncation-rule explanation, and the pairing-naming warning are well written and land the “why.”

11.7 Repeated computations & patterns (gas / size)

- **[IMPROVE] Five separate `scalar_inv` modexps in the f_{eval} Horner.** Sets 4/3/2/1/0 each call `scalar_inv` on their own denominator (lines 3136, 3180, 3216, 3245, 3265) — five modexp precompile calls, on top of the Lagrange `batch_invert` (line 1346), i.e. six runtime modexps. The five denominators are independent, so they can be collected into one contiguous array and inverted with a single `batch_invert` (one modexp + a few `mulmods`), then each set’s Montgomery backward pass proceeds unchanged from its own inverted product. Since x_3 is already squeezed (line 1260) before the Lagrange block, they can even join the line-1346 batch once the rotation points (sub-block 1) are hoisted (six modexps $\rightarrow$ one).

```
// BEFORE: 5 modexps
set4: bq := scalar_inv(bp_5) // modexp
set3: bq := scalar_inv(bp_5) // modexp
set2: bq := scalar_inv(bp_3) // modexp
set1: bq := scalar_inv(bp_3) // modexp
set0: dx0_inv := scalar_inv(dx0) // modexp

// AFTER: 1 modexp
// stage the 5 full-products contiguously, one batch_invert,
// then each set's backward pass uses its own inverted product.
success := batch_invert(success, den_array, den_array_end, scratch, r)
```

Saves ~ 4 modexp calls per proof: $\approx 1,354$ gas each under EIP-2565 ($\approx 5.4k$; $\approx 6.8k$ if merged into the Lagrange batch), and 4,064 each under EIP-7883 (Osaka/Fusaka, live on mainnet and Sepolia) ($\approx 16.3k$; $\approx 20.3k$ merged). [\[OPEN\]](#)

- [\[IMPROVE\]](#) **Five near-identical Horner blocks.** The f_{eval} section repeats the same batch-inversion skeleton five times with cardinalities 3, 3, 2, 2, 1 (lines 3116–3268). A parameterised emitter (loop over sets, cardinality as a parameter) would cut the code size and the audit surface substantially — today a reviewer must diff five blocks to confirm they differ only in rotation indices. [\[OPEN\]](#)
- [\[OK\]](#) **Full-precision power accumulators.** x_1 powers (sub-block 2, a rolled 42-iteration loop, line 2915) and x_4 powers (sub-block 9, five unrolled steps, lines 3281–3291) keep a full-precision running product and truncate each stored power — the pattern recommended in §4.8.4. Only the x_1 table is rolled.
- [\[NOTE\]](#) **Fused-MSM staging is verbose.** Sub-block 9 is 78 mcopy+mstore pairs staging the whole final_com MSM (multi-open terms plus the 14 fused linearisation terms). Necessary, but a table-driven emitter (a data array of (src_g1, scalar_expr) pairs) would shrink it and make the term list auditable at a glance. [\[OPEN\]](#)
- [\[NOTE\]](#) **Magic power offsets.** mload(add(X1_POWER_MPTR, 0x180)) etc. appear many times; the offsets are correct but are magic numbers relative to the power table. [\[OPEN\]](#)

11.7.1 Concrete gas profile (EIP-2537)

Counting the runtime precompile calls in the production verifier and pricing them against the EIP-2537 schedule (modexp: EIP-2565) gives a concrete profile. The B64 source has 18 staticcall sites; the three in require_eip2537_precompiles (lines 224, 240, 250) run only at deployment, and the 0x0c site at line 872 is a multi-line call that a one-line grep misses. The fused final MSM dominates.

Where the gas goes. The fused final MSM is **74%** of precompile gas and the final pairing **14.5%**; together they are 88.5%. This is the *good* news: the dominant cost is already a single Pippenger-discounted MSM (the 78-pair discount of 561/1000 is near the asymptotic 519), so there is little left to win there. The remaining optimization surface is small:

- **1-pair MSMs (72k, 10.1%).** Six separate 1-pair G1MSMs each pay the full $k=1$ price (12,000, no discount). [\[CORRECTED\]](#) The earlier count (five) and proposal (one 5-pair MSM at “848/1000”) were wrong: 848 is the $k=3$ entry ($k=5$ is 764), and the MSMs cannot be merged into one because each output point is consumed separately (α is hashed over the accumulator and KZG points, and a 2-pair MSM at 22,776 costs more than a 1-pair MSM plus a G1ADD at 12,375). Sound options: (a) append $-v \cdot G$ and $x_3 \cdot \pi$ to the fused MSM (v, x_3, π are all known in sub-block 9): 80 pairs cost 536,640, i.e. +11.5k against -24.75k, net

Call		Pairs	Formula	Gas
Fused final MSM (0x0c)		78	$78 \cdot 12000 \cdot 561/1000$	525,096
1-pair (0x0c) $\times 6$	G1MSM	1 ea	$6 \cdot 12000$	72,000
G1ADD (0x0b) $\times 4$		—	$4 \cdot 375$	1,500
Final pairing (0x0f)		2	$32600 \cdot 2 + 37700$	102,900
modexp (0x05) $\times 6$		—	$6 \cdot \text{floor}(16 \cdot 254/3)$	8,124
Total precom- pile				709,620

Table 11.2: Runtime precompile gas, EIP-2537 schedule (G1 MSM discount table, pairing 32600k+37700) and EIP-2565 modexp (32-byte operands, 255-bit exponent $r - 2$). The six 1-pair MSMs are the two accumulator validations (lines 827, 872), $-v \cdot G$ and $x_3 \cdot \pi$ (3475, 3486) and the two α -scalings (3531, 3545); the G1ADDs are at 3480, 3490, 3536, 3550. Under EIP-7883 (Osaka/Fusaka, live on mainnet and Sepolia), a modexp with these operands costs $16 \cdot 254 = 4,064$, so the six modexps cost 24,384 and the total is ≈ 725.9 k. The modexps are five `scalar_inv` plus the Lagrange `batch_invert`. EVM overhead (transcript keccak, VM loop, memory) adds on top.

≈ -13.2 k gas (the memory plan and the constructor probe must grow by two pairs); (b) the two scalar-1 validation MSMs are fail-fast only — the later α -scaling MSMs subgroup-check the same points — so dropping them saves 24k at the cost of rejecting bad accumulators later. This is the I4 item. [\[OPEN\]](#)

- **modexps (8.1k; 24.4k under EIP-7883).** Six 256-bit modexps ($\approx 1,354$ gas each under EIP-2565, 4,064 under EIP-7883); batching the five `scalar_inv` into one `batch_invert` (I2) saves ≈ 5.4 k gas (≈ 16.3 k under EIP-7883). Small against the MSM but free to do. [\[OPEN\]](#)

Audit note. The 78-pair MSM is the whole KZG `final_com`, not just the linearization: 63 multi-open commitment terms (weighted by x_1/x_4 powers) + 14 linearization terms (4 quotient limbs + 10 simple-selector fixed commitments, `linearization_term_count`, `lowering/kzg/mod.rs:631`) + 1 for f_{com} (`final_msm_shape`, `lowering/kzg/mod.rs:681`). The rendered comment “final MSM input length from circuit/VK shape: 78 term(s)” (line 3276) states the total. Rust’s `lin_com_len` (`linearization/verifier.rs:58`) is only a `Vec` capacity and does not predict the 78. [\[CORRECTED\]](#)

11.8 Audit-facilitation improvements

The changes that would most help a human cross-check this contract against the Rust verifier *by eye*:

1. **Emit an address legend.** A generated comment block (or a companion `MEMORY_MAP.md`) mapping every scratch address to its semantic role — `0x8540 = advice[0] eval rot0`, `0x8a20 = advice[14] eval rot0`, `0x5a20 = fixed_comms[14]` (the first simple-selector bucket), etc. This converts the 246 raw-hex addresses from a puzzle into a lookup table. [\[OPEN\]](#) not emitted by HEAD; B64 has none.
2. **Name the eval/commitment slots.** Replace raw hex with generated `ADVICE_EVAL_0_MPTR`, `PERM_Z_CUR_3_MPTR`, ...so the Yul reads like the Rust. [\[OPEN\]](#) Caveat: eval addresses consumed by the VM are u16 operands inside the VK-resident `q_program`, so naming them in Yul changes nothing there; this helps only the Yul-rendered blocks (inline prefix, native callbacks, trash suffix, PCS). Bytecode operands need item 1 or 4.

3. **Emit a trace manifest.** Map each VM identity / native-callback position to the identity and its Rust source location (`verifier.rs::...`, gate name), so the y-batch order can be checked against `partially_evaluate_identities` without running the VM. **[PARTIAL]** the data exists as a host API (§11.8.4) but is not emitted with the render.
4. **Disassemble the quotient VM program.** The `q_program` bytecode lives in the VK payload and is invisible in the Solidity. Emitting a human-readable disassembly listing alongside the render lets an auditor review the interpreted identities without a VK dump. **[OPEN]** no disassembler in HEAD (§11.8.2 shows it is easy).
5. **Semantic SSA names.** Have the lowering pass name intermediates after their origin (`perm_left_acc`, `lookup_f_i`) instead of `var7`. **[OPEN]**

11.8.1 Worked example: the labeling asymmetry

The generator *already* labels the VK payload beautifully — every word carries a comment:

```
// Halo2VerifyingKeyFee12345B64.sol (the data side: LABELED)
mstore(add(payload, 0x2c20), 0x...08d3aa97...) // fixed_comms[0].x_hi
mstore(add(payload, 0x2c40), 0xb667bf02...) // fixed_comms[0].x_lo
```

But the *verifier* reads those same commitments as raw offsets into the VK_MPTR region, with no comment:

```
// Halo2VerifierFee12345B64.sol (the code side: UNLABELED)
mcopy(0xaa0, 0x5320, 0x80) // what is 0x5320?
mcopy(0xaa00, 0x5720, 0x80) // what is 0x5720?
```

Resolving it by hand: `VK_MPTR = 0x2700`, so `0x5320 = VK_MPTR + 0x2c20 = fixed_comms[0]` and `0x5720 = VK_MPTR + 0x3020 = fixed_comms[8]` (both staged at lines 3320–3322 as ordinary multi-open terms of point set 0, not selector terms). The information exists on the data side but is lost at the point of use.

What the legend would restore. A generated `MEMORY_MAP` (or inline `#define`-style constants) turns the code side into the same readable form:

```
// generated legend (illustrative)
// 0x5320 = VK_MPTR + 0x2c20 = fixed_comms[0] (queried at x, set 0)
// 0x5720 = VK_MPTR + 0x3020 = fixed_comms[8] (queried at x, set 0)
// 0x5a20 = VK_MPTR + 0x3320 = fixed_comms[14] (simple-selector bucket 0)
// 0x8540 = advice[0] eval @ rot0
// 0x8a20 = advice[14] eval @ rot0
// 0xa320 = fused-MSM staging pair 0
```

With that, the fused-MSM staging in sub-block 9 reads as a term list a human can diff against the Rust multi-open (`poly/kzg/mod.rs`) and, for pairs 41–54, `compute_linearization_commitment` directly — which is exactly the “cross-check with the Rust code by human eyes” the review is meant to enable. The mapping is fully mechanical (the codegen knows every address’s role), so emitting the legend is a low-risk, high-return change. **[OPEN]**

11.8.2 Worked example: disassembling the `q_program`

The compact quotient VM’s identity program is a byte stream in the VK payload (`q_program_mptr = 0x4140`, length `0x11cf`), invisible in the verifier source. The opcode table is fully exposed in the codegen (`vm/mod.rs`), so the program *is* decodable. Decoding the real production bytes gives a human-readable identity listing:

Op	Name	Semantics
0x05	PUSH_MEM_U16	push mload(u16 ptr)
0x06	ADD	top += pop()
0x08	NEG	top = -top
0x0d	MUL_CONST_U8	top *= const[u8]
0x10	ADD_MEM_U16	top += mload(u16 ptr)
0x0b	FOLD_SELECTOR	fold top into selector bucket
0x1b	NATIVE_IDENTITY	delegate one heavy gate identity
0x21	MODARITH7	fused 7-limb foreign-field affine id

Table 11.3: Quotient VM opcode subset (from `vm/mod.rs`).

Real disassembly (first instructions). Decoding the actual `fee_12345` program bytes:

```
[0000] PUSH_MEM_U16 0x8580 ; push mload(0x8580)
[0003] ADD_MEM_U16 0x8aa0 ; top += mload(0x8aa0)
[0006] PUSH_MEM_U16 0x8620 ; spill; top = mload(0x8620)
[0009] NEG ; top = -top
[000a] ADD ; top = (m[0x8580]+m[0x8aa0]) - m[0x8620]
[000b] MUL_CONST_U8 0x00 ; top *= const[0]
[000d] FOLD_SELECTOR 0x020001 ; fold top into selector bucket 2
[0011] NATIVE_IDENTITY 0x0000 ; delegate heavy identity #0
[0014] MODARITH7 <fused 7-limb foreign-field identity>
[0102] FOLD_SELECTOR 0x030001
[0106] MODARITH7 <fused 7-limb foreign-field identity>
...
```

A full decode (re-implementing `decode_byte_quotient_instruction`, `vm/mod.rs:2024`) consumes exactly `0x11cf` bytes as **101** instructions using exactly the 11 rendered opcodes: 12 `PUSH_MEM_U16`, 7 `ADD`, 5 `NEG`, 20 `MUL_CONST_U8`, 2 `ADD_MEM_U16`, 12 `MUL_MEM_U16`, 21 `FOLD_SELECTOR`, 16 `MODARITH7`, 4 `NATIVE_IDENTITY`, and one each of `NATIVE_PERMUTATION/NATIVE_LOOKUP`. Each `FOLD_SELECTOR` boundary is one quotient identity entering the *y*-batch, exactly as §9.8 describes; this program contains no `FOLD_MAIN` (`0x0a` is not rendered), so main-target identities are folded inside the native callbacks and the Yul prefix/-suffix blocks. The `MODARITH7` cases are the foreign-field (7-limb) gate identities that the limb recognizers compressed into one super-instruction.

Why this matters. With a decoder in hand, the “invisible VM bytecode” objection dissolves: an auditor can dump the program to a listing like the above and check the identity *order* and *memory operands* against the Rust `partially_evaluate_identities` directly. The recommendation in §11.8 (emit a disassembly alongside the render) is therefore not aspirational — the encoding is simple enough that the generator could emit this listing for free. [\[OPEN\]](#) HEAD has the decoder (`validate_quotient_program`) but emits no listing.

11.8.3 Worked example: fused linearization MSM ↔ Rust `compute_linearization_commitment`

The hardest thing to audit by eye is the fused final MSM (sub-block 9, 78 staging pairs). It is the `KZG final_com` of the multi-open (`poly/kzg/mod.rs`), with the linearization commitment expanded in place: pairs 0–40 and 55–77 are ordinary multi-open terms (63 commitments weighted by x_1/x_4 powers, plus f_{com} at x_4^5), and pairs 41–54 are a direct, term-by-term render of the Rust `linearization/verifier.rs::compute_linearization_commitment`. The Rust builds the linearization point as two term classes:

```
// Rust: linearization/verifier.rs
identities_points.extend(quotient_limb_commitments);
let mut splitting_pow = F::ONE - *xn;
for _ in 0..quotient_limb_commitments.len() {
    identities_scalars.push(splitting_pow); // (1 - x^n) * x_split^i
    identities_labels.push(CommitmentLabel::NoLabel);
    splitting_pow *= splitting_factor; // x_split = x^(n-1)
}
// ... grouped by column, weighted by y^k:
*grouped_points.entry(*col_idx).or_insert(F::ZERO) += y_pow * eval;
// Some(col) -> fixed_commitments[col] with scalar = sum_k y^k * eval
// None -> expected_eval -= eval (the negated numerator)
```

The generated fused MSM stages exactly these two classes (each pair is (G1 point, scalar) fed to one G1MSM):

```
// Generated: sub-block 9, quotient-limb terms
let lin_query_scalar_41 := mload(add(X1_POWERS_MPTR, 0x540)) // multiopen batch scalar
let lin_cur_scalar_41 := mulmod(lin_query_scalar_41, lin_one_minus_x_n, r)
mstore(0xbd40, lin_cur_scalar_41) // scalar for Q_0
lin_cur_scalar_41 := mulmod(lin_cur_scalar_41, lin_x_split, r)
mstore(0xbde0, lin_cur_scalar_41) // scalar for Q_1 (* x_split)
lin_cur_scalar_41 := mulmod(lin_cur_scalar_41, lin_x_split, r)
mstore(0xbe80, lin_cur_scalar_41) // scalar for Q_2 (* x_split^2)
...
// Generated: selector terms (the grouped y^k * eval per fixed column)
mstore(0xbfc0, mulmod(lin_query_scalar_41, mload(add(SELECTOR_ACC_MPTR, 0x00)), r))
mstore(0xc060, mulmod(lin_query_scalar_41, mload(add(SELECTOR_ACC_MPTR, 0x20)), r))
...
```

Term	Rust	Generated
Quotient	$(1-x^n)*$	<code>lin_query_scalar_41</code>
limb Q_i	<code>splitting_pow</code>	<code>* one_minus_x_n *</code> <code>x_split^i</code>
Fixed col j	<code>sum_k y^k * eval</code> (grouped)	<code>lin_query_scalar_41</code> <code>* SELECTOR_ACC[j]</code>
Fully-eval ($\perp$)	<code>expected_eval -=</code> <code>eval</code>	negated at <code>QUOTIENT_</code> <code>EVAL_MPTR</code>
Batch scalar	applied in multiopen step	<code>lin_query_scalar_41</code> (fused here)

Table 11.4: Term-by-term correspondence: fused MSM $\leftrightarrow$ Rust `compute_linearization_commitment`.

The one structural difference. The Rust applies the KZG multi-open batching scalar in a *later* step (`VerifierQuery::new_linear` $\rightarrow$ multiopen), whereas the generated code fuses that batch scalar (`lin_query_scalar_41 = X1_POWERS[42] = truncate( $x_1^{42}$ )`; point set 0, so no x_4 factor) into the linearization MSM directly. This is a *deviation in structure, not in value*: the product `batch·lin_com` is identical, just computed in one MSM instead of two. The `SELECTOR_ACC[j]` slots are the generated form of the Rust `grouped_points` map — built during the VM’s `FOLD_SELECTOR` steps (and directly by the Yul prefix and the `0x1b` native callbacks) as $\sum_k y^k \cdot \text{eval}$ per simple-selector column.

Audit payoff. With this table, a reviewer can confirm the 14 linearization pairs against the Rust function in minutes: count the 4 quotient limbs, check each scalar is the prior times `x_split`, and check each of the 10 selector scalars is the batch scalar times the bucket accumulator. The remaining 64 pairs are checked against the multi-open (`q_coms` · truncated x_1^i , then truncated x_4^j per point set, `poly/kzg/mod.rs:399–402, 488–508`), so the 78 pairs stop being an opaque blob and become two checkable expansions.

11.8.4 The per-identity trace manifest already exists in the codegen

The single most useful audit aid — a mapping from each opaque VM identity to its named Rust source — is *already implemented* in the codegen as a public API type, `QuotientIdentityManifest` (`src/api.rs:189`), built by `quotient_identity_manifest` (`lowering/diagnostics.rs:74`) and exposed on the builder (`builder/api.rs:121`). It has simply not been emitted alongside the render. The recommendation in §11.8 is therefore a wiring task, not a design task. **[PARTIAL]** host API only; no rendered file or comment block in HEAD or B64.

Each manifest entry carries a stable `global_index` (its position in the global *y*-batch) and a typed source:

```
// api.rs: the manifest entry
pub struct QuotientIdentityManifestEntry {
    pub global_index: usize,          // position in the global y-batch
    pub source: QuotientIdentitySource,
    pub target: QuotientIdentityManifestTarget,
}

pub enum QuotientIdentitySource {
    Gate      { gate_index, gate_name, constraint_index,
                constraint_name, polynomial_index },
    Permutation { identity_index },
    Lookup     { identity_index, lookup_index, lookup_name },
    Trash      { trash_index, trash_name },
}

pub enum QuotientIdentityManifestTarget {
    Main,
    Selector { selector_index, fixed_column },
}
```

The manifest is laid out in four contiguous blocks (`gate_entries` → `permutation_base` → `lookup_base` → `trash_base`), so a given `global_index` falls in exactly one block and resolves to a human-readable name — the gate’s `create_gate` name and `constraint_name`, or `lookup_i`, or the trash gate’s name. The VM checks its own execution manifest (`execution_manifest`, `vm/mod.rs:382`) against the expected identity stream at `vm/mod.rs:450` (`actual.global_index != expected.meta.global_index`, or a source/target mismatch → error); since the diagnostic manifest is built from the same lowering plan, the two cannot silently drift.

Audit payoff. With the manifest emitted, every `FOLD_SELECTOR` boundary and native callback in the disassembly (§11.8.2) can be annotated with the *named* Rust identity it checks — e.g. `[000d] FOLD_SELECTOR 0x020001 → gate name, constraint i`. The opaque identity stream becomes a named checklist that maps one-to-one to the circuit’s gates, lookups, and trash arguments. This is the highest-leverage audit-facilitation change available, and the data is already computed.

11.8.5 VK payload layout: the data-interface audit

The VK contract and the verifier share a fixed, tagged data interface: the payload is a sequence of 32-byte words, each tagged in the VK source, loaded into `VK_MPTR` by `extcodecopy`. A mismatch here is silent and catastrophic, so the verifier *cross-checks* seven header words against its own constants before parsing the proof (B64 lines 934–941: six from commit `e64f2891`, the certificate word from the M7 follow-up line, `e7f736c4 / 7bf45715`).

Header layout (fixed prefix). The first words of the `fee_12345` payload:

0x0000	<code>vk_digest</code>	0x13368f76...203e509f
0x0020	<code>num_instances</code>	19
0x0040	<code>k</code>	21
0x0060	<code>n_inv</code>	0x73eda3b3...00000801
0x0080	<code>omega</code>	0x47c8b581...2d3ea03a

0x00a0	omega_inv	0x5f3ed180...8f26d89e
0x00c0	omega_inv_to_1	0x6a29fe31...999fa4bf
0x00e0	has_accumulator	1
0x0100	acc_offset	11
0x0120	num_acc_limbs	7
0x0140	num_acc_limb_bits	56
0x0160	quotient_manifest_hash	0xd3ddedb9...41050c0d

The bulk sections follow in payload order: the G1/G2 bases (g : 12 words; neg_s_g : 8), `quotient_const` (178), `quotient_program` (143, from payload offset 0x1a40, hence `q_program_mptr` = 0x4140), `fixed_comms` ($108 = 27 \times 4$) and `permutation_comms` ($72 = 18 \times 4$): 533 words = 17,056 bytes = `EXPECTED_VK_PAYLOAD_LENGTH`.

The verifier’s drift check. Immediately after loading, the verifier asserts each header word equals its compiled-in constant:

```

success := and(success, eq(mload(NUM_INSTANCES_MPTR), 19))
success := and(success, eq(mload(K_MPTR), 21))
success := and(success, eq(mload(HAS_ACCUMULATOR_MPTR), 1))
success := and(success, eq(mload(ACC_OFFSET_MPTR), 11))
success := and(success, eq(mload(NUM_ACC_LIMBS_MPTR), 7))
success := and(success, eq(mload(NUM_ACC_LIMB_BITS_MPTR), 56))
success := and(success, eq(mload(QUOTIENT_CERTIFICATE_HASH_MPTR),
    EXPECTED_QUOTIENT_CERTIFICATE_HASH_WORD))
if iszero(success) { revert(0, 0) }
```

The comment states the purpose exactly: “*catch generator drift before calldata parsing chooses a stale schema.*” Codehash pinning protects the VK *runtime bytes*; these word-level checks confirm that the pinned bytes match the verifier’s compiled-in *schema*.

The manifest-hash binding. **[CORRECTED]** The `quotient_manifest_hash` header word is a *quotient certificate* hash over the quotient plan, VM bytecode, constant table and sorted simple selectors (`quotient_certificate_hash_from_parts`), not the hash of `QuotientIdentityManifest`. The verifier never hashes the loaded program on-chain; it only compares the stored word with `EXPECTED_QUOTIENT_CERTIFICATE_HASH_WORD`. Since the codehash pin already covers every VK byte, this adds no runtime tamper resistance; its value is to name the cause of generator drift and to give reviewers an off-chain certificate to recompute. The check is not in HEAD’s (`main@6b3d2096`) templates: it comes from the M7 follow-up line (`e7f736c4 / 7bf45715`, not on `main`), from which B64 was rendered.

Audit payoff. The tagged payload + header cross-check means a reviewer can verify the VK/verifier interface by reading two lists: the VK’s tagged words and the verifier’s `eq(mload(...), const)` assertions. Only 7 of the 12 header words are asserted; `vk_digest`, `n_inv`, `omega`, `omega_inv` and `omega_inv_to_1` are covered only by the codehash pin. This is a concrete, checkable contract between the two generated files — the kind of interface that is usually invisible and therefore a common source of silent bugs, but here is explicit and pinned.

11.8.6 Accumulator public-input decode: a soundness-critical path

The accumulator public inputs arrive as a *shifted, packed* limb encoding (7 limbs $\times$ 56 bits, 4 limbs per 256-bit word). Decoding them into EIP-2537 G1 slots is the single most soundness-critical decode in the verifier: a bug here admits a forged point. The generated decode layers six independent checks.

1. **Packing canonicity** (`check_acc_coord_packing`). Each word’s unused high bits must be zero, else the same coordinate has multiple calldata encodings. Every packed word is bounded by `lt(word, shl(used_bits, 1))` (line 656): 2^{224} for the 4-limb word, 2^{168} for the 3-limb tail.

2. **Identity-flag detection** (`allow_id`). Only `x` may carry the point-at-infinity flag; `y` decodes as a normal shifted coordinate. A cheap prefilter (`calldataload(src) >= base`) gates the flag probe.
3. **Field-range check**. The decoded coordinate must be $\leq p - 1$ in split hi/lo form: `or(lt(hi,BLS_P_HI), and(eq(hi,BLS_P_HI), iszero(gt(lo,BLS_P_MINUS_ONE_LO))))`.
4. **The $p-1 \rightarrow 0$ shift**. The codec maps encoded $p-1$ to decoded zero; all other coordinates are `coord-1`, so one is added back with carry into the high word.
5. **The (0,0) rejection**. EIP-2537 reserves affine (0,0) for the identity. A decoded (0,0) that did *not* come from the canonical identity encoding is rejected: `ok := and(ok, iszero(decoded_zero))`.
6. **128-bit high-word bound**. EIP-2537 pads each 48-byte $\mathbb{F}_p$ coordinate to 64 bytes, so `lt(hi, shl(128,1))` catches reconstructions above 384 bits.

Defense-in-depth, not redundancy. Each check closes a distinct forgery class: (1) non-canonical packing, (2) a bogus identity flag on `y`, (3) an out-of-field coordinate, (4) a mis-shifted decode, (5) a forged (0,0) that bypasses the canonical identity path, (6) an over-width reconstruction. The identity path itself is gated twice — the canonical all-zero encoding is preferred first (`is_acc_encoded_identity`), and the coordinate-level flag path additionally requires *both* coordinates to be zero after shifting (line 750), so a half-identity is rejected.

Curve/subgroup check deferred, not skipped. The decode does *not* check the point is on-curve; that is deferred to the G1MSM that routes the decoded point (the EIP-2537 MSM subgroup-checks its inputs). This is the same invariant as the transcript G1 reads (§9.1): decode for canonical form, MSM for group membership. The split is deliberate and documented in the function header.

Audit payoff. The six checks are individually small and locally checkable, and together they close the decode’s forgery space. A reviewer can verify each against the coordinate-codec spec without tracing the whole verifier. This is the strongest example in the artifact of *defense-in-depth done right* — and the place where a reviewer should spend the most scrutiny, because it is the one decode that turns attacker-controlled public-input bytes into a pairing operand.

11.8.7 Worked example: final pairing assembly — the terminal soundness gate

Every check in the verifier ultimately collapses into one pairing equation. This is the gate that decides accept/reject, so it deserves a full worked trace.

The KZG identity. The verifier must check

$$e(\underbrace{[\text{final}] - v G + x_3 \pi}_{\text{commitment side}}, [1]_2) = e(\pi, [s]_2), \quad (11.1)$$

where `final_com` is the fused 78-pair MSM output (§11.8.3), v is the multi-open evaluation $v = \sum_i x_4^i q_{\text{eval},i} + x_4^5 f_{\text{eval}}(\mathbf{v_MPTR}, \text{lines } 3292\text{--}3297)$, x_3 the (truncated) f_{com} evaluation point, and π the KZG opening-proof commitment (the last proof G1, line 1300). **[CORRECTED]** (earlier text called v the instance evaluation and π the public-input commitment.) The `ec_pairing` helper checks $e(\text{arg0}, [1]_2) \cdot e(\text{arg1}, -[s]_2) = 1$, so `arg0` must be the commitment side and `arg1` must be π .

The naming trap (third instance). The slots are named for the *dual-MSM accumulator* (left = π , right = combined), *not* the pairing argument order:

```
PAIRING_LHS_MPTR := pi
PAIRING_RHS_MPTR := final_com - v*G + x3*pi
```

so the call passes them *swapped*: `ec_pairing(success, PAIRING_RHS_MPTR, PAIRING_LHS_MPTR)`. The code comment spells this out (lines 3555–3567), and the call is at line 3569. This trap has bitten reviewers before, and the comment is the only thing keeping it from being a soundness bug in a future edit.

Assembling the RHS point. Lines 3471–3493 build $[final] - vG + x_3\pi$ with two one-pair G1MSMs and two G1ADDs, reusing scratch 0x80/0x100:

```
mcopy(PAIRING_LHS_MPTR, PI_MPTR, 0x80)          // LHS = pi
mcopy(0x80, G1_BASE_MPTR, 0x80)
mstore(0x100, addmod(0, sub(r, mload(V_MPTR)), r)) // -v
staticcall(gas(), 0x0c, 0x80, 0xa0, 0x80, 0x80) // 0x80 = -v*G
mcopy(0x100, FINAL_COM_MPTR, 0x80)
staticcall(gas(), 0x0b, 0x80, 0x100, 0x80, 0x80) // 0x80 = final - v*G
mcopy(0x100, PI_MPTR, 0x80)
mstore(0x180, mload(X3_MPTR))
staticcall(gas(), 0x0c, 0x100, 0xa0, 0x100, 0x80) // 0x100 = x3*pi
staticcall(gas(), 0x0b, 0x80, 0x100, 0x80, 0x80) // 0x80 = final - v*G + x3*pi
mcopy(PAIRING_RHS_MPTR, 0x80, 0x80)
```

Randomized accumulator batching — not naive multiplication. The IVC accumulator carries its *own* pairing equation. The generated code explicitly refuses to just multiply the two equations:

“We do not simply multiply the two pairing equations together: two bad equations could cancel.”

Instead it derives a verifier-local randomizer over the fully materialized pairing inputs,

$$\alpha = H(\text{"pairing-batch-acc-kzg"} \parallel \text{kzg_rhs} \parallel \text{kzg_lhs} \parallel \text{acc_rhs} \parallel \text{acc_lhs}) \bmod r, \quad \alpha=0 \Rightarrow 1, \quad (11.2)$$

where H is `keccak256` over 0x220 bytes (lines 3514–3523; the 21-byte tag is one `mstored` 32-byte word, i.e. 3 zero bytes, the tag, 8 zero bytes), and checks the combined equation

$$e(\text{kzg_rhs} + \alpha \text{acc_rhs}, G_2) \cdot e(\text{kzg_lhs} + \alpha \text{acc_lhs}, -sG_2) = 1. \quad (11.3)$$

If *either* original equation is bad, Eq. (11.3) holds for at most one $\alpha \in \mathbb{F}_r$ — so a forged proof that satisfies one equation but not the other is rejected except with probability $1/r$. The $\alpha=0 \Rightarrow 1$ guard prevents a zero draw from silently dropping the accumulator equation.

Audit payoff. This is the single most important place to spend human scrutiny, and the generated code makes it checkable: the KZG identity is stated in a comment, the swap is explained, the batching rationale is spelled out, and the randomizer domain is a literal ASCII tag. A reviewer can verify Eq. (11.1) against the Rust `verify_proof` final pairing and confirm the accumulator batching matches Eq. (11.3) without tracing the 3,500 lines that precede it.

11.8.8 Production transcript squeeze-order verification

The Fiat-Shamir binding chain is the backbone of soundness: every challenge must be squeezed over exactly the bytes the Rust prover absorbed, in the same order. A single reordering silently breaks the binding. This subsection verifies the *generated* contract’s `absorb/squeeze` sequence against the Rust `verify_proof` squeeze order line-by-line.

The squeeze order matches. The production contract’s challenge squeezes, in order, are:

```
theta  <- squeeze_to L1095 (after advice commitments)
beta   <- squeeze_to L1113 (after lookup multiplicities)
gamma  <- squeeze_to L1114
trash  <- squeeze_to L1173 (after perm Z + lookup helper/Z commitments)
y      <- squeeze_to L1190 (after trashcan commitments)
x      <- squeeze_to L1213 (after quotient-limb commitments)
x1, x2 <- squeeze_to L1247-1248 (after evals)
x3     <- squeeze_to L1260 (after f_com), then masked to 128 bits
x4     <- squeeze_to L1294 (after q_evals)
pi     absorbed      L1300 (no further squeeze)
```

[CORRECTED] (the earlier listing placed y after f_{com} and x after π .) The Rust squeezes in the *same* order (plonk/verifier.rs: theta L150, beta L167, gamma L172, trash_challenge L194, y L215, x L304; then poly/kzg/mod.rs: x1/x2 L363–364, x3 L455, x4 L484). The generated sequence is a faithful replay.

The trace-ID red herring. The Rust transcript assigns each value a *trace ID* that does *not* follow squeeze order: trash_challenge is squeezed 4th but carries ID 12, while y (5th) and x (6th) carry IDs 10 and 11. This is a **labeling artifact**, not a soundness issue — what binds the proof is the *squeeze order* and the *bytes absorbed before each squeeze*, both of which match. A reviewer who mistakes trace-ID numbering for squeeze order could wrongly flag a non-existent mismatch; the production schedule above is the ground truth.

Pre-instance absorbs also match. Before any challenge, the production absorbs VK_DIGEST (L1008), then the committed-instance identity as 128 zero bytes (L1023–1027), then the instance count (19, L1035), then the 19 range-checked instance words — the same prefix the Rust verifier builds (plonk/verifier.rs:70–103). The VK digest is absorbed *after* the header cross-check (§11.8.5) but the digest itself is never re-absorbed from a mutable slot, so the binding is to the pinned VK.

Audit payoff. A reviewer can verify the entire Fiat-Shamir chain by reading the ordered common_word/common_uncompressed_g1/squeeze_to calls in the generated contract and confirming they match the Rust squeeze sequence. The generated code makes the chain explicit (one call per absorb/squeeze, in order), so the binding is auditable as a single linear read — no hidden state. The one absorb that bypasses the helpers is the committed-instance identity (four raw mstores and a manual buf_len bump, L1023–1027); a reviewer must include it.

11.8.9 Quotient VM ISA coverage and fail-closed dispatch

The codegen defines a 31-opcode quotient ISA (Q_OP_*, vm/mod.rs L560–590), but the fee_12345 program emits only a subset. The production VM’s dispatch is a **switch** over the emitted opcodes with a fail-closed **default**.

ISA vs. emitted subset. The codegen ISA spans 0x01–0x22 with 0x17, 0x18 and 0x1a unassigned (31 opcodes): pushes (PUSH_CONST, PUSH_MEM_*), arithmetic (ADD/MUL/NEG), folds (FOLD_MAIN/FOLD_SELECTOR), constant/memory forms, the ADD_MUL_* family, the native delegates (NATIVE_PERMUTATION 0x19, NATIVE_IDENTITY 0x1b, NATIVE_LOOKUP 0x1f), the limb-aware LIN7/BILIN7_*, POW5, MODARITH7, and AFFINE_SUM. The fee_12345 VM implements 11 cases: 0x05 PUSH_MEM_U16, 0x06 ADD, 0x08 NEG, 0x0b FOLD_SELECTOR, 0x0d MUL_CONST_U8, 0x10/0x11 ADD/MUL_MEM_U16, 0x19/0x1b/0x1f the native delegates (0x1b with its own u16 sub-switch, cases 0–3 plus a reverting **default**, lines 2251–2528), and 0x21 MODARITH7 (dispatch loop lines 1692–2564). These are exactly the opcodes the program uses (§11.8.2): the switch is pruned to the program’s opcode set (quotient_program_usage, vm/mod.rs:1970–1988). The

gap is expected — the ISA is the full language; a given circuit’s quotient program uses only what its identities need.

Fail-closed dispatch. The switch ends with:

```
// Invalid generated bytecode should fail closed. 0x1a intentionally lands here.
default {
    revert(0, 0)
}
```

Any opcode outside the implemented 11 — a malformed byte, a future-incompatible codegen output, or the reserved `0x1a` — reverts rather than silently advancing the PC. This is the correct default: a VM that silently skipped an unknown opcode would let a truncated or tampered program “succeed” by doing less work. The explicit comment that `0x1a` intentionally lands here shows the reserved-opcode space is deliberate, not accidental.

Audit payoff. The ISA coverage is a concrete, checkable property: a reviewer can diff the codegen’s `Q_OP_*` table against the VM’s `case` labels and confirm (a) every emitted opcode has a case, and (b) the `default` reverts. The fail-closed default is the single most important safety property of the VM — it converts “unknown bytecode” from a potential soundness hole into a guaranteed revert. It is complemented by the post-loop asserts `if iszero(eq(q_pc, q_end)) revert` and `if q_has_top revert` (lines 2569–2570, commit 33f78d2b), which catch an operand over-read or a partial expression left live.

11.8.10 Deploy-time smoke tests: catching a broken chain before first proof

The constructor runs `require_eip2537_precompiles()` (lines 203–254) before checking and storing the VK address, exercising the Cancun/EIP-2537 runtime prerequisites (MCOPY probe: commit b53e3ebe; full-size shapes: 2b2bf494) so an incompatible chain/fork reverts *at deployment* rather than on the first real proof. This is a deployment-safety property worth documenting because it converts a class of runtime failures into a deploy-time guarantee.

The four probes.

1. **MCOPY availability.** `mstore/mcopy/ mload` round-trip of `0x1234`. A non-Cancun fork reverts here because the verifier relies on MCOPY for proof-time staging.
2. **G1ADD identity round-trip.** `G1ADD(id, id)` must return the 128-byte identity. Catches a missing precompile or a non-standard success shape.
3. **Worst-case G1MSM.** A `0x30c0`-byte all-zero MSM (the *full 78-pair length* the verifier actually uses) must return identity. This is the key probe: it exercises the *largest* MSM input the verifier emits, not a one-pair smoke call, so a chain whose precompile exists but cannot handle the real input size fails at deploy.
4. **PAIRING two-pair check.** `PAIRING_CHECK([(id,id),(id,id)])` must return `true` with a 32-byte return — matching the runtime two-pair KZG pairing input size.

Why the worst-case length matters. A naive smoke test calls the precompile with a minimal input and passes on a chain that later runs out of gas or hits an undocumented size limit on the real 78-pair MSM. By sizing the probe to `0x30c0` (78×160 bytes, the exact fused final-MSM length, §11.8.3), the constructor proves the chain accepts and prices the verifier’s actual worst-case input shape before any user pays for a failed verification. The same logic applies to the pairing probe (two-pair, the real size). Identity/zero inputs exercise shape, pricing and return length, not arithmetic on real points.

Audit payoff. The smoke tests are a compact, readable constructor block that a reviewer can confirm covers the EIP-2537 precompiles the verifier calls at runtime (G1ADD, G1MSM, PAIRING) plus MCOPY, each at its largest runtime input size. **[CORRECTED]** It does *not* probe MODEXP (0x05, six runtime calls); that is low-risk (EIP-198, Byzantium) but means “every runtime precompile” is not literally true. This is the kind of defense-in-depth that is easy to under-test; here it is explicit and sized correctly.

11.8.11 Lagrange basis evaluation: the multiplicative-subgroup math in code

The permutation argument needs the Lagrange basis $L_i(x)$ on the multiplicative subgroup $H = \{\omega^0, \dots, \omega^{n-1}\}$. The generated code computes the 29 values it needs (rotations $-10 \dots 18$) with a single batch inversion — a textbook math-to-code correspondence worth showing in full.

The formula. For the coset-free subgroup with $\omega^n = 1$,

$$L_i(x) = \frac{x^n - 1}{n \omega_i^{n-1} (x - \omega_i)} = \frac{(x^n - 1) \omega_i}{n (x - \omega_i)}, \quad (11.4)$$

where the second equality uses $\omega_i^{n-1} = \omega^{i(n-1)} = \omega^{-i}$ (since $\omega^n = 1$), so $1/\omega_i^{n-1} = \omega_i$. The code’s $l_i_common = (x^n - 1) \cdot n^{-1}$ and the per- i factor $\omega_i/(x - \omega_i)$ match Eq. (11.4) exactly.

Batch inversion of all denominators. Rather than invert each $(x - \omega_i)$ separately, the code writes every denominator *and* $x^n - 1$ into a contiguous block and inverts the whole set in one `modexp` (line 1346):

```
// First pass writes denominators (x - omega_i) for every
// Lagrange value needed below, then appends x^n - 1. The
// batch inversion pass turns all of them into inverses in one
// modexp call.
for { let pow_of_omega := mload(OMEGA_INV_TO_L_MPTR) } ... {
    mstore(mp_ptr, addmod(x, sub(r, pow_of_omega), r)) // x - omega_i
    pow_of_omega := mulmod(pow_of_omega, omega, r)
}
mstore(mp_ptr_end, x_n_minus_1)
success := batch_invert(success, X_N_MPTR, ..., r)
```

This is the Montgomery-trick amortization: 29 denominators (0x3a0 bytes) plus $x^n - 1$, i.e. 30 inversions for the price of one `modexp`. The rotation seed `OMEGA_INV_TO_L` = ω^{-10} starts the walk at the correct ω -power so the loop emits the exact ω_i sequence the formula needs.

The derived scalars. After inversion the code forms $L_i(x)$, the blinding sum, and the instance evaluation:

- $l_last = L_{-(b+1)}(x) = L_{n-10}(x)$ (slot 0; $b = 9$ blinding rows), $l_0 = L_0(x)$ (slot 10) — the boundary Lagrange terms used by the permutation first/last-row checks. **[CORRECTED]** (was L_{n-1} ; Rust uses `l_i_range(x, xn, -(b+1)..=0)`, `plonk/verifier.rs:444-449`.)
- $l_blind = \sum_{j=1}^b L_{-j}(x)$ over the $b = 9$ *negative-rotation* terms (slots 1–9) — the midnight-proofs blinding identity (line 1361–1367).
- $instance_eval = \sum_i L_i(x) \cdot pub_i$ — the public-input polynomial evaluated at x (line 1380), formed only after the instance words were range-checked and absorbed in transcript order.

Audit payoff. The Lagrange computation is a direct rendering of Eq. (11.4): a reviewer can check the ω_i walk, the batch-inversion block, and the three derived scalars against the Rust `EvaluationDomain::l_i_range` (`poly/domain.rs:448`, called at `plonk/verifier.rs:345` and `:444`) without any obscure transformation. The batch inversion is the one non-obvious step, and its comment states the intent (“one `modexp` call”) so the amortization is visible rather than hidden.

11.8.12 Truncated powers: the 128-bit challenge reduction (faithful but flagged)

Three of the PCS challenges are *truncated to 128 bits* before use: `x3` itself (masked right after its squeeze, line 1270), and every stored power of `x1` and `x4` (the running products stay full-width). `x2` is used full-width, as in Rust (`poly/kzg/mod.rs:480`). **[CORRECTED]** (earlier text listed `x2` as truncated.) This is a deliberate, faithful translation of a Rust `truncate` function — but one whose own doc comment warns about the security margin. It is the single most important security-target item in the review and deserves the exact correspondence on the page.

The Rust source. Under the `truncated-challenges` feature (`utils/arithmetic.rs` L246–257):

```
pub(crate) fn truncate<F: PrimeField>(scalar: F) -> F {
    let nb_bytes = F::NUM_BITS.div_ceil(8).div_ceil(2) as usize;
    let bytes = scalar.to_repr().as_ref()[..nb_bytes].to_vec();
    let bi = BigUint::from_bytes_le(&bytes);
    F::from_str_vartime(&BigUint::to_string(&bi)).unwrap()
}
pub(crate) fn truncated_powers<F: PrimeField>(base: F)
    -> impl Iterator<Item = F> { powers(base).map(truncate) }
```

For the BLS12-381 scalar field (255 bits) this keeps the lower $32/2 = 16$ bytes = 128 bits of the little-endian representation.

The Solidity translation. The generated code masks each x_1 power to 128 bits (line 2918; the x_4 powers use the same pattern unrolled at lines 3281–3291, and x_3 is masked at line 1270):

```
acc := mulmod(acc, x1, r)
mstore(p, and(acc, 0xffffffffffffffffffffffffffffffff))
```

The `and` with $2^{128}-1$ is exactly the Rust `truncate`: retain the low 128 bits, discard the high 128. The correspondence is exact — the codegen lowers `truncated_powers` to a `mulmod` + `and` mask.

Security analysis (the Rust’s own warning). The Rust doc comment states it plainly: “128 bits may not be enough entropy depending on the application. For example, it makes a collision attack feasible with 2^{64} memory and $\sim 2^{64}$ operations.” The verifier is *faithful* to the Rust (the same truncation happens on both sides, so proofs verify), but the soundness of the *system* is set by the truncated challenges. No written soundness argument for this configuration exists in the repository, and this chapter does not supply one. **[CORRECTED]** Bit-level figures quoted in earlier drafts are withdrawn.

Assessment. This is not a translation bug — it is a **security-target decision** inherited from the Rust. The recommendation (§10.1) stands: if the deployment targets 128-bit soundness, the `truncated-challenges` feature should be disabled (full-width challenges) or the truncation re-examined against the actual threat model. The generated code is correct; the question is whether the truncation is *intended* for this deployment. **[OPEN]** the feature is enabled for this render (`solidity-verifier/Cargo.toml` feature `truncated-challenges`); disabling it must be done on the prover and verifier together.

11.8.13 Instance canonical-range check: where Solidity is *more* defensive than Rust

The public-instance range check is a rare case where the generated Solidity adds a guard the Rust verifier does not need — and the reason is the untyped ABI boundary.

The Rust gets canonicity for free. The Rust verifier signature takes instances as typed field elements:

```
instances: &[&[F]],
```

An `F` is a canonically-reduced scalar-field element by construction — the type system guarantees every value is already $< r$. There is no `lt(r)` check in the Rust because a non-canonical instance simply cannot be represented.

The Solidity must check it explicitly. The generated verifier receives instances as raw `uint256[] calldata` — untrusted 256-bit words that may be any value, including $\geq r$. So the codegen inserts a canonical-range gate before transcript absorption (L1045), and since commit 1e21311c the loop is followed immediately by `if iszero(success) { revert(0, 0) }` (L1050), so a bad instance fails before any proof bytes are parsed:

```
let inst_be := calldataload(instance_cptr)
// Public inputs are BLS12-381 scalar-field elements. They
// must be canonical before transcript absorption; accepting
// non-canonical encodings would admit transcript aliases.
success := and(success, lt(inst_be, r))
buf_len := common_word(buf_len, inst_be)
```

The threat: transcript aliasing. If the verifier absorbed a non-canonical instance $v \geq r$ without checking, the transcript would hash the raw bytes of v , but the field arithmetic downstream would use $v \bmod r$. Each field element would then have two or three accepted `uint256` encodings ($2^{256}/r \approx 2.2$), each with its *own* challenge sequence. A malicious prover can run the prover over a transcript that absorbs $v + r$ and obtain a proof that verifies for the word $v + r$. An application that keys on raw instance words (a nullifier, a replay key, a state root compared with `==`) would then see a “new” value for the same statement. The `lt(inst_be, r)` gate closes this by rejecting any non-canonical encoding before it reaches the transcript.

Assessment. This is a [OK] finding and a point in the generated code’s favor: the codegen correctly recognizes that the Rust’s type-system guarantee does not survive the ABI boundary, and adds the explicit check the Rust doesn’t need. The check is cheap (one `lt` per instance). It is folded into the running `success` flag and enforced by the `revert` at L1050, so it cannot be skipped. (The later `if iszero(success)` at L1316 and its comment “carries deferred canonicity failures from public instance reads” are now redundant and stale.) A reviewer cross-checking against the Rust should note that the absence of a range check in the Rust is *not* a missing check in the Solidity — it is the Solidity compensating for a weaker input type.

11.8.14 G1 point validation: padded-uncompressed design coupling and deferred subgroup checks

The G1 transcript-encoding is a case where the Rust and Solidity were co-designed: the Rust serializes points into a form that lets the EVM verifier skip expensive work, and the Solidity’s checks are named in the Rust’s own comment.

The Rust serialization. `Hashable<Keccak256>` for `G1Projective::to_input` (implementors.rs L283–298) converts to affine, takes the 96-byte uncompressed $x_{be}(48) \parallel y_{be}(48)$, and lays it out as 128 bytes with a 16-byte zero pad per coordinate:

```
let mut out = vec![0u8; 128];
if !bool::from(aff.is_identity()) {
    let raw = G1Affine::to_uncompressed(&aff); // x_be(48)||y_be(48)
    let bytes: &[u8] = raw.as_ref();
```

```

    out[16..64].copy_from_slice(&bytes[0..48]); // x in 64-byte slot
    out[80..128].copy_from_slice(&bytes[48..96]); // y in 64-byte slot
}
out // identity = 128 zero bytes

```

The design intent (Rust’s own words). The Rust comment states the coupling explicitly: *”This format lets the EVM verifier skip the 384-bit sign-bit ladder it would need if we were hashing the 48-byte compressed encoding; the EVM can just memcpy the calldata words verbatim into the keccak buffer (after masking the 16-byte zero pads to defeat transcript malleability).”* The “masking the 16-byte zero pads” corresponds to the Solidity’s high-bits check, with one difference: the Solidity *rejects* a non-zero pad rather than masking it. Rejecting is the correct choice; B64 L382–385 notes that normalizing the pad before hashing would let several calldata encodings share one transcript. (The Rust comment still names the helper `common_g1_uncompressed`; the emitted name is `common_uncompressed_g1`.)

The Solidity validation. `common_uncompressed_g1` (L396–417) performs three checks before the verbatim memcpy:

1. **High-bits-zero** (L401–402): `shr(128, x_hi_word)` must be zero — the top 128 bits of each high word must be the zero pad. This defeats transcript malleability: a non-zero pad would encode a different transcript than the canonical padded form.
2. **Coordinate-canonical** (L406–411): each coordinate must be $\leq p - 1$ via a split hi/lo comparison `lt(hi, BLS_P_HI) || (eq(hi, BLS_P_HI) && lo ≤ BLS_P_MINUS_ONE_LO)` — i.e. `coord < p`, a canonical base-field element.
3. **Verbatim memcpy** (L415): `calldatacopy(buf_len, cptr, 0x80)` — the 128 calldata bytes go into the keccak buffer unchanged, matching the Rust’s padded layout.

Deferred on-curve / subgroup checks. Notably, `common_uncompressed_g1` does *not* check that the point satisfies $y^2 = x^3 + 4$ or lies in the prime-order subgroup. That check is **deferred to the EIP-2537 precompiles**: the comment at L238–239 states *”the production verifier uses G1MSM both for commitments and as the subgroup validator for absorbed proof points.”* Per EIP-2537, G1MSM and PAIRING enforce subgroup membership (rejecting non-subgroup points), whereas G1ADD does not.

Why the lazy check is sound. Every G1 point absorbed into the transcript is eventually consumed by a G1MSM (the fused linearization) or the PAIRING call — both of which subgroup-check. A malicious non-subgroup point absorbed into the transcript would pollute the challenge derivation, but the subsequent G1MSM/PAIRING reverts, so verification fails rather than falsely accepting. The transcript pollution is harmless because the bad point cannot pass the point-of-use check. This is a deliberate **lazy-validation** design: the transcript absorbs the canonical *encoding* (cheap coordinate checks), and the expensive on-curve/subgroup check happens at the precompile. All 33 absorbed commitments other than π go through the fused 78-term G1MSM (L3460); π is validated by the one-pair G1MSM computing $x_3\pi$ (L3486) and by the pairing. The generator enforces this coverage: see §11.8.22.

Assessment. This is a [OK] finding and a strong example of the three-way correspondence: the Rust’s serialization format is chosen to make the Solidity’s memcpy cheap, the Rust comment names the Solidity’s malleability defense, and the subgroup check is correctly deferred to the precompiles that enforce it. A reviewer should note that the absence of an explicit on-curve check in `common_uncompressed_g1` is *not* a missing check — it is a deliberate deferral to the G1MSM/PAIRING subgroup enforcement.

11.8.15 Triple-layered proof-length validation: declared size, total calldata, and walk-cursor end-check

The proof blob’s length is validated three independent ways before any cryptographic work — a defense-in-depth gate that catches a wrong-shaped proof at three different points. Two earlier guards sit underneath them. `solc`’s own ABI decoder for the `bytes calldata/uint256[] calldata` arguments already reverts if either `offset + length` runs past `calldatasize()` (checked in the via-IR output). The generated head check at L305 pins the two ABI head words, which requires the proof head to be `0x40` and the instances head to be `NUM_INSTANCE_CPTR-4`. Without that pin, the fixed cursors below would not point at the length words.

Layer 1 — declared proof length (L954). The proof `bytes` length word must equal the generated constant:

```
success := and(success, eq(0x1e60, calldataload(PROOF_LEN_CPTR)))
success := and(success, eq(19, calldataload(NUM_INSTANCE_CPTR)))
```

The proof must be exactly `0x1e60` (7776) bytes and the instance count exactly 19 — the shape this verifier was generated to parse.

Layer 2 — total calldata size (L961). The whole calldata must be exactly the selector + proof + instance-array length + instance words, with *no trailing bytes*:

```
// Calldata must contain exactly the ABI selector, proof bytes,
// instance-array length, and generated number of instance
// words. Any trailing bytes fail closed.
success := and(success, eq(calldatasize(), add(INSTANCE_CPTR, 0x0260)))
```

This rejects trailing bytes smuggled past the instance array — bytes that a sloppier parser might misread as part of the proof or instances.

Layer 3 — walk-cursor end-check (L1312). After the verifier walks every proof component (advice, lookup, permutation-Z, trash and quotient G1s, the evaluation and q -eval scalars, f_{com} and the final π), the cursor must land *exactly* on the instance-array boundary:

```
// This is redundant with the generated proof length today, but
// makes future proof-layout drift fail closed.
// ... If proof_cptr lands anywhere else,
// some section was under-read or over-read.
if iszero(eq(proof_cptr, NUM_INSTANCE_CPTR)) { revert(0, 0) }
```

It confirms that the sum of all the per-component byte advances ($0x780 + 0x100 + 0x300 + 4 \cdot 0x80 + 0x80 + 0x200 + 0xcc0 + 0x80 + 0xa0 + 0x80 = 0x1e60$) equals exactly the declared proof length, so *no component was skipped or double-counted by the generator*. **[CORRECTED]** This is not a check on proof data. Every advance is a compile-time constant, so the walk is data-independent and no proof can be “mis-tiled”. Once Layer 1 holds, Layer 3 is a tautology at runtime, as the emitted comment (L1304–1307) itself says. Its value is as a fail-closed assertion against *codegen drift*, the same class of guard as the generator’s own layout assertions (`abi/proof.rs`).

Why three layers. Each layer catches a different class of malformed proof:

- Layer 1 catches a wrong *declared* length (truncated or padded proof bytes).
- Layer 2 catches *trailing* bytes beyond the declared structure.
- Layer 3 catches a *generator* bug in which the emitted walk’s constant advances no longer sum to the emitted `0x1e60` (a skipped or duplicated section). No calldata can trigger it, because the walk is data-independent.

Layers 1–2 are cheap pre-checks that fail before any transcript work. Layer 3 is an end-check that ties the *generated* walk to the *generated* length. Layers 1–2 alone stop a wrong-shaped proof from reaching the pairing; Layer 3 guards against the generator emitting an inconsistent parser.

Assessment. This is a **[OK]** finding — a defense-in-depth length gate. The runtime checks on proof data are the two layers plus the ABI head/decoder guards. The third layer is a codegen-consistency assertion, and all of them revert before cryptographic work. A reviewer can confirm the proof layout by summing the per-component advances and checking they equal `0x1e60` and land on `NUM_INSTANCE_CPTR` — a concrete arithmetic cross-check against the Rust proof serialization.

11.8.16 Multi-layered VK pinning: constructor, per-proof codehash re-check, and the INVALID-byte data-only trick

The verifier binds itself to a specific verifying-key runtime through a layered pinning scheme that defends against VK-swap attacks at three points, plus a clever data-only encoding of the VK itself.

Layer 1 — constructor pin (L260–271). At deployment the constructor refuses to store a VK whose code doesn’t match the generated constants:

```
constructor(address authorizedVk) {
    require_eip2537_precompiles();
    require(
        authorizedVk.code.length == EXPECTED_VK_LENGTH
        && authorizedVk.codehash == EXPECTED_VK_CODEHASH,
        "invalid vk"
    );
    AUTHORIZED_VK = authorizedVk;
}
```

The VK’s *length* and *codehash* are both pinned, so a different VK runtime (even one of the same length) is rejected at construction.

Layer 2 — per-proof codehash re-check (L911–923). On *every* proof, the verifier re-checks the VK’s code size and hash before trusting it:

```
// Re-check the pinned VK dependency on every proof. The
// constructor check catches normal deployment mistakes, while
// this fresh check hardens forks or same-transaction edge
// cases where code at the authorized address could differ
// from the runtime originally pinned by this verifier.
if iszero(and(
    eq(extcodesize(vk), EXPECTED_VK_LENGTH),
    eq(extcodehash(vk), EXPECTED_VK_CODEHASH_WORD)
)) { revert(0, 0) }
```

This is the defense against a VK-swap. `AUTHORIZED_VK` is an immutable address, and after EIP-6780 its code can change only in narrow cases. One is a VK that was created, `SELFDESTRUCT`d and re-created via `CREATE2` within its creation transaction. The other is an irregular state change on a fork. In either case the per-proof `extcodehash` check catches the change before any payload is read. **[CORRECTED]** The earlier examples (“chain-reorg”, “malicious proxy”) do not apply. A reorg replays the constructor check. A proxy’s own runtime never changes, and `extcodecopy` reads the proxy’s code, not its implementation’s.

Layer 3 — the INVALID-byte data-only trick (L924–927). The VK runtime’s byte 0 is the `INVALID` opcode (`0xFE`), so the VK contract *cannot be called to execute its payload* — it is a data-only contract. The verifier reads the payload by skipping byte 0:

```
// Runtime byte 0 is INVALID so direct calls cannot execute the
// payload. Copy from byte 1 into VK_MPTR to reconstruct the
// exact payload layout used by the embedded branch.
extcodecopy(vk, VK_MPTR, 0x01, EXPECTED_VK_PAYLOAD_LENGTH)
```

This means the VK’s codehash commits to a contract that is inert as executable code — the only way to consume the payload is the `extcodecopy` the verifier performs, closing a class of “call the VK to do something unexpected” attacks.

Layer 4 — header cross-check (L934–940). After loading the payload, the verifier cross-checks the VK’s header words against its *own* compiled constants:

```
success := and(success, eq(mload(NUM_INSTANCES_MPTR), 19))
success := and(success, eq(mload(K_MPTR), 21))
success := and(success, eq(mload(HAS_ACCUMULATOR_MPTR), 1))
success := and(success, eq(mload(ACC_OFFSET_MPTR), 11))
success := and(success, eq(mload(NUM_ACC_LIMBS_MPTR), 7))
success := and(success, eq(mload(NUM_ACC_LIMB_BITS_MPTR), 56))
success := and(success, eq(mload(QUOTIENT_CERTIFICATE_HASH_MPTR),
    EXPECTED_QUOTIENT_CERTIFICATE_HASH_WORD))
```

The comment states the purpose: *“Codehash pinning protects the external VK address; these checks catch generator drift before calldata parsing chooses a stale schema.”* Even if a VK with the right codehash somehow had wrong header values, the cross-check catches the mismatch before the parser acts on it.

Why four layers. Each defends a different threat:

- Layer 1 (constructor): wrong VK at deployment.
- Layer 2 (per-proof codehash): VK changed after deployment (fork / same-transaction `SELFDESTRUCT + CREATE2`).
- Layer 3 (INVALID byte): VK payload can’t be executed as code, only read as data.
- Layer 4 (header cross-check): generator drift between the VK payload and the verifier’s schema.

Together they make a VK-swap or VK-drift attack require changing the codehash (caught by L1–2), or changing the payload without changing the codehash (impossible), or executing the payload as code (blocked by L3), or drifting the header (caught by L4).

Assessment. This is a **[OK]** finding — a thorough, layered VK-pinning scheme. The per-proof `extcodehash` re-check (L2) is the standout: it’s a defense-in-depth measure that costs one `extcodesize` and one `extcodehash` per proof (plus the `extcodecopy`) and closes the residual post-deployment VK-swap window. The INVALID-byte data-only trick (L3) is an elegant encoding that makes the VK inert as code. A reviewer can confirm the pinning by checking the four constants (`EXPECTED_VK_LENGTH`, `EXPECTED_VK_CODEHASH_WORD`, the INVALID byte at offset 0, and the header cross-checks) against the deployed VK contract.

11.8.17 Transcript initialization and domain separation: empty start with the VK digest as the separator

A natural question for a Fiat-Shamir transcript is whether it carries an explicit domain-separator (a protocol label like “Halo2-v1”) so that transcripts from different protocols can’t be reused. The answer here is subtle and worth documenting.

The transcript starts empty. `transcript_init` (L354–357) sets the buffer to the empty state:

```
function transcript_init() -> buf_len {
    // Empty transcript buffer starts exactly at TRANSCRIPT_MPTR.
    buf_len := TRANSCRIPT_MPTR
}
```

There is **no explicit protocol-label domain separator** absorbed at init. This is faithful to the Rust: the Rust `Keccak256::init()` (`implementors.rs` L191) is `Keccak256::new()` — an empty Keccak state.

Domain separation comes from the VK digest. The first thing absorbed into the transcript is the verifying key itself. The Rust verifier calls `vk.hash_into(transcript)` as its first action (`verifier.rs` L70), and the trace records it as trace ID 1 with the label `"vk_digest"`:

```
// Hash verification key into transcript
vk.hash_into(transcript)?;
// trace ID 1 = "vk_digest"
```

Because every VK has a unique digest (committing to its circuit shape, fixed commitments, and program), the VK digest acts as an *implicit* domain separator: transcripts under different VKs diverge from the very first absorb, so a proof made for one VK can never be replayed against another.

The delegation model. The verifier’s own doc comment (L275–278) is explicit that richer domain separation is *delegated* to the application layer:

“Application contracts must bind the meaning of those instances separately: state roots, program identifiers, expected IVC outputs, chain/domain separation, and any protocol-specific authorization are outside this raw verifier ABI.”

So the raw verifier provides domain separation at the *VK* level (via the digest) but not at the *protocol/chain* level — the application contract is responsible for binding the instances to a specific chain, program, or IVC context.

Assessment. This is a **[NOTE]** finding — faithful to the Rust (empty start + VK-digest-first), but a point a reviewer should understand rather than assume. The absence of an explicit protocol-label separator is *not* a missing check (the VK digest provides VK-level separation), but it does mean the raw verifier is a **VK-scoped** primitive, not a protocol-scoped one. If the deployment needs cross-chain or cross-program transcript separation beyond what the VK digest provides, the application layer must add it (e.g. by binding a chain-id or program-hash into the instances). This is a design boundary, not a defect — but it’s the kind of assumption that should be stated explicitly in a security review, because a deployer who assumes the verifier provides chain-domain-separation would be mistaken.

11.8.18 The airtight terminal return path: revert-on-failure pairing + pre-check

The single most important soundness question in any verifier is: *can the contract ever return `true` on an invalid proof?* The terminal return path (L3568–3576) answers this, and the answer hinges on a subtle but load-bearing property of the `ec_pairing` helper.

The terminal sequence. The last lines of the verifier are:

```

if iszero(success) { revert(0, 0) }
success := ec_pairing(success, PAIRING_RHS_MPTR, PAIRING_LHS_MPTR)
// Success path is terminal. Invalid inputs have already reverted,
// so the Solidity ABI observes `true`.
mstore(RETURN_MPTR, 1)
return(RETURN_MPTR, 0x20)

```

At first glance this looks dangerous: `mstore(RETURN_MPTR, 1)` writes `true` with **no check after the `ec_pairing` call**. If `ec_pairing` returned `false` on a failed pairing, the verifier would return `true` anyway — a catastrophic soundness break.

Why it’s safe: `ec_pairing` reverts, it doesn’t return `false`. The `ec_pairing` helper (L534) is a *revert-on-failure* helper, not a boolean-returning one:

```

function ec_pairing(success, lhs_mptr, rhs_mptr) -> ret {
    ret := success
    if iszero(ret) { leave }
    // ... lay out two (G1, G2) pairs via MCOPY ...
    ret := staticcall(gas(), 0x0f, scratch, 0x0300, scratch, 0x20)
    ret := and(ret, eq(returndatasize(), 0x20))
    ret := and(ret, mload(scratch))
    if iszero(ret) { revert(0, 0) } // <-- reverts on failure
    ret := 1
}

```

The helper validates the pairing result three ways before returning: (1) the `staticcall` to the pairing precompile (0x0f) succeeded, (2) the return data is exactly 32 bytes, and (3) the returned value is `true` (`mload(scratch)`). If *any* of these fails, the helper **reverts** — it never returns `false`. So the only way `ec_pairing` returns is with `ret = 1` (pairing valid) or via the early `leave` when the input `success` was already 0.

The pre-check closes the leave gap. The early `leave` (when input `success = 0`) would return 0 without doing the pairing — which, combined with the unchecked `mstore(1)`, would be a bug. But the caller’s `if iszero(success) revert` at L3568 (immediately before the call) guarantees `success = 1` when `ec_pairing` is invoked, so the `leave` path is never taken in the final call. The two together form an airtight gate:

- **Pre-check (L3568):** if any prior check failed (`success = 0`), revert before the pairing.
- **Revert-on-failure (L551):** if the pairing itself fails, revert inside `ec_pairing`.
- **Terminal `mstore(1)` (L3575):** reached only when `success = 1` *and* the pairing passed — i.e. only on a fully-valid proof.

If the assembly block ever fell through instead, `verifyProof` has no Solidity statement after the block. Its unnamed `bool` return would therefore be the default `false`, never `true`.

Assessment. This is a [OK] finding — the terminal return path is sound. The `ec_pairing` helper’s revert-on-failure design is the load-bearing property: it makes the unchecked `mstore(RETURN_MPTR, 1)` safe because the only way to reach it is through a successful pairing. The three-way pairing validation (`staticcall success + returndatasize + mload`) is thorough defense-in-depth.

Audit note. A reviewer verifying this must confirm the revert-on-failure property of `ec_pairing` (L551) — it is *not* obvious from the call site (L3569), which looks like a boolean assignment. This is a case where the soundness argument depends on a helper’s internal behavior that the call site doesn’t reveal. A clearer pattern would be to make the revert explicit at the call site (e.g. a comment or a post-call `if iszero(success) revert`), or to name the helper

`require_pairing` to signal that it reverts rather than returns. As written, a hasty reader could mistake `success := ec_pairing(...)` for a boolean assignment that gets ignored — a readability trap around a soundness-critical line. **[OPEN]** In B64 the long comment before the call (L3555–3567) explains only the argument order. The helper’s doc line (L532–533) calls it a “final EIP-2537 pairing wrapper” and does not mention the revert. A post-call `if iszero(success) { revert(0, 0) }` costs a few gas and would make the call site self-evidently fail-closed.

11.8.19 The memory-safe promise: scratch at 0x80, never bump 0x40, terminal block

Every assembly block in the verifier is annotated `assembly ("memory-safe")` (L204, L304, L313). This annotation is a *promise to the Solidity compiler* that the assembly block does not corrupt live Solidity memory — if the promise is violated, the compiler may have placed live Solidity variables in memory the assembly overwrites, causing subtle soundness bugs. The verifier honors this promise with a three-part discipline stated explicitly in the `@dev` comment (L283–289):

“The generated verifier uses absolute Yul memory addresses instead of Solidity’s free-memory pointer, but generated scratch starts at 0x80 so Solidity’s reserved memory prefix is preserved. The main assembly block remains terminal: accepted proofs return from assembly and all rejected inputs revert. Do not inline this body into Solidity code that continues executing after verification without reviewing the memory strategy.”

Part 1 — scratch starts at 0x80. Solidity reserves the memory prefix 0x00–0x7f: scratch space (0x00–0x3f), the free-memory pointer (0x40–0x5f), and the zero slot (0x60–0x7f, the initial value of empty dynamic memory arrays). By starting all generated scratch at `TRANSCRIPT_MPTR = 0x80`, the verifier never touches a reserved word (`VerifierMemoryLayout::validate()` rejects any region below 0x80). **[CORRECTED]** That alone does *not* make the block memory-safe. The Solidity rules allow a memory-safe block only scratch 0x00–0x3f, memory Solidity allocated, and memory *at or above the free-memory pointer’s value on entry*. “Above 0x80” equals “above the FMP” only if the compiler has not reserved memory of its own. Under via-IR it has (see below).

Part 2 — never bump the free-memory pointer. The verifier uses *absolute* addresses (0x80, 0x2700, 0x4140, ...) and never reads or writes `mload(0x40) / mstore(0x40, ...)`. It does not allocate via the free-pointer discipline; it uses a fixed, pre-planned layout. This means the assembly cannot accidentally clobber a Solidity-allocated object by bumping the pointer past it — there is no pointer interaction at all.

Part 3 — the block is terminal. This is the load-bearing part. The main assembly block (L313) ends in `return(RETURN_MPTR, 0x20)` or `revert(0, 0)` — it never *falls through* to Solidity code that continues executing. This is what makes the absolute-address strategy safe: because the assembly never returns control to Solidity, the compiler never needs to reason about what the assembly left in memory. If the block *did* fall through, the absolute-address scratch usage would violate the `memory-safe` contract — Solidity code continuing after the assembly would find its memory model broken. Terminality protects only code that runs *after* the block. The Solidity documentation states the memory-safety rules apply “even if the assembly block reverts or terminates”, because the optimizer also relies on them *inside* the block.

The via-IR stack-to-memory reservation. The annotation exists so that solc’s via-IR pipeline may move Yul variables to memory (test `generated_heavy_assemblies_keep_memory_safe_for_via_ir`, `src/lowering/tests.rs` L2127). That pipeline has actually been

used. B64 is compiled with the production settings: the nightfall `foundry.toml` verifier profile (solc 0.8.35, `via_ir`, 200 runs, Cancun), reproduced here with solc 0.8.35 and 0.8.28. Its runtime begins `PUSH2 0x08e0 PUSH1 0x40 MSTORE`, not `0x6080604052`. The compiler reserved `[0x80, 0x8e0)` as 67 words of spill slots, and the optimized IR shows them there. For example, the quotient block’s `let y := mload(Y_MPTR)` (L1510) lives at memory `0x300`, and trash-suffix temporaries live at `0x580–0x8c0`. The generator’s absolute regions sit inside that reservation: the transcript buffer at `0x80`, PCS scratch `0x80–0x1a0`, the accumulator batch buffer `0x100–0x320`, and the `ec_pairing` scratch `0x300–0x600`. For this compile the live ranges happen to be disjoint in time. The transcript buffer is dead before the quotient block spills; the PCS/pairing scratch is written after it; and honest proofs verify. No corruption is observed. But that disjointness is an accident of solc’s slot assignment. Nothing checks it, and it can change with the compiler version, optimizer settings or a template edit.

The explicit warning. The `@dev` comment’s “Do not inline this body into Solidity code that continues executing after verification” is a direct warning that the terminal-block property is a *precondition* of the `memory-safe` annotation. A future maintainer who refactored the verifier to return to Solidity after verification (e.g. to compose with other logic) would break the memory-safe contract unless they reviewed the memory strategy. The comment names `docs/MEMORY_LAYOUT.md` as the reference.

Assessment. [CORRECTED] This was graded [OK]; it should be a [RISK]. The three-part discipline (scratch at `0x80`, no free-pointer interaction, terminal block) is internally consistent and well documented. But with the production via-IR build, the `memory-safe` promise is *formally violated*: the block writes below the entry FMP (`0x8e0`) into compiler-owned spill slots. Correctness then rests on unchecked live-range disjointness, not on the annotation. (The constructor smoke test at L204 likewise overwrites the constructor-argument copy at `0xa0`. This is benign only because solc loads `authorizedVk` onto the stack first.)

Audit note [IMPROVE]. The terminal-block property is load-bearing but *implicit* — it depends on every code path in the assembly ending in `return/revert`, which a reviewer must verify by tracing all paths (the fail-closed VM dispatch and the `revert-on-failure` helpers make this tractable, but it’s manual). A stronger guarantee would be a codegen-time assertion that the assembly block has no fall-through path, or a comment at the block boundary restating the terminal invariant. [PARTIAL] The comment exists: L314 (“This block owns the call-frame memory and remains terminal”) and L3573–3574. There is still no codegen-time no-fall-through assertion. A fall-through would return `false`, not `true` (§11.8.18). [OPEN] The larger gap is the via-IR reservation above. Sound fixes, in order of preference: (a) plan every absolute region above a fixed base (e.g. `0x1000`) and assert `iszero(gt(mload(0x40), BASE))` at block entry, failing closed; (b) add a CI check on the compiled runtime (initial `mstore(0x40, ...)` value $\leq$ lowest planned address) under the pinned solc and flags; (c) reduce Yul variable pressure until solc no longer spills. Dropping the annotation is not an option: with 67 spill slots needed today, via-IR would very likely fail with stack-too-deep.

11.8.20 Calldata bounds safety: exact-length pinning prevents silent zero-reads

The verifier is a **hand-rolled calldata parser** — it reads the proof with raw `calldataload/calldatacopy` rather than through Solidity’s ABI-decoded slices. (solc still runs its calldata-argument checks on entry, rejecting an offset + length beyond `calldatasize()`.) This is a deliberate design choice (cheaper, no decoder overhead), but it carries a hazard: `calldataload` past the end of calldata does *not* revert — it silently returns 0. An unchecked

out-of-bounds read would feed zeros into the transcript as if they were real proof data, a subtle soundness hole. The verifier closes this with an **exact-length pin**.

The exact-length check. Before any proof parsing, the verifier pins the total calldata size to the exact expected value (L959–962):

```
success := and(
    success,
    eq(calldatasize(), add(INSTANCE_CPTR, 0x0260))
)
```

This asserts the calldata is *exactly* the ABI selector + proof bytes + instance-array length + 19 instance words — no missing words, no trailing bytes. The comment (L948) states the intent: “total calldata length has no missing or trailing words” and (L958) “Any trailing bytes fail closed.”

Why this makes the parser bounds-safe. The parser advances through the proof in fixed increments (0x780 advice + 0x100 lookup + 0x300 perm-Z + ...+ 0x80 π) that sum to the pinned proof length 0x1e60. Because the total calldatasize is checked *exactly* against INSTANCE_CPTR + 0x0260, and the parser’s increments are generated to sum to that same length, every calldataload/calldatacopy after the L965 revert is **guaranteed in-bounds**. B64 has 45 such sites on 44 lines (31 loads and 14 copies). The few reads before L965 are at fixed header offsets: the ABI head words (L305) and the two length words (L954–955). They are the guards themselves: a zero read from short calldata fails the equality and reverts. There is no code path that feeds a past-the-end zero into the transcript.

The three-way shape pin. The bounds safety is actually a three-way consistency check (L954–962):

- `eq(0x1e60, calldataload(PROOF_LEN_CPTR))` — the proof-length word in the ABI header is exactly 7776 bytes.
- `eq(19, calldataload(NUM_INSTANCE_CPTR))` — the instance-array length is exactly 19.
- `eq(calldatasize(), add(INSTANCE_CPTR, 0x0260))` — the total calldata is exactly the expected size.

All three must agree. A mismatch (e.g. a truncated proof whose length word lies about its size) fails the total-size check and reverts before any parsing. This is the bounds-safety gate: the parser never trusts the length word alone — it cross-checks against the actual `calldatasize()`.

Assessment. This is a **[OK]** finding — the hand-rolled parser is bounds-safe because the exact-length pin guarantees every read is in-bounds. The three-way shape pin (proof-length word + instance count + total calldatasize) is a thorough consistency check that catches truncated, padded, or mis-shaped calldata before any parsing. The fail-closed discipline (revert on any mismatch) means there is no silent zero-read path.

Audit note. The bounds-safety argument depends on the parser’s fixed increments summing exactly to the pinned length — a property that holds by construction (the codegen generates both the increments and the length constant from the same proof shape) but that a reviewer must confirm by summing the increments. This is the same triple-layered proof-length validation documented in §11.8.15, but here framed as the *bounds-safety* property: the exact-length pin is what makes the hand-rolled parser safe against the silent-zero-read hazard that raw `calldataload` introduces. A reviewer auditing the parser should verify the increment-sum equals the pinned length — if they ever diverged (a codegen bug), an out-of-bounds read would become possible.

[ADDRESSED] The walk end-check at L1312 turns any such divergence into a revert. Whatever was read in the meantime is discarded, because the check runs before any Lagrange/quotient/PCS work. The generator also builds the layout cursor-by-cursor with assertions (`abi/proof.rs`), and `src/lowering/tests.rs` L358/L387 pin the repacked and rendered proof lengths to `proof_layout.proof_len`.

11.8.21 The modexp zero-input guard: a subtle soundness defense a naive implementation misses

The verifier computes scalar inverses via the EIP-198 modexp precompile (address 0x05): `scalar_inv(x)` (L327) computes $x^{-1} = x^{r-2} \bmod r$ by Fermat’s little theorem. The modexp precompile has a subtle behavior that a naive implementation would miss, and the verifier guards against it explicitly.

The hazard: `modexp(0, r-2, r)` returns 0, not an error. The EIP-198 modexp precompile does *not* error on a zero base — `modexp(0, k, m)` returns 0 for any $k \geq 1$ (only 0^0 yields 1). So a naive `scalar_inv(0)` would return 0 as if it were a valid inverse. But 0 has no multiplicative inverse in $\mathbb{F}_r$ — a zero “inverse” silently propagating through the verifier’s field arithmetic (e.g. in a Lagrange denominator or a batch-invert product) could break soundness by making a constraint appear satisfied when it isn’t.

The guard. The verifier explicitly rejects zero before calling modexp (L328–330):

```
// Zero has no multiplicative inverse in Fr; callers rely on a
// revert here rather than a bogus modexp result.
if iszero(x) { revert(0, 0) }
```

This is a critical defense-in-depth measure: it converts the precompile’s silent-zero behavior into a hard revert. The comment makes the intent explicit — callers *rely* on this guard, meaning the verifier’s correctness depends on it.

The EIP-198 frame and return-size check. The modexp frame is laid out as [`base_len`, `exp_len`, `mod_len`, `base`, `exponent`, `modulus`] with all three lengths 0x20 (L334–339), and the return is checked for exactly 32 bytes (L341: `if iszero(eq(returndatasize(), 0x20)) revert`). This guards the *variable-length return* hazard: modexp’s return length equals the modulus length, so a mis-sized modulus would produce a mis-sized return — the check catches it.

The memory-reuse optimization. A notable detail: the verifier reuses the *dead transcript buffer* just below `VK_MPTR` for the modexp frame (L324–326), rather than a fixed post-VK address. The comment explains why: a fixed post-VK address “can collide with live PCS scratch when the VK payload becomes smaller.” This is a memory-layout safety optimization — reusing a buffer that’s provably dead at that point avoids a collision class that a fixed address would introduce.

Assessment. This is a **[OK]** finding — the modexp return handling is correct and the zero-input guard is a well-executed defense against a subtle precompile behavior. The three-part handling (zero-input guard + return-size check + memory-reuse) shows careful attention to the precompile’s actual semantics rather than assuming ideal behavior.

Audit note. The zero-input guard is the kind of defense that’s easy to overlook because the precompile doesn’t signal an error — a reviewer must know that `modexp(0, k, m) = 0` (not a

revert) to appreciate why the explicit `if iszero(x) revert` is necessary. The verifier’s comment (“callers rely on a revert here rather than a bogus modexp result”) documents this dependency, which is good practice. A reviewer auditing the field arithmetic should confirm that every `scalar_inv` call site either (a) can never receive zero by construction, or (b) relies on this guard — if a call site received zero and the guard were removed, the resulting zero-inverse would be a soundness bug. **[ADDRESSED]** All five `scalar_inv` call sites (L3136, L3180, L3216, L3245, L3265; PCS denominators) go through the guard. The only other modexp user, `batch_invert` (L1346), has its own zero guards: the single-element path at L466 and the zero total product at L498. Both return 0 into `success`, which is then enforced at L1397.

11.8.22 The G1 on-curve check deferral and the codegen-time guarantee that closes it

A subtle soundness question: the transcript helper that absorbs G1 points does *not* check that they’re on the curve. The verifier’s soundness therefore rests on a deferral — and a codegen-time guarantee that the deferral is always honored. This is a three-part trust boundary worth documenting explicitly.

Part 1 — the transcript helper checks coordinates, not the curve. `common_uncompressed_g1` (L396) performs only coordinate-canonical checks: high-bits-zero (`shr(128, x_hi_word)`) and field-range ($x \leq p - 1$ via the `BLS_P_HI/BLS_P_MINUS_ONE_LO` comparison). It does **not** verify the point satisfies the BLS12-381 curve equation $y^2 = x^3 + 4$. The comment (L387–391) is explicit:

“This helper does not run an independent curve/subgroup check. Instead, ProtocolPlan::validate rejects generated plans where an absorbed proof commitment would not later be consumed by an EIP-2537 G1MSM or pairing path, and those precompiles perform the curve/subgroup validation.”

Part 2 — the precompile does the curve check. The EIP-2537 precompiles enforce on-curve and subgroup membership, but *not uniformly*: `G1ADD` (0x0b) checks only that inputs are on the curve and does **not** subgroup-check, while `G1MSM` (0x0c) and the pairing precompile (0x0f) **do**. So the deferral is sound only if the absorbed point reaches a `G1MSM` or pairing — not merely a `G1ADD`. This asymmetry is the crux: a point that only ever flows through `G1ADD` would never be subgroup-checked.

Part 3 — the codegen-time guarantee closes the boundary. The load-bearing link is a codegen-time check. **[CORRECTED]** The emitted comment attributes it to `ProtocolPlan::validate`. That function only enforces the *logical* precondition that every absorbed advice commitment is opened by the PCS (`protocol/mod.rs` L778–792). The concrete check is `kzg::validate_absorbed_g1_precompile_coverage` (`kzg/mod.rs` L602–619), invoked from `validate_generator_invariants` (`plan.rs` L245). It first checks that the tracked absorbed-G1 count equals the proof layout’s G1 count. It then requires every absorbed proof-G1 memory handle to appear in `precompile_validated_g1s` (L529–565). That list holds the PCS query commitments routed into the final `G1MSM` (quotient limbs expanded), plus f_{com} and π . Rendering fails otherwise. Unit tests cover both directions (`kzg/mod.rs` L1805, L1819). The list is a model derived from the same PCS plan that emits the MSM, not a scan of the rendered Yul. This is a static (compile-time) invariant that the runtime relies on. If the codegen ever emitted a plan that absorbed a point without routing it to a curve-checking precompile, the verifier would accept an unvalidated point — a soundness hole. The codegen’s `validate` is what makes the runtime deferral safe.

Why defer at all? An on-curve check alone needs only a few 381-bit field multiplications, but in Yul those require multi-limb arithmetic or modexp calls per point. The subgroup check is the expensive part: it needs a scalar multiplication by r (or an endomorphism test) in software, or an extra precompile call per point. Deferring to the precompile means the curve check happens *for free* as part of the MSM/pairing the verifier already performs — the point is validated by the same precompile call that uses it, with no extra cost. The deferral is a gas optimization that’s sound only because of the codegen guarantee.

Assessment. This is a [OK] finding — the deferral is sound, but it’s a *distributed* soundness argument spanning three components (transcript helper + precompile semantics + codegen validate). The verifier is correct, but the correctness proof is not local to any one function.

Audit note [RISK]. This is the most subtle soundness dependency in the verifier: the runtime’s safety depends on a *codegen-time* invariant (`validate_absorbed_g1_precompile_coverage`) that is invisible in the generated contract. A reviewer reading only the Solidity sees a transcript helper that doesn’t check the curve, and must trust that the codegen guarantees every absorbed point reaches a curve-checking precompile. If the codegen’s validate had a bug (e.g. it failed to catch a point that only reaches `G1ADD`, not `G1MSM`), the verifier would silently accept on-curve points outside the prime-order subgroup (`G1ADD` still rejects off-curve ones). This is a case where the generated artifact’s soundness cannot be fully verified by reading the artifact alone — it requires auditing the codegen’s validate logic too. A defense-in-depth improvement would be a deploy-time smoke test that feeds a known off-curve point through the full verifier path and confirms it reverts (the current smoke tests check precompile availability but not the end-to-end off-curve revert). [PARTIAL] A deploy-time version cannot be built, because it would need a valid proof. The generator’s test suite already does this end to end: `every_proof_g1_rejects_off_curve_coordinates` (`src/test.rs` L2408) substitutes a canonical off-curve point at every proof-G1 offset of a deployed verifier and asserts a revert. `src/test.rs` L133–142 checks that `G1MSM` rejects such a point. Still missing is an on-curve, *non-subgroup* mutation, the case only the `G1MSM`/pairing subgroup check catches. Note also that any mutated point changes the transcript, so these end-to-end reverts do not by themselves show *which* check fired.

11.8.23 The VM cached-top stack discipline: overflow/underflow prevented by codegen-time bound

The quotient VM is a stack machine, and stack machines have two classic hazards: **overflow** (push past the allocated region) and **underflow** (pop from an empty stack). The verifier’s VM avoids both with a *cached-top* discipline and a codegen-time bound — but, like the on-curve deferral, the safety rests on a codegen-time invariant rather than a runtime check.

The cached-top design. The VM maintains three registers (L1653–1655):

- `q_top` — the cached top-of-stack value.
- `q_has_top` — whether `q_top` currently holds a value (0 = empty stack).
- `q_sp` — the memory stack pointer for *spilled* values (those below the cached top).

Push (L1709–1710): if `q_top` is already live, the old value is written to `q_sp` and `q_sp` is advanced by one word; the new value becomes `q_top`. Pop (binary `ADD`, `case 0x06`, L1719; B64 renders no `MUL 0x07` case): `q_sp` moves back by one word and the spilled value is combined with `q_top`. This is a standard “one register + memory spill” stack that avoids a memory round-trip for the common push-then-immediately-pop pattern.

Overflow is prevented by a codegen-time bound. The comment (L1658–1661) states the invariant: *“q_sp’s registered range must cover the interpreted operand stack plus any native callback scratch that reuses this base.”* The codegen computes the maximum stack depth of the emitted program and registers a memory range that covers it. Because the program is *generated* (not attacker-controlled: its bytes live in the VK payload, pinned by the VK codehash and the quotient certificate hash), it cannot push more values than the codegen allocated for — overflow is impossible by construction. There is no runtime overflow check because the trusted program provably never overflows.

Underflow is prevented by construction too. A binary ADD/MUL pops by moving `q_sp` back. If the stack were empty, this would underflow below the base. But the codegen only emits binary opcodes when there are operands on the stack (the expression tree guarantees this), so underflow is impossible by construction. The comment (L1484–1488) reinforces this: *“Expression opcodes leave one value in q_top; FOLD_MAIN or FOLD_SELECTOR consumes it and advances the global y-batch position. Native callback opcodes are only emitted at empty-stack boundaries.”*

Assessment. This is a [OK] finding — the cached-top stack discipline is correct and the overflow/underflow prevention is sound. The design is efficient (the cached top avoids a memory round-trip for the common case) and the codegen-time bound makes the stack safe.

Audit note [RISK]. Like the on-curve deferral (§11.8.22), the stack safety rests on a *codegen-time* invariant (the registered range covers the max stack depth) that is invisible in the generated contract. A reviewer reading the VM sees `q_sp := add(q_sp, 0x20)` with no bounds check, and must trust that the codegen’s static analysis correctly bounded the stack. If the codegen had a bug that emitted a program exceeding the registered range, the VM would silently corrupt adjacent memory (the native callback scratch region). This is a second instance of the same pattern: the generated artifact’s safety depends on codegen-time static analysis that the runtime doesn’t re-verify. A defense-in-depth improvement would be a codegen-time assertion that the max stack depth is within the registered range (which the codegen likely does internally, but which isn’t visible in the artifact), or a runtime bounds check on `q_sp` (at a gas cost). The current design trusts the codegen — reasonable for a trusted generator, but a reviewer must audit the codegen’s stack-depth analysis to fully verify the artifact. [ADDRESSED] The codegen-time assertion exists. `validate_quotient_program` (quotient_numerator/vm/mod.rs L2003–2020) re-decodes the *finalized* bytes and rejects underflow (L2258–2270), wrong fold/native boundary depth, and end-of-program leaks. `finish()` asserts that the result equals the builder’s `max_stack` (L1211–1216). The `quotient_stack` region is sized from it (quotient.rs L249–257) and checked for overlap by the memory-layout validator, and `src/lowering/tests.rs` L370–374 re-asserts the coverage. Since commit 33f78d2b the artifact also asserts termination at runtime: `q_pc == q_end` and an empty stack (L2569–2570). [OPEN] A runtime `q_sp` upper-bound check is still absent, by design.

11.8.24 The FOLD_SELECTOR gap mechanism: sparse y-batch position tracking mirroring Rust’s reverse fold

The batch-identity folding (eqs (2.22) and (2.25)) requires each identity’s contribution to land at a specific *y-power* so the verifier’s accumulated sum matches Rust’s reverse fold exactly. The VM’s FOLD_SELECTOR opcode (0x0b) tracks this y-batch position with a clever **gap mechanism** — a sparse-accumulator optimization worth documenting.

The packed operand. The FOLD_SELECTOR instruction packs two operands into three bytes (L2532–2539):

- **high byte:** the selector bucket index (`q_sel_idx`).
- **low u16:** the y -power gap since this selector’s previous contribution (`q_sel_gap`).

The gap mechanism. The opcode does two things (L2549–2558):

```
// Global accumulator * y (keeps main-identity y-batch position)
mstore(0xa320, mulmod(mload(0xa320), y, r))
let q_target_ptr := add(SELECTOR_ACC_MPTR, shl(5, q_sel_idx))
let q_sel_acc := mload(q_target_ptr)
if q_sel_gap {
    // Advance only this selector's local accumulator by y^gap
    q_sel_acc := mulmod(q_sel_acc, mload(add(0xa360, shl(5, q_sel_gap))), r)
}
mstore(q_target_ptr, addmod(q_sel_acc, q_eval, r))
```

The **global** accumulator (0xa320) is multiplied by y every step — this keeps the main-identity y -batch position advancing in lockstep. The **selector bucket’s local** accumulator is advanced by y^{gap} using a precomputed table at 0xa360 (indexed by the gap), then the evaluated value is added.

Why a gap? Sparse buckets. The comment (L2553–2555) explains: *“Selector buckets are sparse in the global identity stream. Precomputed y^{gap} advances only this selector’s local accumulator.”* Because a given selector contributes only occasionally to the global stream, instead of advancing *every* bucket by y at each step (expensive), the codegen precomputes the y -power gaps between consecutive contributions and stores them in a table. At runtime, only the contributing bucket is advanced by its gap — a sparse-update optimization that avoids touching buckets that didn’t contribute.

The correspondence to Rust’s reverse fold. The comment (L2546–2548) makes the faithfulness link explicit: *“The global fully-evaluated accumulator is still multiplied by y so later main identities land at the same y powers as Rust’s reverse fold.”* The gap mechanism is the codegen’s encoding of the y -power distances that Rust’s reverse fold computes implicitly — the verifier’s accumulated y -powers must match Rust’s exactly for the batch identity to verify, and the gap table is how the codegen guarantees that match without recomputing the full y -power sequence at runtime.

Assessment. This is a [OK] finding — the gap mechanism is a correct and efficient encoding of the sparse y -batch position tracking. The precomputed gap table (0xa360) turns what would be a dense per-step update of all buckets into a sparse per-contribution update of one bucket, a meaningful gas optimization that preserves the exact y -power correspondence to Rust’s reverse fold.

Audit note. The gap mechanism’s correctness depends on the codegen computing the gaps correctly — the y^{gap} table at 0xa360 must contain the exact y -power distances between consecutive contributions of each selector bucket. A reviewer cross-checking against Rust should verify that the gap values match the y -power distances in Rust’s reverse fold. This is another codegen-time correctness dependency (like the stack-depth bound and the on-curve deferral), but here the runtime is doing real arithmetic with the gap values. A mis-computed gap changes the verifier’s y -weights relative to Rust’s, so an *honest* proof fails the final pairing. The bug therefore surfaces as a completeness failure in the end-to-end and trace-differential tests rather than as silent acceptance. It is fail-safe in the sense that it cannot ship unnoticed, although a verifier with wrong weights would enforce a different identity combination.

11.8.25 The native callback boundary reset: defense against stack-state leakage

The VM’s native opcodes — 0x19 permutation, 0x1b heavy identity, 0x1f lookup — are “markers” that jump to generated Yul callbacks. A subtle safety question: when a callback runs, does it inherit a dirty VM stack from the previous expression? The verifier answers this with an explicit **boundary reset**.

The reset. Each native callback case begins by clearing the VM stack state (permutation L1934–1941; lookup L2051–2057; heavy identity L2247–2249):

```
// Native callbacks are identity-boundary opcodes. They
// must not inherit any partially evaluated VM stack
// state from the previous expression.
q_top := 0
q_has_top := 0
// ...
q_sp := 0xa980
```

The cached top is zeroed, the “has top” flag is cleared (stack empty), and the stack pointer is reset to the base (0xa980). The callback starts with a clean slate.

Why it matters. The comment (L1931–1933) states the invariant: *“Native callbacks are identity-boundary opcodes. They must not inherit any partially evaluated VM stack state from the previous expression.”* If a callback inherited a dirty stack (a leftover `q_top` or a mis-positioned `q_sp`), its fold logic could operate on wrong values — producing a wrong accumulator contribution. The explicit reset guarantees each callback starts clean, regardless of what the previous expression left behind. (The callbacks read only their own memory scratch, not `q_top`. What the reset really does is discard any leftover state and re-base the scratch table at 0xa980.)

Defensive vs. load-bearing. The VM’s expression evaluation is supposed to leave the stack empty at an identity boundary (the comment L1487–1488: *“Native callback opcodes are only emitted at empty-stack boundaries”*), so the reset is *defensive* — the stack should already be clean. **[CORRECTED]** It is not a benign no-op, however. If a codegen bug left a partially evaluated identity on the stack, the reset would *silently drop* that identity without folding it or advancing `y`. The verifier would then enforce a different identity set. That would show up as an honest-proof failure, but it is masking, not safety. The fail-closed form is `if q_has_top { revert(0, 0) }` at callback entry, the same check the VM already runs once at program end (L2570).

The scratch-table base. A related detail: the callback uses `program.stack_mptr` (0xa980) as its *scratch-table base*, not as a conventional VM stack (L1936–1940). The comment notes: *“The Rust memory planner must reserve enough words for structured_permutation_scratch_words(meta) whenever this opcode can appear.”* This is another codegen-time memory reservation — the callback’s scratch table must be sized by the Rust planner, and the VM reuses the same base address as the stack (safe because the stack is empty at the callback boundary).

Assessment. This is a **[OK]** finding — the native callback boundary reset is correct given the empty-stack invariant, which the generator enforces. The explicit clearing of `q_top/q_has_top/q_sp` re-bases each callback’s scratch. A leak would be masked rather than rejected (see above).

Audit note. The reset’s defensive nature means a reviewer should confirm the complementary invariant: that the VM *does* leave the stack empty at identity boundaries (so the reset is truly redundant, not masking a real leak). **[ADDRESSED]** The generator enforces it twice. The builder asserts `stack_depth == 0` before each native marker (`quotient_numerator/vm/mod.rs` L1165–1168, L1181, L1194–1197). The finalized-bytecode validator rejects any native opcode reached with a non-empty stack (L2241–2247). The runtime reset is therefore redundant. **[OPEN]** Replacing it by (or adding) a `q_has_top` revert would turn the masking case into a hard failure at about 5 gas per callback.

11.8.26 The VK payload as a self-consistent, documented byte layout

The VK contract’s runtime is `INVALID || payload`, and the payload is a **self-describing byte layout** whose total length is pinned and whose sections are contiguous — a property that makes the payload auditable by arithmetic alone.

The documented section map. The VK contract’s header comment (`Halo2VerifyingKeyFee12345B64.sol` L19–37; excerpt L28–37) lays out the sections:

```
/// word 9  : num_acc_limbs
/// word 10 : num_acc_limb_bits
/// word 11 : quotient_manifest_hash
/// word 12..15 : G1_BASE      (4 words, EIP-2537 padded)
/// word 16..23 : G2_BASE      (8 words, EIP-2537 padded)
/// word 24..31 : NEG_S_G2_BASE (8 words, EIP-2537 padded)
/// word 32..31 + Q_PAYLOAD      : quotient VM constants + packed bytecode
/// word 32 + Q_PAYLOAD ..       : fixed_comms (4 words each)
/// word 32 + Q_PAYLOAD + 4*N_FIXED .. : permutation_comms (4 words each)
```

The arithmetic is self-consistent. The payload length is pinned at `EXPECTED_VK_PAYLOAD_LENGTH = 17056` (533 words). The sections sum exactly:

- **Header:** 32 words (0x0000–0x03FF) — tagged scalars + `G1_BASE` + `G2_BASE` + `NEG_S_G2_BASE`.
- **Q_PAYLOAD:** 321 words (0x0400–0x2C1F) — 178 quotient-VM constant words + 143 packed-program words.
- **fixed_comms:** 108 words (27 points × 4) starting at 0x2C20.
- **permutation_comms:** 72 words (18 points × 4) starting at 0x39A0.
- **Total:** 32 + 321 + 108 + 72 = 533 words = 17056 bytes.

The last `mstore` in the deploy routine is at `payload + 0x4280` (`permutation_comms[17].y_lo`), and `0x4280 + 32 = 17056` — the payload length exactly matches the last word’s end. A reviewer can verify the layout by this single arithmetic check: *last mstore offset + 32 = EXPECTED_VK_PAYLOAD_LENGTH*.

Why this matters for audit. The self-consistent layout means a reviewer does not need to trust the codegen’s section sizing — they can verify it by arithmetic. The three pins (`EXPECTED_VK_LENGTH`, `EXPECTED_VK_CODEHASH`, `EXPECTED_VK_PAYLOAD_LENGTH`) plus the contiguous section map form a closed system: any byte tweak changes the codehash (caught at deploy), any length change breaks the `extcodecopy` length pin (caught per-proof), and the section offsets are derivable from the documented map. This is **audit-friendly by construction** — the layout is verifiable without re-running the codegen.

Deviation (readability). The doc comment’s word `32..31 + Q_PAYLOAD` notation is a minor readability quirk — it means “`Q_PAYLOAD` words starting at word 32” but reads awkwardly (the `31 +` is the end-offset base, not a count). A clearer notation would be `word 32 .. 31+Q_PAYLOAD` with an explicit `Q_PAYLOAD = 321` stated. This is cosmetic, not a correctness issue. **[OPEN]** The B64 VK (L34) still uses the original notation.

Assessment. This is a **[OK]** finding — the VK payload is a self-consistent, documented byte layout whose total length is pinned and whose sections are contiguous and arithmetically verifiable. The layout is auditable by arithmetic alone, a genuine audit-facilitation property.

11.8.27 EIP-2537 precompile input encoding: correct by construction

Every EIP-2537 precompile call must present its inputs in the exact padded uncompressed format the precompile expects. The generated verifier’s two heavy precompile call sites — the fused G1MSM and the final pairing — both lay out their inputs correctly, and the arithmetic of the input lengths confirms it.

The fused G1MSM encoding. The linearization MSM (L3460) is:

```
success := staticcall(gas(), 0x0c, 0xa320, 0x30c0, FINAL_COM_MPTR, 0x80)
success := and(success, eq(returndatasize(), 0x80))
```

The input length `0x30c0 = 12480 bytes = 78 pairs × 160 bytes`, where each pair is a 128-byte G1 point (64-byte x padded + 64-byte y padded) followed by a 32-byte scalar. This is exactly the EIP-2537 G1MSM input format — the arithmetic confirms 78 pairs fit the declared length. The 78 terms are the 33 absorbed proof commitments other than π plus the 27 fixed and 18 permutation VK commitments.

The pairing encoding. The `ec_pairing` helper (L534–553) lays out two (G_1, G_2) pairs:

```
// [lhs_g1 (0x80) | G2_BASE (0x100) | rhs_g1 (0x80) | NEG_S_G2_BASE (0x100)]
let scratch := 0x0300
mcopy(scratch, lhs_mptr, 0x80)
mcopy(add(scratch, 0x80), G2_BASE_MPTR, 0x100)
mcopy(add(scratch, 0x180), rhs_mptr, 0x80)
mcopy(add(scratch, 0x200), NEG_S_G2_BASE_MPTR, 0x100)
ret := staticcall(gas(), 0x0f, scratch, 0x0300, scratch, 0x20)
```

The input length `0x0300 = 768 bytes = 2 pairs × 384 bytes`, where each pair is a 128-byte G1 point followed by a 256-byte G2 point (two 64-byte x limbs + two 64-byte y limbs, each left-padded). This is exactly the EIP-2537 PAIRING input format.

The MCOPY gas optimization (already applied). The comment (L539–542) documents a concrete gas win: “*Cancun MCOPY (3 + 3 · words gas) replaces what used to be a 4-step mstore chain for each G1 (~60 gas) and an 8-iter mstore loop for each G2 (~240 gas). Net saving here is ~500 gas per ec_pairing call.*” The verifier uses `mcopy` to move the G1/G2 points into the pairing scratch buffer in bulk, rather than word-by-word `mstore` — a meaningful optimization that the codegen already applies.

Assessment. This is a **[OK]** finding — both precompile input encodings are correct by construction, confirmed by the input-length arithmetic ($78 \times 160 = 0x30c0$ for the MSM; $2 \times 384 = 0x300$ for the pairing). The MCOPY bulk-copy optimization is a concrete gas saving already realized.

Audit note. The input-length constants (0x30c0, 0x0300) are the key to auditing the encoding: a reviewer can verify the format by dividing the length by the per-pair size (160 for MSM, 384 for pairing) and checking the pair count matches the expected number of terms. If the encoding had the wrong pair size, the length wouldn't divide evenly — a cheap arithmetic check that catches stride errors without running the precompile. **[CORRECTED]** Length arithmetic says nothing about *padding*, which does not change the length. Padding and coordinate canonicity are enforced by EIP-2537 itself (a non-zero top 16 bytes or a coordinate $\geq p$ is an error), and upstream by `common_uncompressed_g1` for proof points.

11.8.28 The transcript absorb order: VK digest first, committed_pi identity fix

The Fiat-Shamir transcript's absorb order must match Rust's exactly — any reordering changes every downstream challenge. The generated verifier's opening absorbs match Rust's `verifier.rs` schedule, and the code documents a **deviation fix** in the identity encoding.

The opening absorb order. The transcript opens with (L1001–1035):

1. **VK digest** (L1008): `buf_len := common_word(buf_len, mload(VK_DIGEST_MPTR))` — the first absorb, matching Rust's `vk.hash_into(transcript)` as trace ID 1. The comment (L1006–1007): *"This digest commits to the verifier key / constraint system before any proof material is read."*
2. **committed_pi = G1 identity** (L1018–1028): 128 zero bytes, absorbed *before* the instance count, matching Rust's `committed-instances` loop (`verifier.rs` L90–94). The Solidity hard-codes the identity. The generator accepts only `CommittedInstance CommitmentKind::Identity (src/builder/api.rs` L17–18).
3. Then the instance count (L1035; Rust L98), instances, and proof material follow in the Rust order.

The committed_pi identity encoding fix. The comment (L1012–1016) documents a corrected deviation: *"Under the patched `Hashable<Keccak256>::to_input` (see `midfall/proofs/src/transcript/implementors.rs`), the identity hashes as 128 zero bytes (EIP-2537 (0,0) convention), NOT the 48-byte ZCash compressed form `0xc0||47*0x00` that the previous emitter produced."* This is a concrete faithfulness correction — the earlier emitter used the ZCash compressed identity encoding, which would have produced a different transcript digest than the Rust verifier's EIP-2537 padded-uncompressed identity. The fix aligns the generated transcript with Rust's identity encoding.

Why the order matters. The VK digest being absorbed *first* is load-bearing: it binds every subsequent challenge to the specific constraint system, so a proof crafted for one VK cannot be replayed against another. The `committed_pi` identity being absorbed *before* the instance count matches Rust's schedule — if the order were swapped, the challenge derived after the instances would differ, breaking verification.

Assessment. This is a **[OK]** finding — the transcript absorb order matches Rust's schedule (VK digest first, then `committed_pi` identity, then instance count), and the identity encoding is correctly the EIP-2537 128-zero-bytes form. The documented fix from the ZCash compressed form is a concrete faithfulness correction.

Audit note. The absorb order is the single most fragile part of the transcript — a reviewer should verify each absorb against Rust’s `verifier.rs` line-by-line. The generated code’s explicit comments (L1002–1017) make this cross-check tractable: each absorb is annotated with its Rust counterpart and the encoding convention. This is good audit-facilitation — the transcript schedule is documented at the point of use, not just in a separate table.

11.8.29 The accumulator pairing batching: Schwartz–Zippel soundness

When the verifier has both a KZG pairing equation and an accumulator pairing equation to check, naively multiplying them together is unsound — *“two bad equations could cancel”* (L3500–3501). The verifier instead uses a randomized batching with a Schwartz–Zippel soundness argument.

The combined equation. After all four G1 pairing inputs are fixed, the verifier derives a randomizer α and checks (L3505–3506):

$$e(\text{kzg_rhs} + \alpha \cdot \text{acc_rhs}, [1]_2) \cdot e(\text{kzg_lhs} + \alpha \cdot \text{acc_lhs}, -[s]_2) = 1$$

The Schwartz–Zippel argument. The comment (L3508–3509) states the soundness: *“If either original equation is bad, this combined equation holds for at most one α in $\mathbb{F}_r$.”* By bilinearity the left-hand side is $T_{\text{kzg}} \cdot T_{\text{acc}}^\alpha$ in the order- r target group, where T_{kzg} and T_{acc} are the two original pairing products. If *either* is $\neq 1$ (not only when both are), then $T_{\text{kzg}} T_{\text{acc}}^\alpha = 1$ has at most one solution $\alpha \bmod r$: exactly one if $T_{\text{acc}} \neq 1$, none if $T_{\text{acc}} = 1$. This is the exponent-linear analogue of Schwartz–Zippel. Because α is a hash output, a prover who grinds q candidate proofs succeeds with probability about $q/r \approx q \cdot 2^{-254.9}$ — negligible.

The α derivation. The α is Fiat–Shamir over the fully materialized pairing inputs (L3514–3522): the batch buffer holds `domain || KZG_rhs || KZG_lhs || acc_rhs || acc_lhs` (0x0220 bytes), and `acc_pair_alpha := mod(keccak256(batch_ptr, 0x0220), r)`. Binding α to all four pairing inputs means the prover cannot choose α after seeing the inputs — the randomizer is unpredictable and tied to the exact points being batched.

The $\alpha = 0 \rightarrow 1$ fix. The comment (L3520–3521) documents a subtle correctness guard: *“Replace the negligible zero draw with one so the accumulator equation cannot be accidentally dropped.”* If $\alpha = 0$, the combined equation reduces to just the KZG equation (the accumulator terms vanish), which would let a bad accumulator pass. Replacing $\alpha = 0$ with $\alpha = 1$ ensures the accumulator is always included in the check — a defense against the negligible-but-catastrophic case where the hash happens to produce $\alpha = 0$.

The G1MSM + G1ADD implementation. The batching is implemented with two one-pair G1MSM calls (to compute $\alpha \cdot \text{acc_rhs}$ and $\alpha \cdot \text{acc_lhs}$) followed by two G1ADD calls (to add them into the KZG points) (L3528–3551). This is the natural way to compute $\text{kzg} + \alpha \cdot \text{acc}$ with the available precompiles.

Assessment. This is a [OK] finding — the randomized batching is a correct Schwartz–Zippel soundness argument. The α is properly bound to all pairing inputs via Fiat–Shamir, and the $\alpha = 0 \rightarrow 1$ guard prevents the accumulator from being accidentally dropped.

Audit note. The batching is the most subtle soundness-critical code in the verifier — a reviewer should verify the Schwartz–Zippel argument holds (the combined equation is linear in α in the exponent) and that α is bound to all four inputs. The explicit comment (L3500–3509) makes the argument auditable at the point of use, which is good audit-facilitation for such a subtle check.

11.8.30 The LogUp lookup callback: sum-of-reciprocals via prefix/suffix products

The LogUp helper identity is the cleared-denominator form of $h = \sum_i 1/f_i$: $h \cdot \prod_j f_j - \sum_i \prod_{j \neq i} f_j = 0$ (eq (2.19)). The verifier therefore needs the partial products $\prod_{j \neq i} f_j$. The Rust verifier computes them as $(\prod_j f_j) \cdot f_i^{-1}$ with `f.invert().unwrap()` (logup.rs L456–465). In the EVM each inversion would be a `modexp` call. The generated callback instead uses the **prefix/suffix product trick**, which yields the same partial products with multiplications only.

The compressed lookup terms. The callback first compresses each lookup row into a field element and adds β (L2078–2086):

```
let q_lookup_shared_compressed := addmod(var1, q_lookup_shared_tail, r)
mstore(add(q_lookup_f, q_lookup_shared_off),
        addmod(q_lookup_shared_compressed, q_lookup_beta, r))
```

Each $f_i = \text{compressed_row}_i + \beta$ is stored in the `q_lookup_f` scratch array.

The prefix/suffix product trick. The callback computes prefix products $\Pi_i^{\text{pre}} = \prod_{j < i} f_j$ and suffix products $\Pi_i^{\text{suf}} = \prod_{j > i} f_j$ (L2091–2100), then the sum $\sum_i \Pi_i^{\text{pre}} \cdot \Pi_i^{\text{suf}}$ (L2101–2104):

```
q_lookup_sum := addmod(q_lookup_sum,
    mulmod(mload(add(q_lookup_prefix, ...)),
        mload(add(q_lookup_suffix, ...)), r), r)
```

This sum equals $\sum_i \prod_{j \neq i} f_j$ ($= (\prod_j f_j) \cdot \sum_i 1/f_i$ when no f_j is zero) — the numerator of the sum of reciprocals, computed with only multiplications (no `modexp`). The helper identity is then $h \cdot \prod_j f_j - \text{sum}$ (L2105), which ties the committed helper h to $\sum_i 1/f_i$. The accumulator constraint (L2109–2123) consumes $\sum h$. One difference from Rust: the prefix/suffix form is total, whereas Rust’s `invert().unwrap()` would *panic* if some $f_j = 0$. A prover can supply evaluations at x that make this happen, which turns into a native-verifier crash rather than a clean rejection. This is not a soundness issue, but it is a Rust/Solidity behavioural divergence.

The active-row mask. The callback also computes a Lagrange-based **active-row mask** (L2068): `q_lookup_active := addmod(1, sub(r, addmod(L_LAST + L_BLIND, r)), r) = 1 - (Llast + Lblind)`. This mask switches off the *accumulator* constraint (L2120, L2226) on the last usable row and the blinding rows, where the accumulator is not constrained, exactly as Rust’s `active_rows` (logup.rs L415, L469–477). The boundary term $(L_0 + L_{\text{last}}) \cdot Z$ (L2067, L2073) and the helper identity are unmasked, as in Rust.

Assessment. This is a [OK] finding — the prefix/suffix product trick is a correct and efficient way to compute the sum of reciprocals without field inversions. The active-row mask matches Rust’s placement.

Audit note. The prefix/suffix product structure is subtle — a reviewer should verify that $\sum_i \Pi_i^{\text{pre}} \cdot \Pi_i^{\text{suf}} = \sum_i \prod_{j \neq i} f_j$ (the standard identity). The generated code’s loop structure (prefix forward, suffix backward, sum element-wise) makes this auditable, but the algebra is non-obvious at a glance. This is a case where the optimization (avoiding `modexp`) makes the code harder to read but the identity is standard.

11.8.31 The permutation boundary conditions: Lagrange-masked start/end/-continuity

The permutation argument’s soundness depends on boundary conditions that the verifier enforces via **Lagrange basis masks** — L_0 activates the first-row conditions, L_{last} activates the last-row

condition, and the masks are zero elsewhere so the conditions only bind at the intended rows.

The three boundary conditions. The permutation callback enforces (L2004–2017):

1. **First-row start** (L2004): $L_0 \cdot (1 - Z_{\text{cur}})$ — when $L_0 = 1$ (first row), this forces $Z_{\text{cur}} = 1$ (the permutation accumulator starts at 1).
2. **Last-row condition** (L2007–2008): $L_{\text{last}} \cdot (Z_n^2 - Z_n) = L_{\text{last}} \cdot Z_n(Z_n - 1)$, where Z_n is the *last set's* product evaluated at x (`q_perm_z_cur + 0xa0`, set 5 of 6). When $L_{\text{last}} = 1$ (the last usable row), this forces $Z_n \in \{0, 1\}$ (a boolean condition on the final accumulator).
3. **Cross-set continuity** (L2011–2017): for each permutation set $i \in [1, 6)$, $L_0 \cdot (Z_i(x) - Z_{i-1}(\omega^{\text{last}}x))$. At the first row, set i 's product must equal set $i-1$'s product at the last usable row (`q_perm_z_last`, the ω^{last} evaluations), chaining the sets into one running product (eq (2.15)).

Why Lagrange masks. The L_0 and L_{last} basis polynomials are 1 at their target row and 0 at all other rows (on the evaluation domain). Multiplying the boundary condition by the mask means the condition only contributes to the batched identity at the intended row — the verifier doesn't need a separate "check this row" branch; the mask does it algebraically. This is the standard PLONK boundary-enforcement technique, and the generated code applies it faithfully.

The y-batch interleaving. Each boundary term is folded into the global accumulator with a $\times y$ step (L2005–2006, L2009–2010, L2015–2016), keeping the boundary conditions at distinct y -powers in the batched identity — the same y-batch discipline as the main gate identities (§11.8.24).

Assessment. This is a [OK] finding — the permutation boundary conditions are correctly enforced via Lagrange masks. The three conditions (start, end, continuity) match the standard PLONK permutation boundary requirements, and the mask-based enforcement is the idiomatic technique.

Audit note. The $Z_n(Z_n - 1)$ last-row condition is a *boolean* condition (allowing $Z_n \in \{0, 1\}$), not the stricter $Z_n = 1$ some formulations use. A reviewer cross-checking against Rust should verify this matches the Rust verifier's boundary condition — if Rust enforces $Z_n = 1$ strictly, the boolean relaxation here would be a deviation. The Lagrange-mask structure makes the conditions auditable, but the exact boundary polynomial ($Z_n^2 - Z_n$ vs $Z_n - 1$) is a detail worth confirming against the Rust source. [CORRECTED] There is no deviation. Rust enforces the same boolean condition, `(z_1.square() - z_1) * 1_last` (`plonk/permutation.rs` L210–215). This is the standard halo2 form (eq (2.14)), and the start and continuity terms match L204–229 of the same file.

11.8.32 The heavy-gate callback: u16 index dispatch with manifest validation

Some gate identities are too complex to express as a sequence of simple VM opcodes — they are *whole expressions* with many multiplications and additions. The VM handles these with the `0x1b NATIVE_IDENTITY` callback, which dispatches to a generated Yul sub-case via a **u16 native callback index**.

The u16 index dispatch. The callback reads a u16 operand (L2243–2244):

```
let q_native_idx := shr(240, mload(q_pc))
q_pc := add(q_pc, 2)
```

The top 16 bits of the program counter word hold the native callback index, which selects a generated sub-case via a `switch` (L2251). Each sub-case is a heavy gate identity emitted as native Yul (a polynomial identity over advice columns).

The manifest validation. The comment (L2240–2242) documents a codegen-time invariant: *“The manifest validates that callback indexes appear in generated order and target existing switch cases.”* The quotient identity manifest ensures that every emitted `0x1b` instruction’s index corresponds to a real generated sub-case — no dangling index can dispatch to a missing case. This is another codegen-time correctness dependency (like the stack-depth bound and the on-curve deferral), but here the validation is explicit in the manifest.

The stack clear. Like the other native callbacks (§11.8.25), the heavy-gate callback clears the interpreter stack before dispatching (L2245–2249): `q_top := 0; q_has_top := 0; q_sp := 0xa980`. Heavy identities are whole expressions that own their scratch layout, so they start with a clean stack.

Assessment. This is a **[OK]** finding — the heavy-gate callback’s u16 index dispatch is correct, and the manifest validation ensures no dangling index. The stack clear is consistent with the other native callbacks.

Audit note. The u16 index space (65536 possible values) is larger than the number of generated sub-cases, so a reviewer should verify the manifest’s “target existing switch cases” validation is enforced at codegen time — if a malformed index could reach the switch, the default case must fail closed. The generated switch’s default behavior (revert vs. fall-through) is worth confirming. **[ADDRESSED]** B64 renders four sub-cases (0–3) and `default { revert(0, 0) }` (L2528). The outer opcode switch also reverts by default (L2561–2563). At codegen, `execution_manifest` rejects a native marker with no callback body (`quotient_numerator/vm/mod.rs` L415–418). `validate_execution_manifest` (L436), called from `quotient.rs` L458, checks the expanded stream against the source identity order.

11.8.33 The MODARITH7 opcode: a dispatch optimization, not a math change

The `0x21` MODARITH7 opcode is the VM’s most complex instruction — a “byte-only fused affine 7-limb foreign-field/ECC identity.” A reviewer might worry it introduces custom arithmetic; the comment clarifies it is purely a **dispatch optimization**.

What it fuses. The opcode computes (L1770–1777):

```
// maybe_cond * (
//   c
//   + sum LIN7 blocks
//   + sum BILIN7_ROW blocks
//   + sum BILIN7_PAIRWISE blocks
//   + sum coeff[k] * mload(ptr[k])
//   + sum coeff[k] * mload(lhs[k]) * mload(rhs[k])
// )
```

This fuses six arithmetic patterns (constant, linear, bilinear row, bilinear pairwise, single-term, double-term) into a single dispatch — what would otherwise be many separate VM opcodes.

The dispatch-optimization clarification. The comment (L1778–1780) is the key audit point: *“It is a dispatch/operand-load optimization only; all coefficients still come from the generated quotient constant table.”* The MODARITH7 opcode does *not* introduce custom arithmetic — the

coefficients are the same constant-table values that would appear if the identity were emitted as separate opcodes. The fusion saves dispatch overhead (one opcode instead of many), not a different computation.

The flag-gated operands. Two flag bits control the operand layout (L1782–1786): bit 0 gates the whole identity by a memory condition word (for conditional identities); bit 1 seeds `q_acc` from a constant-table word (for identities with a standalone constant term). The flags let one opcode cover the conditional/unconditional and with/without-constant variants without separate opcodes.

Production usage (decoded). Decoding the 4,559-byte `q_program` from the B64 VK payload (101 instructions) gives **16** MODARITH7 instructions: 12 with flags `0x03` (condition + constant) and 4 with flags `0x02` (constant only); none uses flags `0x00` or `0x01`. Their count headers carry `LIN7`, `BILIN7_ROW`, mem and product blocks, but the **pairwise count is 0 in all 16** — the `BILIN7_PAIRWISE` loop (L1859–1883) is rendered but never executed by the pinned production program.

Assessment. This is a **[OK]** finding — the MODARITH7 opcode is a correct dispatch optimization that fuses multiple arithmetic patterns without changing the math. The coefficients remain in the constant table, so the identity’s semantics are identical to the unfused form.

Audit note. The fusion makes the opcode’s behavior less obvious at a glance — a reviewer must understand the six fused patterns and the flag-gated operands to verify the identity. The comment’s explicit “dispatch optimization only” clarification is good audit-facilitation: it tells the reviewer the math is unchanged, so they can focus on whether the fusion correctly reproduces the unfused computation rather than re-deriving the identity.

11.8.34 The `q_pow5` shortcut: an addition-chain optimization recognized by the codegen

The generated verifier includes a `q_pow5` helper that computes x^5 with an optimized multiplication chain. The comment clarifies this is a **codegen shortcut**, not a verifier rule — the Rust verifier only sees an expression tree.

The addition-chain optimization. The helper (L1415–1419):

```
function q_pow5(x) -> z {
  let q_r := FR_MODULUS
  let x2 := mulmod(x, x, q_r)
  z := mulmod(x, mulmod(x2, x2, q_r), q_r)
}
```

This computes $x^2 = x \cdot x$, then $x^4 = x^2 \cdot x^2$, then $x^5 = x \cdot x^4$ — **3 multiplications** instead of the naive 4 ($x \cdot x \cdot x \cdot x \cdot x$). This is the standard addition-chain for exponent 5.

The codegen-recognition clarification. The comment (L1411–1414) is the key audit point: *“The Rust verifier only sees this as an Expression tree from `vk.cs.gates`; the generator emits `q_pow5` after recognizing five equal multiplicative factors. It is a codegen shortcut for x^5 , not a separate verifier rule.”* The generator pattern-matches $x \cdot x \cdot x \cdot x \cdot x$ in the expression tree and replaces it with the optimized helper. The verifier’s semantics are unchanged — the same value is computed, just with fewer multiplications. In B64 the helper is called only from the post-VM trash suffix (six sites, L2597–2622); the VM’s own `0x20 POW5` opcode does not occur in the production `q_program` and has no rendered case.

Assessment. This is a [OK] finding — the `q_pow5` helper is a correct addition-chain optimization recognized by the codegen. The math is identical to the naive x^5 ; only the multiplication count is reduced.

Audit note. The codegen’s pattern-recognition (five equal multiplicative factors $\rightarrow$ `q_pow5`) is a compiler-style optimization. A reviewer should verify the recognition is correct — that the helper is only emitted when the expression truly is x^5 (five identical factors), not when the factors differ. The comment’s explicit “codegen shortcut, not a verifier rule” clarification is good audit-facilitation: it tells the reviewer the optimization is transparent to the verifier’s semantics. [ADDRESSED] Verified: both recognizers require exactly five factors that are all equal — `pow5_base_expr` (`yul_emit.rs:808–825`, Expression equality) for the Yul helper and `quotient_pow5_base` (`vm/mod.rs:2692–2703`, structural key equality) for the POW5 opcode.

11.8.35 The post-VM validation: exact bytecode consumption and empty stack

After the VM dispatch loop finishes, the verifier runs a **post-VM validation** that enforces two invariants — the bytecode was fully consumed and the stack is empty. This is a strong fail-closed check on the generator’s output.

The two post-VM checks. After the dispatch loop (`L2569–2570`; added in `33f78d2b`, which closed `L-03` in `docs/audit.md`):

```
if iszero(eq(q_pc, q_end)) { revert(0, 0) }
if q_has_top { revert(0, 0) }
```

- `q_pc == q_end`: the program counter must land *exactly* at the end of the bytecode. If the final opcode over-reads operands (advancing `q_pc` past `q_end`), this check reverts. It cannot see an under-read: the loop only exits once `q_pc ≥ q_end`, so a short operand read instead desynchronises the decoding of the following bytes and is caught only if a desynchronised byte hits the default arm or leaves a live value.
- `q_has_top == 0`: the stack must be empty. If a partial expression is left live (a dangling `q_top`), this check reverts.

Why it matters. The comment (`L2565–2568`) explains: *“The VK-pinned bytecode must end exactly at `q_end` and every identity must have been consumed by a fold/native callback. This catches malformed generator output whose final opcode over-reads operands or leaves a partial expression live.”* These checks catch generator bugs that would otherwise silently produce a wrong accumulator — a malformed final opcode that doesn’t consume its operands, or an expression that doesn’t fold into the accumulator.

Defense-in-depth. The post-VM validation is a belt-and-suspenders check on top of the fail-closed dispatch default (which reverts on unknown opcodes). Even if the bytecode is well-formed opcode-by-opcode, the post-VM checks ensure the *overall* execution consumed everything correctly — a global invariant that per-opcode checks can’t catch.

Assessment. This is a [OK] finding — the post-VM validation is a correct and important fail-closed check. The exact-endpoint and empty-stack invariants catch malformed generator output that per-opcode checks would miss.

Audit note. The post-VM checks are a good example of a *global* invariant that complements the *local* fail-closed dispatch. A reviewer should verify both layers are present: the dispatch default reverts on unknown opcodes (local), and the post-VM checks enforce the global consumption/emptiness invariants. Together they catch *framing* errors; a well-framed but semantically wrong program (wrong pointer or constant index) would still execute silently — that class is excluded only by the codegen-side validator (§11.8.53) and the VK codehash / quotient-certificate pin (L920–923, L940).

11.8.36 The trash-constraint post-VM suffix: batched with powers of τ

After the main VM loop, the verifier runs a **structured post-VM suffix** that evaluates the trash-argument identity. The placement is a size/gas choice (see the comment below), not because the trash constraints are inexpressible in the VM. Inside the identity the constraint expressions are batched with powers of the trash challenge τ .

The suffix’s role. The comment (L2572–2578) explains: *“Structured post-VM suffix. The current default uses this for regular trash constraints: it is smaller than fully unrolled Yul and cheaper than interpreting every trash operation. ...These generated blocks run after `q_pc` reaches `q_end`, but they still participate in the same identity order and write into the same numerator / selector accumulators.”* The suffix is a codegen middle-ground — more compact than unrolled Yul, cheaper than VM-interpreting each trash operation.

The τ -batching. The suffix loads the trash challenge τ (L2580: `q_trash_tau := mload(TRASH_CHALLENGE_MPTR)`) and folds the trash constraints with powers of τ in a Horner-like accumulation (L2625, 2654, 2667):

```
let var33 := addmod(mulmod(0, q_trash_tau, r), var32, r)
let var60 := addmod(mulmod(var33, q_trash_tau, r), var59, r)
let var72 := addmod(mulmod(var60, q_trash_tau, r), var71, r)
```

Each step multiplies the running sum by τ and adds the next trash constraint — the standard randomized-batching of multiple constraints into one identity, with τ as the randomizer. In B64 there is one trashcan whose eight constraint expressions are folded this way (L2625–2763); the suffix then forms compressed $-(1 - q) \cdot \text{trash}$ (L2764–2767) and folds it with the main-identity rule $A \leftarrow A \cdot y + \text{eval}$ (L2768–2769). This matches `plonk/trash.rs:82–87` (`fold(acc $\cdot\tau$  + eval)`) then compressed $-(1-q) \cdot \text{trash_eval}$.

The τ derivation. The trash challenge τ is squeezed from the transcript in the order $\theta \rightarrow \beta, \gamma \rightarrow \text{trash_challenge} \rightarrow y \rightarrow x$ (L92, L1173). Binding τ to the transcript before y and x means the trash batching is bound to the same Fiat–Shamir chain as the main identities — the prover can’t choose τ after seeing the proof.

Assessment. This is a [OK] finding — the trash-constraint suffix is a correct τ -batched randomized-combination that participates in the same identity order as the main VM. The codegen’s choice of a structured suffix over unrolled Yul or VM interpretation is a reasonable size/gas tradeoff.

Audit note. The trash constraints are “invisible” to the main VM dispatch — they run in a separate suffix block. A reviewer auditing the identity order should check both the VM dispatch AND the post-VM suffix to see the complete identity stream. The comment’s explicit “same identity order” clarification is good audit-facilitation: it tells the reviewer the suffix isn’t a separate check but part of the same batched identity.

11.8.37 The MUL_CONST_U8 opcode: a one-byte compact constant multiply

The VM’s 0x0d MUL_CONST_U8 opcode is a compact encoding that multiplies `q_top` by a constant-table slot using a **one-byte index** instead of the u16 index of 0x0f MUL_CONST — saving one byte per constant multiply in short affine chains (20 occurrences in the production `q_program`).

The compact encoding. The opcode (L1729–1735):

```
case 0x0d {
  // One-byte constant-index multiply, used by short
  // affine chains after an initial PUSH.
  let qconst := byte(0, mload(q_pc))
  q_pc := add(q_pc, 1)
  q_top := mulmod(q_top,
    mload(add(q_const_mptr, shl(5, qconst))), r)
}
```

The operand is a single byte indexing the constant table (`q_const_mptr + 32·index`), not a u16 memory pointer. The comment (L1730–1731): *“One-byte constant-index multiply, used by short affine chains after an initial PUSH.”*

The byte savings. **[CORRECTED]** The alternatives for a constant multiplier are 0x0f MUL_CONST (3 bytes: opcode + u16 slot) or PUSH_CONST_U8+MUL (2+1 bytes, plus a stack spill). MUL_CONST_U8 is 2 bytes (opcode + 1-byte index) — a **1-byte saving** per constant multiply. (PUSH_MEM_U16 pushes a memory word, not a constant, so it is not the relevant comparison.) The size win is modest; the in-place update also avoids a spill/pop.

The constant-table indirection. The one-byte index (0–255) addresses the first 256 constant-table slots. **[CORRECTED]** The codegen does *not* rank constants by frequency: `const_slot` assigns slots in first-use order (vm/mod.rs:1834–1845), and `emit_acc_leaf` (L1761–1792) emits 0x0d when the slot fits a byte and falls back to 0x0f MUL_CONST otherwise. In B64 the table has 178 constants, so every slot fits one byte.

Assessment. This is a **[OK]** finding — the MUL_CONST_U8 opcode is a correct compact encoding that saves contract size. The one-byte index is a reasonable tradeoff for short affine chains whose constant slots fall below 256.

Audit note. The one-byte index space (256 constants) is a codegen-time constraint, but there is nothing to tune: overflow degrades automatically to the u16 form, so the audit check is only that the emitted byte equals the interned slot. The compact form is transparent to the math (same `mulmod` result).

11.8.38 The foreign-field representation: lowered to Fr, not switched

The VM’s limb-aware opcodes handle circuits that use a **foreign field** (a field other than BLS12-381’s $\mathbb{F}_r$). A reviewer might expect the verifier to perform foreign-field arithmetic; the comment clarifies it **does not switch fields** — it evaluates the lowered identity over $\mathbb{F}_r$.

The fused load+arith opcodes. The 0x10 ADD_MEM_U16 and 0x11 MUL_MEM_U16 opcodes (L1737–1750) fuse a short-memory load with an arithmetic operation:

```
// 0x10: q_top := addmod(q_top, mload(q_ptr), r)
// 0x11: q_top := mulmod(q_top, mload(q_ptr), r)
```

The comment (L1738–1739) notes: *“The pointed word is an already range-checked Fr scalar in verifier memory.”*

The foreign-field clarification. The comment (L1751–1766) is the key audit point: *“‘Foreign field’ means the circuit represents elements modulo another modulus m as 7 limbs in base $2^{\text{LOG2_BASE}}$. The verifier does not switch fields; it evaluates the lowered identity over BLS12-381 $\mathbb{F}_r$, using $\mathbb{F}_r$ coefficients equal to $\text{base}^i \bmod m$ or $\text{base}^{i+j} \bmod m$.”*

Why this matters. The foreign-field arithmetic is *lowered* to native $\mathbb{F}_r$ arithmetic at codegen time. The circuit’s 7-limb representation modulo m is compiled into $\mathbb{F}_r$ coefficients ($\text{base}^i \bmod m$), so the verifier only ever does native $\mathbb{F}_r$ `addmod/mulmod` — no foreign-field reduction, no multi-precision arithmetic. This is a significant simplification: the verifier’s arithmetic is uniformly native $\mathbb{F}_r$, regardless of what field the circuit uses.

The Rust source correspondence. The comment cites the Rust source shapes: `partially_evaluate_identities`, `sum_exprs/pair_wise_prod`, `base_powers/double_base_powers` — the codegen recognizes these patterns and lowers them to the limb-aware opcodes with pre-computed $\text{base}^i \bmod m$ coefficients.

Assessment. This is a **[OK]** finding — the foreign-field lowering is a correct and elegant simplification. The verifier’s arithmetic is uniformly native $\mathbb{F}_r$, with the foreign-field semantics captured in the precomputed coefficients.

Audit note. The foreign-field lowering is a subtle codegen transformation — a reviewer should verify the coefficients ($\text{base}^i \bmod m$) are correctly computed and that the lowering preserves the circuit’s foreign-field semantics. The comment’s explicit “does not switch fields” clarification is good audit-facilitation: it tells the reviewer the verifier’s arithmetic is native $\mathbb{F}_r$, so they don’t need to look for foreign-field reduction logic.

11.8.39 The ISA-vs-dispatch discrepancy: an opcode-usage-pruned switch

The VM’s ISA comment lists 0x1c LIN7, 0x1d BILIN7_ROW, and 0x1e BILIN7_PAIRWISE as separate opcodes, but the dispatch has **no cases for them**. **[CORRECTED]** An earlier version of this section attributed this to the three opcodes being “fused into MODARITH7”. That is wrong: they are live standalone opcodes in the generator; the switch is pruned to the opcodes that the VK-pinned program actually contains.

The ISA comment lists them separately. The ISA comment (L1673–1684) lists, among others:

```
// 0x1c LIN7          0x1d BILIN7_ROW
// 0x1e BILIN7_PAIRWISE 0x1f native lookup
```

A reviewer reading this would expect three separate dispatch cases for 0x1c, 0x1d, 0x1e.

Why the dispatch has no such cases. Grepping the dispatch for case 0x1c, case 0x1d, case 0x1e returns **nothing**, and the same holds for 0x01–0x04, 0x07, 0x09, 0x0a, 0x0c, 0x0e, 0x0f, 0x12–0x16, 0x20 and 0x22. The generator computes the set of opcodes present in the finished program (`quotient_program_usage`, `vm/mod.rs:1964–1988`: “the generated evaluator only needs switch arms for opcodes that can actually occur in its embedded program”) and the template wraps every arm in `{%- if program.op_usage.<op> %}` (`QuotientNumeratorBlock.yul`). The standalone limb opcodes are still emitted when a whole expression matches a single limb shape but not the MODARITH7 shape (`emit_expr` tries `POW5` $\rightarrow$ `MODARITH7` $\rightarrow$ standalone limb shape $\rightarrow$

limb decomposition, `vm/mod.rs:1282–1294`). Decoding the B64 `q_program` confirms it contains exactly the 11 rendered opcodes ($0x05 \times 12$, $0x06 \times 7$, $0x08 \times 5$, $0x0b \times 21$, $0x0d \times 20$, $0x10 \times 2$, $0x11 \times 12$, $0x19$, $0x1b \times 4$, $0x1f$, $0x21 \times 16$).

The five-counter block structure. MODARITH7 separately reads five one-byte counters (L1809–1815) that describe the blocks that follow:

```
let q_lin_count := byte(0, q_counts_word) // LIN7 blocks
let q_row_count := byte(1, q_counts_word) // BILIN7_ROW blocks
let q_pairwise_count := byte(2, q_counts_word) // BILIN7_PAIRWISE
let q_mem_count := byte(3, q_counts_word) // single-term
let q_product_count := byte(4, q_counts_word) // double-term
```

The comment (L1806–1808): *“Five one-byte counters describe the blocks that follow. Each block has a fixed-width internal layout, so `q_pc` can advance without per-term tags.”* So the LIN7/BILIN7 patterns exist *both* as blocks within MODARITH7 and as standalone opcodes `0x1c–0x1e`; this render only uses the former.

Assessment. This is a **[NOTE]** finding — the ISA comment documents the full ABI (31 assigned opcodes, `0x01–0x16`, `0x19`, `0x1b–0x22`), while the switch implements the 11 the program uses. The implementation is correct and unused bytes fall to the reverting default, but the rendered source never says the switch is pruned (the explanation lives only in `vm/mod.rs`).

Improvement. **[OPEN]** The original proposal (“note that `0x1c/0x1d/0x1e` are fused into MODARITH7”) would itself be inaccurate. The sound fix is for the generator to render the `op_usage` set next to the ISA summary (“cases rendered for this program: ...; all other ISA opcodes revert”), which it already computes (`QuotientProgramBuild::used_ops`). Same recommendation as Finding V-3.

11.8.40 The BILIN7_PAIRWISE convolution: coefficient indexed by $i+j$

The MODARITH7 opcode’s **BILIN7_PAIRWISE** block implements a weighted 7-by-7 product convolution with a subtle structure: the coefficient is indexed by $i+j$ (the degree of the product term), not by the individual indices (i, j) . This is the structure of a *polynomial multiplication*.

The double-loop convolution. The block (L1861–1883) iterates over all 49 pairs (i, j) :

```
for { let q_i := 0 } lt(q_i, 7) { q_i := add(q_i, 1) } {
  let q_lhs_value := mload(add(q_lhs_base, shl(5, q_i)))
  for { let q_j := 0 } lt(q_j, 7) { q_j := add(q_j, 1) } {
    let qconst := byte(0,
      mload(add(q_coeff_pc, add(q_i, q_j))))
    q_acc := addmod(q_acc,
      mulmod(mulmod(q_lhs_value,
        mload(add(q_rhs_base, shl(5, q_j))), r),
        mload(add(q_const_mptr, shl(5, qconst))), r), r)
  }
}
```

The $i+j$ coefficient indexing. The critical line is `add(q_i, q_j)` — the coefficient index is the *sum* of the two limb indices, not a 2D index. This means the coefficient for the product $\text{lhs}_i \cdot \text{rhs}_j$ depends only on $i+j$, the degree of the product term. The coefficient table has **13 entries** (degrees $0 \dots 12$), matching the degree range of a 7-limb $\times$ 7-limb polynomial product.

Why this is a polynomial convolution. If we view the 7 limbs as coefficients of a polynomial $\text{lhs}(x) = \sum_i \text{lhs}_i x^i$ and similarly for rhs, then the product $\text{lhs}(x) \cdot \text{rhs}(x) = \sum_d c_d x^d$ where $c_d = \sum_{i+j=d} \text{lhs}_i \cdot \text{rhs}_j$. The BILIN7_PAIRWISE block computes the *weighted collapse* $\sum_d w_d c_d$ of this convolution, where the 13-entry table holds the weights $w_d = \text{base}^d \bmod m$ (`double_base_powers, circuits/src/field/foreign/params.rs:59`), indexed by the degree $d = i + j$ — i.e. the foreign-field product $\text{lhs} \cdot \text{rhs}$ with the base powers pre-reduced mod m . The table holds the weights, not the c_d themselves.

Assessment. This is a **[OK]** finding — the BILIN7_PAIRWISE block is a correct weighted polynomial convolution with the $i+j$ coefficient structure. The 13-entry coefficient table matches the degree range of the 7-limb product.

Audit note. The $i+j$ indexing is a subtle but important structural property — a reviewer should verify the coefficient table is correctly indexed by degree (not by a 2D index) and that the 13-entry table covers the full degree range $0 \dots 12$. The convolution structure is the natural lowering of a foreign-field product where the limbs are base- 2^k digits. **[ADDRESSED]** The recognizer enforces this at codegen time: `try_quotient_bilin7_pairwise_shape` (`vm/mod.rs:3252–3329`) requires all 49 (i, j) cells and rejects the shape unless every cell with the same $i+j$ has the same coefficient. Note, however, that in B64 no MODARITH7 instance has a nonzero pairwise count, so production proofs never execute this Yul loop; the encoding is checked only against a Rust re-implementation of the interpreter in unit tests (e.g. `quotient_vm_modarith7_mixed_affine_matches_direct_expr_eval, tests.rs:2566`).

11.8.41 The MODARITH7 escape-hatch blocks: terms outside the 7-limb shapes

MODARITH7’s five-counter structure includes two **escape-hatch blocks** — `q_mem_count` and `q_product_count` — that handle terms that don’t fit the 7-limb LIN7/BILIN7 patterns. These let MODARITH7 express any affine/bilinear combination of u16-addressable memory words (at most 255 terms per category), not just the structured 7-limb shapes.

The single-term linear block (`q_mem_count`). The block (L1886–1896) adds single memory terms:

```
q_acc := addmod(q_acc,
    mulmod(mload(add(q_const_mptr, shl(5, qconst))),
        mload(q_ptr), r), r)
```

Each term is $c \cdot \text{mem}[\text{ptr}]$ — a 3-byte operand (1-byte const index + 2-byte pointer). This handles linear terms that aren’t part of a 7-limb LIN7 block.

The binary-product block (`q_product_count`). The block (L1899–1914) adds binary product terms:

```
q_acc := addmod(q_acc,
    mulmod(mulmod(mload(q_lhs), mload(q_rhs), r),
        mload(add(q_const_mptr, shl(5, qconst))), r), r)
```

Each term is $c \cdot \text{mem}[\text{lhs}] \cdot \text{mem}[\text{rhs}]$ — a 5-byte operand (1-byte const + 2-byte lhs + 2-byte rhs). This handles bilinear products that aren’t part of a 7-limb BILIN7 block.

Why the escape hatch matters. The LIN7/BILIN7 blocks are *optimized* for the common 7-limb foreign-field shapes, but not every term in the quotient polynomial fits those shapes. The escape-hatch blocks make MODARITH7 **complete** for its domain — affine/bilinear sums over

literal u16 memory words with Fr coefficients, an optional constant and one condition factor — falling back to the generic single-term/binary-product form when the structured 7-limb pattern doesn't apply. Terms outside that domain (symbolic memory tokens, unfactorable higher-degree products, > 255 terms in a category) make `quotient_modarith7_affine_shape` return `None` (vm/mod.rs:2846–2877), and the identity is lowered without MODARITH7. In B64 the escape hatches carry real load: 2 instances are pure mem+product (11 mem / 43 products), one has 12 mem / 57 products.

Assessment. This is a [OK] finding — the escape-hatch blocks are a correct completeness mechanism. The five-counter structure (LIN7 + BILIN7_ROW + BILIN7_PAIRWISE + mem + product) covers affine/bilinear sums over memory words, with the structured blocks as optimizations and the escape-hatch blocks as the general fallback.

Audit note. The escape-hatch blocks are the *general case* of the MODARITH7 structure — a reviewer should verify that the structured blocks (LIN7/BILIN7) are correct specializations of the escape-hatch form. If a structured block computes something different from the escape-hatch form for the same term, that's a bug. The escape-hatch blocks are the "ground truth" against which the structured blocks should be cross-checked.

11.8.42 The y-power precomputation table: O(1) lookup for sparse selector folds

The verifier precomputes a **y-power table** at 0xa360 containing $y^1, y^2, \dots, y^{48}$, enabling the FOLD_SELECTOR opcode to look up y^{gap} in **O(1)** instead of computing it in $O(\text{gap})$, and the end-of-stream selector tails to apply y^{tail} the same way. This is a concrete gas optimization for the sparse selector-bucket folds.

The table construction. The table is built in a loop (L1537–1544):

```
let q_y_power := 1
for { let q_y_power_i := 1 }
  lt(q_y_power_i, 49) {
    q_y_power_i := add(q_y_power_i, 1) } {
  q_y_power := mulmod(q_y_power, y, r)
  mstore(add(0xa360, shl(5, q_y_power_i)),
    q_y_power)
}
```

Each iteration multiplies the running power by y and stores it at $0xa360 + 32 \cdot i$. The table holds 48 entries (y^1 through y^{48}), each 32 bytes.

The O(1) lookup. When FOLD_SELECTOR needs to advance a selector bucket by a gap g , it reads y^g directly from the table (L2556: `mload(add(0xa360, shl(5, q_sel_gap)))`) instead of computing y^g via g multiplications. For a gap of g , this saves $g - 1$ `mulmod` operations per selector fold; a gap of 0 (a bucket's first contribution) skips the multiply (L2552). In B64 every fold gap is 0 or 1 (21 VM folds: five 0, sixteen 1; inline/native folds: gap 0 or y^1), so the gap lookups save nothing over a single `mulmod` by y ; the table earns its keep in the selector tails. The generator comment (L1531–1534) gives the actual rationale: it avoids a runtime y^{-1} modexp and per-identity selector-scale maintenance.

Why 48 entries. [CORRECTED] The size is `max(all fold gaps, all selector tail exponents)` (`SelectorFoldPlan::max_power`, quotient.rs:339–351), and in B64 it is set by the *tails*, not the gaps: the ten tail updates (L2779–2818) read y^{48} (bucket 0) down to y^{20} (bucket 9), while the largest fold gap is 1. There is no on-the-fly fallback: the table is sized by construction to cover every exponent the program uses.

Assessment. This is a **[OK]** finding — the y-power table is a correct and effective gas optimization. The $O(1)$ lookup replaces $O(\text{gap})$ multiplication, and the table construction is a single linear pass.

Audit note. The table size (48 entries) is a codegen-time constraint — a reviewer should verify the maximum selector gap *and tail exponent* fit within the table range. **[ADDRESSED]** By construction the loop bound is `max_power+1` over exactly those exponents (`quotient.rs:266–271, 339–351`; template loop bound `selector_max_power + 1`), and the u16 gap operand is asserted at emission (`vm/mod.rs:1141–1149`). There is no runtime bound check, so an out-of-range gap in a hand-edited program would read adjacent memory; the VK codehash pin excludes that.

11.8.43 The FOLD_MAIN structure: main vs. selector folds

[CORRECTED] An earlier version of this section described **FOLD_MAIN** as folding a main identity into *both* the global accumulator and a selector bucket. That is not what the code does, and the snippet it quoted is not **FOLD_MAIN**. The generator has two fold rules (template `QuotientNumeratorBlock.yul`, cases `0x0a/0x0b`; builder `fold_identity`, `vm/mod.rs:1132–1157`):

- **Main** (`0x0a FOLD_MAIN`, identity group `None`): $A \leftarrow A \cdot y + \text{eval on the global accumulator only}$.
- **Selector** (`0x0b FOLD_SELECTOR`): $A \leftarrow A \cdot y$ (advance the shared y-position only), then $B_s \leftarrow B_s \cdot y^{\text{gap}} + \text{eval on the selector bucket only}$.

Neither rule adds the identity value to both accumulators.

How B64 realises them. B64 renders **no case 0x0a**: the production `q_program` contains no **FOLD_MAIN** byte, because every VM-interpreted identity is selector-gated (21 **FOLD_SELECTOR**s, buckets 2–9). Main identities are folded by the same rule inlined in generated Yul: the permutation callback (L2005–2036), the lookup callback (L2074–2228) and the trash suffix (L2768–2769) — 11 `mstore(0xa320, addmod(mload(0xa320), ...))` sites. The code this section previously quoted (L2481–2487) is a *selector* fold inside native heavy identity #2 of the `0x1b` callback:

```
// Global accumulator: advance the y-position only
mstore(0xa320, mulmod(mload(0xa320), y, r))
// Selector bucket 9: advance by y^1 (gap 1), then add
let q_selector_ptr := add(SELECTOR_ACC_MPTR, 0x120)
let q_selector_acc := mload(q_selector_ptr)
q_selector_acc := mulmod(q_selector_acc,
    mload(add(0xa360, 0x20)), r)
mstore(q_selector_ptr,
    addmod(q_selector_acc, mload(0xa980), r))
```

The evaluated identity (at `0xa980`) goes only into bucket 9 (`SELECTOR_ACC_MPTR + 0x120`); the global accumulator at `0xa320` is only multiplied by y .

Why two rules. The global accumulator tracks the y-batch position of *every* identity (so later main identities land on the right power of y , matching Rust’s reverse fold), and accumulates only the fully evaluated (`None`) identities. Selector buckets accumulate only their own identities, each with its own gap-adjusted power; the end-of-stream tails (L2779–2818) then realign every bucket to the final global position.

The gap mechanism. The selector bucket’s gap is encoded in the y-power table lookup (`0xa360 + 0x20 = y1` for `gap=1`). This is the **FOLD_SELECTOR** gap mechanism (§11.8.24) inlined into a native callback.

Assessment. This is a [OK] finding — both fold rules match the generator’s `fold_identity` and the Rust y-batch, and the inlined native/suffix folds reuse the same snippets as the VM arms.

Audit note. A reviewer must check *both* update rules, and must look for main folds outside the VM switch: in B64 FOLD_MAIN semantics exist only as inlined snippets in the 0x19/0x1f callbacks and the trash suffix. Gas remarks about “the FOLD_MAIN case” (e.g. the redundant load/store item in ch. 10) therefore apply to those 11 inlined sites, not to a VM arm.

11.8.44 The IVC accumulator public-input decoding subsystem

Three functions in the production verifier had *no* prior coverage in this report: `is_acc_encoded_identity`, `load_acc_point` and `validate_public_accumulator` (L622, L709, L785). Together with their helpers (`load_acc_coord_shifted`, `is_bls_p_minus_one`, `check_acc_coord_packing`, `load_acc_coord`) they form a self-contained subsystem that decodes the recursive accumulator’s *public inputs* — two G1 points carried as native-field words — back into EIP-2537 padded form. This is the single largest previously-undocumented block in the contract.

Where the encoding comes from (Rust side). The circuit’s foreign-field gadget exposes a base-field point through `AssignedForeignPoint<BLS12-381>::as_public_input`. Each coordinate is committed as *seven radix- 2^{56} limbs of $(coord - 1)$* , packed four-at-a-time into native $\mathbb{F}_r$ words. The x coordinate’s first packed word carries the point-at-infinity flag by adding *one raw radix base 2^{56}* (encoder: `circuits/src/ecc/foreign/ecc_chip.rs:226` over `circuits/src/field/foreign/field_chip.rs:130`). The Solidity decoder is the exact inverse of this encoder; the codegen template `templates/partials/verifier/AccumulatorHelpers.yul` renders it verbatim.

The packed constants, verified. The sentinel constants (L193–199: three packed words plus the BLS_P_HI/BLS_P_MINUS_ONE_LO split) are not magic — they are $(p - 1)$ re-sliced under the radix- 2^{56} , 4-limbs-per-word layout. Writing p for the BLS12-381 base-field modulus:

$$\text{word}_0 = (p - 1) \bmod 2^{224} = \text{0xf38512bf...fffffffffaaaa} \quad (11.5)$$

$$\text{word}_0^{\text{id}} = \text{word}_0 + 2^{56} = \text{0xf38512bf...baf...} \quad (11.6)$$

$$\text{word}_1 = \lfloor (p - 1) / 2^{224} \rfloor = \text{0x1a0111ea...64774b84} \quad (11.7)$$

A direct computation confirms all five: the identity flag is literally the byte `b9→ba` at bits 56–63 of `PACKED_0`, i.e. add one radix base. The split `BLS_P_HI = p >> 256`, `BLS_P_MINUS_ONE_LO = (p - 1) mod 2256` matches the EIP-2537 high/low word convention.

Canonical identity is accepted only in one exact form. The fast path `is_acc_encoded_identity` (L622) requires all four packed words to match the canonical $(\text{word}_0^{\text{id}}, \text{word}_1, \text{word}_0, \text{word}_1)$ pattern. The comment is explicit that this is *deliberately stricter* than “decodes to zero”: the point at infinity has exactly one accepted public-input encoding, and non-canonical zero-like encodings are rejected downstream. This closes the malleability hole where several distinct calldata patterns could each decode to $\mathcal{O}$.

Canonical packing check. `check_acc_coord_packing` (L638) rejects any unused high bits in the final packed word of a coordinate: the last word holds fewer than four limbs, so the word must be $< 2^{\text{used_bits}}$. Without this, the *same* coordinate would have multiple calldata encodings (a classic witness-malleability vector). The check runs before any arithmetic, so non-canonical bytes never reach the curve logic.

The straddle handling in `load_acc_coord_shifted`. Reconstructing a 384-bit coordinate from 56-bit limbs means a limb can straddle the 256-bit low/high word boundary. The loop (L573) splits each limb: bits below 256 go to `lo`, the overflow of a straddling limb is shifted into `hi` via `shr(sub(256, shift), limb)`. The three-way branch (`shift<256` with straddle, `shift<256` without, `shift≥256`) is exhaustive and correct — a reviewer should confirm the `gt(add(shift,bits),256)` boundary is the only place a limb crosses the word split.

Defense-in-depth: every point forced through G1MSM. The most important safety property is in `validate_public_accumulator` (L816–829). The decoded carried LHS point is *always* routed through the G1MSM precompile (0x0c), *even when it is the identity or has scalar 0/1*:

```
// Always route the decoded carried point through G1MSM,
// even for identity points and zero/one scalars. The
// precompile is the on-curve/subgroup validator for this
// public-input point; skipping it would let a malformed
// non-identity point hide behind scalar 0.
mcopy(acc_scratch, ACC_LHS_MPTR, 0x80)
mstore(add(acc_scratch, 0x80), lhs_scalar)
if out {
    out := staticcall(gas(), 0x0c, acc_scratch,
        0xa0, ACC_LHS_MPTR, 0x80)
    out := and(out, eq(returndatasize(), 0x80))
}
```

This is a deliberate soundness choice. A scalar of 0 would otherwise erase a malformed point from the equation, letting a non-curve point pass unnoticed; forcing the MSM means the precompile’s subgroup check is the validator for *every* decoded public point. The same discipline applies to the RHS (L848–855, MSM at L872–880).

Template-vs-render: the collapsed configuration. The codegen template carries two conditionals that the `fee_12345` render collapses: `expected_acc_has_carried_scalars` (false $\Rightarrow$ carried scalars are implicit 1, not read from calldata) and `acc_fixed_bases.len()` (0 $\Rightarrow$ no fixed-base scalar tail). The rendered contract therefore takes the “already-collapsed point pair” branch: LHS and RHS are both carried points with implicit unit scalar, and the RHS MSM is a single pair. A reviewer diffing template against render should note that the *dead* branches (explicit carried scalars, fixed-base tail with negated scalars) are not present in this artifact — they exist for other render configurations.

Deviation: Solidity-only subsystem. This decoder has **no direct counterpart in the Rust verifier**. The Rust verifier operates on native G1 points inside the circuit’s native field; it never sees the packed public-input encoding. The packing is defined by the foreign-field `as_public_input` encoder (the in-repo `circuits` crate, not `proofs/`), and the Solidity side is the inverse. This is a **[NOTE]** rather than a defect: the two sides are inverse by construction, but a reviewer cross-checking against `verifier.rs` will find nothing to match — the cross-check target is the *encoder*, not the verifier. The only Rust-side anchor is the LogUp accumulator constraint (`evaluation.rs:579`), which is a different object.

Improvement: hoist the identity fast-path check. `load_acc_point` (L709) calls `is_acc_encoded_identity` first, and on a miss re-decodes via `load_acc_coord` for both x and y . **[CORRECTED]** The two `check_acc_coord_packing` calls are not redundant (each checks a different coordinate’s words), so “sharing the packing check” saves nothing. The real redundancy is the x identity probe: with `allow_id=1`, `load_acc_coord` (L663–704) runs a probe decode plus the real decode whenever x ’s first word is $\geq 2^{56}$ (the usual case), so a non-identity point costs three 7-limb decode loops, not two. After the fast path has missed, a flagged x can only end in rejection (all-zero decode is exactly the canonical identity already tested; otherwise the unadjusted

coordinate exceeds $p - 1$), so the probe could be skipped on this path. **[OPEN]** The saving is one 7-iteration loop per point, marginal next to the G1MSM round-trip; the current structure favours clarity — a reasonable trade.

Assessment. This subsystem is **[OK]** on soundness: canonical identity accepted in exactly one form, packing canonicity enforced before arithmetic, every decoded point forced through the G1MSM subgroup check, and the x -only identity flag correctly mirrored (y decoded with `allow_id=0`). The **[IMPROVE]** note is purely auditability: because the encoder lives in the `circuits` crate rather than the verifier, the spec’s Rust→Solidity correspondence table has a gap here that can only be closed by reading the foreign-field library, not `verifier.rs`. The packed-constant derivation above is provided so a reviewer can verify the sentinels without that library.

11.8.45 The codegen-side VK payload section map (`vk_payload.rs`)

The runtime reads the VK payload through fixed `*_MPTR` offsets (§11.8.5); the *codegen* side of that contract lives in `src/lowering/layout/vk_payload.rs`, a module with no prior coverage in this report. It is the single source of truth for the payload’s section boundaries, and it is worth reading because it is where the “scattered offset arithmetic” that makes the runtime hard to audit is *prevented* on the generator side.

A typed, validated section map. The module defines a `PayloadSectionKind` enum with five variants — `Header`, `QuotientConstants`, `QuotientProgram`, `FixedCommitments`, `PermutationCommitments` — and an append-only `VkPayloadLayout` that reserves each section at a running word cursor. The module doc states the design intent directly: *“generated byte payloads should have one validated section map instead of scattered offset arithmetic. The Solidity shape remains static and pinned.”*

Three structural guarantees. The layout enforces invariants that the runtime cannot:

1. **Uniqueness.** `reserve()` rejects a second section of the same kind — “duplicate VK payload section” — so offsets can never be ambiguous.
2. **Contiguity.** `validate()` walks the sections and asserts each `word_offset` equals the running cursor, and that the final cursor equals the total. A gap or overlap is a hard codegen error, not a silent runtime misread.
3. **G1 width folding.** `reserve_g1()` converts a commitment *count* into `count × G1_WORDS` (4 words per EIP-2537-padded point) once, so emission sites never re-derive the $4\times$ factor.

The worked example from the module’s own test. `for_vk(31, 3, 5, 7, 2)` produces the section map: `Header@0`, `QuotientConstants@31`, `QuotientProgram@34`, `FixedCommitments@39`, `PermutationCommitments@67`, total 75 words. The fixed-commitment byte offset is $39 \times 0x20$ and the permutation byte length is $2 \times 0x80$ — the same $0x80$ -per-point G1 slot width the runtime’s `extcodecopy` into `VK_MPTR` relies on.

Packed program codec. The byte-oriented quotient program (the 4559-byte VM bytecode) is stored in VK constant words by `PackedProgramCodec`: `encode_words` big-endian-packs bytes with zero padding; `decode_words` (test-only) verifies the padding is zero and the length fits capacity. **[CORRECTED]** Because `decode_words` is `#[cfg(test)]` (`vk_payload.rs:213–233`), the non-zero-padding rejection runs only in the module’s unit tests, not in the render pipeline; the production path relies on `encode_words` zero-padding by construction (L206–210). The runtime never looks at the padding anyway: the VM stops at `q_end = q_program_mptr + 0x11cf`. (B64’s padding is zero: the 143 program words hold 4,576 bytes for a 4,559-byte program.)

Codegen↔runtime correspondence. This module is the *generator* half of the verifier drift check the runtime performs at L934–941. The runtime cross-checks loaded header words against its own constants; the codegen’s `validate()` guarantees those words were laid out contiguously in the first place. Together they form a two-sided pin: the generator cannot emit an inconsistent layout, and the runtime refuses to consume a layout that drifted from the pinned constants. A reviewer verifying the VK payload only needs to check `for_vk`’s five section sizes against the runtime’s `*_MPTR` arithmetic — one table, not scattered offsets.

Assessment. This is a [OK] finding — the typed section map is exactly the kind of structure that makes the generated artifact auditable *at the source*. The [IMPROVE] opportunity is symmetry: the runtime still expresses the same boundaries as raw hex literals rather than deriving them from a rendered section table. If the codegen emitted the section map as named Solidity constants (Header/QuotientConstants/Program/Fixed/ Perm start words), the runtime’s offsets would be self-documenting and the drift check would compare like-for-like. [OPEN] In B64 the header slots have named constants (VK_MPTR...NEG_S_G2_BASE_MPTR, L71–86), but the quotient sections appear only as local literals (`q_const_mptr := 0x2b00`, `q_program_mptr := 0x4140`, L1515/L1518), and the drift check (L934–941) compares header *values*, not section offsets.

11.8.46 The batched identity numerator emitter (`yul_emit.rs`)

`src/lowering/quotient_numerator/yul_emit.rs` (1161 lines) had no prior coverage in this report. It walks the gates / permutation / lookup / trash arguments of the Rust `ConstraintSystem` and emits the Yul lines that compute each identity’s contribution to the batched identity numerator at the evaluation challenge x . [CORRECTED] It does *not* produce the 4,559-byte VM program: the bytecode is emitted by `QuotientProgramBuilder` in `vm/mod.rs` from typed `QuotientExpr` trees. Gate identities reach the VM through `quotient_expr_from_expression` directly from `Expression<Fq>` (`quotient.rs:760`, `vm/mod.rs:2386`); only permutation/lookup/-trash Yul is parsed back into `QuotientExpr` (`quotient.rs:913–919`). The Yul text itself is what B64 renders in the direct prefix, the `0x1b` heavy-identity callbacks, parts of the lookup callback and the trash suffix.

The historical-name clarification (a documented trap). The module doc is explicit that the name `yul_emit` is historical and the object is *not* the quotient polynomial $h(x)$:

“This is not an evaluator for the quotient polynomial $h(x)$. It reconstructs the batched identity numerator $\nu_y(x)$. The generated verifier later stores $-\nu_y(x)$ as the expected opening scalar for the linearized commitment; the commitment side already includes the quotient-limb factor $(1 - x^n)$.”

This is the codegen-side statement of the negated-numerator deviation logged in §10.1: the verifier holds $-\nu_y(x)$ at `QUOTIENT_EVAL_MPTR` and the trace re-negates to match Rust’s positive $\nu_y(x)$. Reading the emitter confirms the sign convention is intentional, not an off-by-sign bug.

Four emitters, one return shape. The permutation, lookup and trash emitters return `Vec<(Vec<String>, String)>` — a pair of (*yul_lines*, *final_var*); `gate_computations_tagged` returns `GateComputation` records carrying the same pair plus tags:

- `gate_computations_tagged` — per-gate `polynomials()`, tagged with the simple-selector index, gate name and constraint name (the same tagging that feeds the trace manifest of §10.8);
- `permutation_computations` — boundary + product equality;
- `lookup_computations` — boundary + helper + accumulator;

- `trashcan_computations` — $\text{compressed} - (1 - q) \cdot \text{trash}$.

The caller in `lowering/quotient.rs` folds each `final_var` into the compact quotient VM state or a native callback body — which is exactly the VM-vs-native split observed in the runtime dispatch (§11.8.9) — though for gates the VM path uses the typed expression, not this text.

The permutation emitter mirrors the Rust four-constraint set. The emitter’s doc comment reproduces the same four constraints as `plonk/permutation.rs::expressions`: $l_0(1 - z_0)$ (first set), $l_{\text{last}}(z_n^2 - z_n)$ (last set), $l_0(z_i - z_{i-1}^{\text{last}})$ (cross-column continuity), and $(1 - (l_{\text{last}} + l_{\text{blind}}))(\text{left}_i - \text{right}_i)$ per set, with δ_{pow} starting at $\beta x \delta^{i \cdot \text{chunk_len}}$ and multiplied by δ each step, where δ is `PrimeField::DELTA`. This is the codegen-side anchor for the permutation boundary conditions reviewed in §11.8.31 — the [RISK] note that the last-row check is boolean ($z_n^2 - z_n$) rather than a strict $z_n = 1$ is faithful to this Rust form.

The source of the opaque SSA names. The `var0`, `var1`, ..., `varN` names that this report repeatedly flagged as an auditability weakness are generated by a single function:

```
/// Return the next `varN` local name.
fn next_var(&self) -> String {
    let count = *self.var_counter.borrow();
    *self.var_counter.borrow_mut() += 1;
    format!("var{count}")
}
```

Each emitted identity resets the counter (L861), so the SSA names are per-identity sequential. The counter is a `RefCell` shared across the recursive expression walk. This is the precise origin of the “opaque SSA names” finding: the names carry no semantic information because the emitter has no semantic label to attach at the point of allocation. The fix is structural — the emitter would need to thread the constraint name into `next_var` — and is the same audit-facilitation change recommended in §11.6. These names appear only in the rendered Yul blocks (direct prefix, native callbacks, trash suffix); the VM bytecode carries no variable names.

Stable literal rendering. `u256_string` renders field elements as the shortest stable hex literal (`0x{x}` for < 64 -bit values, full `0x{value:x}` otherwise). This keeps the emitted program compact and deterministic — a reviewer diffing two renders of the same circuit should see byte-identical literals, which makes generator regressions visible in a plain diff.

Assessment. This is a [OK] finding on correctness — the four emitters mirror the Rust argument structure one-to-one, the sign convention is intentional and documented, and the literal rendering is deterministic. The [IMPROVE] note is the SSA naming: `next_var` is the single point where semantic labels are lost, and threading the constraint name through it would make the inlined Yul blocks (not the VM bytecode, which has no names) self-documenting at the source rather than requiring the per-identity trace manifest to recover the mapping at review time. [OPEN] B64 still renders `var0...varN` (e.g. L1551–1586, L2584–2763).

11.8.47 The multi-set batched KZG `f_eval`: a barycentric rearrangement

The largest repeated-pattern block in the production verifier (L3107–3269) computes the batched-opening scalar f_{eval} over the rotation *point sets*. It was not covered in this report before, and it contains a genuine algebraic rearrangement that a reviewer is expected to verify without being told — so the math is worked out here.

The Rust source. In `poly/kzg/mod.rs:471–482` the verifier folds the point sets *in reverse* with a Horner accumulator in x_2 :

```

let f_eval = point_sets.iter()
  .zip(q_eval_sets.iter())
  .zip(q_evals_on_x3.iter()).rev().fold(
    E::Fr::ZERO,
    |acc_eval, ((points, evals), proof_eval)| {
      let r_poly = lagrange_interpolate(points, evals);
      let r_eval = eval_polynomial(&r_poly, x3);
      // eval = (proof_eval - r_eval) / prod_i (x3 - point_i)
      let den = points.iter().fold(F::ONE,
        |acc, p| acc * &(x3 - p));
      let eval = (*proof_eval - &r_eval)
        * den.invert().unwrap();
      acc_eval * &(x2) + &eval
    });

```

Mathematically, for each set s with points $\{\rho_0, \dots, \rho_{m-1}\}$ and fixed evals $\{e_0, \dots, e_{m-1}\}$, the contribution is

$$E_s = \frac{\text{proof}_s - r_s(x_3)}{\prod_i (x_3 - \rho_i)}, \quad r_s(X) = \sum_i e_i L_i^{(s)}(X), \quad (11.8)$$

and the sets are combined $f_{\text{eval}} = \sum_s E_s x_2^s$: Horner over the *reversed* list makes set 0 the constant term. ([**CORRECTED**] an earlier version wrote x_2^{N-1-s} , which is the forward-Horner weighting.) The Solidity matches: its five sub-blocks consume `Q_EVAL_CPTR+0x80`, ..., `+0x0`, i.e. set 4 first and set 0 last.

The codegen never forms $r_s(X)$. The rendered Solidity does *not* interpolate the polynomial and evaluate it. It distributes the $1/\text{den}$ factor into the barycentric sum. Writing $\text{dx}_i = x_3 - \rho_i$, $\text{lbasis}_i = \prod_{j \neq i} (\rho_i - \rho_j)$ (the barycentric weight), and inverting *all* dx_i and lbasis_i together in one `scalar_inv` per set, the code computes

$$E_s = \text{proof}_s \cdot \text{den}^{-1} - \sum_i e_i \text{dx}_i^{-1} \text{lbasis}_i^{-1}, \quad \text{den}^{-1} = \prod_i \text{dx}_i^{-1}. \quad (11.9)$$

This is exactly Eq. (11.8): since $L_i^{(s)}(x_3) = \text{den} \cdot \text{dx}_i^{-1} \cdot \text{lbasis}_i^{-1}$, multiplying $r_s(x_3) = \sum_i e_i L_i^{(s)}(x_3)$ by den^{-1} cancels the `den` and leaves the per- i product of inverses. A direct numerical check (3-point set, random e_i /proof) confirms both forms agree to the field modulus.

Why the rearrangement. The Rust form needs `lagrange_interpolate` (build the polynomial) plus `eval_polynomial` (Horner it) — $O(m^2)$ work and a temporary polynomial. The Solidity form needs only the barycentric weights and one batch inversion of $2m$ field elements, no polynomial object, no second evaluation pass. It is the same value at lower code and memory cost — a [**IMPROVE**]-class codegen choice that is *already* applied, not a suggestion.

The repeated-pattern structure. The block renders one identical sub-block per point set, parameterised by cardinality (five sets in this profile, of cardinalities 3, 3, 2, 2, 1). Each sub-block:

1. computes dx_i for the set's rotation points;
2. computes lbasis_i as the product of $\rho_i - \rho_j$ over $j \neq i$;
3. runs the prefix-product / `scalar_inv` / suffix-product ladder to invert all dx_i and lbasis_i in one call (the same Montgomery trick as the Lagrange block, §11.8.11);
4. forms E_s per Eq. (11.9) and folds it into f_{eval} with x_2 .

The cardinality-1 set degenerates to a single dx_0^{-1} with no `lbasis` (a single point's barycentric weight is 1), which is why the last sub-block is shorter.

Audit note. This is the block the report’s “five near-identical Horner blocks” observation refers to at the code-size level (§13.7). The repetition is *not* redundant computation — each set has different points and evals — but it *is* repetitive code that a parameterised emitter (cardinality as a loop bound) could shrink. The soundness obligation for a reviewer is to confirm, per set, that the lbasis_i products exclude $j = i$ and that the $Q_EVAL_SET_MPTR$ offsets line up with the set’s eval order. The barycentric identity above is the cross-check: a reviewer can recompute E_s directly from Eq. (11.8) for one set and confirm it matches the rendered E_s .

Assessment. This is a [OK] finding on correctness — the barycentric rearrangement is algebraically exact (numerically verified) and faithful to the Rust fold. The [IMPROVE] note is code size: the per-set sub-blocks are emitted with the cardinality unrolled; a loop-parameterised emitter would reduce the rendered contract size while preserving the identical per-set math. [OPEN] B64 still renders five unrolled sub-blocks (L3117–3268).

11.8.48 The permutation product-equality constraint and the δ -pow chunk mechanism

§11.8.31 covered the three *boundary* conditions of the permutation callback. The fourth — and principal — constraint, the running **product equality**, was not yet documented. It is the core of the permutation argument and contains the δ -pow chunk mechanism that a reviewer must understand to verify the identity.

The Rust specification. The codegen’s own doc comment (`yul_emit.rs:230–243`) states the fourth constraint:

$$(1 - (l_{\text{last}} + l_{\text{blind}})) \cdot (\text{left}_i - \text{right}_i) = 0 \quad (11.10)$$

per permutation set i , where

$$\text{left}_i = z_{i,\text{next}} \cdot \prod_{c \in \text{chunk}_i} (\text{eval}(c) + \beta s(c) + \gamma) \quad (11.11)$$

$$\text{right}_i = z_{i,\text{cur}} \cdot \prod_{c \in \text{chunk}_i} (\text{eval}(c) + \delta_{\text{pow}} + \gamma) \quad (11.12)$$

with δ_{pow} starting at $\beta x \delta^{i \cdot \text{chunk_len}}$ and multiplied by δ each column step, and δ the field’s `PrimeField::DELTA` constant.

The rendered callback. The Solidity (L2018–2038) mirrors this exactly. The per-set base is seeded once:

```

mstore(q_perm_delta_base_ptr,
  mulmod(mload(BETA_MPTR), mload(X_MPTR), r))
for { let q_perm_set := 0 } lt(q_perm_set, 6) { ... } {
  let q_perm_delta_pow := mload(q_perm_delta_base_ptr)
  for { let q_perm_j := q_perm_start } lt(q_perm_j, q_perm_end) { ... } {
    q_perm_left := mulmod(q_perm_left,
      addmod(addmod(v, mulmod(BETA, s, r), r), GAMMA, r), r)
    q_perm_right := mulmod(q_perm_right,
      addmod(addmod(v, q_perm_delta_pow, r), GAMMA, r), r)
    q_perm_delta_pow := mulmod(q_perm_delta_pow, delta, r)
  }
  q_perm_eval := mulmod(
    addmod(1, sub(r, addmod(L_LAST, L_BLIND, r)), r),
    addmod(q_perm_left, sub(r, q_perm_right), r), r)
  mstore(q_perm_delta_base_ptr,
    mulmod(mload(q_perm_delta_base_ptr), q_perm_delta_chunk, r))
}

```

The active-row mask $1 - (L_{\text{last}} + L_{\text{blind}})$ zeroes the constraint on the last and blinding rows, exactly as Eq. (11.10) requires.

The δ -pow chunk offset, verified. The subtle part is how the per-set starting δ_{pow} is produced without an exponentiation. The codegen precomputes $\text{q_perm_delta_chunk} = \delta^{\text{chunk_len}}$ and advances the base by `*= delta_chunk` at the end of each set. A direct check confirms the constant: $\delta^{\text{chunk_len}} = \delta^3 = 0\text{x}42850883\text{...b}3982923$, matching the rendered literal. So set i ’s base is $\beta x \cdot (\delta^{\text{chunk_len}})^i = \beta x \delta^{i \cdot \text{chunk_len}}$ — the exact δ_{pow} start the Rust specifies, reached with one `mulmod` per set instead of a modular exponentiation.

The scratch-table gathering. Before the identity computation, the callback stages the permutation operands into contiguous scratch tables (`q_perm_vals`, `q_perm_sigmas`, `q_perm_z_cur`, `q_perm_z_next`, `q_perm_z_last`) via generated load loops (L1957–2002; comment at L1936–1940). The comment is explicit that `q_sp` is *not* a conventional VM stack here — the Rust memory planner must reserve `structured_permutation_scratch_words(meta)` whenever this opcode can appear. This is the same “codegen-time invariant invisible at runtime” theme: the scratch size is a generator contract, not something the running code can self-check.

Audit note. The product equality is the constraint that actually proves the permutation is a bijection (the boundary conditions only anchor the accumulator endpoints). A reviewer verifying the permutation argument must check: (i) the β, γ randomizers are the same in left and right (they are — both read `BETA_MPTR/GAMMA_MPTR`); (ii) the δ_{pow} advance is $\times \delta$ per column and $\times \delta^{\text{chunk_len}}$ per set (verified above); (iii) the active-row mask excludes exactly the last and blinding rows. The chunk structure (6 sets $\times$ 3 columns = 18 columns) is visible in the loop bounds `q_perm_num_cols=18, q_perm_num_sets=6, q_perm_chunk_len=3`.

Assessment. This is a [\[OK\]](#) finding — the product equality is a faithful rendering of the Rust constraint, and the δ -pow chunk mechanism is algebraically exact (the $\delta^{\text{chunk_len}}$ constant verified numerically). The [\[NOTE\]](#) is that this constraint was previously undocumented in the report while its three boundary siblings were covered — a reviewer relying on §11.8.31 alone would have missed the actual bijectivity proof.

11.8.49 The fail-closed dispatch chain: every unknown opcode reverts

A recurring [\[RISK\]](#) note in this report was “verify the default fails closed.” This section resolves it by direct inspection: the generated VM has a **three-layer fail-closed chain**, and no unknown opcode, over-running program, or partial expression can pass silently. (Tampering with the bytecode itself is excluded earlier, by the VK codehash pin at L920–923.)

Layer 1: the main opcode switch. The VM dispatch switch (L1700–2563, inside the loop at L1692) ends with an unconditional default:

```
// Invalid generated bytecode should fail closed.
// 0x1a intentionally lands here.
default {
    revert(0, 0)
}
```

Any of the 256 possible opcode bytes outside the 11 rendered cases (0x05/0x06/0x08/0x0d/0x10/0x11/0x21/0x19/0x1f/ 0x1b/0x0b) reverts immediately. The comment names 0x1a as an intentional fall-through target, confirming the default is a designed sink, not an oversight.

Layer 2: the heavy-gate u16 sub-dispatch. The 0x1b native-identity case extracts a 16-bit callback index (`q_native_idx := shr(240, mload(q_pc))`) that addresses a 65536-entry space but only has a handful of generated sub-cases (four in B64: 0–3, each invoked once by the program). That inner switch also ends with `default { revert(0, 0) }` (L2528), so a `q_native_idx` with no generated sub-case reverts rather than falling through. The manifest

validation (§10.8) guarantees generated indexes target existing cases; this default is the runtime backstop if that invariant is ever violated.

Layer 3: the post-VM terminal guards. After the dispatch loop, two guards prove the program was consumed exactly:

```
if iszero(eq(q_pc, q_end)) { revert(0, 0) }
if q_has_top { revert(0, 0) }
```

The first forces the program counter to land exactly on `q_end` (no over-read of operands past the end; an under-read shows up only indirectly, as a desynchronised decode — see §11.8.35); the second forces the stack to be empty (no partial expression left live). Together they mean a malformed generator output whose final opcode over-reads, or that leaves a dangling value, cannot reach the pairing.

Why the ISA-vs-dispatch gap is safe. The ISA comment advertises 31 assigned opcodes (0x01–0x16, 0x19, 0x1b–0x22; 0x17, 0x18, 0x1a unassigned) but only 11 have dispatch cases. **[CORRECTED]** The un-dispatched opcodes are absent because the generator renders arms only for opcodes present in the VK-pinned program (`quotient_program_usage`, `vm/mod.rs:1964–1988`), and the B64 program contains exactly the 11 rendered ones — not because 0x1c/0x1d/0x1e/0x20/0x22 are “fused into MODARITH7” or FOLD_MAIN “is a post-VM suffix” (§11.8.39, §11.8.43). The point for soundness is that *even if* one of these were emitted as a live opcode by a generator bug, Layer 1 reverts it. The discrepancy documented in §11.8.39 is therefore a readability concern, not a soundness hole.

Assessment. This is a **[OK]** finding that *upgrades* the prior **[RISK]** notes: the fail-closed property is confirmed at all three layers (opcode switch, u16 sub-dispatch, post-VM terminal guards). A reviewer no longer needs to chase each dispatch arm individually — the defaults are present and the terminal guards close the loop. The **[IMPROVE]** opportunity is to make the chain *visible* in the source: a single `fail_closed()` helper called from all three sites (with a distinct revert reason per layer) would let a reviewer confirm the property by reading one function rather than three scattered `revert(0,0)`s, and would give on-chain distinguishability between “bad opcode,” “bad native index,” and “bad program end” — currently all three revert with empty returndata. **[OPEN]** B64 still uses bare `revert(0, 0)` at all three layers (L2562, L2528, L2569–2570). The change is sound but costs a few bytes of code per site.

11.8.50 The MODARITH7 operand byte-encoding (reading the `q_program` by hand)

§11.8.33 explained *what* MODARITH7 fuses; this section documents *how each term is encoded in the bytecode*, which is what an auditor actually needs to read the 4559-byte `q_program` with human eyes. The operand bit-packing was verified by direct computation.

The big-endian load convention. `mload(q_pc)` loads 32 bytes with the byte *at* `q_pc` as the most-significant byte. So a 3-byte operand $[b_0\ b_1\ b_2]$ occupies bits 248–255, 240–247, 232–239, and `byte(0, word) = b_0`, and `(shr(232, word), 0xffff) = (b_1 << 8) | b_2`. Every operand layout below follows this convention.

The five term encodings.

- **LIN7** ($7 \times 3 = 21$ bytes per block), each limb term $[c\text{-idx} \mid \text{ptr}_{16}] \Rightarrow \text{q_acc} += c[c\text{-idx}] * \text{mload}(\text{ptr})$. Verified: bytes 11 22 33 decode to `c_idx=0x11`, `ptr=0x2233`.

- **BILIN7_ROW** (2 + 7×3 bytes): a `lhs` pointer (u16), then seven $[c\text{-idx} \mid \text{rhs}_{16}]$ terms $\Rightarrow q_acc += lhs * \sum_i c[i] * rhs[i]$.
- **BILIN7_PAIRWISE** (4 + 13 bytes): $[lhs_base_{16} \mid rhs_base_{16}]$ followed by a 13-byte coefficient table indexed by $i + j \Rightarrow$ the 7×7 convolution $\sum_{i,j} c[i+j] lhs_i rhs_j$. Verified: bytes AA BB CC DD decode to `lhs_base=0xAABB`, `rhs_base=0xCCDD`.
- **mem term** (3 bytes): $[c\text{-idx} \mid ptr_{16}] \Rightarrow c * mload(ptr)$ (the escape-hatch single term, §11.8.41).
- **product term** (5 bytes): $[c\text{-idx} \mid lhs_{16} \mid rhs_{16}] \Rightarrow c * mload(lhs) * mload(rhs)$. Verified: bytes 11 22 33 44 55 decode to `c-idx=0x11`, `lhs=0x2233`, `rhs=0x4455`.

Why the 13-entry pairwise table. The 7×7 convolution has 49 products but only 13 distinct $i + j$ values (0...12), so the coefficient table is 13 bytes, not 49 — the coefficient for cell (i, j) is read at `q_coeff_pc + (i+j)`. This is the same $i+j$ -indexing noted in §11.8.40; the byte layout makes the 49→13 compression explicit. A reviewer can confirm the table length (13) against the maximum $i + j = 6 + 6 = 12$.

The fixed-width advance. Because each block/term type has a fixed byte width (LIN7 21, ROW 2+21, PAIRWISE 4+13, mem 3, product 5; matching `quotient_modarith7_op_len_checked`, `vm/mod.rs:2136–2185`), `q_pc` advances by a constant per term with no per-term tag byte — the five counters in the counts word (§11.8.41) tell the interpreter how many of each type follow. This is what makes the encoding compact: the term *type* is implied by which counter loop is executing, not repeated per term.

Audit note. This is the cross-check surface for the `q_program` bytes. A reviewer verifying a rendered verifier by hand should: (i) decode the counts word to learn the term mix; (ii) for each LIN7/mem term confirm the 3-byte $[c\text{-idx} \mid ptr]$ layout and that `ptr` points at a live memory slot; (iii) for BILIN7_PAIRWISE confirm the 13-byte table length matches $2 \cdot 7 - 1$; (iv) confirm the gate condition (flag bit 0) is applied *last* (L1916–1920) so every subterm shares the same selector. The bit-packing verified above means the decoding is mechanical, not interpretive.

Assessment. This is a [OK] finding — the operand encoding is a faithful, compact big-endian packing (verified numerically) that preserves the exact coefficient/pointer semantics of the unfused identity. The [IMPROVE] note is that the byte-level layout is invisible in the rendered contract: a reviewer must reverse-engineer the bit shifts from the `shr/and` idioms. Emitting a one-line operand comment per term type in the generated Yul (as this section does) would make the `q_program` self-describing at the source. [PARTIAL] B64 does render one semantic comment per block type (L1822, L1837, L1859–1860, L1885, L1898) and byte-layout comments for simple opcodes (e.g. 0x05, 0x10, 0x0b), and the template carries a per-opcode byte-layout table (`QuotientNumeratorBlock.yul:188–340`) — but that table is a Jinja `{# #}` comment and is not rendered, and no byte layout is printed for the MODARITH7 term blocks.

11.8.51 The codegen emitter that produces the MODARITH7 bytes

§11.8.50 documented how the runtime *reads* the MODARITH7 operand bytes. The mirror-image question — how the codegen *writes* them — is answered by `emit_modarith7_shape` in `quotient_numerator/vm/mod.rs` (L1514–1589), and the two agree byte-for-byte. This is the cleanest codegen↔runtime correspondence in the whole system, because the emitter’s push order *is* the interpreter’s read order.

The push sequence. The emitter writes, in order (abridged; each slot is interned via `const_slot` then narrowed with `u8::try_from(...).expect(...)`):

```
self.bytes.push(Q_OP_MODARITH7);          // 0x21
let mut flags = 0u8;
if shape.cond.is_some() { flags |= Q_MODARITH7_FLAG_COND; } // 0x01
if shape.constant != U256::ZERO { flags |= Q_MODARITH7_FLAG_CONST; } // 0x02
self.bytes.push(flags);
if let Some(cond) = shape.cond { self.u16(cond as usize); }
if shape.constant != U256::ZERO { self.bytes.push(const_slot(shape.constant)); }
// five one-byte counts
self.bytes.push(lin_count); self.bytes.push(row_count);
self.bytes.push(pairwise_count); self.bytes.push(mem_count);
self.bytes.push(product_count);
// then the term blocks, same order as the interpreter reads
```

This is exactly the layout the Solidity decodes: opcode, flags byte, optional cond u16, optional const slot, the five counts word, then LIN7/ROW/PAIRWISE/mem/product blocks.

The flag decision is structural. The two flag bits are set from the identity’s *shape*, not from any runtime value: `FLAG_COND` iff the identity has a condition factor (`shape.cond.is_some()`), `FLAG_CONST` iff it has a nonzero standalone constant (`shape.constant != ZERO`). The codegen inspects the expression tree and chooses the compact encoding; the interpreter just follows the flags. This is the “codegen-time invariant invisible at runtime” theme again — the flag *means* something the generator knew and the VM only obeys.

Coefficient interning via `const_slot`. Every coefficient in every term block is passed through `const_slot(coeff)`, which interns the field element into the quotient constant table and returns its slot index; the emitted byte is that index. This is why §11.8.33’s comment says “all coefficients still come from the generated quotient constant table” — the byte is a table index, not the value. The same coefficient appearing in many terms occupies one table slot, which is the space saving.

The u16 pointer helper. `self.u16(ptr)` writes a memory pointer as two bytes, matching the interpreter’s `and(shr(232,word),0xffff)` read. The emitter and interpreter agree on the 2-byte big-endian pointer width without a shared schema — the correspondence is enforced by tests: a Rust re-implementation of the byte interpreter (`eval_quotient_program_artifacts_for_test`, `tests.rs:3005`) is checked against direct expression evaluation, and the fixture suites under `tests/` run real proofs through rendered verifiers.

Codegen-time count validation. Each count is wrapped in `u8::try_from(...).expect("modarith7 ... count checked")` (L1535–1541). The codegen *refuses to emit* a program whose term count exceeds 255 per category — a codegen-time invariant that guarantees the one-byte count fields never overflow. In practice the `expect` is a backstop: the recognizer already returns `None` for > 255 blocks or terms (`vm/mod.rs:2846–2851`, `2875–2877`), so an oversized identity falls back to other lowering rather than panicking. The runtime does not re-check (it trusts the pinned bytecode).

Assessment. This is a [OK] finding — the emitter and interpreter are byte-for-byte consistent, the flag decision is structural, coefficients are interned (the space saving that justifies the fusion), and the count fields are validated at generation time. The [NOTE] is that this correspondence is the *strongest* audit lever available for the VM: a reviewer who reads `emit_modarith7_shape` side-by-side with the interpreter’s read order can verify the entire fused-identity encoding mechanically, without executing anything. Pairing this section with §11.8.50 gives both halves of that cross-check.

11.8.52 The fusion decision: when the codegen fuses vs emits separate opcodes

The rendered `q_program` is a *mix* of fused MODARITH7 opcodes and separate arithmetic opcodes (B64: 16 MODARITH7 among 101 instructions). Why some identities fuse and others don't is decided by a two-level gate that was not previously documented — and understanding it is necessary to predict what the rendered program should look like for a given circuit.

Level 1: the shape/eligibility test (`vm/mod.rs`). **[CORRECTED]** An earlier version placed Level 1 in `yul_emit.rs`'s `should_fuse` (`yul_emit.rs`:894–899). That predicate only decides whether a sum is flattened into constant + scaled terms when emitting *Yul text* (`evaluate_sum_with_coeff_and_const`). It has no bearing on MODARITH7: gate identities reach the VM through the typed `quotient_expr_from_expression`, not through that text. The real semantic gate is the eligibility check at the end of `quotient_modarith7_affine_shape` (`vm/mod.rs`:2879–2897, §11.8.56): the expression must decompose completely into limb blocks, mem/product terms and a constant; it must contain a limb block *or* be a conditional sum with products and at least two components; and an unconditional single component is rejected.

Level 2: the encoding gate (`vm/mod.rs`). Even a semantically-fusable identity only becomes MODARITH7 if the compact byte encoding is available:

```
fn try_emit_modarith7_shape(&mut self, expr: &QuotientExpr) -> bool {
    if !self.limb_vm_ops { return false; }
    let Some(shape) = quotient_modarith7_shape(expr) else { return false; };
    if !self.modarith7_shape_has_u8_const_slots(&shape) { return false; }
    self.emit_modarith7_shape(shape);
    true
}
```

Three conditions must hold: the limb-VM is enabled, the expression matches the MODARITH7 shape, and **all coefficients fit one-byte constant-table slots**. The last check is a *non-mutating peek* (`peek_u8_const_slots`) — if the coefficients would not fit, the builder is left untouched and `emit_expr` moves on to the next strategy rather than interning coefficients that would bloat the table.

The fallback path. When either gate fails, `emit_expr` (`vm/mod.rs`:1282–1294) tries, in order, a standalone limb opcode (`LIN7/BILIN7_*`, `0x1c–0x1e`), the limb-decomposition peephole, and finally the generic opcode sequence (`PUSH_*`, `MUL`, `ADD`, ..., with the `ADD_MUL_*` product-add peepholes). (`POW5` is tried before MODARITH7.) This is why the rendered program is heterogeneous: fusion is *opportunistic*, taken only when the compact encoding is available without cost. A reviewer who sees a non-fused identity should not read it as a missed optimization — it means the identity's coefficients or shape did not fit the MODARITH7 encoding.

Why the u8-slot peek matters for space. The one-byte count and slot fields are what make MODARITH7 compact. If a coefficient's table slot exceeded 255, the fused encoding would need wider fields and lose its advantage. The `has_u8_const_slots` peek enforces *addressability* before committing: the codegen only fuses when every coefficient lands in a one-byte slot. It is not a size comparison against the unfused form. This is a generation-time decision, invisible in the runtime.

Audit note. This two-level gate is the explanation for the program's structure. A reviewer cross-checking a rendered verifier against the circuit should: (i) for each fused MODARITH7, confirm the identity is a (possibly condition-gated) affine/bilinear sum over memory words (Level 1); (ii) confirm its coefficients are in the low constant-table slots (Level 2); (iii) for non-fused identities, confirm they fail the shape eligibility test or have out-of-range coefficient slots. The gate makes the fusion *predictable* from the circuit, which is what a manual cross-check needs.

Assessment. This is a **[OK]** finding — the fusion decision is a sound two-level gate (semantic worthiness + encoding feasibility) that never fuses when the compact form is unavailable, and the non-mutating peek keeps the constant table clean. The **[IMPROVE]** note is auditability: the gate’s outcome is not recorded in the rendered program, so a reviewer must re-derive “why did this fuse?” from the circuit. Emitting a per-identity fusion-decision annotation in the trace manifest (fused / not-fused + reason) would make the program’s structure self-explaining. **[PARTIAL]** The execution manifest already records each identity’s execution surface (`QuotientExecutionKind`: `Inline` / `Interpreted` / `Native*` / `StructuredTail`, `vm/mod.rs`:345–361), but not which opcode form an interpreted identity was lowered to, or why.

11.8.53 The codegen’s offline bytecode validator (static analysis before render)

Before the generated `q_program` is ever rendered, the codegen runs a **structural offline static analyzer** over its own emitted bytes. This is the codegen-time counterpart to the runtime’s fail-closed dispatch (§11.8.49) — a defense-in-depth layer that catches malformed generator output *before* it reaches the chain. The driver is `validate_quotient_program` (`vm/mod.rs`:2003–2021), called from `QuotientProgramBuilder::finish` (L1206–1225), which panics on any error and also asserts that the validated stack depth equals the builder’s own accounting. Two validators work together: a length validator and a stack-effect simulator (plus memory-token validation for `0x03/0x04`).

The length validator. `quotient_modarith7_op_len_checked` (L2109–2188) walks the emitted bytes and verifies the layout is self-consistent:

- The flags byte is checked against the known-bit mask: `flags & !(COND | CONST) != 0` is rejected — an unknown flag bit is a hard error, not silently ignored.
- The optional `cond u16` and `const slot` advance only when their flag bits are set (matching the emitter).
- Each term block’s total length is computed with `checked_quotient_len_mul` (L2318), which uses Rust’s `checked_mul` and errors on overflow — a malformed count cannot silently wrap the cursor past the program end.

The function returns the total opcode length, so the caller can verify the whole program’s byte layout is consistent.

The stack-effect simulator. `apply_quotient_stack_effect` (L2191–2255) models the VM stack depth for every opcode, with explicit boundary requirements that mirror the runtime’s expectations:

- **Push opcodes** (`PUSH_*`, `LIN7`, `BILIN7_*`, `MODARITH7`): `depth += 1`, and `max_stack` is tracked for the depth bound.
- **Binary ops** (`ADD`, `MUL`): require `depth ≥ 2`, then `depth -= 1`.
- **Unary ops** (`NEG`, `ADD_CONST`, `MUL_CONST`, `AFFINE_SUM`, `POW5`, ...): require `depth ≥ 1`.
- **FOLD_MAIN / FOLD_SELECTOR**: require **exactly 1** value on the stack (the identity result), then set `depth = 0`.
- **Native callbacks** (`PERMUTATION`, `LOOKUP`, `IDENTITY`): require an **empty stack** (`depth == 0`) — they own their scratch layout, so a non-empty stack at a native callback is a structural error.
- **Unknown opcode**: the catch-all returns an error — the validator rejects any opcode it does not model.

Why this is defense-in-depth. The runtime already fails closed on bad opcodes (§11.8.49). The offline validator adds a *second* layer: it catches structural errors (stack underflow, wrong fold boundary, native callback with a dirty stack, overflowed length) at *generation* time, when they are a generator bug to fix, rather than at *verification* time, when they would reject a valid proof or accept a bad one. The two layers are complementary: the offline validator proves the program is well-formed; the runtime default proves no unknown opcode executes.

The native-callback empty-stack rule. The requirement that native callbacks see an empty stack (`depth == 0`) is the static counterpart to the runtime `q_top := 0; q_has_top := 0` reset each callback performs (e.g. §11.8.48). The validator proves the reset is *sound* — the stack is already empty at the callback boundary, so the reset is a belt-and-braces clear, not a recovery from a dirty state.

Assessment. This is a [OK] finding — the codegen runs a structural static analyzer (length + stack-effect) over its own output before rendering, with overflow-checked arithmetic and explicit boundary requirements. The [NOTE] is that this validator is the strongest evidence of the generator’s *structural* correctness: a reviewer who trusts the validator’s opcode model can trust that every rendered program is well-framed and stack-balanced, without inspecting each render. It is not a semantic check: it does not verify constant-slot indices against the table length, u16 pointers against the memory plan, or selector index/gap operands. Those are correct by construction of the emitter and are covered by the equivalence tests, not by this pass. The [IMPROVE] note is that the validator’s opcode model must be kept in sync with the emitter and the interpreter — a three-way correspondence (emitter, validator, interpreter) that is maintained by the test suite but not by a single source of truth. [PARTIAL] Opcode numbers and fixed byte lengths already come from one table (QUOTIENT_OPCODE_TABLE, vm/mod.rs:673–862), which feeds both the validator and the template constants (`template_constants.quotient_vm.op.*`). The dynamic layouts (MODARITH7, AFFINE_SUM, runs) and the stack effects are still hand-written separately in the emitter, the validator, the Yul template and the Rust test interpreter.

11.8.54 Constant interning: hash-consing the quotient constant table

The coefficient interning referenced in §11.8.51 is implemented as a **hash-consing table** with a non-mutating peek that makes the fusion gate (§11.8.52) clean. Three functions in `vm/mod.rs` implement it.

const_slot: hash-consing insertion.

```
fn const_slot(&mut self, value: U256) -> u16 {
    if let Some(slot) = self.const_slots.get(&value) { *slot }
    else {
        let slot = self.consts.len();
        assert!(slot <= u16::MAX as usize, "too many quotient constants");
        self.consts.push(value);
        self.const_slots.insert(value, slot as u16);
        slot as u16
    }
}
```

A value already in the table returns its existing slot; a new value is appended and assigned the next slot. This is classic hash-consing: **identical coefficients occupy one slot**, so a constant reused across many identities (e.g. the 2^{56} base powers in foreign-field lowering, §11.8.38) is stored once. This is the concrete space saving that makes the byte-indexed fused encoding viable.

The u16 slot space with a hard assert. Slots are u16, with `assert!(slot <= u16::MAX)` — the generator refuses to build a program with more than 65536 distinct constants (slots 0–65535).

This is a codegen-time bound (the generator panics, never silently truncates). The binding VK-size limit is tighter and separate: `consts.len()` must fit the reserved `quotient_const_words` (`plan.rs:266`, `quotient.rs:161`). B64 uses 178 slots.

peek_u8_const_slots: the non-mutating batch simulation. The fusion gate needs to know whether a batch of coefficients *would* fit one-byte slots *without* inserting them. The peek simulates the insertion with a local `pending` map:

```
let mut next_slot = self.consts.len();
let mut pending = HashMap::new();
for value in values {
    let slot = if let Some(s) = self.const_slots.get(value).copied() { s as usize }
        else if let Some(s) = pending.get(value).copied() { s }
        else { let s = next_slot; next_slot += 1; pending.insert(*value, s); s };
    slots.push(u8::try_from(slot).ok()?);
}
```

The `pending` map tracks values that *would* be newly interned, so a value repeated within the batch gets the same simulated slot as it would on real insertion. If any simulated slot exceeds 255, the peek returns `None` and the fusion gate falls back — **without having mutated the table**.

Why the peek is essential for a clean fallback. The doc comment (L1854–1856) states the invariant precisely: the peek “lets limb opcode recognition fail cleanly before mutating the constant table, so fallback lowering sees the same builder state it would have seen if recognition had never been attempted.” Without this, a failed fusion attempt would have already interned the coefficients, bloating the constant table for a program that doesn’t use the fused encoding — a space regression caused by a rejected optimization. The peek makes the fusion decision *reversible*.

Assessment. This is a [OK] finding — the constant table is a correct hash-consing interner (deduplication saves space), the u16 slot space is hard-bounded by an assert, and the non-mutating peek makes the fusion gate cleanly reversible. The [NOTE] is that the peek’s `pending-map` simulation must exactly mirror `const_slot`’s insertion semantics — if the two ever diverge (e.g. a new dedup rule added to one but not the other), the peek could predict slots that the real insertion wouldn’t assign, causing a fusion that later fails the u8 check at emit time. The test suite guards this, but it is a subtle coupling. [PARTIAL] A related failure did occur and was fixed in 6b3d2096: in the limb-decomposition and product-add peephole the peek ran *before* the residue/base was emitted, and that emission could intern new constants and push a coefficient past slot 255, panicking in `u8::try_from(...).expect(...)`. Both paths now re-check after the intervening emission and fall back to generic ops (`vm/mod.rs:1356–1381`, 1615–1650; regression test `quotient_vm_limb_decomposition_survives_const_table_overflow`). The MODARITH7 and whole-expression limb paths peek and emit back to back, so they are not exposed to that staleness. The peek/`const_slot` semantic coupling itself remains.

11.8.55 Run compaction: a peephole optimizer over the emitted bytecode

After the initial emit, the codegen runs a **second-pass peephole optimizer**, `compact_quotient_runs` (L1879–1954), that scans the emitted bytes and replaces long adjacent runs of fused add-mul opcodes with a single counted superinstruction. This is a genuine compiler optimization pass — not part of the lowering decision, but a post-hoc byte-level rewrite that shrinks the program. In B64 it is a no-op: the production program contains no 0x12/0x13 terms to compact and no 0x15/0x16/0x22 superinstructions.

The two compaction forms. The optimizer detects a maximal run of adjacent affine-add ops (`ADD_MUL_CONST_U8_MEM_U16` = linear, `ADD_MUL_MEM_MEM_CONST_U8` = product) and applies one of two forms:

- **Mixed `AFFINE_SUM`** (L1909–1925): when the run contains *both* linear and product terms and the total $\geq$ `run_compaction_min_len`, emit `Q_OP_AFFINE_SUM` followed by a u16 `lin_count` and u16 `product_count`, then all the linear operands grouped together, then all the product operands grouped together.
- **Homogeneous counted `RUN`** (L1926–1943): when the run is all the *same* op and $\geq$ `run_compaction_min_len`, emit a counted run opcode (`RUN_ADD_MUL_MEM_MEM_CONST_U8` or `RUN_ADD_MUL_CONST_U8_MEM_U16`) with a u16 count followed by the repeated operands.

If neither threshold is met, the run is copied verbatim (L1944–1946).

The commutativity reordering. In the mixed `AFFINE_SUM` form the operands are *reordered* — all linear operands first, then all product operands — rather than kept in original order. This is sound because the accumulator update is `q_acc += term` for each term, and addition is commutative and associative, so the final sum is order-independent. The comment (L1882–1884) states this explicitly: “addition is commutative and each term only mutates the same accumulator.” This is the algebraic justification for the grouping.

The min-length threshold. Short runs are *not* compacted — the superinstruction header (opcode + u16 count(s)) would cost more bytes than it saves on a short run. The `run_compaction_min_len` threshold is 4 (`QUOTIENT_VM_RUN_COMPACTION_MIN_LEN`, `vm/mod.rs:533`). **[CORRECTED]** It is not a byte-exact net-win test. A homogeneous `RUN` of n terms saves $n - 3$ bytes (drop n opcode bytes, add 3 header bytes), a win from $n=4$. The mixed `AFFINE_SUM` header is 5 bytes, so at $n=4$ it is one byte *larger* and at $n=5$ break-even. It may still save interpreter dispatch gas, but the “only when smaller” reading is inaccurate for the mixed form.

The `u16::MAX` run cap. The inner scan breaks when `lin_count` or `product_count` reaches `u16::MAX` (L1904–1906), capping a single superinstruction at 65535 terms per category so the u16 count field never overflows. A longer run is split into multiple superinstructions.

Assessment. This is a **[OK]** finding — the run compaction is a correct peephole optimizer that preserves the logical operation stream (the comment’s explicit guarantee), with the commutativity reordering justified by the accumulator’s additive structure, a min-length threshold (a byte win for homogeneous runs; not for 4–5-term mixed runs), and a u16 cap preventing count overflow. The **[IMPROVE]** note is that this second-pass rewrite is invisible in the rendered program: a reviewer reading the `q_program` sees the compacted superinstructions but not the pre-compaction op stream they were compacted from, making it harder to trace a superinstruction back to the original identity terms. Emitting the pre-compaction op sequence in the trace manifest would make the compaction auditable. **[OPEN]** This is not implemented, but it is moot for B64, whose program contains no compacted superinstructions.

11.8.56 Greedy shape recognition: how an expression is matched to MODARITH7

§11.8.52 showed the fusion *gate*; this section shows the *recognition* that feeds it — `quotient_modarith7_affine_shape` (L2774–2914), a greedy pattern-matcher that decomposes an expression tree into the five MODARITH7 term categories.

The condition-factor split. Before shape matching, `quotient_condition_factor` (L2758–2770) peels an outer `cond * inner` when `cond` is a literal memory load, moving any scalar factor attached to `cond` into `inner`:

```
if let Some((coeff, ptr)) = quotient_mem_term(lhs) {
    return Some((ptr, quotient_scaled_term_expr(coeff, rhs)));
}
```

This is what populates the `FLAG_COND` operand (§11.8.51): the gate condition is extracted as a pointer, and its scalar coefficient is folded into the inner expression so the gate multiplies the whole sum.

The greedy extraction order. The core loop extracts the *richest* sub-shape first, removing its used terms and repeating:

1. **Pairwise** (`try_quotient_bilin7_pairwise_subshape`) — the 7×7 convolution, the most compact encoding, so it is claimed first.
2. **Factored row** (`try_quotient_factored_bilin7_row_subshape`) — a bilinear row with a factored left-hand side.
3. **Plain row** (`try_quotient_bilin7_row_subshape`).
4. **LIN7** (`try_quotient_lin7_subshape`) — the simplest shape, claimed last.

This is a greedy set-cover: the encoding that packs the most terms per byte wins first, leaving the simpler shapes for the remainder. The order is a space-optimization heuristic, not a semantic requirement — any order that covers the terms produces a correct program; this order is *intended* to minimize the byte count (a heuristic — see the assessment below).

The leftover grouping. Terms not absorbed by a limb shape are grouped into the escape-hatch categories: `grouped_mem` (single memory terms, keyed by pointer) and `grouped_products` (binary products, keyed by (lhs,rhs)), with coefficients *summed* for matching keys and then **sorted by pointer** (L2871–2874). The sort makes the output *deterministic* — the same expression always produces the same bytecode, which is essential for reproducible VKs and byte-for-byte test comparisons.

The fusion-eligibility validation. The final gate (L2875–2897) decides whether the recognized shape is worth fusing:

- Each category must fit `u8::MAX` (255) terms.
- `component_count` = `lin` + `rows` + `pairwise` + `mem` + `products` + constant.
- Fusion requires a **limb component** (some `lin`/`rows`/ `pairwise` present) *or* **sparse conditional products** (a condition present, products present, and `component_count` ≥ 2).
- A single component with no condition is *not* fused (`cond.is_none()` && `component_count` < 2) — it would add encoding overhead with no benefit.

Assessment. This is a [OK] finding — the shape recognition is a correct greedy matcher with a sensible richest-first order, deterministic sorted output, and a well-defined eligibility gate that refuses to fuse when the encoding wouldn't pay for itself. The [IMPROVE] note is that the greedy order is a heuristic whose optimality is not proven — a different extraction order could in principle produce a smaller program for some expressions (the classic greedy-vs-optimal set-cover gap). For a verifier this is acceptable (the program is generated once and pinned), but a reviewer should not assume the rendered encoding is the *minimal* one, only a compact one.

11.8.57 G1ADD, subgroup cancellation, and whether G1ADD can be eliminated

EIP-2537 is explicit that **G1ADD performs no subgroup check** (it checks only that inputs are on-curve or infinity), whereas G1MSM and the pairing *must* reject off-subgroup inputs. This raises a sharp soundness question: if a G1ADD result flows into the pairing (which subgroup-checks the *result*), can a prover hide an off-subgroup point by making the *sum* land in the subgroup?

The cancellation math: possible in general. Since $\#E(\mathbb{F}_p) = h \cdot r$ with $\gcd(h, r) = 1$, the curve group decomposes as a direct product $E(\mathbb{F}_p) \cong G_1 \oplus E(\mathbb{F}_p)[h]$. Every point splits $P = P_r + P_h$ into its r -part and its cofactor-part. Then

$$\text{G1ADD}(P, Q) = (P_r + Q_r) + (P_h + Q_h), \quad (11.13)$$

which lies in G_1 iff $P_h + Q_h = 0$. A prover can choose $Q = S - P$ with $S \in G_1$: both P and Q are off-subgroup, yet $P + Q = S \in G_1$. **So yes — a G1ADD can return an in-subgroup point from off-subgroup inputs, and G1ADD’s on-curve check would not flag it.**

In this contract: not reachable. The contract never feeds an *unchecked* proof point into a G1ADD. Every G1ADD operand is either a **G1MSM output** (or a VK constant), or a point that an earlier G1MSM on the same **success** chain has already subgroup-checked. G1MSM subgroup-checks its inputs, so its output is guaranteed in G_1 (cofactor part = 0). **[CORRECTED]** An earlier draft said “every operand is a G1MSM output”. That is literally false for L3550: PAIRING_LHS is π copied raw at L3471, and it is safe only because the G1MSM(x_3, π) at L3486 validated π first. The pairing precompile also subgroup-checks its G1 inputs, so a cofactor component that survived a G1ADD would still be rejected. The four runtime G1ADDs (the fifth 0x0b call, L224, is the constructor smoke test on identity inputs):

Line	Operation	Operand provenance
L3480	G1ADD(final_com, A)	final_com = G1MSM (78 pairs, L3460); $\mathcal{A} = \text{G1MSM}(-v, G)$
L3490	G1ADD(B, C)	$B \in G_1$ (above); $C = \text{G1MSM}(x_3, \pi)$
L3536	G1ADD(PAIRING_RHS, M)	PAIRING_RHS $\in G_1$; $M = \text{G1MSM}(\alpha, \text{ACC_RHS})$
L3550	G1ADD(PAIRING_LHS, M)	PAIRING_LHS = raw π (L3471), subgroup-checked by the G1MSM at L3486; $M = \text{G1MSM}(\alpha, \text{ACC_LHS})$

Because a G1MSM output has *zero* cofactor component, it *cannot* cancel a proof point’s P_h — and there is no off-subgroup operand present to cancel with. Every G1ADD here is $G_1 + G_1 \rightarrow G_1$. The pattern is deliberate: **G1MSM is the subgroup gate; G1ADD operates only downstream of it.** Raw points (final_com’s inputs, π , ACC_RHS, ACC_LHS) are each forced through a G1MSM (which checks them) *before* any G1ADD touches them.

On-curve: doubly safe. G1ADD *does* check on-curve on its inputs, and the sum of two on-curve points is on-curve (group closure). The pairing re-checks on-curve too. So on-curve is enforced at every step — no gap.

The transcript: deferred but sound. The transcript absorbs raw proof points with only a range check (common_uncompressed_g1, $x, y < p$), *not* a subgroup check — so challenges are derived from unchecked bytes. But every absorbed point is subsequently passed through a G1MSM or the pairing, which **reverts** on an off-subgroup point. A prover who injects a small-subgroup point into the transcript cannot produce an *accepting* proof: the gate rejects it before completion. The deferred check is sound *provided every absorbed point flows to a checking precompile*.

The load-bearing invariant (the real finding). The safety rests on a structural invariant that is **not locally enforced**:

- Every *G1ADD* operand is a *G1MSM* output / VK constant.
- Every transcript-absorbed point flows to a *G1MSM* or the pairing.

The contract does not assert these at runtime — it trusts the generator. If a future codegen change ever emitted a *G1ADD* with a **raw proof-point** operand (two prover-controlled points added), the cancellation attack becomes live: the prover could make the sum land in G_1 while one input is an unchecked small-subgroup point. **Recommendation:** the generator should *assert* the invariant at emit time — statically verify every *G1ADD*’s operands are MSM outputs/constants, and that every transcript-absorbed commitment appears in the final MSM/pairing input set — turning “trust the structure” into a machine-checked property. **[PARTIAL]** The second bullet is already asserted at generation time: `ProtocolPlan::validate` (`src/lowering/protocol/mod.rs:778ff`, logical PCS coverage) and `validate_absorbed_g1_precompile_coverage` (`src/lowering/kzg/mod.rs:602--619`, concrete memory handles, run from `validate_generator_invariants` at `plan.rs:245`; unit tests `kzg/mod.rs:1815--1826`); B64 L387–391 documents this. Both checks run on the generator’s plan model (memory handles), not on the emitted Yul; the end-to-end test mutates every proof *G1* off-curve (`src/test.rs:2408`), but no test feeds an on-curve, off-subgroup point, the only case *G1ADD*’s own on-curve check would let through. The first bullet (*G1ADD* operand provenance) has no check: the *G1ADD* sites are hand-emitted (`kzg/mod.rs:1735--1760`, `FinalPairing.yul:41,55`).

Can we eliminate *G1ADD* entirely? Yes in principle — a sum of scalar-multiplications is exactly what a *G1MSM* computes — but the cost depends on the α -ordering. Three variants (RHS side including the $\alpha \cdot \text{ACC_RHS}$ term; $\text{G1MSM} = k \cdot 12000 \cdot \text{discount}_k / 1000$, $\text{G1ADD} = 375$):

Variant	G1ADDs	RHS gas	Trade-off
A (current)	3	$\approx 562,221$	78-pair MSM, then $3 \times (\text{MSM-1} + \text{ADD})$
B (fold $-vG$, $x_3\pi$ into 80-pair MSM)	1	$\approx 549,015$	saves $\sim 13.2\text{k}$, no soundness change
C (α known early, 81-pair MSM)	0	$\approx 542,376$	saves $\sim 6.6\text{k}$ more, but changes FS binding

[CORRECTED] An earlier version priced A at 549,846 (two *G1ADD*s, omitting the α term that B and C include), which made B look like a $\sim 0.8\text{k}$ saving. The $-vG$ and $x_3\pi$ terms have scalars known before the final MSM, so they fold into it cheaply (Option B, eliminates 2 *G1ADD*s with no soundness impact): the two extra pairs cost 11,544 inside the 78-pair MSM versus 24,750 for $2 \times (\text{MSM-1} + \text{ADD})$. The generator’s comment (`kzg/mod.rs:1701--1704`) only rejects a *standalone* 3-pair MSM, not this fold. The $\alpha \cdot \text{ACC}$ terms are the blocker: α is derived *after* the pairing inputs are materialized (Fiat-Shamir over the materialized points, to stop the KZG and accumulator equations cancelling), so they cannot fold into the early MSM. As a standalone 2-pair MSM the α -add costs 22,776 vs **MSM+ADD 12,375** — i.e. **G1ADD at 375 gas is the cheap glue**, and folding it into a small MSM is a $\sim 10.4\text{k}$ loss. Full elimination (Option C) requires making α available before the final MSM, which changes the Fiat-Shamir randomizer binding — a soundness-sensitive re-derivation. It is feasible: the transcript state after π already determines all four pairing inputs (the accumulator points are decoded from absorbed instances), so α can hash that state plus ACC_LHS/RHS ; the $K \cdot A^\alpha$ argument of §11.8.66 still needs to be re-stated for it. Option C removes only the RHS *G1ADD*s; the LHS $\pi + \alpha \cdot \text{ACC_LHS}$ add (L3550) stays unless it too becomes a 2-pair MSM (+10.4k).

Assessment. This is a **[OK]** finding on the current artifact (cancellation is mathematically possible but structurally unreachable), with a **[RISK]** on the un-asserted G1ADD half of the invariant and a **[IMPROVE]** path: adopt Option B now ($\sim 13.2\text{k}$ gas, removes 2 G1ADDs, no soundness change), and consider Option C only with a careful randomizer re-derivation. **[OPEN]** HEAD still emits MSM-1 + G1ADD pairs (kzg/mod.rs:1706--1770, FinalPairing.yul:41,55). Notably, **eliminating G1ADD entirely also eliminates the cancellation concern** — with no G1ADD, there is no unchecked-add for the attack to exploit; the G1MSM checks everything. That makes G1ADD elimination a defense-in-depth win on top of the gas saving.

11.8.58 Static analysis tooling run: results and false-positive triage

Four tools were run against the generated verifier and VK contracts. The headline is a **meta-finding**: the contract’s assembly-heavy structure largely defeats the automated tools — stock Slither crashes, one produces false positives on assembly patterns, one finds nothing — which is precisely why the manual deep-read of this report was necessary. The soundness surface lives in inline Yul that the tools cannot analyze. (A later, patched Slither run, `nightfall_4_PV/audit/slither-2026-09-29`, does complete; see the table.)

Tool-by-tool results.

Tool	Outcome	Findings & triage
solc 0.8.35	compiles <i>only</i> with <code>--via-ir</code>	Stack-too-deep without <code>viaIR</code> (real build constraint). Unused-param warnings on <code>proof/instances</code> (read in assembly). Runtime 21,253 B (via-IR, 200 runs, Cancun; re-measured) — under EIP-170 24 KB ($\sim 3.3\text{KB}$ headroom). Yul analyzer: no diagnostics, but the via-IR runtime starts <code>PUSH2 0x08e0 PUSH1 0x40 MSTORE</code> : spilled variables take <code>[0x80,0x8e0)</code> , which overlaps the raw scratch (§11.8.19).
slither 0.11.6	crashes unpatched; runs with patches	Stock: RecursionError in <code>IR→SSA (add_phi_origins)</code> , recursion per dominator-tree level on long straight-line Yul. [CORRECTED] not unfixable: with local patches 0001-0003 it completes (<code>verifyProof = 2,816</code> CFG nodes); <code>divide-before-multiply</code> $\times 4$ FP; one real dead store (<code>q_perm_eval</code> , L2003-2004); 5 truly unused constants; all 18 <code>staticcalls</code> check <code>returndatasize</code> .
aderyn (88 det.)	2 High / 3 Low, all triaged	H-1 shift order FP (<code>shl(used_bits,1)=2^{used_bits}</code> correct, L656). H-2 <code>return</code> in Yul FP (intended terminal / constructor return). L-1 <code>PUSH0</code> (L2 note). L-2 unspecific pragma (valid). L-3 unused state var $\times 76$: 71 FP (assembly refs invisible), 5 true (see below).
semgrep (rules)	(68 0 findings)	Solidity ruleset targets high-level patterns (reentrancy, <code>tx.origin</code>); none apply to an assembly verifier.

The aderyn and semgrep rows could not be re-checked: no raw output is archived with the audit evidence. The one archived semgrep run (1.176.1, `p/smart-contracts`; register CF-1019) reported 14 performance-category findings, not zero, so the semgrep result depends on the ruleset.

Why the tools miss the real surface. Aderyn’s own L-3 note is the giveaway: *“No analysis has been performed to see if any inline assembly references it.”* Of the 76 “unused” constants, 71 are named slots the Yul depends on; five are genuinely dead (`CHALLENGE_MPTR`, `Q_COM_MPTR`, `G1_IDENTITY_MPTR`, `QUOTIENT_RETURN_MPTR`, `TRACE_U256_MPTR`; none is referenced by any code, and the `G1_IDENTITY_MPTR` comment at L144–149 describes “four mloads below” that do not exist). Stock Slither’s SSA pass recurses once per dominator-tree level and overflows on this long straight-line Yul. Semgrep’s Solidity ruleset has no concept of precompile-call correctness or transcript binding. **Every genuine soundness question in this contract — the G1ADD cancellation of §11.8.57, the transcript absorb order, the fail-closed dispatch — is invisible to all four tools’ stock detectors.** This is not a tool defect; it is the expected consequence of putting the verifier logic in assembly.

Actionable items the tools *did* surface.

- **Pin the pragma** (aderyn L-2): `^0.8.24` → an exact version for reproducible builds. [\[OPEN\]](#) Still `^0.8.24` in `templates/contracts/Halo2Verifier.sol:2` and `Halo2VerifyingKey.sol:3`. Pick one version: the generator pins `solc 0.8.30` (`docs/reference/REPRODUCIBLE_BUILDS.md:9`), Nightfall builds with `0.8.35` (`foundry.toml:16`).
- **Document the `--via-ir` requirement:** the contract will not build with the default codegen (stack-too-deep); the build recipe must set `viaIR: true` with the optimizer. [\[PARTIAL\]](#) Documented in `REPRODUCIBLE_BUILDS.md:10` and enforced for the verifier only by Nightfall’s `foundry.toml:30--51`; not stated in the artifact header. The VK contract must *not* be built via-IR (`foundry.toml:57--61` reports a 57-byte stub instead of the payload).
- **Silence the unused-param warning** by commenting the Solidity-level names or noting they are consumed via `calldataload` in assembly — a readability fix that also stops reviewers chasing a phantom. [\[OPEN\]](#) (`solc 0.8.35` still warns on L294–295).
- **PUSH0 / EVM target** (aderyn L-1): confirm the deployment chain supports Shanghai+ PUSH0, or pin an earlier EVM target. [\[CORRECTED\]](#) Moot: the verifier already needs Cancun (`mcopy`) and Prague (EIP-2537), both after Shanghai, and `solc` rejects `mcopy` below Cancun, so an earlier target cannot be pinned.
- **Remove the five dead constants and the stale G1_IDENTITY_MPTR comment**, and the `q_perm_eval` dead store (patched Slither). [\[OPEN\]](#)

Assessment. This is a [\[NOTE\]](#) finding: the tooling run is *clean of soundness-relevant true positives* (the only true positives are Info-level hygiene: five dead constants and one dead store), but its value is the triage — confirming that the apparent “High” findings are false positives and that the tools cannot reach the assembly soundness surface. The [\[IMPROVE\]](#) note is that a generated contract of this shape should ship with a `via-ir` build manifest and pinned pragma so the tooling/CI story is reproducible, and that teams should not read “slither crashed / semgrep clean” as assurance — the real assurance is the manual cross-check this report provides. The patched Slither run adds one mechanically checked property worth keeping: every precompile result is size-checked and the `success` flag is monotone up to the revert at L3568.

11.8.59 The VM scratch memory map: a concrete legend (the I9 fix demonstrated)

The single highest-leverage readability fix (I9) is to make the VM’s raw-hex scratch self-describing. This section quantifies the gap and shows the concrete legend the generator should emit.

Named vs anonymous memory. The contract has **57 named `_MPTR` constants** (64 with the seven `_MPTR_BASE`, 68 with the four `_CPTR` cursors) covering the high-level flow — VK header slots (`VK_DIGEST_MPTR=0x2700`, `K_MPTR=0x2740`, ...), challenges (`THETA_MPTR...X4_MPTR` at `0x69a0--0x6ac0`), and pairing inputs (`PAIRING_LHS/RHS_MPTR`). These are auditable (five of them are dead, §11.8.58). But inside the VM dispatch body (L1692–2564) there are only **9 distinct named `_MPTR` slots (21 references) vs 46 distinct raw-hex `mload/mstore` addresses (153 occurrences)** — the witness and permutation scratch is anonymous.

The witness columns are non-uniformly packed. The witness columns `a_i` are loaded from raw-hex addresses that do *not* follow a simple $\text{base} + i \cdot \text{stride}$ rule — current-row and next-row values are interleaved with meaningful gaps:

Col	cur-row	next-row	Col	cur-row	next-row
a_0	0x8540	0x85e0	a_8	0x86a0	0x8800
a_1	0x8560	0x8600	a_9	0x86c0	—
a_2	0x8580	0x8620	a_{10}	0x86e0	—
a_3	0x85a0	0x8760	a_{11}	0x8700	—
a_4	0x85c0	0x8780	a_{12}	0x8720	—
a_5	0x8640	0x87a0	a_{13}	0x8740	—
a_6	0x8660	0x87c0	a_{14}	0x8a20	—
a_7	0x8680	0x87e0			

Note the gaps: $a_4=0x85c0$ then $a_5=0x8640$ (skips the `0x85e0--0x8620` next-row block), and $a_{13}=0x8740$ then $a_{14}=0x8a20$ (a large jump; “—” means no rotation-+1 load exists). There is no per-column formula: each address is `REVERSED_EVALS_MPTR + 0x20 · j`, where j is the query’s position in the proof’s evaluation stream (the spill loop, L1224–1241), so recovering column and rotation needs the circuit’s query order. The gaps are load-bearing (they hold the next-row values of earlier columns). The loads are already bound to descriptive SSA names (`let a_3_next_1 := mload(0x8760)`, L2305); what is missing is one table that maps names to addresses.

The concrete fix: emit a legend. The generator already knows the semantic role of every scratch slot (it assigned them). It should emit them as named constants rather than raw hex, e.g.:

```
// Witness columns (current row)
uint256 internal constant WITNESS_A0_CUR = 0x8540;
uint256 internal constant WITNESS_A1_CUR = 0x8560;
// ...
// Witness columns (next row, rotation +1)
uint256 internal constant WITNESS_A0_NEXT = 0x85e0;
uint256 internal constant WITNESS_A1_NEXT = 0x8600;
// Permutation / lookup scratch
uint256 internal constant PERM_Z_CUR      = 0xae00;
uint256 internal constant LOOKUP_F       = 0xa980;
```

With these, the VM body reads `let a_0 := mload(WITNESS_A0_CUR)` instead of `mload(0x8540)`, and a reviewer can cross-check the witness layout against the circuit’s column order by reading the constant block — not by tracing 46 hex literals through the dispatch. The legend must also record aliasing: `0xa980` is the VM operand-stack base (`q_sp`, L1667), which the permutation and lookup callbacks reuse as scratch (`q_perm_vals`, L1947; `q_lookup_f`, L2061), so a single `LOOKUP_F` name would mislabel it. `PERM_Z_CUR=0xae00` matches the existing local `q_perm_z_cur` (L1949). It should also mark `[0x80,0x8e0)` as taken by via-IR variable spills (§11.8.19).

Assessment. This is a [\[IMPROVE\]](#) finding with a concrete, mechanical fix. The high-level memory map is already named (good); the gap is the VM’s internal witness/permutation scratch,

which is anonymous and non-uniformly packed. Emitting named scratch constants (or a generated legend comment block) is a pure codegen change — it does not alter the bytecode semantics at all, only the source readability — and it is the single highest-leverage change for making the contract cross-checkable against the Rust verifier by human eyes. The generator has the information; it just isn’t emitting it. **[OPEN]** No legend or scratch constants in HEAD templates or in B64.

11.8.60 The verifier is VK-data-generic within one circuit shape: `block_size` lives in the VK payload, not the code

A second generated variant (`block_size_8`, B8) was reviewed alongside B64. Diffing them produced a striking result that reframes what “generated verifier” means.

The verifier differs by 2 lines; the VK by 332. Normalizing the contract name, the B8 (as committed in Nightfall 5fa1f06e) and B64 *verifier* contracts are **byte-identical except two lines**:

```
- contract Halo2VerifierFee12345B8 {
+ contract Halo2VerifierFee12345B64 {
- EXPECTED_VK_CODEHASH_WORD = 0x336d1c24...aa00de4;
+ EXPECTED_VK_CODEHASH_WORD = 0xad26a475...23a6c4e1;
```

Every circuit parameter baked into the contract — the permutation δ (0x8634d0aa...), `q_perm_num_cols=18`, `q_perm_num_sets=6`, `q_perm_chunk_len=3`, the VM, the pairing assembly — is **identical**. The block-size variation lives entirely in the **VK payload (332 differing diff lines, 166 per side)**. **[CORRECTED]** These are the contract name, `vk_digest`, 24 of the 27 fixed-column commitments and 17 of the 18 permutation commitments (165 payload words). The header ($k = 21$, 19 instances, accumulator parameters), domain constants, G1/G2 bases, quotient constants and bytecode, and the quotient certificate hash are *identical*: B8 and B64 share one constraint-system shape and differ only in fixed-column data. (The staged, uncommitted B8 in the working tree re-renders `vk_digest` and `fixed_comms[9]`, so its pin is 0x10677213...f7f0.)

The verifier is shape-specialized, not a generic interpreter. **[CORRECTED]** The earlier claim that the contract reads all circuit-specific data from the VK is wrong. The VK supplies the domain constants (ω , ω^{-1} , n^{-1} , ω^{-10}), the base points, the quotient VM constants and bytecode (via `q_program_mptr`, L1518), and the fixed/permutation commitments. The verifier bakes in the rest: k (let `k := 21`, L1322 and L2846; the VK’s k at `K_MPTR` is only cross-checked, L935), the instance count and proof length (L954–961), the calldata and memory layout, the permutation parameters (L1946–1955), the program length (`q_end`, L1665) and the native callbacks. The header says so itself (L11–13: “It is not a generic verifier”). B8 and B64 differ only in **EXPECTED_VK_CODEHASH_WORD** because they share that shape; the codehash is the trust anchor that binds the shape-specialized code to one VK.

The codehash binding is enforced twice. The VK binding is checked at both abstraction levels:

- **Solidity** (L267): `authorizedVk.codehash == EXPECTED_VK_CODEHASH` — the authorized VK contract’s codehash must match the pinned constant.
- **Yul** (L922): `eq(extcodehash(vk), EXPECTED_VK_CODEHASH_WORD)` — the runtime re-checks the VK’s `extcodehash` before reading its payload.

So the verifier only accepts proofs for the VK whose runtime codehash matches the pinned constant. A swapped or tampered VK fails the codehash check before its payload is read.

Audit implications (this is the value).

- **One contract to review, many VKs of the same shape.** Reviewing the B64 verifier *is* reviewing the B8 verifier — same code. The per-block-size variation is hash-bound *data*, not code to re-audit. This holds only while k , the instance count, the gates, lookups, permutation and quotient program stay the same; any of those changes the verifier code.
- **The VK payload is the security-critical surface.** Since the verifier trusts the VK payload (after the codehash check), the VK’s integrity is load-bearing. The codehash check is the single trust anchor — its correctness (both levels) is the most important thing to verify, and it is.
- **The “circuit-specialized” framing is accurate.** [CORRECTED] The header’s “circuit-specialized verifier” (L5) and “not a generic verifier” (L11) describe the code correctly: it is specialized to one constraint-system shape. What is generic is the fixed-column data within that shape. A reviewer should read it as “shape-specialized verifier + pinned VK data”.

Assessment. This is a [OK] finding — one shape-specialized code path with a double-checked codehash anchor over the VK data is a clean, auditable architecture. The earlier [NOTE] proposing to rename the header to “generic Halo2/KZG verifier bound to a pinned VK” is withdrawn [CORRECTED]: that name would be false. At most the header could add “VK data (fixed and permutation commitments) varies within this shape”. Reviewers should still focus their per-block-size scrutiny on the VK payload and the codehash check, not on re-reading the (identical) verifier code.

11.8.61 The precompile-availability check tests acceptance but not rejection

`require_eip2537_precompiles()` (L203–254) is genuinely good defensive engineering: it probes each precompile with the *actual maximum input sizes* the verifier uses — a 78-pair G1MSM (0x30c0 bytes), a two-pair pairing (0x0300 bytes) — at deploy time, so a chain missing a precompile fails at construction rather than during a proof. The MCOPY probe (L211–213) correctly catches non-Cancun forks. This is the right idea, well executed.

The gap: every probe tests the accepting case only. All three precompile checks assert the *trivially-true* result:

- `G1ADD(identity, identity) → identity` (L224).
- `G1MSM(all-zero) → identity` (L240).
- `PAIRING( $e(\mathcal{V}, \mathcal{V}) \cdot e(\mathcal{V}, \mathcal{V}) = 1$ ) → true` (L250–252).

None tests that a precompile **correctly rejects** a non-trivial case. The pairing is the dangerous one: the check verifies the precompile returns `true` when the product *is* 1, but never verifies it returns `false` when the product *is not* 1. A pairing precompile with broken semantics that **always returns 1** would pass this deploy check and then silently accept every proof — a total soundness break — because the trivial case is answered correctly.

Why this matters despite low practical risk. On a standard EVM chain the EIP-2537 precompile is correct, so the practical risk is low. But the check’s own comment (L249) claims it catches “obviously incompatible semantics.” An always-true pairing *is* obviously incompatible, and the check misses it. The defense-in-depth is therefore incomplete relative to its stated goal — which is exactly the kind of gap a deploy-time validator is supposed to close.

Concrete fix: add a negative pairing test. The BLS12-381 generator points are *not* in memory at construction: `G1_BASE_MPTR=0x2880` and `G2_BASE_MPTR=0x2900` are filled only by the per-proof `extcodecopy` (L927), and `require_eip2537_precompiles()` runs before the VK pin (L264). **[CORRECTED]** So either hardcode the standard generators, or pass `authorizedVk` to the probe, run it after the codehash `require`, and copy payload words 12–23 (`G1_BASE||G2_BASE`, runtime bytes `0x181..0x300`) out of the VK:

```
// Negative test: a single non-trivial pair must NOT verify.
// e(G1_gen, G2_gen) != 1, so the precompile must return 0.
// Payload words 12..23 = G1_BASE (0x80) || G2_BASE (0x100);
// runtime byte 0 is the INVALID prefix, hence offset 0x181.
extcodecopy(authorizedVk, pair_scratch, 0x181, 0x180)
if iszero(staticcall(gas(), 0x0f, pair_scratch, 0x180, scratch, 0x20)) {
    revert(0, 0)
}
if eq(mload(scratch), 1) { revert(0, 0) } // must be FALSE
```

Symmetrically, a cancelling pair $e(G_1, G_2) \cdot e(G_1, -G_2) = e(G_1, \mathcal{K}) = 1$ must return **true**, confirming the precompile distinguishes the two. An analogous negative probe (an off-curve G1 input that must revert) would harden the G1ADD/G1MSM checks too, but the pairing is the critical one because it gates the final soundness decision.

Assessment. This is a **[IMPROVE]** finding — the existing check is good and the fix is small (a few lines in the constructor, zero runtime cost since it runs once at deploy). The principle generalizes: **a validator that only tests acceptance is half a validator.** Every “does it work?” probe should be paired with a “does it reject what it must?” probe, especially for a precompile whose **true** return is the verifier’s final accept signal. **[OPEN]** The constructor probe is unchanged in HEAD (`templates/partials/verifier/PrecompileSmoke.sol`; 2b2bf494 and b53e3ebe only added `gas()` forwarding and the MCOPY probe). The generator’s test suite already has the negative vectors (`src/test.rs:100--183`: $e(G_1, G_2) \rightarrow \text{false}$, bilinearity, off-curve G1MSM rejected), but only against its bundled Prague revm, not the deployment chain. The repository’s own audit notes ask for the same (`midfall docs/audit.md:49--68`, M-02).

11.8.62 Verified input-handling correctness (audit-surface reduction)

A high-value service to the auditor is confirming which paths are *verified correct*, so attention can focus elsewhere. The input-handling and modular-inversion paths were traced by hand and are sound.

ABI/proof-shape validation (L954–965) — complete. Before any transcript work, the verifier enforces, fail-closed:

- proof bytes length = `0x1e60` (7,776 B) — exactly the generated layout;
- instance-array length = 19 — the generated public-input count;
- total `calldatasize` = `INSTANCE_CPTR + 0x260` — no missing or trailing words.

All three are folded through **success** and enforced with a single **revert** (L965) before the parser walks raw calldata. Together with the ABI-head guard at L305 (proof offset `0x40`, instances offset `NUM_INSTANCE_CPTR - 4`), a truncated, extended, or mis-shaped proof cannot reach the transcript. This is a complete envelope check. Each public instance is then range-checked $< r$ as it is absorbed, and the verifier reverts right after the instance loop (L1037–1049; fail-fast since 1e21311c).

G1 transcript range-check (L396–417) — correct. `common_uncompressed_g1` enforces $x, y < p$ via the two-word comparison $x_hi < \text{BLS_P_HI} \vee (x_hi = \text{BLS_P_HI} \wedge x_lo \leq \text{BLS_P_MINUS_ONE_LO})$. Traced at the boundaries: $x = p$ reverts, $x = p - 1$ passes, the top-128-bit padding is rejected (`shr(128, x_hi)`). The check is exactly canonical field element. (On-curve / subgroup is deferred to the MSM/pairing, as in §11.8.57 — sound because every absorbed point flows to a checking precompile.)

Batch inversion (L448–530) — correct, including the zero case. The Montgomery batch inversion is the classic place a zero denominator silently corrupts every inverse. This implementation guards it correctly: the forward pass computes the total product g_p , and `iszero(gp) { ret := 0 }` (L498) catches any zero element (since r is prime, $g_p = 0 \iff$ some denominator is 0). The backward pass was traced for $n \in \{1, 2, 3\}$:

- $n=1$: fast path (L464), single `modexp`, zero-checked.
- $n=2$: loop skipped, `inv_first` = $g^{-1}a_1 = 1/a_0$, `inv_second` = $g^{-1}a_0 = 1/a_1$. ✓
- $n=3$: loop recovers $1/a_2$, then `inv_first` = $1/a_0$, `inv_second` = $1/a_1$. ✓

A zero denominator yields `ret=0` (a degenerate proof is rejected, fail-closed) — never a silently wrong inverse. The special-casing of the first two elements in the backward pass is subtle but correct.

Squeeze (L422–430) — standard. `mod(h0, r)` reduces the 256-bit Keccak digest into $\mathbb{F}_r$ exactly as the Rust transcript does (`BigUint::from_bytes_be(...) % modulus, proofs/src/transcript/implementors.rs:29`); no truncation at this layer (the truncation is the separate `truncated-challenges` feature, §12.3). **[CORRECTED]** It is not bias-free: $2^{256}/r \approx 2.2$, so each residue has two or three preimages and no challenge value has probability above $3/2^{256} \approx 1.36/r$. That factor is negligible for the soundness bounds.

Assessment. This is a **[OK]** credit. These are the paths where a subtle bug would be catastrophic (a wrong inverse or a missed length check breaks everything downstream), and they are correctly implemented with explicit zero-guards and fail-closed shape checks. For the auditor: **these paths can be trusted**; the residual audit risk lies in the VM dispatch and the memory-map legibility (§11.8.59), not in input handling.

11.8.63 The accumulator codec: the hardest-to-audit subsystem

The IVC accumulator public-input codec (`load_acc_coord_shifted`, `check_acc_coord_packing`, `load_acc_coord`, `load_acc_point`, L573–772) is the single hardest part of the contract to audit by hand. It is *correct and defensively layered*, but its identity-flag logic is subtle enough that a reviewer must hold a lot of state to be confident. This is a readability finding, not a soundness bug.

What it does. A G1 accumulator point is encoded as seven radix- 2^{56} limbs of $(\text{coord} - 1)$, four limbs per native $\mathbb{F}_r$ word. The x coordinate’s first packed word carries an **identity flag** by adding one radix base ($+2^{56}$), so the point-at-infinity has a distinguishable public-input form. The decoder must: reject non-canonical high bits, detect the identity flag, reconstruct the 384-bit coordinate across the 256-bit word boundary (handling a limb that straddles it, L595–599), range-check $< p$, and map the $p-1$ sentinel back to zero.

The layered validation (good defense-in-depth).

- `check_acc_coord_packing` (L638): every packed word must be $< 2^{\text{used_bits}}$, so unused high bits are zero — one coordinate, one encoding.
- Field range (L683): reconstructed coord $\leq p - 1$.
- Padding (L703): `hi` $< 2^{128}$ (EIP-2537 64-byte Fp padding), catching impossible >384 -bit reconstructions.
- Canonical identity (L622): the point-at-infinity is accepted *only* in the exact four-word form, deliberately stricter than “decodes to zero.”

The subtle part: the identity-flag probe. `load_acc_coord` (L665) probes the identity flag by subtracting one radix base from the first word and checking whether the adjusted coordinate is $p - 1$. This is correct — a non-identity $x < p$ cannot satisfy $\text{reconstruct}(W - \text{base}) = p - 1$ without $x = p + \text{base}$, which is out of field — but the argument is non-obvious. **[CORRECTED]** It is local and does not rely on the IVC equation: the only packed value the probe can flag is $(p - 1) + 2^{56}$, which as an ordinary coordinate would fail the field-range check (L683); once flagged, the point is accepted only if both coordinates decode to zero (L750), which is exactly the canonical identity encoding of L622. So every accepted point has exactly one public-input encoding. **A reviewer must reconstruct this argument from scratch** to be confident the probe can’t be fooled — exactly the kind of load-bearing subtlety that a worked example would eliminate.

Concrete fix: a worked encode/decode example. The codec should ship, in comments, a single concrete example: a known accumulator point, its seven radix- 2^{56} limbs of $(\text{coord} - 1)$, the identity-flag $+2^{56}$ on x ’s first word, and the resulting four packed words — plus the identity encoding side by side. A reviewer could then verify the decoder against a worked case instead of re-deriving the algebra. A companion test vector (encode $\rightarrow$ decode $\rightarrow$ assert round-trip for identity and a non-identity point) would make the subtlety machine-checked.

Assessment. This is a **[IMPROVE]** readability finding. The codec is correct and well-guarded, but it is the place where a human auditor will spend the most minutes per line, because the identity-flag probe requires reconstructing a non-obvious soundness argument. A worked example and a round-trip test vector convert “I have to re-derive this” into “I can check this in ten seconds” — the highest-leverage readability fix for the accumulator path specifically. **[PAR-TIAL]** End-to-end negative tests exist (`tests/ivc_keccak_solidity.rs:1418--1447`: non-canonical limb packing, malformed LHS/RHS points with zero scalar) plus a template check (`src/lowering/tests.rs:1191`); there is no worked example and no identity/non-identity round-trip vector.

11.8.64 The VK trust boundary: codehash pin + header cross-checks

The verifier’s VK-load phase (L910–941) is a model of a well-defended trust boundary, and it is worth spelling out *why* its two checks are not redundant — a reviewer who dismisses the header cross-checks as “already covered by the codehash” would be wrong.

Level 1 — the codehash pin (external tampering). On *every* proof (not just the constructor), the verifier re-checks `extcodesize(vk) = EXPECTED_VK_LENGTH` $\wedge$ `extcodehash(vk) = EXPECTED_VK_CODEHASH_WORD` (L920–923). This pins the entire VK runtime by hash, so a wrong or tampered contract at `AUTHORIZED_VK` reverts before any payload is read. The re-check on every proof (not only at construction) hardens the fork / same-transaction edge cases the generated comment names (L911–915), where code at the address could differ from what the constructor

saw. It is defense-in-depth rather than a reachable path: the VK runtime starts with `INVALID` and can never execute `SELFDESTRUCT`, so its code cannot change after deployment. (An earlier “reorg” rationale does not apply: a reorg that drops the VK drops the verifier deployed after it too.)

Level 2 — the header cross-checks (generator drift). After `extcodecopy`, the verifier asserts seven loaded header words against its own baked constants (L934–940): `num_instances=19`, `k=21`, `has_accumulator=1`, `acc_offset=11`, `num_acc_limbs=7`, `num_acc_limb_bits=56`, and the quotient-certificate hash. These are **not** implied by the codehash. The codehash pins the VK’s bytes; the verifier’s constants are baked into the *verifier* independently. If the generator ever emitted a verifier whose constants disagreed with the VK it produced (say the VK says $k = 22$ but the verifier’s constant is 21), the codehash would still pass — it pins the VK, not the agreement between the two artifacts — and the verifier would mix a stale constant with a live payload field. The header cross-checks are exactly the guard that catches this cross-artifact drift before calldata parsing commits to a schema. Note that generator `HEAD` (`VkLoading.yul:82--87`) emits only the first six checks; the quotient-certificate check (L940) comes from the M7 follow-up generator line that rendered B64 (introduced in `e7f736c4`, 2026-06-07; not on `main`).

The `INVALID`-byte handling (verified consistent). The VK runtime is `INVALID || payload`; the constructor returns `0x42a1 = 17,057` bytes, and `EXPECTED_VK_LENGTH = EXPECTED_VK_PAYLOAD_LENGTH + 1 = 17,056 + 1` (L44–45). The verifier copies `extcodecopy(vk, VK_MPTR, 0x01, PAYLOAD_LENGTH)` — from byte 1, skipping the `0xfe` — so the reconstructed payload layout matches the embedded render exactly. The byte-0 `INVALID` also means a direct call to the VK halts exceptionally rather than executing payload bytes as code.

Constructive note on coverage. The cross-check covers the seven fields the comment flags as parser/domain/accumulator-critical. An auditor should confirm this set is *complete* — i.e. every verifier constant that mirrors a VK payload field is cross-checked. **[CORRECTED]** The quotient program length is *not* self-contained in the verifier: `q_end = q_program_mptr + 0x11cf` (L1665), `q_program_mptr` (L1518) and `q_const_mptr` (L1515) are verifier constants that must match where and how long the program and constant table are in the VK payload. None has its own cross-check; they are covered indirectly by the quotient-certificate cross-check (L940), whose hash binds the VM bytecode and constant table (VK NatSpec L45–48), and at generation time by `plan.rs:259--281` (program-length drift, bytecode and constant-table fit). Making that explicit (a one-line comment per cross-checked field stating *why* it is checked, and which unchecked constants the certificate covers) would let the auditor verify coverage without re-deriving which fields are mirrored. **[OPEN]** The emitted comment (L929–933) gives one collective reason only.

Assessment. This is a **[OK]** credit with a small **[NOTE]**. The two-level defense is genuinely good engineering — the header cross-checks close a gap the codehash cannot. The only improvement is documentation: state per-field why each cross-check exists so the coverage argument is checkable rather than implicit.

11.8.65 The proof-parser consumption check (verified consistent)

The hand-rolled proof parser walks the raw proof bytes with a cursor `proof_cptr`, and ends with a single consumption assertion (L1312): `if iszero(eq(proof_cptr, NUM_INSTANCE_CPTR)) { revert(0, 0) }`. I verified this arithmetic end-to-end and it is exactly consistent — a useful credit, plus one readability caveat.

The check is complementary, not redundant. The generator’s own comment (L1306) calls it “redundant with the generated proof length today.” That undersells it. The proof-length check (L954, `eq(0x1e60, calldataload(PROOF_LEN_CPTR))`) validates the *calldata* length; the consumption check validates the *parser’s walk*. They catch different bugs: a wrong *calldata* length fails the first; a generator bug that makes the section sizes sum to something other than the proof length fails the second even when the *calldata* length is correct. The consumption check is the guard that makes *parser* drift fail closed.

Verified arithmetic. The parser’s sections are nine looped ranges (advice, lookup multiplicities, permutation *Z*, two lookup-helper loops, trashcans, quotient limbs, evaluations, `q_evals`) plus four single G1 reads (two lookup accumulators, `f_com`, π). Summing the loop bounds ($0x780 + 0x100 + 0x300 + 3 \times 0x80 + 0x200 + 0xcc0 + 0xa0 = 0x1c60$) plus the four single reads ($4 \times 0x80 = 0x200$) gives $0x1e60$ — exactly the proof length. And $PROOF_CPTR + 0x1e60 = 0x64 + 0x1e60 = 0x1ec4 = NUM_INSTANCE_CPTR$, so the cursor lands precisely on the instances length word. The parser consumes the whole proof and nothing more.

Readability caveat. This verification was *hard by hand*: the section sizes are hardcoded hex spread across nine `for`-loops (whose bounds are section *totals*, not per-iteration amounts) and four single reads, and it is easy to double-count a loop-body advance as a separate section (I did, twice, before classifying correctly). A reviewer cannot eyeball that the parser consumes exactly $0x1e60$. **Concrete fix:** emit a section-size manifest comment next to the parser — `// consumed: 0x780 advice + 0x100 lookup_m + 0x300 perm_z + 2 x (0x80 lookup_helper + 0x80 lookup_acc) + 0x80 trashcan + 0x200 quotient + 0xcc0 evals + 0x80 f_com + 0xa0 q_evals + 0x80 pi = 0x1e60 = proof length` — so the auditor checks the sum against the proof-length constant in one line instead of tracing loop structure. **[OPEN]** `HEAD TranscriptProofParser.yul` emits section headers but no size manifest; `ProofCalldataLayout (src/lowering/abi/proof.rs)` already holds the numbers.

Assessment. This is a **[OK]** credit with a **[IMPROVE]** readability note. The consumption check is genuinely good defense-in-depth (it closes a parser-drift gap the length check cannot), and it is verified consistent. The only improvement is making the section-size sum checkable at a glance rather than requiring a manual loop-structure trace.

11.8.66 The final pairing: sound randomized batching, confusing helper parameter names

The final soundness gate (L3465–3569) batches the KZG opening equation and the IVC accumulator equation into a single pairing using a verifier-local randomizer α . The construction is **sound and well-motivated** — but the slot naming is a genuine footgun that a three-line comment has to rescue.

The randomization is sound (verified by algebra). Naively multiplying two pairing equations together is unsafe: two *bad* equations can cancel to a true product. The verifier instead checks, after all four G1 inputs are fixed,

$$e(kzg_rhs + \alpha acc_rhs, [1]_2) \cdot e(kzg_lhs + \alpha acc_lhs, -[s]_2) = 1.$$

By bilinearity this is $K \cdot A^\alpha = 1$ where K is the KZG equation’s product ($= 1$ iff KZG holds) and A the accumulator’s. Tracing every case:

- both good: $1 \cdot 1^\alpha = 1$ — passes (correct).
- KZG bad, ACC good: $K \cdot 1 = K \neq 1$ — fails for *all* α (cannot be satisfied).

- KZG good, ACC bad: $A^\alpha = 1$ only at $\alpha = 0$ — and the `if iszero(acc_pair_alpha) { acc_pair_alpha := 1 }` guard (L3523) closes exactly this case, so a dropped accumulator is impossible.
- both bad: $A^\alpha = K^{-1}$ has exactly one solution mod r ; the Fiat–Shamir α hits it with probability about $1/r \approx 2^{-255}$ (at most $\approx 3/r$ once the $0 \rightarrow 1$ remap and the `mod r` bias of §11.8.62 are counted).

So a bad proof passes with negligible probability ($\lesssim 3/r$), and the $\alpha=0 \rightarrow 1$ guard is load-bearing (it is the only thing preventing the ACC-bad case from slipping through at $\alpha = 0$). This is a textbook-correct randomization, and the guard shows the authors thought about the degenerate draw.

The footgun: helper parameters vs. slot names. `ec_pairing` checks $e(\arg0, [1]_2) = e(\arg1, [s]_2)$, so `arg0` must be the $[1]_2$ -side point (the combined `final_com` $-vG + x_3\pi$) and `arg1` must be π . But the slots are named `PAIRING_LHS_MPTR := pi` and `PAIRING_RHS_MPTR := combined` — the *opposite* of the helper’s argument order — because the names follow the dual-MSM accumulator, not the helper. The call is therefore `ec_pairing(success, PAIRING_RHS, PAIRING_LHS)` (swapped, L3569). This is correct, but a reviewer who assumes “LHS pairs with the first argument” would misread the equation entirely. The generator had to add a three-line comment (L3565–3567, inside the L3555–3567 block) to explain the swap — a smell that the names are fighting the helper. **[CORRECTED]** The names are not “backwards” relative to the Rust reference: `DualMSM` is built as `left =  $\pi$ , right = final_com - vG + x3 $\pi$` (`proofs/src/poly/kzg/mod.rs:554--558`), and `DualMSM::check` pairs `left` with $[s]_2$ first and `right` with $-[1]_2$ (`msm.rs:284--296`). The Solidity slots copy that faithfully; the inversion is in the Solidity helper, whose first parameter (named `lhs_mptr`, L534) is paired with `G2_BASE =  $[1]_2$` and whose second with `NEG_S_G2_BASE`.

Concrete fix. Rename the slots to match the pairing role rather than the accumulator history: `PAIRING_G1_SIDE_MPTR` (the $[1]_2$ -paired combined point) and `PAIRING_S_SIDE_MPTR` (the $[s]_2$ -paired π), or simply `PAIRING_ARG0/ARG1`. Then the call reads `ec_pairing(success, G1_SIDE, S_SIDE)` with no swap comment needed, and the equation is checkable by name. This is a pure codegen rename with no runtime effect. Two amendments: “`G1_SIDE`” is ambiguous (both operands are $G1$ points), so prefer names after the $G2$ partner, e.g. `PAIRING_WITH_G2_MPTR` / `PAIRING_WITH_NEG_S_G2_MPTR`; and the smallest change that keeps the Rust `left/right` correspondence (mirrored by the Rust trace labels `pairing_lhs/pairing_rhs`, `proofs/src/poly/kzg/mod.rs:563,568`) is to rename the helper’s parameters (`lhs_mptr` $\rightarrow$ `g2_base_side_mptr`, `rhs_mptr` $\rightarrow$ `neg_s_g2_side_mptr`). **[OPEN]** Unchanged in HEAD (`FinalPairing.yul:79`).

Assessment. This is a **[OK]** credit on the math (the randomized batching is sound and the $\alpha=0$ guard is correct) with a **[IMPROVE]** on the naming — the most confusing readability trap in the pairing block, and the only thing explaining it is a comment. **[CORRECTED]** A misread cannot ship silently: swapping the call would check $e(\pi, [1]_2) = e(\text{combined}, [s]_2)$, which rejects every honest proof, so the happy-path fixture and E2E tests fail at once.

11.8.67 Vanishing polynomial and rotation points (verified)

The Lagrange/vanishing computation (L2844–2867) and the rotation-point computation (L2883–2905) are the algebraic core of the quotient check. Both are **correct**; the rotation points carry a readability caveat.

The vanishing squaring walk is correct (verified). The verifier computes x^n and x^{n-1} in one walk ($n = 2^k$, $k = 21$): `x_pow_2i` tracks x^{2^i} and `x_pow_2i_minus1` tracks x^{2^i-1} , updated as `minus1 ← minus12 · x` and `pow ← pow2`. The invariant holds by induction: $(x^{2^i-1})^2 \cdot x = x^{2^{i+1}-1}$. I simulated the walk against $r = \#\mathbb{F}_r$ for $k \in \{1, 2, 4, 21\}$: `x_pow_2i` = x^n and `x_pow_2i_minus1` = x^{n-1} in every case, and `one_minus_x_n` = $1 - x^n = -(x^n - 1) = -Z_H(x)$ exactly. The sign convention ($-Z_H$ rather than Z_H) is consistent with how the quotient numerator is assembled and is covered by the differential trace.

Rotation points: correct but the offset is implicit. The PCS opens at four distinct rotations $\{0, +1, -1, -10\}$, computed by multiplying x by ω / ω^{-1} : `ROT+0x40` = x , `ROT+0x60` = $x\omega$, `ROT+0x20` = $x\omega^{-1}$, and `ROT+0x0` = $x\omega^{-10}$ (ten ω^{-1} multiplications, L2893+L2895–2903). Two things make this hard to audit by eye:

- the rotation *offset* is implicit in the *count* of ω^{-1} multiplications — a reviewer must count them to confirm the offset is -10 ;
- the stores are *emitted* out of rotation order (`0x40, 0x60, 0x20, 0x0`), so the block doesn't telegraph which slot is which rotation. **[CORRECTED]** The memory layout itself *is* in rotation order: `slot = ROT_POINTS_MPTR + 32 · (index in the sorted rotation list)`, i.e. $-10, -1, 0, +1 \mapsto 0x0, 0x20, 0x40, 0x60$ (`kzg/mod.rs:840--852`); only the walk (forward, then back from x) is out of order.

A wrong multiplication count would query the wrong rotation — caught by the differential CI, but not by a human skimming the block. **Concrete fix:** annotate each store with its rotation — `// rot 0 -> ROT+0x40, // rot +1 -> ROT+0x60, // rot -1 -> ROT+0x20, // rot -10 -> ROT+0x0 (x * omega_inv^10)` — so the count and the mapping are checkable without tracing the multiply chain. Better still, -10 is `rotation_last` and the VK already carries ω^{-10} as `OMEGA_INV_TO_L` (header word 6; checked numerically against the B64 VK), which the Lagrange block already uses (L1338); `mulmod(x, mload(OMEGA_INV_TO_L_MPTR), r)` replaces the nine further multiplications with one and removes the counting. **[OPEN]** The rotation block in HEAD emits only “// N distinct rotation(s)” (`kzg/mod.rs:860--862`).

Assessment. This is a **[OK]** credit on the algebra (both computations verified correct) with a **[IMPROVE]** readability note on the rotation points. The vanishing walk is a clean, verified piece of the verifier; the rotation block is correct but asks the reader to count multiplications and follow out-of-order stores into an in-order map — both fixable with per-store rotation comments.

11.8.68 Power tables are precomputed, not recomputed (efficiency credit)

A natural worry for a hand-rolled verifier is *repeated computation* — recomputing x^i at every use site. The generated contract does the opposite: it precomputes power tables once and reads them, which is both efficient and easier to audit (one computation site per table).

X1 powers: one write site, 67 read sites. The x_1 power table is computed in a single `pre-compute 43 x1 power(s)` loop (L2910–2920) — one seed `mstore` of $x_1^0 = 1$ (L2912) and **one** in-loop `mstore` (L2918), which stores the 128-bit-truncated power while `acc` keeps full precision — and then read at **67** direct `mload` sites across the PCS batching, plus three pointer-walking loops (L2971, L3051, L3089). If the generator had inlined the exponentiation at each use site instead, that would be $\sim 67\times$ the work; the table collapses it to one walk plus cheap memory reads.

x4 powers: one truncating walk, reused as locals. The x_4 powers are not a memory table but a single local walk (L3281–3291): `x4_pow_full` keeps *full* precision for the next multiplication, while each `x4_pow_i` is the 128-bit-truncated value used for batching. This is exactly the truncated-challenges rule (§12.3) applied correctly — full-precision accumulator drives the recurrence, truncated value is the security-relevant operand — and the five locals are reused across the *v*-batching (L3293–3297) and the fused-MSM staging (L3414+). Computed once, never re-computed.

Why this matters for audit. The “one computation site per table” property is what makes the power handling auditable: a reviewer checks the *single* precompute loop and the truncation rule once, rather than verifying an exponentiation at every of the 67 use sites. The repetition that *does* exist is cheap `mloads` from a verified table, not re-derived arithmetic. This is the generator doing the right thing — and it is the kind of property a human auditor wants to confirm holds globally (it does here).

Assessment. This is a [OK] credit. The *repeated-computation* concern is not realized in this render: powers are precomputed (X1: one loop / 67 read sites; x4: one walk reused), so the contract trades a little memory for a large reduction in recomputation and a much smaller audit surface. The only residual note is that the truncation of `x4_pow_i` should be cross-checked against the `x1` truncation rule to confirm the two use the same 128-bit mask — they do (0xffffffffffffffff in both), but stating the shared mask as a named constant (`TRUNC_MASK_128`) in the emitted code would make that consistency self-evident. [PARTIAL] The generator already has that single source: one Rust constant `TRUNC_MASK_128` (`kzg/mod.rs:819`) feeds both emission sites (L987, L1565), and `src/lowering/tests.rs:2117,2121` assert both; only the rendered Solidity shows the bare literal.

11.8.69 The trust root does not trace-test the exact production VK

[CORRECTED] An earlier draft of this section said that no test exercises the production *shape*. For the shape parameters that is wrong. What is still true is narrower: no test trace-checks the exact production VK. The finding still refines C2 (§12.2). The differential trace harness is the verifier’s primary assurance, and it runs on a sibling of the shipped artifact, never on the artifact itself.

What the synthetic fuzzer covers. `supported_shape_evm_cases()` (`test.rs` L528) is **seven** synthetic circuits, **every one at $k = 5$** , varying *features*: next-rotation, second-phase, permutation, lookup, complex/additive selectors, fixed-scale. Case 7 (“full mixed lookup permutation shape”) combines all of them, so the feature *combinations* are exercised. The random-seed PBT (`pbt_transcript_differential_fuzzer_matches_solidity_trace`, L706) is also $k = 5$. The trace-equivalence assertion (L1060–1102) is genuinely strong: bidirectional id-matching plus byte-for-byte data equality against the Rust oracle.

What the real-circuit trace jobs cover. Two other tests run the same trace comparison on real `ZkStdLib` circuits. The first is Poseidon at $k = 6$ (`native_midfall_verifier_trace_matches_solidity_trace`, `test.rs` L1553; `POSEIDON_K`, L62). The second is the **IVC Keccak decider** at $k = 20$ (`tests/ivc_keccak_solidity.rs: DECIDER_K` L936, trace assertion L1320, plus 13 adversarial calldata mutations at L1339–1457, commit 6abffd2b). A local render of that decider (the untracked `target/ivc-keccak-solidity-dump/`) matches B64 in almost every shape parameter:

- the same permutation parameters, `q_perm_num_cols=18`, `q_perm_num_sets=6`, `q_perm_chunk_len=3` (B64 L1953–1955);

- the same 0x1e60-byte proof layout: 15 advice, 2 multiplicity, 6 perm- Z , $2 \times (1 \text{ helper} + 1 \text{ acc})$, 1 trashcan, 4 quotient-limb commitments, 102 evals and 5 q_evals;
- the same 78-pair final MSM;
- the same 27 fixed and 18 permutation VK commitments;
- the same 178-word quotient constant set;
- a 0x11cf-byte q_program with an identical opcode histogram.

The RSA ($k = 12$), SHA ($k = 13$) and hybrid-MT ($k = 13$) fixtures add positive and negative E2E runs on further real circuits (8178242b, 51f912c2). A grep of test.rs therefore does *not* find $k = 5$ everywhere: $k = 6, 12, 13, 19$ and 20 all appear across the crate’s tests. The one test named “production” (test.rs L1430) is positive-only, but it is not the only production-like coverage.

What is still not covered.

- $k = 21$. The decider is $k = 20$. What differs at $k = 21$ is the `let k := 21` squaring walks (L1322, L2846) and the VK’s ω and n^{-1} words. All of these are k -parametric code that the $k = 20$ run already exercises.
- **The 19-instance public-input layout.** The IVC decider has 14 instances. The in-crate Sepolia deployments/sepolia/moonlight-wrap render (19 instances, $k = 20$) is exercised only positively, by scripts.
- **The embedded-quotient render mode.** The IVC job renders `RenderQuotient::ExternalPinned` (ivc_keccak_solidity.rs L1139).
- **The generator version.** B64 is not a main-HEAD render. It comes from the M7 follow-up generator line (e7f736c4, 2026-06-07; also 7bf45715 on benchmark_256_trees), which has the 2026-05-19 hardening commits but not 6b3d2096. Its quotient-certificate check (L51, L940) does not exist in main HEAD.
- **CI execution.** The poseidon-trace-equivalence and ivc-trace-equivalence jobs are defined in proofs/solidity-verifier/.github/workflows/ci.yml (L59, L117). GitHub Actions executes only the repository-root .github/workflows, and the root ci.yml has no solidity-verifier job. In this monorepo the gates therefore do not run automatically unless a standalone mirror runs them.

Concrete fix. A synthetic $k = 21$ ShapeFuzzCase adds little. The IVC decider already covers the 18/6/3 permutation parameters, and the synthetic ShapeFuzzCircuit cannot reproduce the ZkStdLib gate set anyway. The high-value step is to run the existing trace harness on the **exact fee-12345 VK** with a native wrap proof, wherever that circuit’s proving key lives. That means rendering with `RenderDiagnostics{trace: true}`, capturing `solidity_trace` natively and applying `assert_trace_equivalence_and_required_coverage`. The second step is to wire the crate’s jobs into the root workflow so that the existing gates actually run.

Assessment. [PARTIAL] Downgraded from [RISK] MAJOR to a [IMPROVE] MINOR residual. The production shape is exercised by the $k = 20$ IVC decider’s trace job, including its permutation, lookup, PCS and opcode mix. The gap is the exact VK ($k = 21, 19$ instances, embedded quotient, generator lineage), together with the fact that the gate is not wired into the monorepo’s CI.

11.8.70 `f_eval`: Lagrange interpolation with inlined batch inversion

The f_{eval} computation (L3107–3269) is the most complex auditable arithmetic in the contract outside the VM. It is a Lagrange interpolation over five point-sets combined by a Horner fold over x_2 . The four sets of cardinality 3, 3, 2 and 2 each get a *manually inlined* Montgomery batch inversion. The cardinality-1 set 0 uses a single `scalar_inv` (L3265). The computation is **correct** (the batch inversion was verified), but it is the second-hardest thing in the file to read.

What each set does. For a set of rotation points $\{p_0, p_1, p_2\}$ the block computes the Lagrange interpolation of the q -evaluations at x_3 . It forms the differences $dx_i = x_3 - p_i$ and the basis denominators $\ell_i = \prod_{j \neq i} (p_i - p_j)$. It then applies a single `scalar_inv` to the total product $dx_0 dx_1 dx_2 \ell_0 \ell_1 \ell_2$, with a backward pass that recovers every dx_i^{-1} and ℓ_i^{-1} . Finally it computes $\text{eval} = q_{\text{eval}} \cdot \text{den}^{-1} - \sum_i \text{set}_i \cdot dx_i^{-1} \cdot \ell_i^{-1}$. The five sets are folded with $f_{\text{eval}} \leftarrow f_{\text{eval}} \cdot x_2 + \text{eval}$.

The inlined batch inversion is correct (verified). The $bp_0..bp_5$ prefix-product and bq backward pass are the Montgomery trick done by hand. I traced it against $\mathbb{F}_r$: all six recovered inverses ($dx_0^{-1}..dx_2^{-1}$ and $\ell_0^{-1}..\ell_2^{-1}$) match $\text{pow}(v, r-2, r)$. The one expensive inversion per set is therefore sound: there is no per-element modexp and no silent wrong inverse.

The readability cost. The batch-inversion pattern repeats **four times**. The two cardinality-3 blocks are about 42 lines each and the two cardinality-2 blocks about 28 lines each; the set-0 block is 5 lines. The blocks differ only in which rotation points the set uses and which `Q_EVAL_SET/Q_EVAL_CPTR` offsets they read. A reviewer must verify the Lagrange basis and the backward pass four times, and confirm that each set selects the right rotation-point subset. The reversal (set 4 first) is not arbitrary codegen. It mirrors the Rust `.rev().fold(...)` Horner in `multi_prepare` (`poly/kzg/mod.rs:471–481`), but nothing in the Yul says so. (`REVERSED_EVALS_MPTR` is unrelated: it is the quotient-VM eval spill region.) Correctness is carried by the differential trace, since f_{eval} is the required trace id 31. A human cross-check is still expensive.

Concrete fix. Factor the per-set interpolation into one named Yul helper, `lagrange_set_eval(x3, p0, p1, p2, qeval, set_base) -> eval`, and emit five calls to it. The batch inversion and the basis algebra then live in *one* audited place instead of four near-identical copies, and the reversal becomes an explicit loop over a set list. This is the same “one computation site” principle that made the power tables auditable (§11.8.68), applied to the interpolation. One caveat: a helper that calls `scalar_inv` itself keeps five modexps. The gas-optimal cross-set batch inversion (§11.7) needs the helper to take pre-inverted dx_i^{-1} and ℓ_i^{-1} , or all sets’ denominators need to go through one `batch_invert` over a memory list.

Assessment. This is a **[OK]** credit on correctness (the inlined batch inversion is verified) with a **[IMPROVE]** readability note **[OPEN]**. Four copies of a subtle interpolation is the kind of repetition that turns a one-page audit into a four-page one, and a single helper collapses it.

11.8.71 VM fail-closed boundary, and the hybrid execution model

The VM’s boundary checks are a genuine credit. They also expose that the verifier is not *just* a VM: it is a **hybrid** of an interpreted bytecode VM and emitted Yul, and that split matters for the audit.

The fail-closed boundary is strong (credit). Three checks close the VM (L2561–2570):

- `default { revert }` (L2561): any opcode not in the switch reverts, so the VM cannot execute arbitrary bytecode. The comment notes that `0x1a` intentionally lands here, so a stray opcode is caught rather than silently skipped.

- `if iszero(eq(q_pc, q_end)) { revert }` (L2569): the program counter must land exactly on the end of the VK-pinned bytecode. The loop condition `lt(q_pc, q_end)` already guarantees $q_{pc} \geq q_{end}$ on exit, so in practice this catches a final instruction whose operands over-read past `q_end`. It cannot catch a program that is merely “short”.
- `if q_has_top { revert }` (L2570): every identity must have been consumed by a fold or callback, so a partial expression left live fails.

Combined with the VK codehash pin (§11.8.64), this means the VM can only ever run the exact generated program, and a malformed program cannot silently pass. This is exactly the posture you want for an interpreter over a pinned blob.

The hybrid model: inline prefix, VM (with native callbacks), post-VM suffix. The numerator is evaluated in *three* emitted segments. The **direct inline prefix** (L1547–1648) is Yul emitted before the VM loop. The interpreted `q_program` loop (L1692–2564) includes native Yul callbacks for the permutation (0x19), the lookup (0x1f) and four heavy gates (0x1b, sub-cases 0–3). The **post-VM suffix** (L2572+) is generated Yul for the “regular trash constraints”, chosen because it is “smaller than fully unrolled Yul and cheaper than interpreting every trash operation” (L2572–2574). All three write into the same numerator and selector accumulators in the same *y*-batch order. A reviewer who wants to enumerate “what constraints does this verifier check?” must read the opaque bytecode, the callbacks, the prefix *and* the suffix. The contract text has no single manifest tying them to the constraint set.

Why this is an auditability cost, and what already exists. The hybrid is a sensible gas optimization, but it deepens C1: the constraint set is spread over several emission modes. The earlier claim that a constraint missing from every segment “would only be caught by the final pairing failing on a prover who exploits the gap” was **[CORRECTED]**. Dropping an identity changes $\nu_y(x)$ at a random x , so *honest* proofs fail the pairing, and any positive E2E test at that shape catches it. The trace harness also requires every per-identity id (range 30,000–40,000, test.rs L1722–1727). The generator already builds the manifest this section asks for. HEAD’s `QuotientProgramPlan::execution_manifest()` tags every identity `Inline`, `Interpreted`, `Native{Permutation,Lookup,Identity}` or `StructuredTail` (vm/mod.rs:343–432). It is validated against the source identity stream at render time (quotient.rs:458), and a host-side `QuotientIdentityManifest` carries the gate and constraint names (api.rs:189). B64’s generator lineage also hashes this into `EXPECTED_QUOTIENT_CERTIFICATE_HASH_WORD` (L51), which is checked against the VK header at L940. **Remaining fix:** render that manifest as a comment block listing each identity, its name and its execution mode, so that a reader of the contract sees it too.

Assessment. This is a **[OK]** credit on the fail-closed boundary with a **[IMPROVE]** note on the hybrid model **[PARTIAL]**. The boundary is well-defended, and a machine manifest exists and is validated. It is simply not emitted into the contract, which compounds the VM abstraction critique for human readers.

11.8.72 Challenge-schedule cross-check against the Rust verifier

The Fiat–Shamir challenge order is the single most soundness-critical sequence in the verifier: a reordering silently changes every challenge. I cross-checked the contract’s full absorb/squeeze order against the Rust `plonk/verifier.rs`, and it matches.

The contract squeezes exactly ten challenges. Following every `squeeze_to` in order gives $\theta \rightarrow \beta \rightarrow \gamma \rightarrow \text{trash} \rightarrow y \rightarrow x \rightarrow x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_4$ (L1095–1294). That is the PLONK challenge block $(\theta, \beta, \gamma, \text{trash}, y, x)$ followed by the KZG multi-open block $(x_1..x_4)$, which is exactly

the Rust schedule. `verifier.rs` squeezes θ (L150), β (L167), γ (L172), trash (L194), y (L215) and x (L304), and `multi_prepare` adds x_1, x_2 (`poly/kzg/mod.rs:363–364`), x_3 (L455) and x_4 (L484). The proof material the contract absorbs between these squeezes also matches the Rust’s read order: advice comms, lookup multiplicities, perm- Z products, lookup helpers and acc, trashcan comms, quotient-limb comms, evals, `f_com`, `q_evals` and the opening proof π .

No user challenges — and that is correct. The Rust’s phase loop (L112–144) squeezes *user* challenges at `USER_CHALLENGE_TRACE_BASE = 1000` only if the circuit declares them. The production circuit declares none, so the contract emits **no** user-challenge squeezes. The “User phase 1” block (L1076) reads advice commitments and proceeds straight to θ . A reviewer who expects to see user challenges squeezed between the advice comms and θ (as the generic Rust loop would for a multi-phase circuit) may briefly think they are missing. They are correctly absent because this circuit is single-phase with no custom challenges. The ten protocol and KZG challenges are trace ids 7–16. The required-coverage check asserts nine of them (`test.rs` L1702–1711). Trash (id 12) is traced only when the circuit has trashcans, so it is left out of the required list, though the bidirectional id match still compares it when present.

Assessment. This is a **[OK]** credit. The soundness-critical challenge schedule is consistent with the Rust across both the PLONK and KZG halves, and the absence of user challenges is a correct consequence of the circuit shape, not an omission. The one readability note **[OPEN]**: the “User phase 1” comment (L1076–1079) still says only that counts are “generated from the protocol plan”. It could state “*no user challenges for this circuit*” explicitly, so a reviewer doesn’t have to re-derive from the Rust that the missing squeezes are intentional.

11.8.73 Gas: two 1-pair MSMs where one 2-pair MSM would do

The verifier’s gas is dominated by precompiles. Under EIP-2537, the 78-pair `final_com` MSM alone costs $78 \times 12,000 \times 0.561 = 525,096$ gas, and the 2-pair final pairing costs $2 \times 32,600 + 37,700 = 102,900$. The batched-numerator section (inline prefix, VM and suffix) is about 24% of the IVC-bench total: 313,762 of 1,300,701 gas (`docs/plans/FURTHER_OPTIMISATIONS.md`). The final-pairing RHS is built with **two separate 1-pair G1MSMs** that could be batched. **[CORRECTED]** This saves far less than an earlier draft claimed, because EIP-2537 MSM pricing has no per-call fixed cost.

The current computation (L3471–3492). The final pairing needs $\text{RHS} = \text{final_com} - vG + x_3\pi$. The contract builds it as:

1. `G1MSM([G], [-v])` $\rightarrow -vG$ (1-pair, L3475)
2. `G1ADD(final_com, -vG)` $\rightarrow \text{final_com} - vG$ (L3480)
3. `G1MSM([pi], [x3])` $\rightarrow x_3\pi$ (1-pair, L3486)
4. `G1ADD(prev, x3*pi)` $\rightarrow \text{final_com} - vG + x_3\pi$ (L3490)

That is **2 G1MSMs + 2 G1ADDs**. The accumulator batching that follows (L3497–3553) adds two more 1-pair MSMs and two G1ADDs. Those cannot be folded, because the randomizer α is a hash of the materialized pairing inputs.

The optimization, priced correctly. $-vG$ and $x_3\pi$ are two independent scalings that are simply summed. A single 2-pair MSM computes them together, `G1MSM([G, pi], [-v, x3])` $= -vG + x_3\pi$, followed by one G1ADD with `final_com`. This replaces **2 MSMs + 2 ADDs with 1 MSM + 1 ADD**. EIP-2537 prices a k -pair G1MSM as $k \cdot 12,000 \cdot d_k / 1000$ with $d_1 = 1000$ and $d_2 = 949$ (`revm g1_msm.rs DISCOUNT_TABLE`). Two 1-pair MSMs therefore cost 24,000 and

one 2-pair MSM costs 22,776. The saving is $1,224 + 375$ (one G1ADD) $\approx 1.6\text{k}$ gas of precompile fees, plus one fewer `staticcall`. That is about 0.1% of the 1,291,730 gas measured for a fresh proof on the $k = 20$, 19-instance Sepolia wrap verifier (`deployments/sepolia/moonlight-wrap/README.md`). It is not the “ $\sim 18\text{k}$ gas, $\sim 2\%$ ” claimed earlier.

The version worth doing: fold into the big MSM. `final_com` is itself the output of the 78-pair MSM at L3460, and both extra scalars are already known before that call: v is computed at L3292–3297 in the same sub-block, and x_3 is fixed. Appending the $(G, -v)$ and (π, x_3) pairs gives an 80-pair MSM costing $80 \times 12,000 \times 0.559 = 536,640$. That replaces $525,096 + 24,000 + 750 = 549,846$, a saving of about **13.2k gas** ($\approx 1\%$) plus four `staticcalls`. This mirrors the Rust, which adds `scaled_pi` into the right-hand DualMSM (`poly/kzg/mod.rs:544–558`). Two costs follow. Trace renders record `final_com` as the required trace id 33 (test.rs L1714), so they must keep materializing it separately. And the constructor’s worst-case G1MSM smoke test and the 0xa320 scratch window must grow from 0x30c0 to 0x3200 bytes.

Assessment. This is a [\[IMPROVE\]](#) gas finding [\[OPEN\]](#). The premise is right (two 1-pair MSMs and two ADDs, L3475–3490). The 2-pair merge is worth only $\approx 1.6\text{k}$ gas. Folding both pairs into the `final_com` MSM is the change worth making, at $\approx 13\text{k}$ gas, and it also removes four precompile calls and their `success` gates.

11.8.74 Lookup (logup) accumulator read matches the Rust verifier

The logup lookup argument is a soundness-critical check I had not yet cross-checked against the Rust. The contract’s read of the lookup commitments matches the Rust `logup/verifier.rs` exactly.

What the contract reads. For each lookup (the production has two), the contract reads `nb_chunks` helper commitments followed by one accumulator Z commitment (L1133–1161). It copies them into separate memory regions (`LOOKUP_HELPER_COMMS` vs `LOOKUP_Z_COMMS`) because the quotient and PCS schedules address them differently. The accumulator Z is read with the same `common_uncompressed_g1` range check as every other G1 (L1147), so a malformed accumulator encoding is rejected.

Cross-check against the Rust. The Rust `logup/verifier.rs` reads the accumulator commitment at L98 (`transcript.read()`), then the evaluations Z_{eval} (L146) and $Z(\omega x)_{\text{eval}}$ (L152) in the eval phase. The contract mirrors this split. The Z commitment is read in the commitment phase (L1147–1150), Z_{eval} and $Z(\omega x)_{\text{eval}}$ are read in the eval section, and the logup identity itself is evaluated in the VM. The helper/accumulator ordering (helpers first, then acc) matches the Rust’s “`nb_chunks` helper commitments and one accumulator commitment” (L75).

Assessment. This is a [\[OK\]](#) credit. The lookup accumulator read order, the helper/accumulator split and the G1 range check all match the Rust logup verifier, and the logup identity is consumed by the VM (the `q_has_top` check, §11.8.71, ensures no identity is left unchecked). The one readability note [\[OPEN\]](#): the contract’s “lookup 0 / lookup 1” comments don’t name the *semantic* lookup each corresponds to (for example, which `LookupArgument` in the circuit). An auditor therefore can’t tell *which* lookup is being verified without cross-referencing the circuit. This is a naming improvement for the generator’s emitted comments.

11.8.75 Permutation-argument read matches the Rust verifier

The copy-constraint (permutation) argument is the third soundness-critical argument. The contract’s read of the permutation product commitments and the VM’s identity check match the

Rust `permutation/verifier.rs`.

What the contract reads. The permutation product commitments are read in a loop of 0x300 bytes, i.e. 6 G1 points (L1119–1126). Each goes through `common_uncompressed_g1`, so every perm-Z point is range-checked, and is copied into `PERM_Z_COMMS_MPTR_BASE`. The comment (L1116–1117) correctly notes that these are used both by the permutation identities in the quotient numerator and later by the PCS openings.

Cross-check against the Rust. The Rust `permutation/verifier.rs` reads the product commitments in `read_product_commitments` (L43–59), then the evaluations `permutation_product_eval` (L109), `permutation_product_next_eval` (L115), and `permutation_product_last_eval` (L121) in the eval phase. The contract mirrors this. The *Z commitments* are read here (L1119–1126) and the evals in the eval section, and the VM’s permutation identity consumes `q_perm_z_cur/q_perm_z_next/ q_perm_z_last` (L1949–51). That is the same cur/next/last triple the Rust evaluates. The “last” eval exists for 5 of the 6 sets, as the Rust’s `iter.len() > 0` guard requires (the loop bound at L1997 is 5). The `q_has_top` check (§11.8.71) ensures the permutation identity is consumed, not left dangling.

Assessment. This is a [OK] credit. The permutation product read (6 points, range-checked), the cur/next/last eval triple and the VM identity consumption all match the Rust permutation verifier. The readability note is the same as for the lookup [OPEN]: the contract’s “permutation Z products” comment names the *mechanism* but not the *copy constraints* it enforces, so an auditor can’t see which permutation is being verified without the circuit. Across all three arguments (permutation, lookup and the gate identities), the contract’s read order and the VM’s identity consumption are consistent with the Rust. The soundness-critical structure is faithful; the remaining gap is human legibility, not correctness.

11.8.76 Public-input (instance) binding and the ABI-shape guard

The public inputs are the verifier’s only caller-supplied data besides the proof, so their binding is soundness-critical. The contract’s instance handling is a credit. A cheap ABI-shape guard rejects malformed layouts before any generated memory work, and the instances are absorbed into the transcript in the same order the Rust reads them.

The ABI-shape guard (L305). Before any generated work, the contract checks `calldataload(0x04) == 0x40` (the `proof` head points at the bytes payload) and `calldataload(0x24) == NUM_INSTANCE_CPTR - 0x04` (the `instances` head points at the generated instance array). The comment (L301–303) is explicit: the verifier is “a hand-rolled calldata parser”, and failing here “keeps malformed dynamic-argument layouts from being interpreted as a valid Midfall proof stream”. This is exactly the right posture for a hand-rolled parser: validate the ABI pointers before trusting them.

Instance absorption matches the Rust. The Rust `verifier.rs` receives `instances` (and `committed_instances`) as arguments. It absorbs the VK digest (L70), then each committed instance (L90–94), then each instance column’s length followed by its values (L96–101). The contract follows the same order. It absorbs the VK digest (L1008), the identity `committed_pi` as 128 zero bytes, the length word 19 (L1035), and then the 19 instance words from `INSTANCE_CPTR = 0x1ee4`. Each word is checked canonical ($< r$, L1045) and the loop fails fast at L1050 (1e21311c). This happens *before* any proof commitment is read. It is not the position of the `pi` absorb in §11.8.72, which is the KZG opening proof π and has nothing to do with public inputs. [CORRECTED] The accumulator is *read from* instance words 11–18 (`INSTANCE_CPTR+0x160`, L796). It is decoded from radix-2⁵⁶ limbs (§11.8.63) into `ACC_LHS/ACC_RHS` memory (L785–885).

before the transcript starts. Nothing is “re-serialized into the `instances` array”: the array is absorbed verbatim. The instance count is pinned at every call by `eq(19, calldataload(NUM_INSTANCE_CPTR))` (L955), and the exact-`calldatasize` check (L961) ensures that a caller cannot smuggle extra words.

Assessment. This is a [OK] credit. The public-input binding is guarded (the ABI-shape check runs before any memory work), count-pinned, canonicity-checked, and absorbed in the same transcript order as the Rust. The one readability note is that the contract’s `instances` are opaque scalars. The NatSpec (L274–278) correctly warns that “application contracts must bind the meaning of those instances separately: state roots, program identifiers, expected IVC outputs, chain/domain separation”. The verifier itself does not (and cannot) know what the instances *mean*. That semantic binding is the application’s responsibility, and the contract is right to flag it.

11.8.77 `q_com` / KZG multi-open: the fused final MSM

The KZG multi-opening reduction is the last soundness-critical computation. The contract fuses the quotient commitment (q_{com}) into the final MSM scratch rather than materializing it separately. This is a deliberate design with a readability cost.

The fused MSM (L3460). The contract computes `final_com` with a single 78-pair G1MSM over a scratch region (0xa320, length 0x30c0 = 78 pairs). Each pair is a G1 *commitment* with the scalar $x_1^i \cdot x_4^j$ (truncated powers), for example `mstore(0xd320, mulmod(mload(add(X1_POWERS_MPTR, 0x80)), x4_pow_4, r))` at L3456. The linearization pairs (the quotient limbs and the selector commitments) and f_{com} (scalar `x4_pow_5`, L3457–3458) are also in it. The MSM output is therefore the KZG multi-open reduction $\sum_j x_4^j q_{\text{com},j} + x_4^5 f_{\text{com}}$. The comment (L134–137) is explicit that “Q_COM materialization is currently fused into the final MSM scratch”. The quotient commitment is never written to `Q_COM_MPTR` as a standalone point; it is folded into the same MSM that produces `final_com`.

Cross-check against the Rust. The header comment (L8–10) points to `transcript/implementors.rs` and `poly/kzg/mod.rs`. The Rust `multi_prepare` reduces the per-set q_{com} and f_{com} with powers of x_4 into a single commitment (`mod.rs:488–508`), which is exactly what the fused MSM does. The contract’s x_4 -power scaling (the `x4_pow_i` table, §11.8.68) matches the Rust’s multi-open reduction coefficients, and the final pairing (§11.8.66) consumes `final_com` as the reduced commitment. The fused MSM is therefore a faithful KZG multi-open.

The readability cost. The fusion means the quotient commitment, a named object in the protocol, has no named home in the production contract. `Q_COM_MPTR` is declared (L137) but never written, and the actual q_{com} lives only as a few of the 78 MSM pairs. [PARTIAL] Two things mitigate this. First, the constant is not an unexplained dead name: its comment (L134–136) says it “intentionally aliases `Q_EVAL_SET_MPTR` and has zero reserved capacity until a future emitter starts writing `Q_COM_MPTR`”. Second, trace renders *do* materialize every per-set q_{com} (`lowering/kzg/mod.rs:1159–1170`), and the differential test requires the “PCS `q_com` point-set commitments” id range 40,000–41,000 (test.rs L1728–1733). The decomposition is therefore checked against the Rust even though the production render never names it. **Concrete fix** [OPEN]: either (a) emit a comment annotating which MSM pairs are the q_{com} contribution of which point set, or (b) drop the marker constant. Option (c), materializing q_{com} in production, is unnecessary, because trace renders already do it for verification.

Assessment. This is a [OK] credit on correctness (the fused MSM is a faithful KZG multi-open, and the per-set q_{com} is trace-checked) with a [IMPROVE] readability note. Fusing q_{com} into the MSM is a sensible gas choice, but the production source should annotate the MSM pairs by point set.

11.8.78 NatSpec quality: good on mechanics, silent on the trust model

The generated contract carries 58 lines of NatSpec, which is more than most hand-written verifier templates. It documents the curve differences (BLS12-381 vs BN254), the transcript model, the deployment smoke tests, the success-or-revert failure policy, and the memory strategy (absolute Yul addresses, scratch starting at 0x80). The `verifyProof` NatSpec explicitly warns that “application contracts must bind the meaning of those instances separately: state roots, program identifiers, expected IVC outputs, chain/domain separation, and any protocol-specific authorization are outside this raw verifier ABI”. That is a correct and important boundary statement.

What is missing: the trust model. [PARTIAL] The NatSpec does say that the contract “is not a generic verifier” and is generated for one `VerifyingKey` (L11–13), and that the VK runtime length and codehash are pinned (L38–39). It never states the consequence an auditor needs: **the verifier is only as trustworthy as its pinned VK**. The VK is a trusted input. If the VK is wrong (wrong circuit, wrong parameters, wrong commitments), the verifier will faithfully accept proofs for the wrong statement. A generated `@dev` block should say this explicitly:

```
/// @dev TRUST MODEL: this verifier is a faithful translation of the Rust
/// verifier for ONE specific VerifyingKey. The VK is a TRUSTED input:
/// the verifier does not (and cannot) check that the VK corresponds to
/// the intended circuit. Auditors must verify the VK out-of-band:
/// 1. Regenerate the VK from the trusted circuit definition and
///    confirm its runtime keccak equals EXPECTED_VK_CODEHASH_WORD.
/// 2. The VK was generated by the trusted generator (commit below).
/// 3. The generator's trace-equivalence suite covers the generator on
///    its test circuits, not this VK; run the trace render on this VK.
/// A wrong VK with a matching codehash is indistinguishable from a
/// correct one at the contract level.
```

(Two items in an earlier draft of this block were amended. A codehash cannot “match circuit parameters”; it can only be reproduced by regeneration. And the CI never sees the fee-12345 VK (§11.8.69), so the block must not claim that CI passed “for this VK”.)

What is missing: the soundness level. [OPEN] The NatSpec does not mention that x_3 , x_1^i and x_4^i are truncated to 128 bits (§11.8.12), and no soundness level for that configuration is stated anywhere. An auditor evaluating whether the verifier meets a 128-bit security target needs this number at the contract level, not buried in a spec chapter.

What is missing: the generator provenance. [OPEN] The NatSpec says the contract “ports the verifier flow from” the Rust files (L7–10). It does not state that **the contract is a derived artifact and the Rust generator is the source of truth**, and it does not say which generator produced it. An auditor who finds a bug in the generated Solidity needs to know three things: (a) the bug is in the generator, not in hand-written code; (b) fixing it means fixing the generator and re-rendering, not editing the contract; (c) which generator commit to fix. The last point is concrete here. B64 contains a quotient-certificate constant (L51) that main HEAD’s generator does not emit, so the artifact cannot be tied to a commit without outside knowledge (§11.8.79).

Assessment. The NatSpec is **good on mechanics** (how the contract works) but **silent on the trust model** (what the auditor must verify before trusting a deployment). This is a readability and auditability gap, not a correctness gap: the trust model is correct, it just isn’t documented at

the contract level. Adding the three missing `@dev` blocks (trust model, soundness level, generator provenance) would make the contract self-documenting for an auditor without changing a line of logic.

11.8.79 The VK codehash is a raw hex constant with no circuit-parameter provenance

The constructor pins the VK by codehash (L265--269), and the Yul body re-checks it at every `verifyProof` call (L920--923), which is correct defence in depth. But the pinned constant is a bare hex string:

```
uint256 internal constant EXPECTED_VK_CODEHASH_WORD =
    0xad26a4756f14289c40673c500189a7e4aab500e6f452581b262796b123a6c4e1;
```

An auditor reading the *verifier* source cannot tell which circuit parameters produced this constant. The generator *knows* them: `k` is read at `artifacts.rs:223`, `num_permutation_zs` at L234, and `num_fixed` via `vk.fixed_comms.len()` (L139). None of it is emitted alongside the codehash. **[PARTIAL]** Part of it is already recoverable. The companion VK source labels every header slot inline (`Halo2VerifyingKeyFee12345B64.sol` L69–80: `// num_instances, // k, ...`). The verifier cross-checks `num_instances=19`, `k=21`, the accumulator parameters and the quotient-certificate hash against the loaded VK header on every call (L934–941, `e64f2891`). What is missing is a summary next to the codehash, plus the generator commit.

Concrete fix. The generator should emit a provenance comment block alongside the codehash constant. The values below are the real B64 ones:

```
// VK provenance (generated; do not hand-edit):
// circuit: k=21, num_instances=19 (accumulator at 11..18),
//          advice_comms=15, fixed_comms=27 (10 simple selectors),
//          permutation: 18 columns in 6 Z-sets, lookups=2,
//          trashcans=1, quotient_limbs=4
// fee:     12345
// block:   64
// generator: solidity-verifier @ <commit>
// The codehash below is Keccak256 of the VK runtime bytes
// (INVALID || payload). To verify: regenerate the VK from
// the circuit definition and confirm the digest matches.
uint256 internal constant EXPECTED_VK_CODEHASH_WORD =
    0xad26a4756f14289c40673c500189a7e4aab500e6f452581b262796b123a6c4e1;
```

This is a comment-only change, with zero soundness risk, and it turns the codehash from an opaque magic number into a self-documenting provenance anchor. **[CORRECTED]** An earlier draft of this block used invented parameters (“12 fixed, 8 advice”) and also proposed a `rendered: <timestamp>` line. A timestamp would break deterministic rendering (`standard_plonk_render_is_deterministic_for_same_seed`, `test.rs` L1454). When solc CBOR metadata is enabled, any source-text change also changes the verifier’s runtime bytecode. The generator commit is sufficient.

Why this matters for auditability. The codehash is the **root of the trust chain**. The constructor pins it, the Yul body re-checks it, and the entire verifier’s correctness depends on it matching the intended circuit. If the auditor cannot determine which circuit the codehash corresponds to, they cannot verify the trust chain. B64 shows the problem concretely: its `EXPECTED_QUOTIENT_CERTIFICATE_HASH_WORD` (L51) is emitted by the M7 follow-up generator line (`e7f736c4`, 2026-06-07), not by main HEAD (`6b3d2096`), and nothing in the artifact says so. The generator has this information; it just doesn’t emit it. This is the same “information exists but is discarded at the emit boundary” pattern as the raw-hex addresses (Fix 1) and the `varN` names (Fix 2): the generator knows the semantic content but renders only the raw value.

11.8.80 All 34 revert paths are silent — no diagnostic information on failure

The generated contract contains 34 `revert()` calls, and **every single one is `revert(0, 0)`**: a silent revert with no reason data. Ten of them are in the constructor’s deployment smoke test, and 24 are reachable from `verifyProof`. The only reason string in the entire contract is “invalid vk” in the constructor’s `require` (L268). When a proof fails verification, the caller and auditor therefore get **no diagnostic information about which check failed**.

The failure taxonomy is invisible. [CORRECTED] The 34 silent reverts cover the following failure classes. Line numbers were re-verified; an earlier draft mislabelled L941, L965, L981 and L1050 as “precompile failures” and omitted L551 and L1397.

- **Deployment smoke-test failures** (L213–252, constructor only): MCOPY unavailable, G1ADD/G1MSM/PAIRING precompile failures, wrong return sizes.
- **VK dependency failures**: runtime length or codehash mismatch (L923), and VK header words disagreeing with the generated constants (L941).
- **ABI and calldata shape violations**: bad dynamic heads (L306); a proof length, instance count or `calldatasize` that is not the generated one (L965); a parser cursor that does not end at `NUM_INSTANCE_CPTR` (L1312).
- **Public-accumulator decode or validation failure** (L981), including its G1MSM subgroup checks.
- **Non-canonical scalars**: public instances (L1050, with the now-redundant deferred check at L1316), proof evaluations (L1233) and `q_evals` (L1286).
- **G1 encoding failures** (L401–410): a non-zero 16-byte pad in either coordinate’s high word, or a coordinate $\geq p$. On-curve and subgroup membership are left to the EIP-2537 calls.
- **Inversion failures**: a zero input to `scalar_inv` (L330), the `modexp` call failing or returning a short result (L340–341), and the Lagrange batch inversion failing or hitting a zero denominator (L1397).
- **VM failures**: an unknown opcode (L2562) or native index (L2528); an over-read past `q_end` (L2569); a value left live on the stack (L2570).
- **Final checks**: any PCS or accumulator-batch precompile failure (L3568), and a pairing that is false or fails (L551).

All of these collapse into the same silent `revert(0, 0)`. An auditor debugging a failed verification cannot distinguish “the proof is invalid” from “the chain doesn’t support EIP-2537” from “the quotient program has a bug” without instrumenting the contract or reading the bytecode. One mitigation already exists off-chain: the *trace* render emits a per-step log stream and compares it with the Rust. The last matching trace id localises the failure, and one trace-only path even reverts with its id as data (`QuotientAndLinearization.yul:123–126`).

Concrete fix. [OPEN] Emit a per-class error code. The `Error(string)` snippet in an earlier draft of this section was wrong. It wrote the selector with `mstore(0, 0x08c379a0)`, which puts it in bytes 28–31, so `revert(0, 100)` would start with four zero bytes. It also placed the string at offset 64, overlapping the length word at 36–67. A correct `Error(string)` needs `mstore(0, shl(224, 0x08c379a0))` and the data at offset 68. Custom errors are the better fit in Yul, because they are 4 bytes and cost about 10 bytes of code per site:

```
// e.g. error NonCanonicalScalar(); selector precomputed by the generator
mstore(0, shl(224, NON_CANONICAL_SCALAR_SELECTOR))
revert(0, 4)
```

Revert data is safe here: nothing secret is revealed, and it only costs gas on the failure path. The one trade-off is that several sites deliberately aggregate checks through `success` (L965, L3568). Per-check codes there need extra branches on the success path, so class-level codes per revert site are the pragmatic choice. (An unmerged branch, `solidity-verifier-wrap-adversarial-suite-and-spec`, already introduces a typed-error taxonomy (bffffb4be). Neither main HEAD nor B64 has it.)

Why this matters for auditability. The production verifier is a black box that either returns `true` or reverts silently. This is the same “information exists but is discarded at the emit boundary” pattern as the raw-hex addresses (Fix 1), the `varN` names (Fix 2) and the VK codehash provenance (§11.8.79). The generator knows which check is failing, because it emits the check, but renders only the silent revert.

Severity. MINOR. This is a debuggability and auditability gap, not a correctness or soundness issue. The silent reverts are correct, since they reject invalid inputs; they just don’t explain themselves. In a production deployment where proofs are verified by automated systems, the lack of error codes makes incident response harder: a failed verification cannot be triaged without replaying it against a trace render.

11.8.81 Helper-function deep review: the nine primitives that do the real work

Nine of the contract’s 15 Yul helper functions carry essentially all of the cryptographic weight. They are `scalar_inv`, `common_uncompressed_g1`, `batch_invert`, `ec_pairing`, and the four-function accumulator-decoding stack (`load_acc_coord_shifted`, `check_acc_coord_packing`, `load_acc_coord`, `load_acc_point`) topped by `validate_public_accumulator`. Each was read against its Rust source. The verdict is that *all nine are correct*. Some are correct in a way a human auditor cannot easily confirm, and one diverges semantically from the Rust it is named after. Two were initially flagged as resting on “coincidental” invariants **[CORRECTED]**: on re-check, both hold by construction (H-6, H-8). The findings follow, in severity order.

Finding H-1 **[RISK] — `batch_invert` does not have the semantics of the `ff::batch_invert` it is named after**

The generated `batch_invert` is named for, and structurally resembles, the Montgomery-trick batch inversion in the `ff` crate that the Rust verifier uses. But the two disagree on the single most important edge case: **what happens when a denominator is zero**. The Rust *skips* zeros and keeps going:

```
// ff-0.13.1/src/batch.rs (the Rust verifier's batch inversion)
for (tmp, p) in tmp.into_iter().rev() {
    let skip = p.is_zero();
    let tmp = tmp * acc;
    acc = F::conditional_select(&(acc * *p), &acc, skip); // zero skipped
    *p = F::conditional_select(&tmp, p, skip);           // zero left as zero
}
```

The generated Yul *fails the entire batch* the moment the running product hits zero:

```
// Halo2VerifierFee12345B64.sol L498
if iszero(gp) {
    ret := 0
    leave
}
```

This is a genuine semantic divergence, and the naming makes it a trap. An auditor who knows `ff::batch_invert` will assume a zero denominator is tolerated and silently left as zero, and will not expect the whole Lagrange block to abort. **It is not a soundness bug; in fact the Yul is the safer of the two.** The only call site (L1346) inverts the Lagrange denominators $(x - \omega^i)$ plus $x^n - 1$. A zero there means x is in the evaluation domain, which makes the proof invalid, and the Yul’s fail-closed behaviour is exactly right. The Rust (`poly/domain.rs:463`) returns all-zero L_i in that case and continues. The divergence is unreachable in practice. The main challenge x is a full-width squeeze (only x_3 and the x_1/x_4 powers are truncated, L1261–1270), so landing in the order- 2^{21} domain has probability about $2^{21}/r \approx 2^{-234}$ per attempt. An earlier draft instead attributed this to a “ 2^{128} truncated-Keccak collision”. The function name still promises `ff` semantics it does not deliver.

Concrete fix [OPEN]: rename to `batch_invert_failclosed` (or `batch_invert_strict`) and add a one-line comment stating the difference from `ff::batch_invert` explicitly. The current header comment (L439–447) describes the algorithm but not the zero policy. If the skip-zero behaviour is ever actually wanted, it must be a different function; do not silently change this one.

Finding H-2 [IMPROVE] — `ec_pairing`’s parameter names are inverted relative to the Rust `left/right`

The Rust final check (`msm.rs:284`) is $e(\text{left}, s \cdot g_2) \cdot e(\text{right}, -g_2) = 1$, i.e. $s \cdot \text{left} = \text{right}$. The Yul wrapper is:

```
function ec_pairing(success, lhs_mptr, rhs_mptr) -> ret {
    // pair1 = (lhs_mptr, G2_BASE)
    // pair2 = (rhs_mptr, NEG_S_G2_BASE)
    mcopy(scratch, lhs_mptr, 0x80)
    mcopy(add(scratch, 0x80), G2_BASE_MPTR, 0x100)
    mcopy(add(scratch, 0x180), rhs_mptr, 0x80)
    mcopy(add(scratch, 0x200), NEG_S_G2_BASE_MPTR, 0x100)
    ...
}
```

The single call site (L3569) is `ec_pairing(success, PAIRING_RHS_MPTR, PAIRING_LHS_MPTR)`. So the parameter named `lhs_mptr` is fed `PAIRING_RHS`, and `rhs_mptr` is fed `PAIRING_LHS`. Matching the G_2 bases through the equation shows the Yul checks $PAIRING_RHS = s \cdot PAIRING_LHS$. That equals the Rust’s $s \cdot \text{left} = \text{right}$ under the mapping $PAIRING_LHS \rightarrow \text{left}$, $PAIRING_RHS \rightarrow \text{right}$. The pairing is **correct**; the trace-equivalence ids 27, 28 and 35 confirm it. **[PARTIAL]** An earlier draft said a reader must “propagate G_2 bases through the pairing equation by hand”. That overstates it: the call site carries a 12-line comment (L3555–3566) stating exactly this mapping, including “the historical LHS/RHS naming follows the dual MSM accumulator ... Pass them swapped to `ec_pairing`”. What remains is that the function’s own parameter names contradict their use.

Concrete fix [OPEN]: rename the parameters to match the G_2 base they are paired with, e.g. `ec_pairing(success, g1_with_g2, g1_with_neg_s_g2)`. The call-site comment then becomes unnecessary.

Finding H-3 [IMPROVE] — `scalar_inv` hardcodes its scratch to a magic 0x2600

`scalar_inv` writes its modexp frame at a hardcoded `let p := 0x2600` (L331). The comment explains the *reason* (reuse the dead transcript buffer just below `VK_MPTR`), but the address itself is an unnamed magic number. I verified it is safe today. The transcript’s high-water mark is the 102-eval segment, $0x80 + 0x20 + 102 \cdot 0x20 = 0xd60$ (the pre- θ segment ends at `0xb20`). An earlier draft said “ $\approx 0x800$ ”. With `VK_MPTR=0x2700`, the 0xc0-byte frame at 0x2600 is far from any live data. The same pattern applies to `0xa1e0`, which serves as `SELECTOR_ACC_MPTR`, `BATCH_INV_SCRATCH_MPTR`, the accumulator `acc_scratch` (L808) and the constructor MSM smoke scratch (L234). **[PARTIAL]** The earlier claim of “no overlap check” is wrong at the generator level.

The memory planner allocates each of these as a named, phase-scoped region (`scalar_inv_scratch` at `layout/memory.rs:661–666`, `batch_invert_scratch` at `L774–781`). Its `validate()` rejects any overlap between regions whose lifetimes intersect (`L234–280`, enforced at render in `render/models.rs:708, 834`). It also has an explicit drift check for this scratch (`L989–995`). Commit 6b3d2096 additionally fixed a transcript-buffer under-sizing that could overrun `VK_MPTR` for multi-phase shapes; B64 is single-phase and not affected. What is *not* checked is phase disjointness in time, which holds by construction.

Concrete fix [OPEN]: render the planner’s region names as constants (`SCALAR_INV_SCRATCH_MPTR`, ...). The template currently emits the hex literal (`AssemblyHelpers.yul:10`). The build-time overlap check already exists.

Finding H-4 [OK] (with a readability caveat) — `common_uncompressed_g1` is faithful to the Rust

This one is genuinely correct and worth crediting. The Rust `to_input` (`implementors.rs:283`) builds the 128-byte EIP-2537 padded form with explicit zero pads and encodes the identity as 128 zero bytes. The Yul copies the four calldata words *verbatim* after (a) rejecting non-zero top-16-bytes in each `_hi` word and (b) rejecting coordinates $\geq p$. For canonical points the verbatim copy equals the Rust’s constructed bytes, and the identity case matches too. The top-16-byte rejection is the essential anti-malleability guard: without it, an attacker could perturb the transcript while keeping the same on-curve point. The only complaint is readability. The field-range test is a hand-rolled two-word comparison,

```
if iszero(or(lt(x_hi, BLS_P_HI),
             and(eq(x_hi, BLS_P_HI), iszero(gt(x_lo, BLS_P_MINUS_ONE_LO)))))) {
    revert(0, 0)
}
```

and it is repeated verbatim for x and y . A named `fp_lt_p(hi, lo)` helper would make the intent obvious and remove a copy-paste surface. The comparison is used three times (`L406, L409, L683`), not four as an earlier draft said. [OPEN]

Finding H-5 [IMPROVE] — the accumulator decoder uses complementary ifs

`load_acc_coord_shifted` reconstructs the 384-bit coordinate from seven radix-2⁵⁶ limbs and splits it at the 256-bit boundary. The split is correct, but the two mutually exclusive branches are written as two separate ifs (`L593, L601`):

```
if lt(shift, 256) {
    lo := add(lo, shl(shift, limb))
    if gt(add(shift, bits), 256) { hi := add(hi, shr(sub(256, shift), limb)) }
}
if iszero(lt(shift, 256)) { // == NOT the first condition
    hi := add(hi, shl(sub(shift, 256), limb))
}
```

Both predicates are evaluated on every iteration, and the second is literally the negation of the first. [CORRECTED] This is not a codegen artifact. The function is a static template (`AccumulatorHelpers.yul:39–49`), and Yul has no `else`. The idiomatic single-test form is `switch lt(shift, 256) case 0 {...} default {...}`. The cost is a few gas per limb. Not a bug: the arithmetic is right. It is noise that makes a 384-bit limb reassembly slightly harder to audit. [OPEN]

Finding H-6 [NOTE] — `check_acc_coord_packing`’s `used_bits >= 256` branch is unreachable

The canonicity check is `ok := and(ok, lt(calldataload(...), shl(used_bits, 1))` (L653–656), guarded by `if lt(used_bits, 256)`. The guard is necessary, because `shl(256,1)=0` would make the test vacuous. When `used_bits >= 256`, no high-bit check is performed. An earlier draft warned that “a future emulation with 5 limbs/word at 56 bits” would silently lose the check. **[CORRECTED]** That configuration cannot be rendered. `AccumulatorEncoding::validate_for_num_instances` rejects every encoding other than exactly 7 limbs of 56 bits (`api.rs:138–148`, `UnsupportedAccumulatorEncoding`), `LIMBS_PER_WORD` is the constant 4 (`layout/mod.rs:139`, unit-asserted at L703–705), and the VK header check re-asserts 7 and 56 on every call (L938–939). The maximum is therefore $4 \times 56 = 224 < 256$. Even for an ≥ 256 -bit word there would be no unused high bits to check, and instances are already range-checked $< r$ (L1045). A `used_bits < 256` assertion would be belt-and-braces only.

Finding H-7 [IMPROVE] — `validate_public_accumulator` carries render-time generality it does not use

The function has `acc_instance_ptr := add(INSTANCE_CPTR, 0x0160)` and `limbs_per_word := 4` (L792, L796). Both are rendered from the VK/config: the offset is `acc_offset=11`, re-checked against the VK header at L937. This is intended specialization, not an unexplained hardcoded. **[CORRECTED]** An earlier draft said both scalar pointers are dead. In fact `lhs_scalar_ptr` (L803) is *used*: it becomes `rhs_instance_ptr` (L835), because the collapsed layout has no scalar word. Only `rhs_scalar_ptr` (L838) is dead, and `solc` removes it. The `acc_msm_len` guard is commented as being “for synthetic render configurations” (L857–859), so the code defends a branch this render never takes. The `pop(lhs_is_id)` and `pop(rhs_is_id)` lines (L815, L847) discard computed identity flags. Together these give the impression of a function generated for a more general case and then specialized.

Concrete fix [OPEN]: emit the offsets as named constants (`ACC_INSTANCE_OFFSET`, `ACC_LIMBS_PER_WORD`) and drop `rhs_scalar_ptr` and the always-true guard when the render is the collapsed-pair case.

Finding H-8 [OK] — the `decoded_zero` rejection is sound by construction

`load_acc_point` rejects a non-identity point that decodes to (0,0) (L762), because EIP-2537 reserves (0,0) for infinity. **[CORRECTED]** An earlier draft called this a “curve coincidence” and said a legitimate point with both coordinates $p-1$ would be wrongly rejected. That misreads the codec. The codec maps *encoded* $p-1$ to *decoded* 0, so encoded $(p-1, p-1)$ is the affine point (0,0). (0,0) lies on no short-Weierstrass curve $y^2 = x^3 + b$ with $b \neq 0$, and that is exactly why EIP-2537 can use it as the infinity encoding. No legitimate point is rejected, and a curve swap cannot introduce the issue without also breaking EIP-2537’s own encoding. The existing comment (L757–761) already states the EIP-2537 rationale.

Summary of the helper review

Function	Verdict	Issue
<code>batch_invert</code>	RISK	Fails-closed on zero; <code>ff</code> skips zeros. Safer but misnamed.
<code>ec_pairing</code>	IMPROVE	Param names inverted vs Rust left/right; mapping documented at call site (L3555–3566).
<code>scalar_inv</code>	IMPROVE	Magic <code>0x2600</code> ; planner region + overlap check exist, name not emitted.
<code>common_uncompressed_g1</code>	OK	Faithful; hand-rolled fp-range check repeated ($\times 3$).
<code>load_acc_coord_shifted</code> <code>check_acc_coord_packing</code>	IMPROVE NOTE	Complementary ifs; use <code>switch</code> . <code>used_bits >= 256</code> branch unreachable (7×56 enforced).
<code>load_acc_coord</code>	OK	Identity-flag probe has no false positives.
<code>load_acc_point</code>	OK	<code>decoded_zero</code> sound by construction ($((0, 0)$ off-curve for $b \neq 0$).
<code>validate_public_accumulator</code>	IMPROVE	One dead pointer (<code>rhs_scalar_ptr</code>), always-true guard.

The through-line matches the rest of this review: *the math is right; some names are not*. H-1 is the one that would actually mislead a soundness review: a `batch_invert` that isn’t `ff`’s. H-2’s inverted parameter names are mitigated by the call-site comment. Both are zero-risk to fix by renaming.

11.8.82 The quotient VM: a pinned bytecode interpreter and its native callbacks

The heart of the contract is not the pairing or the transcript. It is the `q_program` interpreter (registers from L1650, dispatch loop L1692–2564, end checks L2569–2570): a byte-addressed VM that reads a 0x11cf-byte program from the VK payload and turns it into the y -batched constraint numerator $\nu_y(x)$. Decoding the pinned program from the VK source gives **101 instructions using exactly 11 opcodes**: `0x05` $\times 12$, `0x06` $\times 7$, `0x08` $\times 5$, `0x0b` $\times 21$, `0x0d` $\times 20$, `0x10` $\times 2$, `0x11` $\times 12$, `0x19` $\times 1$, `0x1b` $\times 4$, `0x1f` $\times 1$ and `0x21` $\times 16$. There is no `0x0a`: main folds happen inside the callbacks and the inline and trash Yul. This is the single most complex region in the artifact and the one an auditor must trust the most. I read every opcode case against the Rust `partially_evaluate_identities` (mod.rs:488) and `compute_linearization_commitment` (linearization/verifier.rs:45). The interpreter is **correct and its trust model is sound**. The bytecode is pinned by the VK codehash, no proof calldata can alter control flow, and the end-of-program check fails closed. The generator also statically validates the exact pinned bytes before embedding them (`validate_quotient_program`, vm/mod.rs:2003–2021). It is also the region where the “information discarded at the emit boundary” pathology is most severe. The VM is written largely in magic addresses and dead arithmetic, and its opcode summary documents a larger ISA than the one it implements.

Finding V-1 [OK] — the end-of-program check is genuinely strong

```
if iszero(eq(q_pc, q_end)) { revert(0, 0) }
if q_has_top { revert(0, 0) }
```

This is the right pair of assertions. The operand stack must be empty, so no expression is left half-evaluated. The program counter must land *exactly* on `q_end`. The loop condition already ensures $q_{pc} \geq q_{end}$ on exit, so this second check specifically catches a final instruction whose operands over-read past the end. A generator bug that mis-encodes an operand width or

drops a fold is caught here rather than silently producing a wrong ν_y . The same invariants are proven offline for the pinned bytes: no underflow, fold boundaries of exactly one value, and an empty stack at the end (vm/mod.rs:2014–2018, 2231–2246). Credit where due: this is the kind of fail-closed invariant that should be in every VM, and it is.

Finding V-2 [OK] — the y -batch fold order is faithful to the Rust reverse-fold

The Rust batches identities by iterating *in reverse* with an increasing power:

```
// linearization/verifier.rs:74
let mut y_pow = F::ONE;
expressions.iter().rev().for_each(|(col_idx, eval)| {
    *grouped_points.entry(*col_idx).or_insert(F::ZERO) += y_pow * eval;
    y_pow *= y;
});
```

The Yul does the equivalent forward Horner ($A := A*y + eval$) into 0xa320. Forward Horner $\sum_i e_i y^{N-1-i}$ equals the Rust reverse accumulation $\sum_k y^k e_{N-1-k}$: same polynomial, same coefficients. The selector buckets use a precomputed y^{gap} to jump each bucket to the correct global position instead of tracking a global y_pow . That is an honest optimization, not a semantic change. The gate→perm→lookup→trash order matches mod.rs:516–604. Faithful.

Finding V-3 [RISK] — the opcode-summary comment advertises a VM the switch does not implement

The comment block at L1673–1683 lists all 31 defined opcodes: 0x01/0x09 push const, 0x07 mul, 0x1c LIN7, 0x1d BILIN7_ROW, 0x1e BILIN7_PAIRWISE, 0x20 POW5, 0x22 AFFINE_SUM, and so on. The actual switch implements **eleven**: 0x05, 0x06, 0x08, 0x0b, 0x0d, 0x10, 0x11, 0x19, 0x1b, 0x1f, 0x21. Every other listed opcode is *not emitted by this render* and falls through to default { revert(0,0) }. That includes the standalone POW5, LIN7, BILIN7_ROW and BILIN7_PAIRWISE. The generator already prunes the switch to the opcodes the program uses (quotient_program_usage, vm/mod.rs:1970, feeding quotient.rs:174–195). Only the summary is static template text (QuotientNumeratorBlock.yul:172–183).

This is not a soundness bug: unused opcodes revert, and the pinned program never emits them. It is an *audit* hazard. A reviewer cross-referencing the comment against the switch will look for case 0x20 for POW5, not find it, and reasonably wonder whether the S-box terms are computed at all. In fact they are computed by the inline q_pow5() helper (L1415) in the trash suffix, outside the VM. The LIN7 and BILIN7 semantics survive only as sub-blocks of 0x21 MODARITH7. **Fix [OPEN]**: render the summary from used_ops, the same set that already prunes the switch, and mark the rest “ISA, not used in this render”.

Finding V-4 [RISK] — the VM is written largely in magic addresses

In the VM region alone (L1650–2570), 0xa320 appears **50 times** (the global numerator accumulator), 0xa980 **14 times** (the VM operand-stack base *and* the native-callback scratch base), and 0xa360 **3 times** (the y -power table). None of these is a named constant. The single most-referenced value in the entire interpreter is a bare 0xa320:

```
mstore(0xa320, mulmod(mload(0xa320), y, r))
mstore(0xa320, addmod(mload(0xa320), q_perm_eval, r))
```

It appears fifty times over, with no name saying “this is ν_y , the running batched numerator”. The same address is later reused as the PCS q_eval staging table (L2927) and the final-MSM input base (L3460), which is exactly why a phase-specific name matters. The generator has a MemoryRegion.name field for every one of these regions (layout/memory.rs:180) and uses it in overlap-validation error strings, then throws the name away at the emit boundary. This is the

same defect as the raw-hex scratch addresses in §13, concentrated in the hardest code to read. **Fix [OPEN]**: emit `NUMERATOR_ACC_MPTR`, `VM_STACK_MPTR` and `Y_POW_TABLE_MPTR` as named constants. The mechanism already works, as the contract’s 68 named `*_MPTR/*_CPTR` constants show. (An earlier draft said 78.) It just was not applied here.

Finding V-5 [IMPROVE] — selector tail multipliers are unannotated magic offsets

The ten selector-tail updates just after the trash suffix (L2780–2817) read like a ransom note:

```
mstore(q_sel_ptr, mulmod(mload(q_sel_ptr), mload(add(0xa360, 0x0600)), r))
mstore(q_sel_ptr, mulmod(mload(q_sel_ptr), mload(add(0xa360, 0x0400)), r))
```

Those offsets are y^{48} , y^{32} , ...: the gap from each selector’s last contribution to the final batch position. Nothing in the source says *which* selector, *what* gap or *why* that power. The generator knows the selector identity and the gap, because it computed them to emit the offset, but it emits only the raw address. An auditor cannot verify a single tail without independently re-deriving the whole identity stream. **Fix [OPEN]**: emit `// selector `q_range_fixup', tail gap = 48 → y^{48}`  above each line, or emit a named `SEL_TAIL_<name>` constant.

Finding V-6 [IMPROVE] — dead arithmetic: `mulmod(0, ...)` and multiply-by-one

The generic Horner emitter does not constant-fold, so a leading term whose coefficient is zero is still emitted as a live multiply. The generated Yul computes `mulmod(0, ..., r)` five times, all of which are always zero. Four are `mulmod(0, q_lookup_theta, r)` in the lookup callback (L2079–2176) and one is `mulmod(0, q_trash_tau, r)` in the trash suffix (L2625). The nine inline gate-identity blocks (prefix and `0x1b` callbacks) open with `let var0 := 0x1` and close with `mulmod(var0, expr, r)`, a multiply by one that does nothing:

```
let var0 := 0x1
...
let var19 := mulmod(var0, var18, r) // == var18
```

Each is a wasted `mulmod` (8 gas plus stack shuffling) and, worse, wasted reader attention. You have to convince yourself the `var0` really is 1 and the 0 really is 0 every time. A constant-folding pass in the lowering layer would delete all of it. **Fix [OPEN]**: fold constant 0/1 coefficients at emit time, and skip the leading Horner step when the top coefficient is zero.

Finding V-7 [NOTE] — no runtime stack-depth guard in the interpreter

The push path grows `q_sp` without a runtime bound:

```
if q_has_top {
    mstore(q_sp, q_top)
    q_sp := add(q_sp, 0x20)
}
```

There is no runtime check that `q_sp` stays inside the reserved stack region. **[PARTIAL]** An earlier draft rated this [RISK] on the grounds that a generator bug emitting unbalanced pushes would silently walk `q_sp` past the reserved words. That scenario is excluded at build time. `validate_quotient_program` decodes the *finalized*, *pinned* bytes and computes the exact high-water mark (`vm/mod.rs:2003–2021`, 2190–2256). The builder asserts that it equals its own accounting (L1211–1216), and the planner reserves `max(max_stack, callback_scratch)` words at the stack base (`quotient.rs:249–257`), with overlap validated. Because the runtime program *is* those validated bytes (codehash-pinned), an `lt(q_sp, VM_STACK_END)` guard would be defence in depth against a generator whose validator is itself wrong. That is reasonable but low priority, at a few gas per push.

Finding V-8 [NOTE] — native callbacks reset rather than assert an empty VM stack

Every native callback (0x19 permutation, 0x1f lookup, 0x1b heavy identity) begins by resetting the stack:

```
q_top := 0
q_has_top := 0
q_sp := 0xa980
```

The comment (L1931–1932) says native callbacks are “identity-boundary opcodes” that “must not inherit any partially evaluated VM stack state”. The code does not *verify* at runtime that the stack is empty; it just *discards* whatever was there. The callback scratch tables (`q_perm_vals := 0xa980`, `q_lookup_f := 0xa980`) also start at the VM stack base. **[PARTIAL]** The invariant is not “asserted nowhere”, as an earlier draft said. The offline validator rejects any native callback reached with a non-empty stack (`vm/mod.rs:2239–2246`), the builder asserts it at emission (L1165, L1181, L1194), and the shared base is sized for both users (`quotient.rs:249–257`). **Optional hardening:** if `q_has_top { revert(0,0) }` before each callback’s reset, one line per callback, as runtime defence in depth.

Finding V-9 [NOTE] — MODARITH7’s pairwise matrix read (dead in this render)

The `BILIN7_PAIRWISE` block inside `0x21` reads its 13-byte Hankel coefficient matrix with

```
let qconst := byte(0, mload(add(q_coeff_pc, add(q_i, q_j))))
```

for $i, j \in [0, 7)$, so the last read is at offset 12. The `mload` there reads bytes 12...43, past the 13-byte coefficient block, and `byte(0, ...)` keeps only byte 12. This is the same byte-fetch idiom the interpreter uses for every opcode (`byte(0, mload(q_pc))`, L1697), so it is safe. It is also **never executed in production**: all 16 MODARITH7 instructions in the pinned program have a pairwise count of 0 (they use 32 LIN7, 78 ROW, 96 memory and 285 product blocks). A comment would still help readers.

Summary of the VM review

Aspect	Verdict	Issue
End-of-program check	OK	<code>q_pc==q_end</code> (over-read) + empty-stack: strong fail-closed.
y -batch fold order	OK	Forward Horner $\equiv$ Rust reverse-fold; order faithful.
Opcode-summary comment	RISK	Lists 31 opcodes; switch has 11 (pruned via <code>used_ops</code>).
Magic addresses	RISK	<code>0xa320×50</code> , <code>0xa980×14</code> , <code>0xa360×3</code> , unnamed.
Selector tail offsets	IMPROVE	Raw <code>0x0600/0x0400</code> with no name/gap.
Dead arithmetic	IMPROVE	<code>mulmod(0,...)×5</code> , multiply-by-1 in 9 blocks.
Stack-depth guard	NOTE	No runtime bound; depth proven offline on pinned bytes.
Native stack reset	NOTE	Resets, not asserts; empty stack proven offline.
Pairwise over-read	NOTE	Safe <code>byte()</code> idiom; pairwise count 0 in production.

The verdict on the VM: the cryptography is right and the pinned-bytecode trust model is the correct one. The stack-safety invariants are proven offline over the exact pinned bytes, which is stronger than it looks from the Yul alone. What is wrong is that the VM is the *least* auditable code in the contract while being the *most* load-bearing. The remaining audit hazards are the misleading opcode summary and the magic addresses, and both are cheap to fix: render

the summary from `used_ops`, name three addresses and fold two constants. Runtime `q_has_top` and stack-bound guards are optional defence in depth. The generator already has the names, the constant values and the opcode-usage set; it just does not put them in the output. That is the whole story of this codebase in one interpreter.

11.8.83 The PCS section: the KZG multi-open final computation

The PCS region (L2869–3493, preceded by the linearization-scalar prep at L2828–2867) is the KZG multi-open. It consists of ten generated sub-blocks that turn the per-rotation evaluations into the final f_{eval} , the batched opening scalar v , the fused `final_com` MSM, and the pairing inputs. I checked every block against the Rust `multi_prepare` (poly/kzg/mod.rs:321–574). **The math is faithful:** the f_{eval} Lagrange interpolation, the v inner product and the final pairing equation all match the Rust exactly. This review is therefore *not* about correctness. It is about the fact that this is the least readable, most magic-address-dense region in the contract, and the place where the “information discarded at the emit boundary” pathology reaches its worst form.

Finding P-1 [OK] — the f_{eval} Lagrange interpolation is faithful

The Rust computes each set’s contribution as `(proof_eval - r_eval) * den.invert()`, where `r_poly = lagrange_interpolate(points, evals)` (mod.rs:475–479). The Yul expands this to `eval = proof_eval*den_inv -  $\sum_i Q\_SET\_i * dx\_inv\_i * lbasis\_inv\_i$` . These are algebraically identical. With $L_i(x_3) = \prod_{j \neq i} (x_3 - p_j) / (p_i - p_j)$ one gets $r_{\text{eval}} = \text{den_inv}^{-1} \sum_i e_i dx_inv_i lbasis_inv_i$, so $(proof_eval - r_{\text{eval}}) \cdot \text{den_inv}$ is exactly the Yul expression. The reverse Horner over x_2 matches `acc_eval * x2 + eval` under the Rust’s `.rev()` (mod.rs:471–481). Faithful.

Finding P-2 [RISK] — `final_com` is 157 `mcopy/mstore` ops over 229 distinct raw-hex addresses with zero semantic annotation

Sub-block 9 (L3271–3464) builds the 78-term fused MSM as a flat sequence of `mcopy/mstore` pairs:

```
mcopy(0xa320, 0x98e0, 0x80)
mstore(0xa3a0, 1)
mcopy(0xa3c0, 0x9960, 0x80)
mstore(0xa440, mload(add(X1_POWERS_MPTR, 0x40)))
mcopy(0xa460, 0x9d60, 0x80)
mstore(0xa4e0, mload(add(X1_POWERS_MPTR, 0x60)))
```

That is **157 copy/store operations** (78 `mcopy`, 79 `mstore`) over **229 distinct raw-hex operand addresses** in one block, and *not one of them says what it is*. Is `0x98e0` the advice column a_0 at rotation 0? A quotient limb? A selector commitment? Which commitment’s x_1 -power is the scalar `mload(add(X1_POWERS_MPTR, 0x40))`? The generator knows all of this. The Rust carries a `CommitmentLabel` (`Fixed(idx)`, `NoLabel`, `Custom("linearization_poly")`) on every term (mod.rs:379–385, 493; linearization/verifier.rs:98), and the memory planner has a `MemoryRegion.name` for every address. At the emit boundary all of it is thrown away and replaced with a bare `0x98e0`. This is the worst audit hazard in the entire contract: a reviewer checking the linearization MSM by hand must reverse-engineer 78 terms from raw addresses with no legend. **Fix [OPEN]:** emit `// term 12: fixed col `a_3' rot=+1, scalar = x1_pow[2]*x4_pow[3]` above each pair, or emit the address-map appendix (Ch 13 Fix 5) so the hex is at least resolvable.

Finding P-3 **[RISK]** — the f_{eval} batch-inversion is hand-inlined with opaque `bq/bp_N` names, four times over

Each multi-point f_{eval} set inverts its $2n$ ($dx, lbasis$) terms with a Montgomery walk whose running inverse is named `bq` and whose running products are `bp_0..bp_5`:

```
let bq := scalar_inv(bp_5)
let lbasis_inv_2 := mulmod(bq, bp_4, r)
bq := mulmod(bq, lbasis_2, r)
let lbasis_inv_1 := mulmod(bq, bp_3, r)
bq := mulmod(bq, lbasis_1, r)
let lbasis_inv_0 := mulmod(bq, bp_2, r)
bq := mulmod(bq, lbasis_0, r)
let dx_inv_2 := mulmod(bq, bp_1, r)
bq := mulmod(bq, dx_2, r)
...
```

Across the four multi-point sets there are **16 bq reassignments** (5 per cardinality-3 set, 3 per cardinality-2 set), 4 declarations and 40 mentions in total. There are five `scalar_inv` calls in all, the fifth being set 0’s direct inverse (L3265). An earlier draft counted “40 mutations, eight per set”. No comment says “`bq` is the running inverse of the suffix product; each line peels one factor off”. A reviewer verifying the batch inversion has to trace `bq` through the whole backward pass, per set, with names that carry no meaning. The `batch_invert` helper exists (reviewed in §11.8.81) but is not used here, because its interface takes a contiguous memory range while these operands live in Yul locals. The algorithm is the same. **Fix [OPEN]**: a named helper `invert_dx_lbasis(dx0..dxn, lbasis0..lbasisn)` that returns the inverse arrays. Better still, stage all sets’ denominators in memory and make one `batch_invert` call (§11.7). Either way, four copies of this walk collapse into one audited function.

Finding P-4 **[IMPROVE]** — the five f_{eval} set-blocks are near-identical

Sets 4 and 3 have cardinality 3, sets 2 and 1 have cardinality 2, and set 0 has cardinality 1. The two cardinality-3 blocks are identical except for the rotation-point subset they use ($\{p_2, p_3, p_0\}$ vs $\{p_2, p_3, p_1\}$) and their `Q_EVAL_SET_MPTR/Q_EVAL_CPTR` offsets. This is the duplication Ch 13 Fix 6 targets. Note that the repeated part is the per-set Lagrange-basis and batch-inversion code, not the one-line Horner step. It is about 150 lines of copy-paste that a parameterized per-set helper would replace. Every duplicated block is a place where a future edit must be made N times consistently. **[OPEN]**

Finding P-5 **[IMPROVE]** — the rotation points are an unannotated `omega_inv` chain

Sub-block 1 (L2880–2905) computes the four rotation points $x\omega^0, x\omega^1, x\omega^{-1}$ and $x\omega^{-10}$. The last comes from a chain of `mulmod(..., omega_inv)` with no comment about which rotation each is:

```
x_pow_of_omega := mulmod(x_pow_of_omega, omega_inv, r)
mstore(add(ROT_POINTS_MPTR, 0x20), x_pow_of_omega)
x_pow_of_omega := mulmod(x_pow_of_omega, omega_inv, r)
... // eight more times
mstore(add(ROT_POINTS_MPTR, 0x0), x_pow_of_omega)
```

The result is $x\omega^{-10}$, the `rotation_last` = $-(\text{blinding}+1)$ point. (An earlier draft said $x\omega^{-9}$: there are ten `omega_inv` multiplications.) Nothing in the source says “rot -10 ” or which column set uses it; only “// 4 distinct rotation(s)” is emitted. The generator knows the rotation, since it is the `Rotation` in the Rust query, but it emits a bare chain of multiplies. **Fix [OPEN]**: annotate each stored point with its rotation value.

Finding P-6 **[IMPROVE]** — the pairing scratch aliases the transcript base 0x80

Sub-block 10 builds the pairing inputs in scratch at 0x80, 0x100 and 0x180:

```

mcopy(0x80, G1_BASE_MPTR, 0x80)
mstore(0x100, addmod(0, sub(r, mload(V_MPTR)), r))
success := staticcall(gas(), 0x0c, 0x80, 0xa0, 0x80, 0x80)

```

0x80 is TRANSCRIPT_MPTR (L62). It is safe here only because the transcript is dead by the time the pairing runs. **[PARTIAL]** This aliasing is not undocumented in the generator. The planner registers it as the phase-scoped region `pcs_pairing_static_working` (layout/memory.rs:626–631), under the stated rule that “the transcript buffer is no longer live once algebra/precompile work begins” (L612–613). The accumulator batch at 0x100 and `ec_pairing`’s scratch at 0x0300 follow the same pattern. What is missing is a name in the emitted Solidity. The planner’s overlap check treats different phases as disjoint, so it would *not* catch a refactor that moved the pairing before the transcript finished. Phase ordering holds by construction. **Fix [OPEN]**: emit the planner’s region as a named PAIRING_SCRATCH_MPTR.

Finding P-7 **[NOTE]** — the rolled `q_eval_set` blocks use `mload(mload(eval_p))` double-indirection

The “rolled” sets (sub-blocks 3, 6 and 7) stage source addresses as data, then double-load:

```

mstore(0xa320, 0x8a20)
...
q_eval_set_0 := addmod(q_eval_set_0, mulmod(mload(mload(eval_p)), pow, r), r)

```

This is a codegen trick to make the Horner loop uniform across sets of different shapes. It works, but `mload(mload(eval_p))` forces the reader to hold the staging indirection in their head. A one-line comment per block (“`eval_p` points to a staged source address; double-load to reach the eval”) would help.

Finding P-8 **[NOTE]** — `k := 21` is a literal, but it is tied to the VK

The linearization scalar walk uses the literal `let k := 21` (L2846), as does the Lagrange block (L1322). **[ADDRESSED]** An earlier draft said no assertion ties this literal to the VK. In fact VK loading checks `eq(mload(K_MPTR), 21)` on every call, before any proof byte is parsed (L935, commit e64f2891). A VK whose *k* disagrees with the rendered literal therefore reverts at L941. No further fix is needed. Naming the literal (DOMAIN_K) would be cosmetic.

Summary of the PCS review

Aspect	Verdict	Issue
f_{eval} Lagrange	OK	Algebraically identical to Rust (<code>proof_eval-r_eval</code>)/den.
v / power walks	OK	$v = \sum(q_evals + f_eval) \cdot x^i$ matches Rust.
<code>final_com</code> MSM	RISK	157 ops / 229 raw-hex addresses, no term labels.
f_{eval} batch-invert	RISK	Opaque <code>bq/bp_N</code> , 16 reassignments, 4× dup.
Five set-blocks	IMPROVE	Near-identical copy-paste (Ch13 Fix 6).
Rotation points	IMPROVE	Unannotated <code>omega_inv</code> chain ($x\omega^{-10}$).
Pairing scratch	IMPROVE	Aliases TRANSCRIPT_MPTR; planner region exists, unnamed in Solidity.
Rolled staging	NOTE	<code>mload(mload(...))</code> double-indirection.
<code>k := 21</code>	OK	Literal, but checked against VK <code>k</code> at L935.

The verdict on the PCS section: the KZG multi-open math is faithful, and fusing the MSM is a legitimate gas optimization. The problem is purely readability. Sub-block 9, the fused `final_com` MSM, is on its own the hardest thing in the contract for a human to audit: 78 terms,

229 raw addresses, no labels and no legend. The generator holds the commitment labels, the rotation indices, the set indices and the memory-region names for every single term, and discards all of them at the emit boundary. Naming the addresses and annotating the MSM terms would turn the worst-audited block in the contract into a readable one, and it is pure codegen plumbing.

11.9 Overall assessment

The artifact is correct, sound, and safe. It is a faithful render of the Rust verifier with no defect found in the reviewed span, strong fail-closed behaviour, pinned dependencies (VK code-hash and header cross-checks, L920–941), and a properly randomised accumulator pairing. The two VM stack items (V-7 and V-8) have no runtime guard. They are not open risks, however: the generator proves stack safety offline over the exact pinned bytes (`validate_quotient_program`), so runtime guards would be defence in depth only. Several helper findings were corrected on re-check: H-6 and H-8 hold by construction, and H-2 is documented at the call site. One latent environmental risk sits outside this span. The production via-IR compile reserves `[0x80, 0x8e0)` for spilled variables, overlapping the raw scratch (§11.8.19).

Assurance is good but aimed at a sibling, not the shipped VK. The trace-equivalence harness covers the production *shape*: the $k = 20$ IVC decider has the same 18/6/3 permutation, the same lookups, the same 78-pair MSM and the same opcode mix. It does not cover the exact fee-12345 VK ($k = 21$, 19 instances, embedded quotient), and B64 was rendered from the M7 follow-up generator line rather than main HEAD. The crate’s CI jobs are also defined in a nested `.github` that the monorepo’s GitHub Actions never executes (§11.8.69).

The gap is human auditability, not machine correctness. The 246 unnamed scratch addresses, the opaque SSA names, the five near-identical f_{eval} set blocks and the invisible VM bytecode together make a line-by-line manual cross-check against the Rust verifier far harder than it needs to be. Much of the missing information already exists in the generator but is not emitted: the region names, the per-identity execution manifest and the used-opcode set. The five improvements in §11.8 (address legend, named slots, trace manifest, VM disassembly, semantic names) would each pay for themselves the first time a human needs to verify a render. On gas, the cross-set f_{eval} batch inversion in §11.7 (one modexp instead of five) and folding the $(G, -v)$ and (π, x_3) pairs into the fused MSM ($\approx 13\text{k}$ gas, §11.8.73) are the two concrete wins. The 2-pair merge is worth only $\approx 1.6\text{k}$.

Recommendation: ship as-is for correctness. First, run the existing trace harness on the exact fee-12345 VK and wire the crate’s jobs into the root CI. Next, prioritise the address legend and named-slot emission as the highest-leverage audit-facilitation change. Finally, fold the cross-set batch inversion into the f_{eval} emitter and the two pairing-RHS pairs into the `final_com` MSM.

Chapter 12

Design critique: what is wrong here, and how to fix it

The preceding chapters establish that the generated verifier is *correct* — no bug was found that breaks soundness on the reviewed artifacts. This chapter is not about correctness. It is about whether the **design** is sound, and it pulls no punches. The thesis: *the architecture optimizes for the wrong axis (contract size) and pays for it in auditability, runtime cost, and unverified trust roots — and it papers over each cost with a band-aid instead of fixing the cause.*

A correct verifier that nobody can audit, that no tool can analyze, whose soundness level under truncated challenges is unstated, and whose trust root is a hand-written interpreter with no formal proof, is a liability even when it has no bug today. Each critique below is tagged **[CRITICAL]** / **[MAJOR]** / **[MINOR]** and paired with a concrete, actionable fix.

Status tags (re-verified). Every item and fix also carries a status tag checked against generator main HEAD 6b3d2096 (2026-07-17) and the B64 render regenerated 2026-07-28: **[AD-DRESSED]** (the current code already does it), **[PARTIAL]**, **[OPEN]**, or **[CORRECTED]** (the critique’s own factual premise was wrong and has been rewritten). B64 was rendered from the M7 follow-up line (e7f736c4), not from main; the mechanisms cited below exist in both, except the quotient-certificate-hash check (B64 L940), which only the render lineage has. `file:line` references are to main HEAD. Line numbers *Ln* refer to Halo2VerifierFee12345B64.sol: 3,579 lines (`wc -l` prints 3,578 because the closing brace has no trailing newline); `verifyProof` spans L293–L3578, i.e. **3,286 lines**.

12.1 C1 — The VM is the wrong abstraction

[MAJOR] **[PARTIAL]** (*downgraded from CRITICAL — see the severity reassessment below*)

The “generated verifier” is not a verifier. It is a **bytecode interpreter** — the VM dispatch loop is ≈ 873 lines of Yul (L1692–2564): ≈ 270 lines of fixed interpreter template (L1692–1929, L2531–2564) plus ≈ 600 lines of circuit-specific native-callback Yul emitted by the generator into the 0x19/0x1f/0x1b arms (L1930–2530) — plus an opaque `q_program` blob of **4,559 bytes stored as 143 hex words in the VK contract**. The actual verification logic is **invisible in the contract**: you cannot read what the verifier checks by reading the contract. You must disassemble 4,559 bytes of bytecode against the VM ISA (31 opcodes defined, `vm/mod.rs:560--590`; only the 11 the program uses are rendered in B64) and mentally execute the VM to recover the semantics. **This is the real cost of the design, and it is large.**

The runtime-cost half of the critique is FALSE (measured) **[CORRECTED]** My original framing claimed the VM is “paid for at runtime.” Measurement refutes the significance of that claim, though the numbers first given here were wrong and are replaced:

- Decoding the VK’s `q_program` with the operand widths of the rendered arms ends exactly at byte 4,559 and yields only **101 instructions** (not 1,139): 16 MODARITH7 macro-instructions carry 4,331 B (95%) of limb-term operands (1,151 terms), the rest are 85 short ops and native-callback markers.
- Dispatch cost $\approx$ `mload` + `byte` + an **11-arm** switch per instruction (not 31-way). Even if every dispatch walked all 11 compare-and-branch steps (≈ 20 gas each), that is $\lesssim 225$ gas per instruction, i.e. $\lesssim 23\text{k}$ gas over 101 dispatches.
- Precompile gas $\approx$ **709.6k** (EIP-2537/EIP-2565 prices as in `revm-precompile` 16.2.0): 78-pair G1MSM 525,096 + six 1-pair G1MSMs (L827, 872, 3475, 3486, 3531, 3545) 72,000 + **one** 2-pair pairing call 102,900 (the accumulator equation is folded into it by a random α , L3497–3553; the other `0x0f` call is the constructor smoke test, L250) + four G1ADDs 1,500 + six modexp inversions $\approx 8,124$ (24,384 under EIP-7883, now live, for a total of $\approx 725.9\text{k}$).
- Measured `verifyProof` gas for B64 is $\approx 1.14\text{M}$ (`nightfall_4_PV/audit/h2v-fee12345/evidence/bundle/logs-rerun/p1_01_abi.log`: 1,140,607). **Dispatch is $\lesssim 2\%$ of total**; precompiles are $\approx 62\%$, not “ $40\times$ ” the rest.

So the *dispatch* tax is negligible. The interpretive cost that remains is inside MODARITH7, which decodes its 1,151 terms at run time (an `mload/byte/shr` per term); it is unmeasured for B64 but bounded well below the precompile floor. A critique that leans on “the VM is slow” is still wrong and should not be made.

The generality-unused point [CORRECTED] The `EXPECTED_VK_CODEHASH_WORD` anchor pins **one VK per verifier deployment** (§11.8.60), and the verifier also hard-codes circuit shape (e.g. `let k := 21` at L1322 and L2846; the pruned 11-arm switch). But the VM was never meant to serve many circuits: the header says “It is not a generic verifier” (L11–13). Its justification is size, because the program lives in VK *data* rather than verifier *code*. So “unused generality” is not a design smell here. What remains is the cost below: the program is opaque.

Steelman (stronger than I first credited). The VM is justified by the 24 KB EIP-170 limit: a fully specialized verifier for this 15-advice-column, degree-5 circuit with all constraints inlined would very likely not fit (B64’s runtime is already 21,253 B). Combined with the $\lesssim 2\%$ dispatch overhead, the VM is a **reasonable engineering choice on size and cost**. The failure is not the VM itself — it is that **the size win was paid for in auditability and never bought back**: the program blob is not human-auditable. A size-driven VM is fine *if* you then make the program legible — which this design does not.

Fix (refocused on the real cost: opacity, not speed).

- **Emit an annotated disassembly appendix.** The generator knows every opcode’s meaning; emit the `q_program` as a commented block (in the VK or a companion file), each instruction tagged with the circuit constraint it enforces. This directly attacks the opacity at zero runtime cost — the single highest-value C1 fix. [PARTIAL]: the host-side `quotient_identity_manifest()` API (`builder/api.rs:121`) already maps every *y*-batch position to its gate/constraint name, and the canonical quotient-certificate hash (identity order, VM bytecode, constant table) is checked on-chain against the VK header (L940); what is missing is an opcode-level listing shipped with the artifact.
- **Formally verify the interpreter** — it is the trust root and the opcode set is bounded. Note the scope: the bounded ISA is the ≈ 270 -line template core; the ≈ 600 lines of native-callback Yul are regenerated per circuit and need per-artifact checking (differential or translation validation), not a one-off ISA proof. [OPEN]

- **Differentially test the shipped program** (§12.14): the VM is data-oblivious, so one passing trace run of B64’s own program shows its whole disassembly is exercised. [\[OPEN\]](#)
- **Document the size budget** in the contract header so a reviewer understands the VM is a 24 KB necessity, not gratuitous indirection. [\[OPEN\]](#)
- **Reconsider the per-VK pinning** if multi-VK reuse was ever intended — otherwise drop the pretense of generality and document the verifier as single-VK. [\[ADDRESSED\]](#): the NatSpec already says “It is not a generic verifier ... generated for one `VerifyingKey`” (L11–13).

Severity reassessment (why MAJOR, not CRITICAL). A CRITICAL rating should mean *a flaw that, left in place, accepts invalid proofs or loses funds*. C1 is not that. The verifier is **correct** — the differential harness passes on the generator’s fixtures and B64 passes Nightfall’s own proof fixtures. The VM is **size-justified** (24 KB limit) and **cheap** ($\lesssim 2\%$ dispatch). What C1 actually is: a **high auditability risk that amplifies the genuinely critical issues**. The opaque program blob and the 3,286-line `verifyProof` do not break soundness — but they make the real soundness risks (C3 truncated challenges with an unstated level, C4 unenforced invariants) *harder for a human to spot*. A risk multiplier is serious, but it is not itself the fire. Rating it CRITICAL conflated “hard to audit” with “broken,” which is the kind of imprecision that erodes a critique’s credibility. MAJOR is the honest grade: fix it because it makes everything else auditable, not because it is on fire.

Remediation plan (prioritized, with effort and impact).

#	Action	Effort	Buys	Status
R1	Emit an annotated disassembly of <code>q_program</code> (opcode-by-opcode, each tagged with the circuit constraint) as a companion file or VK comment block	M	Kills the opacity: a reviewer reads constraints, not hex	[PARTIAL]
R2	Per-artifact differential run : trace-compare each shipped program (B64 included) against Rust in CI; a census of its <code>used_ops</code> (<code>vm/mod.rs:1217</code> ; 11 in B64) shows what it covers	S	Turns “tested some shapes” into “tested the program we ship”	[OPEN]
R3	Split <code>verifyProof</code> (3,286 lines) into named sub-functions (transcript, quotient, pairing, acc)	M	Auditable units; may <i>reduce</i> (not remove) the via-IR dependency, see C11	[OPEN]
R4	Formal model of the VM ISA (≈ 270 -line template core, bounded opcode set)	L	Proves the trust root, not just tests it	[OPEN]
R5	Document the size budget + single-VK framing in the header	S	Sets the reviewer’s mental model right	[PARTIAL]

Do R1 and R2 first — they are small, attack the real cost (opacity + unverified coverage), and together make the contract cross-checkable against the Rust verifier by human eyes. R3 is the structural fix; it helps, but does not by itself remove, the build fragility (C11). R4 is the gold-standard close for the trust root. None of these change the bytecode semantics — they change what the generator *emits*, which is the whole point: the artifact can be made legible without touching what it computes.

12.2 C2 — The trust root is differentially tested, but not formally verified

[MAJOR] [PARTIAL] (*revised downward from CRITICAL after inspecting the CI*)

Every proof this contract accepts is correct *only if the Yul interpreter (template core plus generated callbacks) is correct*. The VM is the actual trust root. **Credit where due:** the generator’s test harness, as wired in `proofs/solidity-verifier/.github/workflows/ci.yml`, is genuinely rigorous — it runs a property-based differential fuzzer (`pbt_transcript_differential_fuzzer_matches_solidity_trace`) with `HAL02_SOLIDITY_RUN_EVM_TESTS=1`, plus dedicated *native-vs-Solidity trace-equivalence* jobs for Poseidon and IVC compiled with `rust-verifier-trace,solidity-trace`. The comparison is bidirectional: every Rust trace id must appear in the Solidity trace, the Solidity trace must emit no id without a Rust oracle, and every shared id must carry equal bytes (`src/test.rs:1060--1103`). That is a high bar. *Correction:* it does *not* run on every PR. GitHub executes only workflows under the repository root’s `.github/workflows`, the nested `ci.yml` is inert in this monorepo, and the root `ci.yml` has no `solidity-verifier` job. The harness exists and passes locally, but nothing gates PRs on it.

The residual gap (why this is still a critique). Differential testing proves equivalence *on the shapes and traces exercised*, not a formal proof for all inputs. Two specific holes:

- **Program-scoped coverage, not opcode-scoped.** The generator renders only the switch arms a program uses (`quotient.rs:194--229`), so B64 contains 11 of the 31 opcodes and every other byte hits the default revert (L2561); an unrendered opcode cannot execute. The VM also has no jumps, and every branch in the dispatch body depends only on program bytes, so one honest run of a program executes *all* of its rendered arms and MODARITH7 sub-blocks. The question is therefore not “untested opcodes” but “has *this* program been differentially tested?”. The closest fixture, the IVC Keccak decider (`tests/ivc_keccak_solidity.rs, k=20`), renders a 4,559-byte program with the same 11-opcode histogram as B64 (checked against the local render in `target/ivc-keccak-solidity-dump`). It is not byte-identical, though: B64’s $k=21$, 19-instance layout, embedded-quotient mode and exact generator revision are never differentially tested, and the other fixtures (Poseidon $k=6$, RSA/SHA/hybrid-MT $k=12-13$) are far smaller.
- **Direct coverage is against a model, not the Yul.** `vm/mod.rs` itself has 2 unit tests, but `src/lowering/tests.rs` has over 20 VM tests, including per-opcode encoder tests for LIN7, BILIN7_ROW, BILIN7_PAIRWISE, POW5 and MODARITH7 (L2296–2750), validator tests (L1521–1559) and a check that every opcode has a template arm (L1445). They execute a Rust model interpreter (`eval_quotient_vm_for_test, L3093`), so they pin the encoder, not the Yul arms; the whole-program test interpreter (`eval_quotient_program_artifacts_for_test, L3005`) panics on dynamic opcodes such as MODARITH7 (L3050), so it cannot evaluate the production program. There is no EVM-level per-arm test.

Fix.

- **Keep the differential harness and wire it into the root CI** — it is the right design, and today nothing gates on it.
- **Differentially test every shipped program:** render a trace build of each production circuit (B64 included) and run the bidirectional trace comparison. Because the VM is data-oblivious, one passing run covers every rendered arm and sub-block. A render-time census (`used_ops, vm/mod.rs:1217`, plus MODARITH7 flag/block usage) makes it cheap to show which fixture covers which artifact. This turns “we tested some shapes” into “we tested the program we ship.”

- **Run the existing Rust-model opcode tests against the Yul arms** (same bytecode, executed in revm), so a Yul-arm bug is caught directly rather than only through a fixture.
- For the residual gap, a **formal model of the VM ISA** (the opcode set is small enough) would close it definitively.

12.3 C3 — Truncated challenges: the soundness level is unstated

[MAJOR] [OPEN] (arithmetic **[CORRECTED]**: the bit-level figures of earlier drafts are withdrawn)

The `truncated-challenges` feature masks `x3` to 128 bits after squeeze (B64 L1270; Rust `poly/kzg/mod.rs:457`) and the powers of `x1/x4` to 128 bits at use (`mod.rs:394, 502`); `x`, `y` and `x2` stay full width. The verifier is faithful to the Rust: the same truncation happens on both sides. The soundness level this leaves is not established here. The library’s own doc comment (`proofs/src/utils/arithmetic.rs:242--245`) warns that 128 bits “may not be enough entropy depending on the application” and cites a collision attack with 2^{64} memory and $\sim 2^{64}$ operations. Earlier drafts of this chapter derived a specific bit level from a Schwartz–Zippel bound; that derivation is withdrawn, and no replacement figure is given.

The real problem: the silence. The critique is about *governance*. The contract ships with **no security-level statement**. The only trace in the artifact is a code comment at L1261–1269 saying the truncation “mirrors midnight-proofs”. The header `NatSpec`, the generator docs and `docs/audit/CODEGEN_ASSURANCE_DOSSIER.md` state no security level. Truncation also buys no EVM gas: EIP-2537 prices an MSM independently of scalar width. It exists for in-circuit verification and is a global crate feature. A tradeoff that is **invisible to the consumer** is indistinguishable from an accident.

Fix.

- **Write down the soundness argument** for the truncated multi-open challenges, and state the resulting level in the contract header.
- **Record a security-team sign-off** that the level is acceptable for this deployment’s threat model. If it is not, the fix is full-width challenges, which cost nothing on-chain for the outermost wrap proof but need per-proof configuration in `midnight-proofs`.
- **Surface the truncation in the provenance header** (C8) so a deployment audit can enumerate which verifiers are truncated.

12.4 C4 — Load-bearing invariants are unenforced

[MAJOR] [PARTIAL] (downgraded from *CRITICAL* — the invariant holds today; the gap is that nothing keeps it holding)

The contract’s safety rests on two invariants:

- *Every G1ADD operand is a G1MSM output, a VK constant, or a proof point that an earlier G1MSM on the same *success* chain has already validated (§11.8.57)* — the thing that makes the subgroup-cancellation attack unreachable. (The stricter wording “MSM output / VK constant” is literally false for L3550: its `PAIRING_LHS` operand is π copied raw at L3471; π is safe because `G1MSM( $x_3, \pi$ )` at L3486 has already subgroup-checked it.) **[OPEN]**: nothing asserts this.
- *Every transcript-absorbed point flows to a checking precompile* — the thing that makes the deferred transcript check sound. **[ADDRESSED]** at codegen time: `validate_generator_`

`invariants` (`plan.rs:238`, run at `plan.rs:164`) calls `kzg::validate_absorbed_g1_precompile_coverage` (`kzg/mod.rs:602`), which fails the render if an absorbed proof G_1 is missing from the precompile-input set. Caveat: it checks the generator’s *model* of those inputs, not the emitted Yul (the model credits π to “final pairing input”, `kzg/mod.rs:560`, although with the accumulator batch π reaches the pairing only through the L3550 G1ADD; its actual subgroup gate is the L3486 G1MSM).

Why this is MAJOR, not CRITICAL (measured). These invariants **hold today** — the G1ADD cancellation attack was traced and shown *unreachable* (§11.8.57): every G1ADD operand is a point already in G_1 (a G1MSM output, or π after its G1MSM check), so no operand carries a cofactor component. There is **no current vulnerability**. A CRITICAL rating requires a live flaw; this is a *defense-in-depth gap* — the safety is real but unprotected. The honest grade is MAJOR.

The real risk: silent future breakage. The danger is not today, it is the **next codegen refactor**. A future change that emits a G1ADD with a raw proof point that no earlier G1MSM has validated turns a mathematically-possible attack into a live one, and **no test or assertion catches it** because the G1ADD half of the invariant was never written down as a check. The safety is a property of the current emission, not an enforced contract on future emission. That is fragile, and fragility in a soundness boundary is worth fixing now.

Fix. Codegen-time assertions that fail the build. The absorbed-point half already exists (above); add the G1ADD half: the generator should statically verify, before rendering, that every G1ADD operand is an MSM output/constant or a point validated by an earlier MSM on the same success chain, and derive both checks from the emitted precompile calls rather than a parallel model. An invariant that isn’t machine-checked is a comment, not a guarantee. This is cheap (a static pass over the emitted precompile graph) and converts “it happens to be safe” into “it cannot be emitted unsafe.”

12.5 C5 — The contract defeats stock automated tools

[MAJOR] [PARTIAL]

Stock slither 0.11.6 *crashes* in the IR→SSA pass (`RecursionError` in `add_phi_origins`); `aderyn` false-positives on assembly patterns (its own L-3 note admits it cannot see assembly references); `semgrep` is blind. A contract that **stock linters cannot analyze** is a contract that can only be audited by expensive manual review — which does not scale and is error-prone. “Semgrep clean” here is not assurance; it is the absence of a capable observer. The assembly-VM choice directly causes this.

Correction (measured). The crash is not caused by switch nesting. SSA construction recurses once per dominator-tree level, and a straight-line Yul body is a dominator *chain*; `verifyProof` has 2,816 CFG nodes. With three small patches (iterative dominator walks, a deep-recursion worker thread, Yul `leave` CFG edges), `slither` analyses B64 completely (`nightfall_4_PV/audit/slither-2026-09-29, runs/generated_B64_final.txt`). Its `unused-state` detector then resolves the assembly references correctly and flags exactly the 5 pointer constants that really are unreferenced (`CHALLENGE_MPTR`, `Q_COM_MPTR`, `G1_IDENTITY_MPTR`, `QUOTIENT_RETURN_MPTR`, `TRACE_U256_MPTR`); it also reports a dead store (L2003–2004), and the audit’s Slither-API pass confirms that all 18 `staticcalls` check `returndatasize`. So `aderyn`’s 76 L-3 hits are 71 false positives and 5 true ones, not all false. The residual critique stands: the tools see Yul arithmetic but nothing of the proof system, so “tool-clean” is still not assurance.

Fix.

- Upstream the three slither patches, and shorten the dominator chain at the source: splitting `verifyProof` into functions (R3 / §13.4) bounds each CFG. Splitting only the VM `switch` would not help.
- Ship a **Solidity-level reference implementation** alongside the assembly, used for tooling and differential testing against the assembly hot loop.
- Do not let “tools can’t run” become the accepted state — a contract that resists all tooling is a contract that resists maintenance.

12.6 C6 — Compiler-internal names leak into the audited artifact

[MAJOR] [OPEN]

The contract carries **163 distinct varN names** (980 uses; 442 `let varN` declarations, because numbering restarts per emitted block and the same name denotes different values in different blocks) and **31 f_N/a_N names** — generator-internal identifiers emitted into the file a human must read. (The `f_N/a_N` names are the one place the generator already passes a semantic hint: fixed/advice column *N*, `yul_emit.rs:1142`.) The generator *knows* the semantic role of every variable (it assigned it) and still emits `var47`. This is the generator being lazy about the audit surface: it optimizes its own emission simplicity over the reviewer’s comprehension.

Fix. Name variables semantically in the emitted code — the generator has the mapping (for gate subexpressions, the gate and constraint names of the quotient identity manifest). `var47` $\rightarrow$ `perm_z_numerator_term`. This is a pure codegen change with zero runtime cost and a large readability win (§13.3). The same applies to the anonymous VM scratch addresses (§11.8.59, §13.2).

12.7 C7 — The pairing helper’s parameter names invert the slot names

[MINOR] [CORRECTED]

`ec_pairing(success, PAIRING_RHS, PAIRING_LHS)` (L3569) passes the slots in the opposite order to the helper’s parameter names `lhs_mptr, rhs_mptr` (L534). The original critique called the slot names “backwards” and the rename “never made”. That is wrong on both counts. The slot names follow the Rust DualMSM: `left =  $\pi$ , right = final_com - vG + x3 $\pi$` (`poly/kzg/mod.rs:554--558`), and the differential trace pins them as ids 27/28 “pairing_lhs”/“pairing_rhs”. Rust checks $e(\text{left}, [s]_2) e(\text{right}, -[1]_2) = 1$ (`msm.rs:284--300`); Solidity checks the equivalent $e(\text{right}, [1]_2) e(\text{left}, -[s]_2) = 1$. The swap is explained at the call site in a 12-line comment (L3555–3567, template `FinalPairing.yul:65--79`). A wrong argument order would reject every honest proof, which every positive test catches, so it is not a latent soundness hazard. Renaming the slots would break the name-level correspondence with Rust and the trace.

Fix. Rename the *helper’s parameters*, not the slots: `ec_pairing(success, g2_one_side_mptr, neg_s_side_mptr)` (`AssemblyHelpers.yul:213`) removes the trap without breaking the correspondence. If semantic slot names are wanted, use `KZG_PI` / `KZG_COMBINED_COMMITMENT`. The earlier suggestion `KZG_EVAL_POINT` was wrong: π is the opening witness commitment, not an evaluation point.

12.8 C8 — No provenance metadata

[MAJOR] [OPEN]

The contract records **no generator version, no Rust commit hash, no generation timestamp**. It does carry content-derived identifiers: the VK’s `vk_digest` (the transcript representation of the constraint system), `EXPECTED_VK_CODEHASH_WORD` (L46), and, in the B64 lineage, `EXPECTED_QUOTIENT_CERTIFICATE_HASH_WORD` (L51). These identify *which circuit/VK* a verifier accepts, but not *which generator build* produced it. For a generated artifact this is unmaintainable: you cannot tell whether a given deployed contract was produced by the current generator or a stale one. When a bug is found in the generator, you have no way to enumerate the affected deployments. The project’s own `docs/audit/CODEGEN_ASSURANCE_DOSSIER.md` already lists the record a production claim needs (Midfall revision, toolchain, Cargo features, solc version and flags); the artifact just does not carry it.

Fix. Embed a provenance header as constants plus a comment: `GENERATOR_VERSION`, `RUST_COMMIT`, `CARGO_FEATURES` (e.g. `truncated-challenges`), `GENERATED_AT`, and name the existing `vk_digest/codehash` words as the circuit id. Unreferenced constants do not change the runtime bytecode, but any source edit changes the CBOR metadata hash unless metadata is omitted (the generator’s builds pass `--no-cbor-metadata`, `src/evm.rs:209`); keep volatile fields such as the timestamp in a sidecar manifest. Make the generator stamp every artifact. This is table stakes for generated code.

12.9 C9 — Magic constants hand-synced with no assertion

[MINOR] [ADDRESSED]

Original claim: `EXPECTED_VK_PAYLOAD_LENGTH = 17056`, the permutation δ , `q_perm_chunk_len` must be kept in sync with the VK by the generator, but nothing in the contract asserts they match the loaded VK.

Re-verified: the length *is* asserted. `EXPECTED_VK_LENGTH = payload + 1` (the `INVALID` prefix) is checked together with the full-runtime codehash at construction (L266–267) and again on every proof (L921–922, template `VkLoading.yul:67--70`); the codehash covers every payload byte. After the copy, the VK header words `NUM_INSTANCES`, `K`, `HAS_ACCUMULATOR`, `ACC_OFFSET`, `NUM_ACC_LIMBS` and `NUM_ACC_LIMB_BITS` are cross-checked against the generator’s constants (L934–939, `VkLoading.yul:82--88`, commit `e64f2891`); the B64 lineage also checks the quotient-certificate hash (L940). δ is not VK data: it is the field constant `Fq::DELTA` (`quotient.rs:1379`). `q_perm_chunk_len` is shape metadata from the same constraint system that the pinned VK digests.

Fix (residual). None needed for soundness. Optionally derive `q_perm_chunk_len` from a VK header word and cross-check it like the others.

12.10 C10 — The negated-numerator sign convention is a footgun

[MINOR] [CORRECTED]

The verifier stores $-\nu_y(x)$ at `QUOTIENT_EVAL_MPTR` (L2823–2824), not at `QUOTIENT_MPTR`, which in this path holds `x_split` and `one_minus_x_n` (L2865–2866). The trace re-negates it for id 36 (`TraceReturn.yul:37`). The original critique said a dropped or doubled negation would “silently” accept wrong proofs or reject right ones. It would not be silent. The negation is not a Solidity trick: it is Rust’s `expected_eval`, into which `compute_linearization_commitment` subtracts the fully evaluated identities. The value is pinned by the differential trace (id 36

against `verifier.rs:508--524`), and every positive end-to-end test fails if the sign flips. The convention is documented three times (L120–122, L1460–1461, L2820–2822). What remains is a naming problem: the constant’s own comment calls `QUOTIENT_EVAL_MPTR` a “legacy name”.

Fix. Rename the slot after what it holds (`LINEARIZATION_EXPECTED_EVAL_MPTR`) and drop the legacy `QUOTIENT_MPTR` name in the fused path. Keep the negation, since it matches the Rust oracle term for term.

12.11 C11 — Build fragility: `via-ir` is load-bearing

[MAJOR] [PARTIAL] (*upgraded from MINOR: see the spill-area paragraph*)

The contract **will not compile with the default codegen** (re-verified with solc 0.8.30 without `--via-ir`: “Stack too deep ... Variable `src` is 1 slot(s) too deep”). A compiler flag is load-bearing for the build. If CI misconfigures, the contract fails to build — which is fail-closed (good) — but it signals the code is operating at the edge of the compiler’s limits.

Already pinned [ADDRESSED]. The generator compiles only with `--via-ir` (`src/evm.rs:206`); the flag set is recorded in `docs/audit/REVIEW_PACKET.md:71` and the assurance dossier; Nightfall’s `foundry.toml` (L31–50) pins `via_ir` per file for the B8/B64 verifiers through `compilation_restrictions`, citing the stack-too-deep error.

The spill area (new, measured). Under `--via-ir` (solc 0.8.30 and 0.8.35, 200 runs) the deployed code starts with `PUSH2 0x08e0 PUSH1 0x40 MSTORE`: solc’s stack-limit evader has reserved the 67-word area `[0x80, 0x8e0)` for Yul locals it moved to memory (for example `y` at `0x300` and the `MODARITH7` pairwise inner-loop counter at `0x80`). The generated code writes the same range as `TRANSCRIPT_MPTR = RETURN_MPTR = 0x80`, the accumulator batch at `0x100–0x320`, and the `ec_pairing` scratch at `0x300–0x600`. So the assembly (“memory-safe”) promise, which the generator makes precisely so `via-IR` can spill (`tests.rs:2127`), is formally broken. It works today only because the spilled locals are live in the quotient phase alone, where no generated write goes below `0x8e0`. That is consistent with every fixture passing, but nothing checks it, and any Yul restructuring (e.g. R3) or solc upgrade reshuffles the spill set.

Fix. Keep `via-ir`; removing it is not realistic. The first legacy error is in the 6-parameter helper `load_acc_coord_shifted` (L573), not in the VM dispatch, and the gas tuning assumes `via-IR`. Instead, close the spill hazard. Either start generated absolute scratch above a fixed margin and add a CI gate that parses the deployed `memoryguard` and fails if it exceeds that base, or make the planner reserve `[0x80, memoryguard)` from a first compilation pass.

12.12 C12 — “Generated” hides that the real artifact is the compiler

[MAJOR] [PARTIAL]

The framing “we generate the verifier” obscures where the complexity actually lives. The contract is small and opaque; the **semantic content is in the generator** — the shape recognizer, the fusion decision, the run compactor, the offline validator (§11.8.53 onward). Auditing the contract does not audit the system; **auditing the generator does**. The VM’s auditability win is an illusion: the complexity was not removed, it was **relocated to a harder-to-audit place**. (The earlier claim that the generator gets the *least* review discipline was too strong. It has ≈ 187 `#[test]` functions, a pre-render invariant pass (`plan.rs:238`), proptest-based property tests, a pinned runtime hash for the IVC fixture (`scripts/check_release_bytecode_sizes.sh`), and

docs/audit/CODEGEN_ASSURANCE_DOSSIER.md. What it lacks is a CI that runs any of this on PRs (§12.2) and the items below.)

Fix. Apply the same review rigor to the generator that you would to a hand-written verifier. Property-based tests over the lowering passes exist in part; still missing are a golden-program corpus with expected disassembly (one pinned hash per shipped artifact, not only the IVC fixture) and a formal model of the VM ISA. Treat the generator as the product and the contract as its output artifact.

12.13 What the design actually gets right

To be fair — and a critique that only lists faults is not credible — the design has genuine strengths that should be preserved:

- **The differential trace-equivalence harness** is the standout. The crate’s own workflow file defines a PBT differential fuzzer and dedicated native-vs-Solidity trace-equivalence jobs (Poseidon, IVC) with bidirectional id-and-value matching. This is more assurance than most verifier projects ship, and it is the reason the “correct” verdict in this review is trustworthy — once it actually gates PRs (it is not wired into the monorepo’s root CI; §12.2).
- **The fail-closed dispatch chain** (§11.8.49) is exemplary: unknown opcodes, unknown sub-cases, and non-terminal VM states all revert. This is the right default and it is consistently applied.
- **The double codehash anchor** (§11.8.60) — checked at both the Solidity and Yul levels — is a clean, defense-in-depth trust binding.
- **The offline bytecode validator** (§11.8.53) — a static analyzer that checks stack effects before rendering — is a genuinely good idea that most codegen projects lack. The same holds for the pre-render invariant pass (`plan.rs:238`: memory non-overlap, absorbed-G1 precompile coverage, PCS/quotient drift), which fails the build rather than emitting.
- **The constant interning and run compaction** are real engineering, not hand-waving.

These show the team *can* do rigorous work. The critique above is that the rigor is applied unevenly: the generator’s internal machinery and its differential harness are strong, while the artifact it produces is left opaque, the trust root is differentially tested but not formally verified, and the produced contract resists stock tooling.

12.14 Measured auditability scorecard

To keep the critique honest, the readability findings are quantified against the B64 artifact (3,579 lines). Numbers, not vibes. Counts are over code with `//` comments stripped. The memory-operand metric is the first argument of `mload/mstore/mcopy` (914 such ops) when that argument is a bare constant: 492 raw `0x...` literals (246 distinct addresses) against 64 named `*_MPTR/*_CPTR`. Counting `add(BASE, off)` forms too gives 514 raw against 176 named. The same metric is used in §13.1.

Metric	Value	Reading
Comment : code lines	47.2%	Good — 1,109 comment vs 2,350 code lines (120 blank)
Named constants	79	Good — 68 pointer constants (*_MPTR/*_CPTR/*_MPTR_BASE), 63 referenced, 5 dead
varN SSA names	163 distinct / 980 uses	Bad — generator names leak (C6)
Raw-hex literals	1,409 (573 distinct)	Bad — 0.60 per code line
Memory-operand naming	11.5% named / 88.5% raw hex	Bad — 64 named vs 492 raw (246 distinct) (I9)
Memory ops	914 (480 / 336 / 98)	mload / mstore / mcopy; scratch traffic is the bulk of the code
verifyProof length	3,286 lines	Bad — god-function (C1)
Nested Yul functions	17	helpers are named (mitigates C6)
Branch points	2 switch / 15 case / 73 if	VM dispatch + guards

What the numbers say. The contract is *heavily commented* (47% — genuinely helpful) and the *high-level* memory map is named (68 pointer constants). But the *working* code is dominated by anonymous scratch: **88.5% of bare-constant memory operands are raw hex**, 163 varN names carry the dataflow, and the whole verifier is one **3,286-line verifyProof**. The comments describe the code; they do not substitute for naming it. A reviewer reads 1,109 lines of prose and then still has to decode 492 raw-hex memory operands (246 distinct addresses: 183 in the final-MSM/quotient scratch from 0xa320, 60 in the decoded-eval buffer from 0x8520, 3 low) and 980 varN references by hand.

The confirmed C2 coverage gap (measured, corrected). The VM ISA defines **31 opcodes** (Q_OP_PUSH_CONST ...Q_OP_AFFINE_SUM, vm/mod.rs:560--590), but a render contains only the arms its program uses. B64 has 11 (0x05, 0x06, 0x08, 0x0b, 0x0d, 0x10, 0x11, 0x19, 0x1b, 0x1f, 0x21); BILIN7_PAIRWISE, POW5 and the other 18 are not rendered and would hit the default revert (L2561), so they cannot be “untested code in production”. B64’s MODARITH7 instructions use the mem/product (“escape-hatch”) blocks heavily and never the pairwise block. The differential test’s required-coverage check (src/test.rs:1697--1743) asserts *semantic verifier stages* — challenge ids 7–11 and 13–16, quotient numerator (36), f_eval (31), final MSM (33), pairing lhs/rhs (27/28), final pairing (35), and trace ranges 30k–42k / 60k–61k — and compares every per-identity value, but it records **no arm or sub-block coverage**. Since the VM is data-oblivious, that is enough for any program that is tested at all. The closest fixture, the IVC Keccak decider, renders the same 11-opcode histogram as B64, so B64’s arms are exercised *in shape*; B64’s exact program, $k=21$ layout and embedded-quotient mode are not. None of it is gated by the monorepo’s root CI. This is the precise, measurable form of C2’s residual: *a tested shape $\neq$ the tested shipped program*.

Composite verdict. Readability is **Fair**: strong comments and a named high-level map, undermined by anonymous scratch (88.5% raw-hex bare memory operands), leaked generator names, a 3,286-line god-function, and a differential test that checks stages but not rendered arms. Every one of these is a codegen-emission choice, not a runtime necessity — the same bytecode could be emitted with named slots, semantic variable names, split functions, and an arm-coverage assertion (split functions would need the spill-area re-check of C11). The gap between “correct” and “auditable” is entirely a matter of what the generator chooses to emit.

12.15 Audit runbook: how to actually audit this contract

The critique says the contract is hard to audit. The constructive capstone is a concrete procedure that makes it auditable *today*, working around the tooling gaps with the assurance the project already provides. A reviewer with a day and this list can reach a sound verdict.

1. **Build integrity.** Compile with `solc --via-ir --optimize` (the contract will not build without `via-ir` — stack-too-deep). Confirm the runtime is <24KB (measured 21,253B with CBOR metadata, 21,200B without, solc 0.8.30, 200 runs). The pragma is *not* pinned (`~0.8.24`), so record the exact compiler (generator: 0.8.30, `src/evm.rs:78`; Nightfall: 0.8.35, `foundry.toml`). Read the deployed prefix `PUSH2 g PUSH1 0x40 MSTORE` and confirm no spilled local is live across generated writes below `g` (C11; `g = 0x8e0` today). *Evidence:* `solc` output, `--bin-runtime` size and prefix.
2. **VK binding (the trust anchor).** Verify `EXPECTED_VK_CODEHASH_WORD` matches the deployed VK's `extcodehash` and length, at *both* the Solidity level (L266–267) and the Yul level (L921–922), and that the header cross-checks (L934–940) match the VK words. A mismatch here breaks everything downstream. *Evidence:* `getCode(vk)` vs the constant.
3. **Read the named skeleton.** Walk the 68 named pointer constants (63 referenced; VK header, challenges `THETA/BETA/GAMMA/Y/X`, pairing inputs). This is the readable high-level map — understand the data flow here before touching the VM.
4. **Transcript schedule cross-check.** Compare the Fiat–Shamir absorb order against the Rust verifier's schedule (§3.9 / §7.6). The order is soundness-critical; a reordering is silent. *Evidence:* the differential trace (step 7) also covers this.
5. **Fail-closed dispatch.** Confirm the three-layer revert chain: main opcode switch default (L2561), heavy-gate `u16` sub-dispatch default (L2528), and the terminal guards (`q_pc==q_end` and `!q_has_top`, L2569–2570). Check that the 11 rendered arms match the decoded program's opcode census. *Evidence:* §11.8.49.
6. **G1ADD operand provenance.** For each of the 4 runtime G1ADDs (L3480, L3490, L3536, L3550), confirm each operand is a G1MSM output, a VK constant, or a proof point already validated by an earlier G1MSM on the same `success` chain (L3550's `PAIRING_LHS` is raw π , validated by the L3486 G1MSM; §11.8.57, §12.4). This is the one manual check that must not be skipped — it is what makes the subgroup-cancellation attack unreachable.
7. **Run the differential harness.** Execute the native-vs-Solidity trace-equivalence jobs (Poseidon + IVC) with `rust-verifier-trace`, `solidity-trace` and `HAL02_SOLIDITY_RUN_EVM_TESTS=1`. This is the strongest single assurance — bidirectional trace matching against the Rust oracle. *Evidence:* the job definitions in `proofs/solidity-verifier/.github/workflows/ci.yml`, which GitHub does not execute in this monorepo, so run them yourself. They validate the *generator* on its fixtures; for B64 itself, render a trace build of the fee circuit and run the same comparison, since nothing else does.
8. **VM scratch legend.** Use the named-constant legend (§11.8.59) or the R1 disassembly to map the ≈ 45 distinct raw-hex addresses in the VM body (L1692–2564) to witness/permutation columns. Without this, the VM body is unreadable.
9. **Tooling triage.** Run `aderyn/semgrep` and clear the *known* false positives: shift-order (H-1), return-in-Yul (H-2), unused-state-var (L-3, except the 5 genuinely dead constants of C5). Run `slither` with the three patches of `nightfall_4_PV/audit/slither-2026-09-29`. Do not treat “semgrep clean” as assurance — it cannot see the assembly (§11.8.58).
10. **Security level.** Confirm the truncation (§12.3) has a written soundness argument, is labeled, and is signed off for the deployment's threat model.

Why this runbook works. It substitutes the project’s *own* strong assurance (the differential harness, the codehash anchor, the fail-closed chain) for the tooling that cannot reach the assembly. The two checks that genuinely require human attention are #2 (the trust anchor) and #6 (the G1ADD provenance) — everything else is either mechanical or covered by the harness. That is a tractable audit surface for a contract that stock linters cannot analyze.

12.16 Critique summary

Critique	Sev	Status	Concrete fix
C1 VM is the wrong abstraction	MAJOR	[PARTIAL]	Annotated disassembly (R1) + arm-coverage assertion (R2); split <code>verifyProof</code> (R3). Runtime-cost claim corrected (101 instr., dispatch $\lesssim 2\%$)
C2 Trust root diff-tested, not formally verified	MAJOR	[PARTIAL]	Wire the harness into root CI; differentially test each shipped program; run the Rust-model opcode tests against the Yul arms
C3 Truncated challenges, level unstated	MAJOR	[OPEN]	Write the soundness argument; state the level in the header + security sign-off
C4 Invariants unenforced (hold today)	MAJOR	[PARTIAL]	Absorbed-point check exists (<code>plan.rs:245</code>); add the G1ADD-provenance check
C5 Defeats stock tooling	MAJOR	[PARTIAL]	Upstream the slither patches; split <code>verifyProof</code> (not the switch); Solidity reference impl
C6 Compiler names leak	MAJOR	[OPEN]	Semantic variable naming in emitted code
C7 Pairing helper parameter names	MINOR	[CORRECTED]	Slot names follow Rust <code>DualMSM</code> ; rename the helper’s parameters only
C8 No provenance	MAJOR	[OPEN]	Stamp generator version, commit, features; sidecar manifest per the dossier
C9 Magic constants unasserted	MINOR	[ADDRESSED]	Length + codehash checked (L266, L921); header words cross-checked (L934–940)
C10 Sign-convention footgun	MINOR	[CORRECTED]	Mirrors Rust <code>expected_eval</code> , pinned by trace id 36; rename <code>QUOTIENT_EVAL_MPTR</code>
C11 via-ir load-bearing	MAJOR	[PARTIAL]	via-IR already pinned; gate the <code>memoryguard</code> spill area against generated scratch
C12 “Generated” hides the real artifact	MAJOR	[PARTIAL]	Golden per-artifact corpus + formal ISA model on top of the existing tests

Bottom line. After measurement, **no critique survived as CRITICAL** — and that is the most important finding of this chapter. The verifier is **correct** (the generator’s differential harness passes against the Rust oracle, though no CI gates on it; B64 passes Nightfall’s own fixtures), and its safety invariants **hold** (the G1ADD cancellation is unreachable). The design’s problems are all **MAJOR auditability and governance concerns**, not soundness breaks: an opaque program blob, a 3,286-line god-function, leaked generator names, an unlabeled security level, a G1ADD invariant that holds by emission rather than by assertion (the absorbed-point half is now asserted), and a via-IR spill area that overlaps generated scratch with nothing checking the live ranges. None of these lose funds today. All of them make the next bug easier to hide and the security posture harder to verify.

This is a favorable result delivered harshly: the team built a *correct* verifier, and the fail-closed chain, the offline validator, and the differential harness prove they *can* do rigorous work. The gap is one of **priority and communication** — the rigor went into the generator’s internals but not into making the produced artifact legible, the security level loud, or the safety invariants

enforced. Almost every fix in this chapter changes only what the generator *emits*, never what it computes (the C11 spill fix may move scratch addresses). Fix the priority, and the design becomes as auditable as it is correct.

Chapter 13

Making the generated contract human-readable: a concrete plan

Chapters 11 and 12 established that the artifact is *correct* but hard for a human to audit. This chapter turns that diagnosis into a concrete, prioritized change plan. Every fix below is anchored to a specific line in the generator (proofs/solidity-verifier/src/, main HEAD 6b3d2096) and shows the *before* (what the renderer emits today) and the *after* (what it should emit). Each fix carries a status tag as in Chapter 12 ([**ADDRESSED**] / [**PARTIAL**] / [**OPEN**] / [**CORRECTED**]); *Ln* refers to Halo2VerifierFee12345B64.sol. The central observation that makes this tractable:

The generator already knows the semantic name of almost every address and value it emits — it just throws the name away at the emit boundary.

Where the names are, and where the hex comes from [**CORRECTED**]. The names exist in the data model: `MemoryRegion.name` (layout/memory.rs:179) and the `fixed_low_memory` table (memory.rs:943) are used only for validation error strings, while `column_eval_var` (yul_emit.rs:1142) is *already* used at emit and produces the `f_N/a_N/i_N` names. Contrary to an earlier draft, `u256_string` (yul_emit.rs:1133) renders *field constants*, not addresses. Addresses reach the Yul through the `Ptr/Value Display impl` (encoding/mod.rs:634--648) and through `format!("{:#x}")` sites (64 in `kzg/mod.rs`, 31 in `quotient.rs`). Value already has a symbolic variant, `Value::Identifier(name, off)` (encoding/mod.rs:590), which renders as `add(name, off)`; that is how `add(X1_POWER5_MPTR, 0x80)` appears in the output. Recovering readability is therefore mostly a matter of *propagating names that already exist* to the emit site, not inventing new information.

13.1 The measured readability debt

Before proposing fixes, the debt in the production Halo2VerifierFee12345B64.sol (3,579 lines; `wc -l` prints 3,578). Address metrics use the definition of §12.14: the first argument of `mload/mstore/mcopy`, comments stripped.

Metric	Count	Verdict
Raw-hex address operands (distinct addresses)	492 (246)	unreadable
Named pointer operands, bare / as <code>add(NAME, off)</code>	64 / 112	good, but minority
Declared pointer constants (referenced)	68 (63)	5 are dead
Opaque <code>varN</code> local names (uses)	163 (980)	unreadable
Lines in <code>verifyProof</code> (L293–L3578)	3,286	unreviewable as one unit
Region names defined in <code>layout/memory.rs</code>	30	<i>one per buffer, unused at emit</i>

The ratio is stark: for every bare named address operand the reader gets, they get ~ 7.7 raw hex ones (~ 2.9 if `add(NAME, off)` forms count as named). The raw operands cluster in two buffers: 327 operands (183 addresses) in the final-MSM/quotient scratch from `0xa320`, and 160 (60) in the decoded-eval buffer from `REVERSED_EVALS_MPTR = 0x8520`; only 5 (3) are low scratch. The fix is to invert that ratio.

13.2 Fix 1 — Named memory addresses (highest impact)

[OPEN] (prototyped on the unmerged branch `41b78eb9`, 2026-08-20, for the final-MSM block only; not in *main* or *B64*)

The problem. A typical emitted block (L3455–3458):

```
mcopy(0xd2a0, 0x9c60, 0x80)
mstore(0xd320, mulmod(mload(add(X1_POWERS_MPTR, 0x80)), x4_pow_4, r))
mcopy(0xd340, F_COM_MPTR, 0x80)
mstore(0xd3c0, x4_pow_5)
```

The reader can see `X1_POWERS_MPTR` and `F_COM_MPTR` are named, but `0xd2a0`, `0x9c60`, `0xd320`, `0xd340`, `0xd3c0` are opaque. They are pairs 76 and 77 of the 78-pair final MSM staged at `0xa320` (point at `0xa320 + 76 · 0xa0`, scalar `+0x80`), and `0x9c60` is `PERM_Z_COMMS_MPTR_BASE + 0x200`, the fifth permutation-product commitment — but nothing says so.

What is half-built, and what is not. `MemoryRegion` (`memory.rs:178`) carries a `name: &'static str` field, and `MemoryMap` holds a `Vec<MemoryRegion>` (L217–218). Its doc comment scopes the name to errors: “*Stable human-readable region name used in validation errors*”. An earlier draft proposed a reverse lookup `name_for_addr(addr)` matching `r.start == addr`. That does not work, for three reasons:

- Regions are one per *buffer* (`"pcs_final_msm"`, `"decoded_evals"`, ...), so an exact-start match names only the base. Every interior slot above — all five addresses — still falls back to hex. No per-slot `MSM_IN_i` regions exist.
- Region names are lowercase planner labels, not declared Solidity identifiers, so emitting one would not compile without a matching constant.
- Addresses are phase-shared (`0x80` is `TRANSCRIPT_MPTR`, `RETURN_MPTR` and `QUOTIENT_RETURN_MPTR`; `0xa1e0` is `SELECTOR_ACC_MPTR` and `BATCH_INV_SCRATCH_MPTR`; `0xa320` is both the final-MSM base and a quotient accumulator), so an address alone cannot pick the right name. The *emit site* knows the role; the reverse map does not.

Concrete change. Name the pointer where it is created, using the symbolic pointer the encoder already supports, and declare the base once:

```
// kzg/mod.rs (sketch): stage final-MSM pairs relative to a named base.
// Value::Identifier(name, off) already renders as add(name, off)
// (encoding/mod.rs:634); Constants.sol declares PCS_FINAL_MSM_MPTR.
fn final_msm_slot(pair: usize, scalar: bool) -> Value {
    let off = pair * 0xa0 + if scalar { 0x80 } else { 0 };
    Value::Identifier("PCS_FINAL_MSM_MPTR", off as isize)
}
```

Do the same for the decoded-eval reads (`REVERSED_EVALS_MPTR + k · 0x20`). A containment lookup (`r.start <= addr && addr < r.start + r.len`) is still useful, but only for the generated legend of Fix 5, where ambiguity can be shown rather than resolved.

After. The same block becomes:

```
mcopy(add(PCS_FINAL_MSM_MPTR, 0x2f80), add(PERM_Z_COMMS_MPTR_BASE, 0x0200), 0x80) // pair 76: perm_z[4]
mstore(add(PCS_FINAL_MSM_MPTR, 0x3000), mulmod(mload(add(X1_POWER5_MPTR, 0x80)), x4_pow_4, r))
mcopy(add(PCS_FINAL_MSM_MPTR, 0x3020), F_COM_MPTR, 0x80) // pair 77: f_com
mstore(add(PCS_FINAL_MSM_MPTR, 0x30a0), x4_pow_5)
```

Now the reader sees “this is the final-MSM staging buffer, pair *i*” without a memory-map lookup. Converting every emitter that addresses the `0xa320` scratch (final-MSM staging in `kzg/mod.rs`, quotient-VM state in `quotient.rs`) and the decoded-eval reads removes 487 of the 492 raw-hex operands; the remaining handful are low scratch.

Cost / risk. Low. The emitted *semantics* are unchanged (a `constant` is substituted at compile time and the via-IR optimizer folds `add(const, const)`). Byte-identical bytecode is expected but should be *checked*, not asserted: the crate’s release check already pins the IVC verifier’s runtime keccak (`scripts/check_release_bytecode_sizes.sh`), which turns “only the source text changed” into a test, and the trace-equivalence harness (§12.2) must still pass unchanged. Neither is gated by the monorepo’s root CI today, so run them explicitly.

Evidence the mechanism already works. The contract already declares 78 `uint256` internal constant names (79 with the `bytes32` codehash; e.g. `PROOF_CPTR = 0x64`, `INSTANCE_CPTR = 0x1ee4`, `NUM_INSTANCE_CPTR = 0x1ec4`) that the Yul body references by name and that compile to the same literals. An earlier draft said the MSM scratch base `0xa320` was the *only* kind of address still emitted raw; that is wrong — the decoded-eval buffer accounts for a third of the raw operands (above). Extending the existing constant mechanism to the scratch regions is not a new capability — it is using one the generator already has. A named `PCS_FINAL_MSM_MPTR` resolves to `0xa320` at compile time, so the argument that bytecode is unchanged is a compile-time substitution argument, confirmed by the pinned hash, not an empirical one.

13.3 Fix 2 — Semantic local names over `varN`

[OPEN]

The problem. 163 distinct local names `var0...var162` (`next_var`, `yul_emit.rs:1125--1129`), declared 442 times because the counter restarts per emitted block, so the same name denotes different values in different blocks. A reader cannot tell `var47` from `var48`.

The fix is already half-built. The generator already has `column_eval_var` (`yul_emit.rs:1142`), which produces semantic names like `f_3`, `a_7_next_1`. The infrastructure for meaningful names exists; it is just not used for the generic `init_var` path.

Evidence the hint mechanism already works. In `yul_emit.rs`, `init_var` (L1113) takes an `Option<String>` name and 23 call sites exercise it: **4** `pass a real hint` (`Some(var_name)` at L1041/L1053 for fixed/advice evals, `Some(column_eval_var("i", col, rot))` at L1059, and `Some(format!("{}", ...))` for challenges at L1062–1065), while **19** `pass None` and fall back to `varN` (L727, 904, 917, 927, 966, 973, 976, 989, 993, 1003, 1005, 1014, 1025, 1033, 1068, 1073, 1079, 1085, 1088). The hinted sites are exactly the leaf reads; the `None` sites are the generic arithmetic — `mulmod`/`addmod`/`negation`/`q_pow5`/constant materialization. So the mechanism is proven in production; the readability gap is precisely those 19 generic-arithmetic sites.

Concrete change. Thread a *hint* through `init_var` and fall back to `varN` only when no hint is available. The useful hints are not protocol-level names: `init_var` only emits *gate-expression subterms* (the vanishing, Lagrange and pairing values live in templates and `kzg/mod.rs` and are already named, e.g. `x_n`, `l_0`, `x4_pow_5`). The hint should come from the gate and constraint names the generator already carries in the quotient identity manifest (`builder/api.rs:121`), plus the term’s role, and must be uniquified, because Yul rejects a second `let` of a name visible in the same scope:

```
// yul_emit.rs: init_var already takes Option<String> for the name.
// Callers that know the gate/constraint should pass a unique hint:
let hint = format!("{}", g[gate]_c[constraint]_prod{}, self.next_suffix());
let (lines, prod) = self.init_var(prod_expr, Some(hint));
// next_var() stays as the last-resort fallback.
```

Cost / risk. Low. `init_var` already accepts an optional name; this is a call-site change. The var-cache keys on the *value*, not the name (L1113–1122), so CSE behavior is unchanged. Uniqueness of hints is the one new obligation.

13.4 Fix 3 — Decompose `verifyProof` into named functions

[OPEN]

The problem. `verifyProof` is 3,286 lines of inline Yul. No reviewer holds that in working memory. The protocol has ~12 named stages (parse, challenges, instance-eval, identity families, linearization, multi-open, final pairing, accumulator) but they are not visible as units.

Concrete change. The renderer already emits per-stage blocks (Ch. 5 documents “PCS blocks”, “Final pairing”, etc.). Wrap each stage in a Yul `function` with the protocol name. Stages should communicate through the existing memory map (`THETA_MPTR...X4_MPTR`, `FINAL_COM_MPTR`, ...) and return only a success flag. Returning nine challenges from one function would add exactly the stack pressure the split is meant to remove.

```
function verify_parse_proof(proof_cpctr) -> ok { ... }
function verify_challenges() -> ok { ... }           // writes THETA_MPTR..X4_MPTR
function verify_instance_eval() -> ok { ... }        // writes INSTANCE_EVAL_MPTR
function verify_identity_families() -> ok { ... }    // writes QUOTIENT_EVAL_MPTR
function verify_kzg_multiopen() -> ok { ... }        // writes FINAL_COM_MPTR, V_MPTR
function verify_final_pairing() -> ok { ... }
```

The top-level `verifyProof` becomes a 15-line table of calls that *reads like the protocol spec*. Each function is independently reviewable and independently testable against the Rust oracle. It also bounds each function’s CFG, which is what stock slither’s SSA pass needs (§12.5).

The seam already exists in the assembly pipeline. The contract body is not one monolithic string — it is assembled from a structured `QuotientComputationBlocks` collection (`plan.rs:68` into `template_parts()`) plus `kzg::computations()` (`artifacts.rs:110--117`). The quotient stages are already separated into a typed block collection before they are concatenated into the single assembly (“memory-safe”) body (the test `generated_heavy_assemblies_keep_memory_safe_for_via_ir`, `tests.rs:2127`, asserts that annotation). So Fix 3 is a mechanical transformation at the assembly seam: wrap each element of the block collection in a named `function` and replace the inline concatenation with a call sequence. The decomposition does not require re-deriving the stages — they are already discrete.

Cost / risk. Medium. Yul function calls add a small stack frame; via-IR’s inliner will probably re-inline single-call functions, so gas should be close to neutral, but that is not guaranteed and the bytecode will change. Two further caveats. First, the restructure changes which locals solc spills into `[0x80, memoryguard)`, an area the generated code also writes (§12.11), so re-check the spill area and live ranges after the change. Second, it does not by itself remove the via-IR requirement: the first legacy-codegen stack-too-deep error is in the helper `load_acc_coord_shifted` (L573). This is the largest change but the single biggest auditability win.

13.5 Fix 4 — Annotate the fused `q_com` MSM pairs

[PARTIAL]

From §11.8.77: the quotient commitment is fused into the 78-pair final MSM (63 multi-open terms, 14 linearization terms — 4 quotient limbs and 10 simple selectors — and `f_com`), and `Q_COM_MPTR` is declared-but-unused. Two cheap fixes:

1. **Drop the dead constant.** `Q_COM_MPTR` (L137) is never written — confirmed: `0 mstore/mload/mcopy` operands reference it. [ADDRESSED] for the comment half: L134–136 already say it “intentionally aliases `Q_EVAL_SET_MPTR` and has zero reserved capacity until a future emitter starts writing `Q_COM_MPTR`”. Deleting it (as the unmerged 41b78eb9 does) remains optional. The same applies to the four other dead pointer constants (`CHALLENGE_MPTR`, `G1_IDENTITY_MPTR`, `QUOTIENT_RETURN_MPTR`, `TRACE_U256_MPTR`; §12.5). `G1_IDENTITY_MPTR`’s comment (L144–148) describes “the four `mloads` below”, which this render does not emit.
2. **Annotate the MSM pairs.** [OPEN]: the staging block (L3298–3458) has no per-pair comments. In the final-MSM input build, emit a comment per pair: `// pair 12: q_com contribution (x4_pow_4)`. With Fix 1’s named addresses this becomes self-documenting.

13.6 Fix 5 — Emit a generated address-map appendix

[OPEN]

Even with Fix 1, some addresses remain raw hex. The generator should emit a machine-generated legend as a comment block at the top of the contract (or a companion `.md` file):

```
// MEMORY MAP (generated; do not hand-edit)
// 0x0064 PROOF_CPTR      proof calldata base
// 0x0080 TRANSCRIPT_MPTR transcript buffer (phase: transcript;
//                      = RETURN_MPTR, QUOTIENT_RETURN_MPTR in later phases)
// 0x0080..0x08e0        solc via-IR spill area (memoryguard)
// 0x1ec4 NUM_INSTANCE_CPTR instance count word
// 0x1ee4 INSTANCE_CPTR  instance array base
// 0x4140 q_program       VM bytecode blob (0x11cf bytes)
// 0x6ea0 FINAL_COM_MPTR  KZG reduced commitment
// 0xa320 PCS_FINAL_MSM_MPTR final MSM input scratch (78 pairs;
//                      quotient-VM scratch in an earlier phase)
// ...
```

This is free to generate (the `MemoryMap` already has every `(name, start, len, lifetime)`, `memory.rs:178--186`; the production contract already declares 68 named pointer constants that could seed the legend) and gives a reviewer a single lookup table. It turns “what is 0xa320?” from a grep hunt into a one-line answer. Two details matter: show each region’s lifetime, because addresses are reused across phases, and include the compiler’s spill area from the compiled `memoryguard`, which would make the C11 overlap (§12.11) visible at a glance.

13.7 Fix 6 — Collapse the five near-identical Horner blocks

[OPEN]

The `f_eval` section (§11.8.70) emits five reversed point-set blocks. In the production contract the `f_eval` block spans L3107–3269 and contains five `f_eval := addmod(mulmod(f_eval, x2, r), eval, r)` steps (L3158, L3202, L3231, L3260, L3267). The Horner step itself is one line; what repeats is the per-set Lagrange interpolation that precedes it. Its size depends on the set’s cardinality (3, 3, 2, 2, 1 at L3116, L3160, L3204, L3233, L3262), so a `horner(coeffs, x)` helper would remove almost nothing. The generator should instead emit one loop-based helper, parameterized by the set’s point count and its `ROT_POINTS/Q_EVAL_SET` offsets, and call it five times. This removes ~150 lines of near-copy-paste and makes the “five sets” structure explicit rather than something the reviewer must pattern-match. It is not bytecode-neutral: a loop costs a little more than unrolled code. It also exposes a gas win: each block calls `scalar_inv` (a `modexp`) once, and one batched inversion across the five sets would save four `modexps` (~5.4k gas under EIP-2565, ~16.3k under EIP-7883).

13.8 Prioritized summary

#	Fix	Impact	Effort	Status
1	Named memory addresses (symbolic <code>Value::Identifier</code> at the emit site)	High	Low	[OPEN]
2	Semantic local names (reuse <code>init_var</code> hint)	High	Low	[OPEN]
3	Decompose <code>verifyProof</code> into named functions	High	Med	[OPEN]
4	Annotate fused <code>q_com</code> MSM pairs	Med	Low	[PARTIAL]
5	Generated address-map appendix	Med	Trivial	[OPEN]
6	Loop helper for the five per-set interpolation blocks	Med	Low	[OPEN]

The unifying principle. Fixes 1, 2, and 5 all share one property: *the information is already in the generator; the renderer just discards it*. A single design change — “every emitted address and local carries its semantic name, with hex as fallback” — recovers most of the lost readability without touching a line of protocol logic. For these three, the compiled bytecode is expected to be unchanged and only the source text gains names. That is why they are low-risk: they cannot introduce a soundness bug, because they cannot change what the contract *does* — only what it *looks like* to the human reading it. The pinned runtime hash makes that claim testable. Fixes 3 and 6 do change the code’s structure: they need the full differential suite and the spill-area re-check of §12.11.

What this does not fix. The fuzz-coverage gap (§11.8.69) and the hybrid VM+Yul model (§11.8.71) are not readability problems and are not addressed here. Readability makes the contract *auditable*; it does not make the production shape *tested*. Both matter, and they are orthogonal workstreams.

Chapter 14

Verification pass over the review chapters

Chapters 9–13 contain many checkable claims: line numbers, counts, gas figures, and statements of the form “nothing checks X ”. Each of those claims was re-checked against the code, and the chapters were corrected in place. This chapter records what the pass was checked against, the single set of measurements every chapter now uses, and the corrections that change a conclusion. Inline, a corrected or re-graded item carries a status tag: **[ADDRESSED]** (implemented, or the requested invariant is enforced), **[PARTIAL]**, **[OPEN]**, or **[CORRECTED]** (the original premise or figure was wrong and has been rewritten).

14.1 Baseline

Object	Version checked
Generator (main)	proofs/solidity-verifier at 6b3d2096 (2026-07-17)
Production artifact (“B64”)	Halo2VerifierFee12345B64.sol (3,579 lines) and Halo2VerifyingKeyFee12345B64.sol, nightfall blockchain_assets/contracts/proof_verification/generated/fee_12345/block_size_64/, regenerated 2026-07-28
B64’s generator	midfall rev bd482567, pinned in nightfall’s Cargo.toml with feature truncated-challenges. Its generator sources and templates are identical to the M7 follow-up commit e7f736c4 (2026-06-07, forked from 51f912c2). <i>Not</i> main: it contains the 2026-05-19 hardening commits but not 6b3d2096, and it adds a quotient-certificate VK-header check (B64 L940).
Rust reference	proofs/src/plonk/verifier.rs, proofs/src/poly/kzg/
Production compile	solc 0.8.35, via-IR, optimizer 200 runs, Cancun (nightfall foundry.toml verifier profile)

The 6b3d2096 fixes (transcript-buffer sizing for multi-phase shapes; the quotient-VM constant-slot overflow fallback) cannot change B64. B64 has a single user phase, and its program renders without tripping the overflow path.

14.2 Reference measurements

Earlier drafts of Chapters 11–13 quoted mutually inconsistent figures. Examples: 3,578 vs. 3,579 lines (`wc -l` undercounts by one: the file has no trailing newline); 69, 76 or 78 named constants; 246, 411 or 243 raw-hex operands; “two pairings, 206k gas”. The chapters now use the following values.

Metric (B64)	Value
Lines; <code>verifyProof</code> span	3,579; L293–L3578 (3,286 lines)
Runtime bytecode (production compile)	21,253 B (EIP-170 limit 24,576 B)
Build without via-IR	fails: stack too deep
<code>internal</code> constant declarations	79, of which 68 are <code>*_MPTR/*_CPTR</code>
<code>varN</code> locals	163 distinct, 980 uses
Raw-hex memory operands (first operand of <code>mload/mstore/mcopy</code> in <code>verifyProof</code>)	246 distinct, 492 uses; 64 uses name a <code>*_MPTR/*_CPTR</code> directly, 112 via <code>add(NAME, off)</code>
Comment lines	1,109
<code>revert</code> sites	34
Yul functions; <code>switch/case/if</code>	17; 2 / 15 / 73
<code>staticcall</code> sites (18)	constructor smoke test: 1 G1ADD, 1 G1MSM, 1 pairing. <code>verifyProof</code> : 3 modexp sites, 4 G1ADD, 7 G1MSM (one fused 78-pair, six 1-pair), 1 two-pair pairing
Runtime precompile gas	≈725.9k under EIP-7883 (live): fused MSM 525.1k, six 1-pair MSMs 72.0k, pairing 102.9k, six modexp executions 24.4k (8.1k under the earlier EIP-2565 pricing, total ≈709.6k), G1ADD 1.5k. Every call forwards <code>gas()</code> .
Fused final MSM (78 pairs)	63 multi-open terms + 14 linearization terms (4 quotient limbs, 10 selectors) + f_{com}
Quotient program	4,559 B (0x11cf), 101 instructions, 178 constants
VM opcodes	31 defined in the generator; 11 used and rendered in B64 (0x05, 0x06, 0x08, 0x0b, 0x0d, 0x10, 0x11, 0x19, 0x1b, 0x1f, 0x21); no FOLD_MAIN (0x0a)

14.3 Corrections that change a conclusion

1. **Risk missing from earlier drafts: the memory-safe promise is broken under via-IR.** The production compile sets the free pointer to 0x8e0 and spills Yul variables into [0x80,0x8e0). That range overlaps the generator’s absolute scratch. Honest proofs verify because the live ranges happen to be disjoint, and nothing checks this (§11.8.19, §9.11.2). The Nightfall audit register already records it (CF-0294); unmerged midfall PR #6 rebases the layout above the spill window. [\[OPEN\]](#)
2. **The differential CI does not gate anything.** The trace and fuzz jobs live in the crate’s nested `proofs/solidity-verifier/.github/workflows/ci.yml`. GitHub runs only root workflows, and midfall’s root `ci.yml` has no solidity-verifier job. The harness exists and passes locally, but “checked on every PR” was false. [\[OPEN\]](#)
3. **Provenance had to be reverse-engineered.** B64 does not come from `main` (§14.1). This pass is itself the evidence for C8 (§12.8).
4. **The transcript-buffer bug fixed in 6b3d2096 was potentially soundness-relevant.** For multi-phase shapes, proof bytes overran into the loaded VK, whose words are read afterwards. It is fixed at codegen time, but there is still no runtime cursor bound (§10.2). B64 is single-phase and unaffected. [\[PARTIAL\]](#)
5. **The production shape is covered more closely than claimed.** The IVC Keccak decider test has B64’s permutation parameters, proof layout, 78-pair MSM, commitment counts, constant table and opcode histogram. It is not the same circuit: $k = 20$ vs. 21, a different instance count, and no embedded-quotient mode. Graded MAJOR, now MINOR (§11.8.69).
6. **The VM is small and runs straight through.** The program has 101 instructions and uses 11 opcodes. Every branch in the dispatch body depends only on program bytes or

constants, so one honest verification executes every instruction and every sub-branch of the pinned program. “Untested opcodes” in the shipped artifact is therefore a non-issue: unused opcodes are not rendered and fall into the `default` revert. The real question is whether *this exact program* has been differentially tested (§11.8.39).

7. **The gas figures were wrong in both directions.** EIP-2537 MSMs have no fixed per-call cost. So merging 1-pair MSMs saves $\approx 1.6k$, not $\approx 18k$, and the α -fold MSMs cannot share a call at all. The real wins are appending $(G, r - v)$ and (π, x_3) to the fused MSM ($\approx 13.2k$) and batching the five `scalar_inv` calls ($\approx 16.3k$ under EIP-7883, which has roughly tripled modexp cost). Separately, `gas()` forwarding lets a rejected proof burn almost the whole transaction (§14.4).
8. **Soundness-level figures are withdrawn.** The review chapters quoted several bit levels for the truncated challenges. None of them is retained, and no replacement figure is given; stating the level needs a written soundness argument (C3). The squeeze’s mod- r reduction is not bias-free: the maximum probability of any value is $\approx 1.36/r$. That is negligible, and it matches Rust exactly.
9. **Several “unchecked” items are checked.** On every proof the verifier re-checks the VK’s code size and codehash and six (B64: seven) header words, fails fast on non-canonical instances, and asserts VM termination (e64f2891, 1e21311c, 33f78d2b). C9 is largely AD-DRESSED.
10. **Framing errors.** The verifier is not VK-generic: $k = 21$ is hard-coded. The pairing slot names follow Rust’s `DualMSM` exactly, and a swapped call would reject every honest proof. The patched slither does run; it finds 5 dead constants and a dead store.
11. **Specification errors outside the review chapters.** (2.7) lacked the ω^i factor. ℓ_{last} is L_{n-b-1} , not L_{n-1} ((2.8)). The Chapter 4 opcode table mislabelled seven opcodes.

14.4 Precompile gas limits

Today (B64). All 18 precompile `staticcalls` forward `gas()`. Commit 2b2bf494 replaced earlier hand-tuned caps with this. A rejecting EIP-2537 call burns all gas it was given, and the EVM gives it about 63/64 of what remains. So one malformed proof point costs the caller nearly the whole transaction: midfall PR #6 measured 29.5M of a 30M limit, against $\approx 1.1M$ for an honest verification. This is a griefing/cost issue, not a soundness one, and the register already tracks it (CF-0646, CF-1008, CF-0722). The upside of `gas()` is that B64 kept working through the EIP-7883 modexp repricing, which roughly tripled the cost of its six inversions.

Proposed (unmerged midfall PR #6). Commit f1840bdf forwards the exact scheduled cost at every site (generated constants `G1ADD_GAS = 375`, `G1MSM_GAS_1PAIR = 12,000`, `PAIRING_GAS_2PAIR = 102,900`, `MODEXP_GAS`, ...). Two pinned quotient-evaluator calls keep `gas()`, because a regular call refunds unused gas. The PR first priced modexp with EIP-2565 only. A later commit, 8e65c962 (MF-1), corrects that to $\max(\text{EIP-2565}, \text{EIP-7883}) = 4,064 (+16)$. However, the committed `fixtures/moonlight-wrap/Halo2Verifier.sol` was regenerated *before* MF-1 and still has `MODEXP_GAS = 1360`. Every modexp in that render would run out of gas on a post-Fusaka chain, so every proof would be rejected. The PR’s tests do not catch this, because its pinned `revm 19` prices modexp with EIP-2565 even under `SpecId::OSAKA`.

Verdict. Bounding the gas is right. Bounding it at the *exact* schedule is not:

- The verifier is immutable. An exact bound turns any upward repricing into a permanent liveness failure of every deployed verifier. The constructor probes only catch repricing that

happens *before* deployment. EIP-7883 is exactly such a repricing, and it was mispriced once already within PR #6.

- A margin keeps the burn bounded at a small multiple of the honest cost instead of $\approx 30M$. For example, use $2\text{--}4\times$ the largest known schedule per site, or a single transaction-level budget checked before the precompile calls.
- Test pricing against the chains actually targeted: a fork test on Sepolia/mainnet state, or a revm version whose Osaka handler implements EIP-7883. Regenerate every committed render after a gas-model change.

The quotient-evaluator exception is sound: a regular call returns its unused gas on failure.

14.5 What still stands

No finding in this pass breaks the soundness of B64 as deployed. The correctness verdict of Chapter 11 stands, and so do these criticisms: the truncated-challenge security level is not stated in the artifact (C3); the G1ADD-operand invariant is not machine-checked (C4, the absorbed-point half is); the artifact carries no provenance (C8); every revert is silent; and the readability debt is real (163 `varN` names, mostly raw-hex scratch operands, a 3,286-line function).

14.6 Do the improvement and strengthening suggestions hold up?

Fact-checking a suggestion is not the same as endorsing it. This section gives a verdict on each proposal in Chapters 10–13. One guiding observation: the quotient program is fixed, codehash-pinned and branch-free with respect to proof data. So any defect in the *pinned* code shows up deterministically on the first honest proof, or in the first differential trace. The highest-value strengthening therefore makes sure the exact production artifact is differentially tested, in CI, from a known generator revision. Runtime guards against states a validated generator cannot emit are worth less.

Suggestion	Verdict	Reason
<i>Strengthening</i>		
Wire the crate's trace/fuzz jobs into root CI	Agree, first	Everything the review credits as assurance depends on it, and today it gates nothing.
Trace-test the exact production VK ($k = 21, 19$ instances)	Agree	The harness exists. Run it on the fee-12345 VK, and regenerate B64 in CI from the pinned rev to diff against the committed file.
Fix the <code>memory-safe</code> /via-IR overlap	Agree, high	Plan scratch above a fixed base and assert <code>mload(0x40) ≤ base</code> on entry, failing closed. Add a CI check on the compiled free-pointer initializer. Dropping the annotation is not viable.
Provenance header (C8)	Agree, re-shaped	Stamp the generator rev, VK digest, feature flags and security level. Put them in NatSpec, not constants, and <i>no timestamp</i> : a timestamp breaks reproducible renders.
State the truncated-challenge soundness level (C3)	Agree	Truncation buys no EVM gas: EIP-2537 prices an MSM independently of scalar width. It exists for in-circuit verification and is a global crate feature. Full width for the outermost wrap proof would cost nothing on-chain, but needs per-proof configuration in midnight-proofs .
Transcript cursor runtime bound	Agree	$< 0.3k$ gas. The generator has already mis-sized this buffer once, and an overrun corrupts VK memory.
Codegen check on G1ADD operand provenance (C4)	Agree	Cheap. It completes the absorbed-point check that already exists.

Suggestion	Verdict	Reason
Negative pairing probe at deploy	Agree	Deploy-time only. It must read the G2 bases from the VK, not from per-proof memory.
Runtime VM stack guards (R3/R4)	Low value	<code>validate_quotient_program</code> proves stack safety over the exact pinned bytes, and a violation would fail every honest proof. Acceptable as a few gas of belt and braces, not a priority.
Formal model of the VM ISA (C2/R4)	Disagree	For a fixed, branch-free program, <i>translation validation</i> is complete and far cheaper. Evaluate the pinned program and the Rust gate expressions at random points (Schwartz-Zippel). The Rust test interpreter (<code>tests.rs</code> L3005) already evaluates MODARITH7. It still needs dynamic-length decoding in its fold scanner and models of the three native callbacks (0x19, 0x1b, 0x1f).
Cover all 31 opcodes / per-opcode tests (C2/R2)	Reshape	Unused opcodes are not rendered. Coverage should be defined over the rendered program, which one honest run already covers completely.
Solidity reference implementation (C5)	Disagree	A second implementation to keep in sync. The Rust verifier is the reference; differential tests are the right tool.
Remove the via-IR dependency (C11)	Disagree	Unrealistic at this Yul size. Pin it; it is already pinned in nightfall's <code>foundry.toml</code> .
<i>Code quality and readability</i> q_program disassembly (R1)	Agree, high	101 instructions fit in a comment block. The decoder already exists in the offline validator.
Named addresses (Fix 1) and address map (Fix 5)	Agree, re-shaped	Render <code>add(REGION, off)</code> for interior offsets (an exact-start lookup names almost nothing). Gate the change on a CI check that the runtime bytecode is identical, ignoring metadata.
Semantic local names (Fix 2, C6)	Agree	A call-site change with no semantic effect, under the same bytecode gate.
Split <code>verifyProof</code> into Yul functions (Fix 3/R3)	Disagree	Function frames add stack pressure, and so more via-IR spills: exactly the mechanism behind the <code>memory-safe</code> overlap. Gas neutrality is unproven. Prefer scoped blocks with banner comments and a generated per-stage line index.
Deduplicate the f_{eval} blocks (Fix 6)	Low value	The duplicated code is the per-set Lagrange/inversion block, and the sets have different sizes (3,3,2,2,1). A loop saves source lines and costs gas.
Drop the negation (C10)	Disagree	A sign error fails every honest proof deterministically. Rename the slot (e.g. <code>NEG_QUOTIENT_EVAL_MPTR</code>) instead.
Rename the pairing slots (C7)	Minor	The slots match Rust. Rename only the helper's parameters.
Typed revert codes	Agree, low	Use 4-byte custom errors for debuggability. The success-path cost is unchanged and there is no soundness value.
Delete the dead constants (Fix 4)	Agree	Five are confirmed dead by slither. <code>Q_COM_MPTR</code> is already commented as a marker.
<i>Gas</i> Fold $(G, r - v)$, (π, x_3) into the fused MSM Batch the five <code>scalar_inv</code> calls	Agree, after CI Agree	$\approx 13.2\text{k}$ ($\approx 1.9\%$), but it touches the terminal soundness gate, so land it only once the differential tests gate PRs. $\approx 16.3\text{k}$ under EIP-7883, low risk, and it leaves <code>scalar_inv</code> dead.
Bound precompile gas (PR #6 f1840bdf)	Agree, re-shaped	Bound every call, but at a margin above the schedule rather than the exact cost (§14.4).

Recommended order. (1) Run the crate's jobs in root CI and add the production-VK trace test plus regenerate-and-diff. (2) Fix the `memory-safe` base and add its CI check. (3) Add the provenance and security-level header. (4) Add the transcript cursor bound and the G1ADD

codegen check. (5) Add translation validation of the pinned quotient program. (6) Do the readability work behind a bytecode-equivalence gate. (7) Take the gas wins last. Steps 1–3 are small, and without them every later change is unverified in practice.